

TUGAS AKHIR - EC234801

QUESTION ANSWERING MULTIMODAL CITRA DAN AUDIO BERBASIS MODEL QWEN DENGAN METODE RETRIEVAL-AUGMENTED GENERATION (RAG)

Defender Artha Widiprasetyo

NRP 5024 22 1032

Dosen Pembimbing

Ir. Arta Kusuma Hernanda, S.T., M.T.

NIP 199620231 1 024

Reza Fuad Rachmadi, S.T., M.T., Ph.D

NIP 19850403201212 1 001

Program Studi Strata 1 (S1) Teknik Komputer

Departemen Teknik Komputer

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Surabaya

2026



TUGAS AKHIR - EC234801

**QUESTION ANSWERING MULTIMODAL CITRA DAN
AUDIO BERBASIS MODEL QWEN DENGAN METODE
RETRIEVAL-AUGMENTED GENERATION (RAG)**

Defender Artha Widiprasetyo

NRP 5024 22 1032

Dosen Pembimbing

Ir. Arta Kusuma Hernanda, S.T., M.T.

NIP 199620231 1 024

Reza Fuad Rachmadi, S.T., M.T., Ph.D

NIP 19850403201212 1 001

Program Studi Strata 1 (S1) Teknik Komputer

Departemen Teknik Komputer

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Surabaya

2026

[Halaman ini sengaja dikosongkan]



FINAL PROJECT - EC234801

**MULTIMODAL QUESTION ANSWERING ON IMAGES
AND AUDIO BASED ON QWEN MODEL USING
RETRIEVAL-AUGMENTED GENERATION (RAG)**

Defender Artha Widiprasetyo

NRP 5024 22 1032

Advisors

Ir. Arta Kusuma Hernanda, S.T., M.T.

NIP 199620231 1 024

Reza Fuad Rachmadi, S.T., M.T., Ph.D

NIP 19850403201212 1 001

Undergraduate Study Program of Computer Engineering

Department of Computer Engineering

Faculty of Intelligent Electrical and Informatics Technology

Sepuluh Nopember Institute of Technology

Surabaya

2026

[Halaman ini sengaja dikosongkan]

LEMBAR PENGESAHAN

QUESTION ANSWERING MULTIMODAL CITRA DAN AUDIO BERBASIS MODEL QWEN DENGAN METODE RETRIEVAL-AUGMENTED GENERATION (RAG)

TUGAS AKHIR

Diajukan untuk memenuhi salah satu syarat
memperoleh gelar Sarjana Teknik pada
Program Studi S-1 Teknik Komputer
Departemen Teknik Komputer
Fakultas Teknologi Elektro dan Informatika Cerdas
Institut Teknologi Sepuluh Nopember

Oleh: **Defender Artha Widiprasetyo**
NRP. 5024221032

Disetujui oleh Tim Penguji Tugas Akhir:

Ir. Arta Kusuma Hernanda, S.T., M.T.
NPP: 1996202311024

(Pembimbing I)

.....

Reza Fuad Rachmadi, S.T., M.T., Ph.D.
NIP: 19850403201212 1 001

(Pembimbing II)

.....

Dr. Surya Sumpeno, S.T., M.Sc.
NIP: 19690613199702 1 003

(Penguji I)

.....

Prof. Dr. I Ketut Eddy Purnama, S.T., M.T.
NIP: 19690730199512 1 001

(Examiner II)

.....

Dr. Supeno Mardi Susiki Nugroho, S.T., M.T.
NIP: 19700313199512 1 001

(Examiner III)

.....

Mengetahui,
Kepala Departemen Teknik Komputer FTEIC-ITS

Dr. Arief Kurniawan, S.T., M.T.
NIP. 19740907200212 1 001

SURABAYA
JULY, 2026

[Halaman ini sengaja dikosongkan]

APPROVAL SHEET

MULTIMODAL QUESTION ANSWERING ON IMAGES AND AUDIO BASED ON QWEN MODEL USING RETRIEVAL-AUGMENTED GENERATION (RAG)

FINAL PROJECT

Submitted to fulfill one of the requirements
for obtaining the Bachelor of Engineering degree at
Undergraduate Study Program of Computer Engineering
Department of Computer Engineering
Faculty of Intelligent Electrical and Informatics Technology
Sepuluh Nopember Institute of Technology

By: **Defender Artha Widiprasetyo**
NRP. 5024221032

Approved by the Final Project Examiner Team:

Ir. Arta Kusuma Hernanda, S.T., M.T.
NPP: 1996202311024

(Advisor I)

.....

Reza Fuad Rachmadi, S.T., M.T., Ph.D.
NIP: 19850403201212 1 001

(Advisor II)

.....

Dr. Surya Sumpeno, S.T., M.Sc.
NIP: 19690613199702 1 003

(Examiner I)

.....

Prof. Dr. I Ketut Eddy Purnama, S.T., M.T.
NIP: 19690730199512 1 001

(Examiner II)

.....

Dr. Supeno Mardi Susiki Nugroho, S.T., M.T.
NIP: 19700313199512 1 001

(Examiner III)

.....

Acknowledged,
Head of Computer Engineering Department FTEIC-ITS

Dr. Arief Kurniawan, S.T., M.T.
NIP. 19740907200212 1 001

SURABAYA
JULY, 2026

[Halaman ini sengaja dikosongkan]

PERNYATAAN ORISINALITAS

Yang bertanda tangan di bawah ini:

Nama Mahasiswa / NRP : Defender Artha Widiprasetyo / 5024221032
Departemen : Teknik Komputer
Dosen Pembimbing / NIP : Ir. Arta Kusuma Hernanda, S.T., M.T. / 1996202311024
Reza Fuad Rachmadi, S.T., M.T., Ph.D. / 19850403201212 1
001

Dengan ini menyatakan bahwa Tugas Akhir dengan judul “Question Answering Multimodal Citra dan Audio Berbasis Model Qwen dengan Metode Retrieval-Augmented Generation (RAG)” adalah hasil karya sendiri, bersifat orisinal, dan ditulis dengan mengikuti kaidah penulisan ilmiah.

Bilamana di kemudian hari ditemukan ketidaksesuaian dengan pernyataan ini, maka saya bersedia menerima sanksi sesuai dengan ketentuan yang berlaku di Institut Teknologi Sepuluh Nopember.

Surabaya, July 2026

Mengetahui
Dosen Pembimbing

Mahasiswa

Ir. Arta Kusuma Hernanda, S.T., M.T.
NIP. 1996202311024

Defender Artha Widiprasetyo
NRP. 5024221032

[Halaman ini sengaja dikosongkan]

STATEMENT OF ORIGINALITY

The undersigned below:

Name of student / NRP : Defender Artha Widiprasetyo / 5024221032
Department : Computer Engineering
Advisor / NIP : Ir. Arta Kusuma Hernanda, S.T., M.T. / 1996202311024
Reza Fuad Rachmadi, S.T., M.T., Ph.D. / 19850403201212 1
001

Hereby, I declare that the Final Project entitled “Multimodal Question Answering on Images and Audio Based on Qwen Model Using Retrieval-Augmented Generation (RAG)” is the result of my own work, is original, and is written in accordance with the principles of scientific writing.

If in the future any discrepancy with this statement is found, I am willing to accept sanctions in accordance with the regulations applicable at Sepuluh Nopember Institute of Technology.

Surabaya, July 2026

Acknowledged
Advisor

Student

Ir. Arta Kusuma Hernanda, S.T., M.T.
NIP. 1996202311024

Defender Artha Widiprasetyo
NRP. 5024221032

[Halaman ini sengaja dikosongkan]

ABSTRAK

Nama Mahasiswa : Defender Artha Widiprasetyo
Judul Tugas Akhir : Question Answering Multimodal Citra dan Audio Berbasis Model Qwen dengan Metode Retrieval-Augmented Generation (RAG)
Pembimbing : 1. Arta Kusuma Hernanda, S.T., M.T.
2. Reza Fuad Rachmadi, S.T., M.T., Ph.D

Perkembangan *Large Language Model* (LLM) telah mendorong pemanfaatan kecerdasan buatan dalam sistem *question answering*. Namun, LLM memiliki keterbatasan karena pengetahuan yang digunakan bersifat statis dan berpotensi menghasilkan jawaban yang tidak sesuai fakta atau *hallucination*. Untuk mengatasi permasalahan tersebut, metode *Retrieval-Augmented Generation* (RAG) digunakan dengan menggabungkan kemampuan generatif model bahasa dan mekanisme pencarian informasi dari basis pengetahuan eksternal. Penelitian ini bertujuan untuk merancang, mengimplementasikan, dan mengevaluasi sistem *question answering* berbasis model Qwen dengan metode RAG pada lingkungan komputasi terbatas.

Sistem yang dikembangkan menggunakan basis pengetahuan berbasis dokumen tekstual yang diproses melalui tahapan ekstraksi teks, *chunking*, embedding, dan pengindeksan menggunakan FAISS. Model Qwen digunakan untuk menghasilkan jawaban, sedangkan input multimodal berupa citra dan audio digunakan sebagai skenario pengujian serta konteks tambahan dalam proses penjawaban. Pengujian dilakukan pada beberapa skenario, yaitu *text-only RAG*, *image-only*, *audio-only*, *text-image*, dan *text-audio*. Selain itu, penelitian ini membandingkan konfigurasi model FP16 dan INT4 untuk mengetahui pengaruh kuantisasi terhadap kualitas jawaban, efisiensi komputasi, dan *latency*.

Evaluasi dilakukan menggunakan metrik BLEU, ROUGE-L, *context precision*, *context recall*, *cosine similarity*, WER, dan *latency*. Hasil sementara menunjukkan bahwa sistem RAG mampu mengambil konteks yang relevan dan menghasilkan jawaban yang cukup sesuai dengan dokumen sumber. Konfigurasi INT4 berpotensi menurunkan kebutuhan memori, tetapi tidak selalu menghasilkan waktu respons yang lebih cepat karena dipengaruhi oleh dukungan perangkat keras, proses dekuantisasi, dan kemungkinan *CPU offload*. Dengan demikian, penelitian ini diharapkan dapat memberikan rekomendasi konfigurasi sistem *question answering* berbasis Qwen dan RAG yang optimal antara kualitas jawaban dan efisiensi komputasi.

Kata Kunci: *Question Answering*, Qwen, *Retrieval-Augmented Generation*, RAG, Multimodal, FP16, INT4.

[Halaman ini sengaja dikosongkan]

ABSTRACT

Name : Defender Artha Widiprasetyo
Title : Multimodal Question Answering on Images and Audio Based on Qwen Model Using Retrieval-Augmented Generation (RAG)
Advisors : 1. Arta Kusuma Hernanda, S.T., M.T.
2. Reza Fuad Rachmadi, S.T., M.T., Ph.D

The development of Large Language Models (LLMs) has encouraged the use of artificial intelligence in question answering systems. However, LLMs have limitations because their knowledge is static and may produce factually incorrect answers or hallucinations. To address this issue, Retrieval-Augmented Generation (RAG) is used by combining the generative capability of language models with information retrieval from an external knowledge base. This research aims to design, implement, and evaluate a question answering system based on the Qwen model and the RAG method in a limited-computing-resource environment.

The developed system uses a textual document-based knowledge base processed through text extraction, chunking, embedding, and indexing using FAISS. The Qwen model is used to generate answers, while multimodal inputs such as images and audio are used as testing scenarios and additional context in the answering process. The evaluation is conducted in several scenarios, including text-only RAG, image-only, audio-only, text-image, and text-audio. In addition, this research compares FP16 and INT4 model configurations to analyze the impact of quantization on answer quality, computational efficiency, and latency.

The evaluation uses BLEU, ROUGE-L, context precision, context recall, cosine similarity, WER, and latency metrics. Preliminary results show that the RAG system can retrieve relevant contexts and generate answers that are sufficiently aligned with the source documents. The INT4 configuration has the potential to reduce memory usage, but it does not always produce faster response time because it is affected by hardware support, dequantization processes, and possible CPU offloading. Therefore, this research is expected to provide recommendations for an optimal Qwen and RAG-based question answering system configuration by balancing answer quality and computational efficiency.

Keywords: Question Answering, Qwen, Retrieval-Augmented Generation, RAG, Multimodal, FP16, INT4.

[Halaman ini sengaja dikosongkan]

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa atas segala rahmat, karunia, dan kemudahan yang telah diberikan sehingga penulis dapat menyelesaikan penyusunan Tugas Akhir yang berjudul *Question Answering Multimodal Citra dan Audio Berbasis Model Qwen dengan Metode Retrieval-Augmented Generation (RAG)*.

Tugas Akhir ini disusun sebagai salah satu syarat untuk memperoleh gelar Sarjana Teknik pada Program Studi S-1 Teknik Komputer, Departemen Teknik Komputer, Fakultas Teknologi Elektro dan Informatika Cerdas, Institut Teknologi Sepuluh Nopember. Penelitian ini membahas pengembangan dan evaluasi sistem *question answering* berbasis model Qwen dengan metode *Retrieval-Augmented Generation (RAG)*, khususnya dalam konteks pemrosesan input teks, citra, dan audio serta analisis performa sistem pada lingkungan komputasi terbatas.

Dalam proses penyusunan Tugas Akhir ini, penulis menyadari bahwa banyak pihak telah memberikan dukungan, arahan, bantuan, dan doa. Oleh karena itu, penulis mengucapkan terima kasih kepada:

1. Tuhan Yang Maha Esa atas segala rahmat, kesehatan, kekuatan, dan kelancaran yang diberikan kepada penulis selama proses penyusunan Tugas Akhir ini.
2. Keluarga tercinta yang senantiasa memberikan doa, dukungan moral, semangat, dan motivasi kepada penulis dalam menyelesaikan studi dan penyusunan Tugas Akhir ini.
3. Bapak Ir. Arta Kusuma Hernanda, S.T., M.T., selaku Dosen Pembimbing I yang telah memberikan bimbingan, arahan, masukan, serta dukungan selama proses penyusunan dan pengembangan penelitian ini.
4. Bapak Reza Fuad Rachmadi, S.T., M.T., Ph.D., selaku Dosen Pembimbing II yang telah memberikan bimbingan, saran, dan masukan berharga dalam penyusunan Tugas Akhir ini.
5. Seluruh dosen dan tenaga kependidikan Departemen Teknik Komputer ITS yang telah memberikan ilmu, fasilitas, dan bantuan selama penulis menjalani masa studi.
6. Teman-teman dan pihak lain yang tidak dapat disebutkan satu per satu karena telah memberikan bantuan, dukungan, dan semangat selama proses penyusunan Tugas Akhir ini.

Penulis menyadari bahwa Tugas Akhir ini masih memiliki keterbatasan dan kekurangan. Oleh karena itu, penulis terbuka terhadap kritik dan saran yang membangun untuk perbaikan di masa mendatang. Akhir kata, semoga Tugas Akhir ini dapat memberikan manfaat bagi pembaca, pengembang sistem, serta penelitian selanjutnya di bidang *question answering*, *Large Language Model*, dan *Retrieval-Augmented Generation*.

Surabaya, Mei 2026

Defender Artha Widiprasetyo

[Halaman ini sengaja dikosongkan]

DAFTAR ISI

ABSTRAK	i
ABSTRACT	iii
KATA PENGANTAR	v
DAFTAR ISI	vii
DAFTAR GAMBAR	xiii
DAFTAR TABEL	xvii
1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Batasan Masalah atau Ruang Lingkup	3
1.4 Tujuan	3
1.5 Manfaat	4
2 TINJAUAN PUSTAKA	5
2.1 Penelitian Terdahulu	5
2.1.1 Penelitian Yin et al. tentang Multimodal Large Language Model	5
2.1.2 Penelitian Wang et al. tentang Qwen2-VL	5
2.1.3 Penelitian Chu et al. tentang Qwen2-Audio	6
2.1.4 Penelitian Lewis et al. dan Li et al. tentang Retrieval-Augmented Generation	6
2.1.5 Penelitian Gu dan Qin tentang Qwen dan RAG	6
2.1.6 Penelitian tentang Indeks Vektor dan Efisiensi Retrieval	7
2.1.7 Posisi Penelitian	7
2.2 Dasar Teori	8
2.2.1 Large Language Model (LLM)	8
2.2.2 Tokenisasi, Batas Konteks, dan Generasi Teks	9
2.2.3 Multimodal Language Model	10
2.2.4 Sistem Question Answering	11
2.2.5 Retrieval-Augmented Generation (RAG)	11
2.2.6 Model Qwen	12
2.2.7 Preprocessing Data	13
2.2.8 Embedding Teks dan Representasi Semantik	14

2.2.9	FAISS sebagai Indeks Vektor	15
2.2.10	Chunking Dokumen	15
2.2.11	Konfigurasi Representasi Numerik Model	16
2.2.12	Kebutuhan Memori GPU	18
2.2.13	Latency dan Pengukuran Waktu Inferensi	18
2.2.14	Metrik Evaluasi Sistem	19
2.2.15	Matriks Evaluasi Penelitian	20
3	DESAIN DAN IMPLEMENTASI	23
3.1	Desain Penelitian	23
3.1.1	Ruang Lingkup Pengembangan Sistem	23
3.1.2	Tahapan Penelitian	24
3.1.3	Diagram Metodologi Penelitian	26
3.2	Lingkungan Implementasi dan Pengujian	27
3.2.1	Spesifikasi Perangkat Keras	27
3.2.2	Spesifikasi Perangkat Lunak	27
3.2.3	Model yang Digunakan	28
3.3	Data Penelitian	28
3.3.1	Dokumen Basis Pengetahuan	28
3.3.2	Ringkasan Data Uji	30
3.3.3	Data Uji Text-Only RAG	30
3.3.4	Data Uji Image-Only	31
3.3.5	Data Uji Audio-Only	32
3.3.6	Data Uji Text-Image RAG	33
3.3.7	Data Uji Text-Audio RAG	34
3.3.8	<i>Ground Truth</i>	34
3.4	<i>Preprocessing</i> Data	35
3.4.1	<i>Preprocessing</i> Dokumen	36
3.4.2	<i>Preprocessing</i> Citra	36
3.4.3	<i>Preprocessing</i> Audio	36
3.5	Pembangunan Indeks Pengetahuan	36
3.5.1	Konfigurasi Utama: <i>Structure-Aware Fixed-Size Chunking</i>	37
3.5.2	Konfigurasi Pembandingan: <i>Recursive Chunking</i>	38
3.5.3	Statistik Hasil <i>Chunking</i>	39
3.5.4	<i>Embedding</i> dan Normalisasi Vektor	39
3.5.5	FAISS dan Metadata	39
3.6	Perancangan Sistem	40
3.6.1	Arsitektur Sistem	40
3.6.2	Mekanisme <i>Retrieval</i> dan <i>Reranking</i> Berbasis Domain	42
3.6.3	<i>Backend</i> dan <i>Endpoint</i> API	42

3.6.4	Manajemen Model dan Memori GPU	42
3.7	Implementasi <i>Pipeline</i> Sistem dan Pengujian	43
3.7.1	<i>Text-Only RAG</i>	43
3.7.2	<i>Text-Image RAG</i>	44
3.7.3	<i>Text-Audio RAG</i>	44
3.7.4	<i>Pipeline</i> Pengujian <i>Image-Only</i>	45
3.7.5	<i>Pipeline</i> Pengujian <i>Audio-Only</i>	45
3.8	Konfigurasi Representasi Numerik Model	46
3.8.1	FP32	46
3.8.2	FP16	46
3.8.3	4-bit NF4	46
3.9	Desain Pengujian	46
3.9.1	Skenario Evaluasi	46
3.9.2	Konfigurasi Eksperimen	47
3.9.3	Kontrol Eksperimen	48
3.9.4	Validasi Konfigurasi Sebelum Pengujian	48
3.9.5	Evaluasi Kualitas Keluaran	48
3.9.6	Evaluasi Kualitas <i>Retrieval</i>	49
3.9.7	Pengukuran <i>Latency</i>	50
3.9.8	<i>Profiling</i> Kebutuhan Memori GPU	50
3.10	Metode Analisis	50
4	PENGUJIAN DAN ANALISIS	53
4.1	Gambaran Umum Pengujian	53
4.2	Hasil Implementasi Sistem Question Answering	53
4.2.1	Tampilan Antarmuka Sistem	54
4.2.2	Contoh Respons Text-Only RAG	54
4.2.3	Contoh Respons Text-Image RAG	55
4.2.4	Contoh Respons Text-Audio RAG	56
4.2.5	Fitur Manajemen Dokumen	56
4.3	Konfigurasi dan Ruang Lingkup Hasil Pengujian	57
4.4	Hasil Pengujian Non-Retrieval Multimodal	58
4.4.1	Image-Only	58
4.4.2	Audio-Only	61
4.5	Hasil Pengujian RAG dengan Konfigurasi Chunking Utama Fixed-Size	64
4.5.1	Text-Only RAG Fixed-Size	64
4.5.2	Text-Image RAG Fixed-Size	68
4.5.3	Text-Audio RAG Fixed-Size	73
4.6	Pengujian Tambahan Latency pada Konfigurasi Fixed-Size	77
4.6.1	Ringkasan Hasil Pengujian Tambahan Latency	77

4.6.2	Hasil INT8 pada Setiap Skenario Fixed-Size	78
4.6.3	Analisis Latency Text-Only RAG Fixed-Size	79
4.6.4	Analisis Latency Image-Only	80
4.6.5	Analisis Latency Audio-Only	80
4.6.6	Analisis Latency Text-Image RAG Fixed-Size	81
4.6.7	Analisis Latency Text-Audio RAG Fixed-Size	81
4.6.8	Pembacaan FP32 Awal dan FP32 Strict	82
4.7	Hasil Pengujian RAG dengan Konfigurasi Chunking Pembanding Recursive . .	83
4.7.1	Text-Only RAG Recursive	83
4.7.2	Text-Image RAG Recursive	87
4.7.3	Text-Audio RAG Recursive	91
4.8	Perbandingan Konfigurasi Chunking	95
4.8.1	Perbandingan Chunking pada Text-Only RAG	96
4.8.2	Perbandingan Chunking pada Text-Image RAG	97
4.8.3	Perbandingan Chunking pada Text-Audio RAG	99
4.8.4	Ringkasan Perbandingan Konfigurasi Chunking	100
4.9	Hasil Profiling Kebutuhan Memori GPU	102
4.9.1	Profiling Text-Only RAG Fixed-Size	102
4.9.2	Profiling Image-Only	104
4.9.3	Profiling Audio-Only	105
4.9.4	Profiling Text-Image RAG Fixed-Size	107
4.9.5	Profiling Text-Audio RAG Fixed-Size	109
4.9.6	Ringkasan Profiling Memori GPU	110
4.10	Perbandingan Keseluruhan Konfigurasi Representasi Numerik Model	112
4.11	Analisis Trade-Off Kualitas, Latency, dan Memori GPU	117
4.11.1	Trade-Off pada Skenario Teks	117
4.11.2	Trade-Off pada Skenario Citra	118
4.11.3	Trade-Off pada Skenario Audio	118
4.11.4	Trade-Off Memori GPU dan Latency	119
4.12	Keterbatasan Pengujian Tambahan	120
4.13	Rekomendasi Konfigurasi Sistem	120
4.14	Ringkasan Temuan Bab IV	122
5	PENUTUP	123
5.1	Kesimpulan	123
5.2	Saran	123
	DAFTAR PUSTAKA	125
A	HASIL JAWABAN SISTEM TERPILIH	129
A.1	Data Media Uji Terpilih	129

A.2 Hasil Jawaban Skenario Image-Only	130
A.3 Hasil Jawaban Skenario Audio-Only	132
A.4 Hasil Jawaban Skenario Text-Only RAG Fixed-Size	134
A.5 Hasil Jawaban Skenario Text-Image RAG Fixed-Size	134
A.6 Hasil Jawaban Skenario Text-Audio RAG Fixed-Size	136
A.7 Hasil Jawaban Skenario Text-Only RAG Recursive	137
A.8 Hasil Jawaban Skenario Text-Image RAG Recursive	138
A.9 Hasil Jawaban Skenario Text-Audio RAG Recursive	139
A.10 Ringkasan Pengujian Tambahan Fixed-Size INT8 dan FP32 Strict	140
BIOGRAFI PENULIS	143

[Halaman ini sengaja dikosongkan]

DAFTAR GAMBAR

3.1	Tiga dimensi eksperimen penelitian	24
3.2	Tahapan penelitian	25
3.3	Diagram metodologi penelitian	26
3.4	Arsitektur sistem <i>question answering</i> multimodal	41
4.1	Tampilan antarmuka utama sistem <i>question answering</i>	54
4.2	Contoh respons sistem pada mode <i>text-only RAG</i>	55
4.3	Contoh respons sistem pada mode <i>text-image RAG</i>	55
4.4	Contoh respons sistem pada mode <i>text-audio RAG</i>	56
4.5	Tampilan fitur manajemen dokumen pada sistem.	57
4.6	Perbandingan <i>cosine similarity</i> per data pada skenario <i>image-only</i>	58
4.7	Perbandingan <i>latency</i> per data pada skenario <i>image-only</i>	59
4.8	Perbandingan rata-rata <i>cosine similarity</i> pada skenario <i>image-only</i>	60
4.9	Perbandingan rata-rata <i>latency</i> pada skenario <i>image-only</i>	60
4.10	Perbandingan <i>cosine similarity</i> per data pada skenario <i>audio-only</i>	61
4.11	Perbandingan <i>latency</i> per data pada skenario <i>audio-only</i>	62
4.12	Perbandingan rata-rata <i>cosine similarity</i> pada skenario <i>audio-only</i>	63
4.13	Perbandingan rata-rata <i>latency</i> pada skenario <i>audio-only</i>	63
4.14	Perbandingan BLEU-1 per data pada skenario <i>text-only RAG fixed-size</i>	65
4.15	Perbandingan ROUGE-L F1 per data pada skenario <i>text-only RAG fixed-size</i>	65
4.16	Perbandingan <i>context precision</i> per data pada skenario <i>text-only RAG fixed-size</i>	66
4.17	Perbandingan <i>context recall</i> per data pada skenario <i>text-only RAG fixed-size</i>	66
4.18	Perbandingan <i>latency</i> per data pada skenario <i>text-only RAG fixed-size</i>	67
4.19	Perbandingan rata-rata BLEU-1, ROUGE-L F1, <i>context precision</i> , dan <i>context recall</i> pada skenario <i>text-only RAG fixed-size</i>	67
4.20	Perbandingan rata-rata <i>latency</i> pada skenario <i>text-only RAG fixed-size</i>	68
4.21	Perbandingan BLEU-1 per data pada skenario <i>text-image RAG fixed-size</i>	69
4.22	Perbandingan ROUGE-L F1 per data pada skenario <i>text-image RAG fixed-size</i>	69
4.23	Perbandingan <i>context precision</i> per data pada skenario <i>text-image RAG fixed-size</i>	70
4.24	Perbandingan <i>context recall</i> per data pada skenario <i>text-image RAG fixed-size</i>	70
4.25	Perbandingan <i>latency</i> per data pada skenario <i>text-image RAG fixed-size</i>	71
4.26	Perbandingan rata-rata BLEU-1, ROUGE-L F1, <i>context precision</i> , dan <i>context recall</i> pada skenario <i>text-image RAG fixed-size</i>	71
4.27	Perbandingan rata-rata <i>latency</i> pada skenario <i>text-image RAG fixed-size</i>	72
4.28	Perbandingan BLEU-1 per data pada skenario <i>text-audio RAG fixed-size</i>	73
4.29	Perbandingan ROUGE-L F1 per data pada skenario <i>text-audio RAG fixed-size</i>	73

4.30	Perbandingan <i>context precision</i> per data pada skenario <i>text-audio RAG fixed-size</i> .	74
4.31	Perbandingan <i>context recall</i> per data pada skenario <i>text-audio RAG fixed-size</i> .	74
4.32	Perbandingan <i>latency</i> per data pada skenario <i>text-audio RAG fixed-size</i> .	75
4.33	Perbandingan rata-rata BLEU-1, ROUGE-L F1, <i>context precision</i> , dan <i>context recall</i> pada skenario <i>text-audio RAG fixed-size</i> .	75
4.34	Perbandingan rata-rata <i>latency</i> pada skenario <i>text-audio RAG fixed-size</i> .	76
4.35	Perbandingan <i>latency</i> pengujian tambahan pada konfigurasi <i>fixed-size</i> .	78
4.36	Ringkasan skor kualitas utama pada pengujian INT8.	79
4.37	Perbandingan <i>latency</i> tambahan pada <i>text-only RAG fixed-size</i> .	80
4.38	Perbandingan <i>latency</i> tambahan pada skenario <i>image-only</i> .	80
4.39	Perbandingan <i>latency</i> tambahan pada skenario <i>audio-only</i> .	81
4.40	Perbandingan <i>latency</i> tambahan pada <i>text-image RAG fixed-size</i> .	81
4.41	Perbandingan <i>latency</i> tambahan pada <i>text-audio RAG fixed-size</i> .	82
4.42	Perbandingan BLEU-1 per data pada skenario <i>text-only RAG recursive</i> .	83
4.43	Perbandingan ROUGE-L F1 per data pada skenario <i>text-only RAG recursive</i> .	84
4.44	Perbandingan <i>context precision</i> per data pada skenario <i>text-only RAG recursive</i> .	84
4.45	Perbandingan <i>context recall</i> per data pada skenario <i>text-only RAG recursive</i> .	85
4.46	Perbandingan <i>latency</i> per data pada skenario <i>text-only RAG recursive</i> .	85
4.47	Perbandingan rata-rata BLEU-1, ROUGE-L F1, <i>context precision</i> , dan <i>context recall</i> pada skenario <i>text-only RAG recursive</i> .	86
4.48	Perbandingan rata-rata <i>latency</i> pada skenario <i>text-only RAG recursive</i> .	86
4.49	Perbandingan BLEU-1 per data pada skenario <i>text-image RAG recursive</i> .	87
4.50	Perbandingan ROUGE-L F1 per data pada skenario <i>text-image RAG recursive</i> .	88
4.51	Perbandingan <i>context precision</i> per data pada skenario <i>text-image RAG recursive</i> .	88
4.52	Perbandingan <i>context recall</i> per data pada skenario <i>text-image RAG recursive</i> .	89
4.53	Perbandingan <i>latency</i> per data pada skenario <i>text-image RAG recursive</i> .	89
4.54	Perbandingan rata-rata BLEU-1, ROUGE-L F1, <i>context precision</i> , dan <i>context recall</i> pada skenario <i>text-image RAG recursive</i> .	90
4.55	Perbandingan rata-rata <i>latency</i> pada skenario <i>text-image RAG recursive</i> .	90
4.56	Perbandingan BLEU-1 per data pada skenario <i>text-audio RAG recursive</i> .	91
4.57	Perbandingan ROUGE-L F1 per data pada skenario <i>text-audio RAG recursive</i> .	92
4.58	Perbandingan <i>context precision</i> per data pada skenario <i>text-audio RAG recursive</i> .	92
4.59	Perbandingan <i>context recall</i> per data pada skenario <i>text-audio RAG recursive</i> .	93
4.60	Perbandingan <i>latency</i> per data pada skenario <i>text-audio RAG recursive</i> .	93
4.61	Perbandingan rata-rata BLEU-1, ROUGE-L F1, <i>context precision</i> , dan <i>context recall</i> pada skenario <i>text-audio RAG recursive</i> .	94
4.62	Perbandingan rata-rata <i>latency</i> pada skenario <i>text-audio RAG recursive</i> .	94
4.63	Perbandingan kualitas dan <i>retrieval</i> antara konfigurasi <i>fixed-size</i> dan <i>recursive</i> pada skenario <i>text-only RAG</i> .	96
4.64	Perbandingan <i>latency</i> antara konfigurasi <i>fixed-size</i> dan <i>recursive</i> pada skenario <i>text-only RAG</i> .	96

4.65	Perbandingan kualitas dan <i>retrieval</i> antara konfigurasi <i>fixed-size</i> dan <i>recursive</i> pada skenario <i>text-image RAG</i>	97
4.66	Perbandingan <i>latency</i> antara konfigurasi <i>fixed-size</i> dan <i>recursive</i> pada skenario <i>text-image RAG</i>	98
4.67	Perbandingan kualitas dan <i>retrieval</i> antara konfigurasi <i>fixed-size</i> dan <i>recursive</i> pada skenario <i>text-audio RAG</i>	99
4.68	Perbandingan <i>latency</i> antara konfigurasi <i>fixed-size</i> dan <i>recursive</i> pada skenario <i>text-audio RAG</i>	99
4.69	Ringkasan perbandingan kualitas dan <i>retrieval</i> antara konfigurasi <i>fixed-size</i> dan <i>recursive</i>	101
4.70	Ringkasan perbandingan <i>latency</i> antara konfigurasi <i>fixed-size</i> dan <i>recursive</i> . . .	101
4.71	Profiling kebutuhan memori GPU pada skenario <i>text-only RAG fixed-size</i>	103
4.72	Perbandingan <i>latency</i> profiling pada skenario <i>text-only RAG fixed-size</i>	103
4.73	Profiling kebutuhan memori GPU pada skenario <i>image-only</i>	104
4.74	Perbandingan <i>latency</i> profiling pada skenario <i>image-only</i>	105
4.75	Profiling kebutuhan memori GPU pada skenario <i>audio-only</i>	106
4.76	Perbandingan <i>latency</i> profiling pada skenario <i>audio-only</i>	107
4.77	Profiling kebutuhan memori GPU pada skenario <i>text-image RAG fixed-size</i> . . .	108
4.78	Perbandingan <i>latency</i> profiling pada skenario <i>text-image RAG fixed-size</i>	108
4.79	Profiling kebutuhan memori GPU pada skenario <i>text-audio RAG fixed-size</i>	109
4.80	Perbandingan <i>latency</i> profiling pada skenario <i>text-audio RAG fixed-size</i>	110
4.81	Ringkasan kebutuhan memori GPU pada seluruh skenario profiling.	111
4.82	Ringkasan <i>latency</i> pada seluruh skenario profiling.	111
4.83	Perbandingan kualitas keluaran dan kualitas <i>retrieval</i> berdasarkan konfigurasi representasi numerik model.	115
4.84	Perbandingan <i>latency</i> berdasarkan konfigurasi representasi numerik model. . .	115
4.85	Perbandingan kebutuhan memori GPU berdasarkan konfigurasi representasi numerik model, termasuk hasil tambahan INT8 WO A16W8.	116
4.86	Analisis <i>trade-off</i> antara kualitas keluaran, <i>latency</i> , dan kebutuhan memori GPU.	119

[Halaman ini sengaja dikosongkan]

DAFTAR TABEL

2.1	Ringkasan penelitian atau perancangan terdahulu	7
2.2	Matriks evaluasi sistem pada penelitian	20
3.1	Spesifikasi perangkat keras implementasi dan pengujian	27
3.2	Perangkat lunak yang digunakan	27
3.3	Model yang digunakan dalam sistem	28
3.4	Dokumen sumber pada basis pengetahuan	28
3.5	Ringkasan data uji penelitian	30
3.6	Data uji <i>text-only RAG</i>	30
3.7	Data uji <i>image-only</i>	31
3.8	Data uji <i>audio-only</i>	32
3.9	Data uji <i>text-image RAG</i>	33
3.10	Data uji <i>text-audio RAG</i>	34
3.11	Contoh <i>ground truth</i> pada data uji	35
3.12	Statistik hasil konfigurasi <i>chunking</i> setelah penyaringan	39
3.13	Skenario evaluasi sistem	47
3.14	Konfigurasi eksperimen	47
4.1	Ruang lingkup hasil pengujian pada Bab IV	57
4.2	Rata-rata hasil pengujian <i>image-only</i>	59
4.3	Rata-rata hasil pengujian <i>audio-only</i>	62
4.4	Rata-rata hasil pengujian <i>text-only RAG fixed-size</i>	67
4.5	Rata-rata hasil pengujian <i>text-image RAG fixed-size</i>	71
4.6	Rata-rata hasil pengujian <i>text-audio RAG fixed-size</i>	75
4.7	Ruang lingkup pengujian tambahan latency pada konfigurasi <i>fixed-size</i>	77
4.8	Ringkasan latency pengujian tambahan pada konfigurasi <i>fixed-size</i>	78
4.9	Ringkasan kualitas pendukung dan latency pada pengujian INT8 fixed-size	79
4.10	Rata-rata hasil pengujian <i>text-only RAG recursive</i>	85
4.11	Rata-rata hasil pengujian <i>text-image RAG recursive</i>	90
4.12	Rata-rata hasil pengujian <i>text-audio RAG recursive</i>	93
4.13	Ringkasan perbandingan konfigurasi chunking pada seluruh skenario RAG	100
4.14	Ringkasan hasil profiling kebutuhan memori GPU	110
4.15	Ringkasan hasil pengujian non-RAG multimodal berdasarkan konfigurasi representasi numerik model	113
4.16	Ringkasan hasil pengujian RAG <i>fixed-size</i> berdasarkan konfigurasi representasi numerik model	113

4.17 Ringkasan hasil pengujian RAG <i>recursive</i> berdasarkan konfigurasi representasi numerik model	114
4.18 Rekomendasi konfigurasi sistem berdasarkan prioritas penggunaan	121
A.1 Data citra terpilih pada lampiran hasil jawaban sistem	129
A.2 Data audio terpilih pada lampiran hasil jawaban sistem	129
A.3 Hasil jawaban terpilih pada skenario image-only	130
A.4 Hasil jawaban terpilih pada skenario audio-only	132
A.5 Hasil jawaban terpilih pada skenario text-only RAG fixed-size	134
A.6 Hasil jawaban terpilih pada skenario text-image RAG fixed-size	135
A.7 Hasil jawaban terpilih pada skenario text-audio RAG fixed-size	136
A.8 Hasil jawaban terpilih pada skenario text-only RAG recursive	137
A.9 Hasil jawaban terpilih pada skenario text-image RAG recursive	138
A.10 Hasil jawaban terpilih pada skenario text-audio RAG recursive	139
A.11 Ringkasan pengujian tambahan fixed-size INT8 dan FP32 strict	140
A.12 Urutan latency hasil pengujian tambahan fixed-size	141

BAB I

PENDAHULUAN

1.1 Latar Belakang

Institusi pendidikan tinggi memiliki kebutuhan layanan informasi yang kompleks karena informasi akademik, administratif, dan layanan mahasiswa sering tersebar dalam berbagai dokumen, panduan, pengumuman, serta prosedur institusi. Kondisi tersebut dapat menyulitkan pengguna dalam memperoleh informasi secara cepat, konsisten, dan sesuai konteks, terutama ketika pertanyaan yang diajukan berkaitan dengan dokumen atau prosedur tertentu. Dalam konteks ini, sistem *question answering* dapat digunakan untuk membantu proses pencarian informasi karena memungkinkan pengguna menyampaikan pertanyaan dalam bahasa alami dan memperoleh jawaban berdasarkan sumber pengetahuan yang tersedia. Penelitian mengenai penerapan sistem tanya jawab di lingkungan universitas menunjukkan bahwa teknologi tersebut dapat dimanfaatkan untuk mendukung layanan informasi akademik dan administratif [1].

Perkembangan model bahasa berskala besar atau *Large Language Models* (LLM) semakin memperluas kemampuan sistem *question answering* dalam memahami pertanyaan dan menghasilkan jawaban berbasis bahasa alami. LLM mampu mendukung berbagai tugas pemrosesan bahasa alami, termasuk *question answering*. Namun, penggunaan LLM secara langsung memiliki keterbatasan karena pengetahuan model bergantung pada data pelatihan dan tidak selalu mencakup informasi spesifik institusi yang bersifat lokal, dinamis, atau tersimpan dalam dokumen tertentu. Selain itu, LLM dapat menghasilkan jawaban yang tidak akurat, tidak sesuai fakta, atau tidak relevan terhadap konteks pertanyaan. Fenomena tersebut dikenal sebagai *hallucination* [2]. Keterbatasan ini menjadi permasalahan penting karena jawaban yang dihasilkan oleh sistem perlu mengacu pada sumber informasi yang benar dan dapat dipertanggungjawabkan.

Salah satu pendekatan yang digunakan untuk mengatasi keterbatasan tersebut adalah *Retrieval-Augmented Generation* (RAG). RAG menggabungkan kemampuan generatif LLM dengan mekanisme pengambilan informasi dari basis pengetahuan eksternal. Dalam pendekatan ini, sistem terlebih dahulu mengambil informasi yang relevan dari dokumen sumber, kemudian menggunakan informasi tersebut sebagai dasar dalam menghasilkan jawaban [3]. Dalam konteks pendidikan, penggunaan RAG dinilai relevan karena dapat membantu membatasi jawaban berdasarkan cakupan pengetahuan tertentu dan mengurangi kemungkinan munculnya jawaban yang tidak didukung oleh sumber [4]. Oleh karena itu, RAG sesuai digunakan dalam sistem *question answering* berbasis dokumen institusi.

Meskipun sistem *question answering* berbasis dokumen umumnya menggunakan input teks, kebutuhan pengguna tidak selalu terbatas pada pertanyaan tekstual. Dalam praktiknya, pengguna dapat memberikan informasi tambahan dalam bentuk citra, seperti tangkapan layar, foto dokumen, atau gambar pengumuman, serta dalam bentuk audio. Kondisi tersebut mendorong kebutuhan terhadap sistem *question answering* multimodal yang mampu memproses lebih dari satu jenis input. Model bahasa multimodal memungkinkan sistem memahami informasi lintas modalitas dan menghasilkan keluaran berdasarkan konteks yang lebih luas dibandingkan model berbasis teks saja [5].

Salah satu keluarga model yang mendukung pengembangan sistem tersebut adalah Qwen.

Keluarga model Qwen menyediakan kemampuan pemrosesan teks, citra, dan audio melalui varian model yang berbeda [6], [7], [8]. Kemampuan tersebut memungkinkan pengembangan sistem *question answering* multimodal dengan pendekatan modular, yaitu setiap jenis input diproses menggunakan komponen model yang sesuai, kemudian hasilnya diintegrasikan dalam proses penjawaban berbasis dokumen.

Di sisi lain, penerapan sistem *question answering* multimodal berbasis LLM dan RAG menghadapi tantangan dari sisi kebutuhan sumber daya komputasi. Pemrosesan teks, citra, audio, retrieval dokumen, dan generasi jawaban membutuhkan kapasitas komputasi dan memori yang berbeda. Kondisi ini menjadi penting ketika sistem diterapkan pada lingkungan dengan keterbatasan GPU dan memori. Oleh karena itu, evaluasi sistem perlu mempertimbangkan tidak hanya kualitas keluaran, tetapi juga *latency* dan kebutuhan memori GPU.

Selain konfigurasi model, metode pemotongan dokumen atau *chunking* juga berpengaruh terhadap proses retrieval. Dokumen perlu dibagi menjadi bagian-bagian yang lebih kecil agar informasi yang relevan dapat ditemukan dan digunakan sebagai konteks jawaban. Perbedaan metode *chunking* dapat menghasilkan karakteristik konteks retrieval yang berbeda. Oleh karena itu, pengaruh metode *chunking* terhadap kualitas jawaban dan kualitas retrieval perlu dievaluasi.

Berdasarkan uraian tersebut, penelitian ini berfokus pada pengembangan dan evaluasi sistem *question answering* multimodal citra dan audio berbasis model Qwen dengan metode RAG. Sistem dikembangkan menggunakan arsitektur modular untuk memproses input teks, citra, dan audio serta mengintegrasikannya dengan basis pengetahuan berbasis dokumen tekstual. Evaluasi dilakukan pada beberapa skenario untuk mengetahui pengaruh metode *chunking* dan konfigurasi representasi numerik model terhadap performa sistem. Hasil evaluasi digunakan untuk merumuskan rekomendasi berdasarkan *trade-off* antara kualitas keluaran, *latency*, dan kebutuhan memori GPU pada lingkungan komputasi terbatas.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana merancang dan mengimplementasikan sistem *question answering* multimodal berbasis model Qwen dengan metode *Retrieval-Augmented Generation* (RAG) untuk memproses input teks, citra, dan audio?
2. Bagaimana performa sistem dalam menghasilkan jawaban yang relevan dan sesuai konteks pada skenario berbasis RAG?
3. Bagaimana kemampuan sistem dalam memproses input citra dan audio pada skenario non-retrieval?
4. Bagaimana pengaruh metode *chunking* terhadap kualitas jawaban, kualitas retrieval, dan *latency* pada skenario berbasis RAG?
5. Bagaimana pengaruh konfigurasi representasi numerik model terhadap kualitas keluaran, *latency*, dan kebutuhan memori GPU?
6. Konfigurasi sistem seperti apa yang memberikan *trade-off* terbaik antara kualitas keluaran, *latency*, dan kebutuhan memori GPU?

1.3 Batasan Masalah atau Ruang Lingkup

Agar penelitian ini terarah dan fokus pada tujuan yang telah ditetapkan, ruang lingkup penelitian dibatasi pada beberapa hal berikut:

1. Penelitian ini berfokus pada pengembangan dan evaluasi sistem *question answering* multimodal berbasis model Qwen dengan metode *Retrieval-Augmented Generation* (RAG).
2. Sistem dikembangkan menggunakan arsitektur modular untuk memproses input teks, citra, dan audio.
3. Basis pengetahuan yang digunakan dalam proses retrieval berupa dokumen tekstual, sedangkan citra dan audio digunakan sebagai input pengguna.
4. Pengujian dilakukan pada skenario berbasis RAG dan non-retrieval yang melibatkan input teks, citra, dan audio.
5. Evaluasi sistem mencakup kualitas keluaran, kualitas retrieval, kemampuan pemrosesan multimodal, *latency*, dan kebutuhan memori GPU.
6. Penelitian membandingkan lebih dari satu metode *chunking* dan lebih dari satu konfigurasi representasi numerik model.
7. Pengujian performa sistem dilakukan dalam kondisi *single user*, sehingga hasil *latency* dan kebutuhan memori GPU merepresentasikan pemrosesan satu permintaan pada satu waktu, bukan pengujian beban banyak pengguna secara bersamaan.
8. Penelitian ini tidak membahas pelatihan ulang atau *fine-tuning* model, pengembangan model embedding baru, pembangunan indeks multimodal berbasis citra atau audio, maupun evaluasi UI/UX dan kepuasan pengguna.

1.4 Tujuan

Berdasarkan rumusan masalah yang telah disampaikan, tujuan penelitian ini adalah sebagai berikut:

1. Merancang dan mengimplementasikan sistem *question answering* multimodal berbasis model Qwen dengan metode *Retrieval-Augmented Generation* (RAG).
2. Mengevaluasi performa sistem pada skenario berbasis RAG dalam menghasilkan jawaban yang relevan dan sesuai konteks.
3. Mengevaluasi kemampuan sistem dalam memproses input citra dan audio pada skenario non-retrieval.
4. Menganalisis pengaruh metode *chunking* terhadap kualitas jawaban, kualitas retrieval, dan *latency*.
5. Menganalisis pengaruh konfigurasi representasi numerik model terhadap kualitas keluaran, *latency*, dan kebutuhan memori GPU.
6. Merumuskan rekomendasi konfigurasi sistem berdasarkan *trade-off* antara kualitas keluaran, *latency*, dan kebutuhan memori GPU pada lingkungan dengan sumber daya komputasi terbatas.

1.5 Manfaat

Penelitian ini diharapkan dapat memberikan manfaat bagi berbagai pihak, baik secara teoritis maupun praktis, sebagai berikut:

1. Bagi Penulis

Penelitian ini memberikan kesempatan untuk memperdalam pemahaman mengenai pengembangan sistem *question answering* multimodal berbasis model Qwen dan metode *Retrieval-Augmented Generation* (RAG). Selain itu, penelitian ini memperkuat keterampilan dalam implementasi pipeline RAG, pemrosesan input teks, citra, dan audio, serta evaluasi performa sistem pada lingkungan dengan keterbatasan sumber daya.

2. Bagi Pengembang Sistem

Penelitian ini dapat menjadi acuan dalam membangun sistem *question answering* berbasis dokumen yang mampu memproses input teks, citra, dan audio. Hasil penelitian dapat membantu pengembang dalam memilih metode *chunking* dan konfigurasi representasi numerik model berdasarkan kebutuhan implementasi.

3. Bagi Institusi Akademik

Penelitian ini dapat menjadi dasar pengembangan sistem pendukung akses informasi akademik berbasis dokumen. Melalui pendekatan RAG, informasi yang tersebar dalam dokumen institusi dapat dimanfaatkan sebagai sumber pengetahuan yang lebih mudah diakses melalui sistem *question answering*.

4. Bagi Peneliti Selanjutnya

Penelitian ini dapat menjadi referensi awal dalam pengembangan sistem *question answering* multimodal berbasis LLM lokal, RAG, dan arsitektur modular. Hasil evaluasi dapat digunakan sebagai dasar untuk penelitian lanjutan mengenai optimasi performa, pengembangan metode retrieval, perluasan modalitas input, atau penerapan sistem pada domain institusi lain.

BAB II

TINJAUAN PUSTAKA

Bab ini membahas hasil penelitian atau perancangan terdahulu serta teori dan konsep teknis yang digunakan dalam pengembangan dan evaluasi sistem *question answering* multimodal citra dan audio berbasis model Qwen dengan metode *Retrieval-Augmented Generation* (RAG). Pembahasan penelitian terdahulu digunakan untuk menunjukkan posisi penelitian ini terhadap kajian sebelumnya, sedangkan pembahasan dasar teori mencakup *Large Language Model* (LLM), tokenisasi dan generasi autoregresif, model bahasa multimodal, sistem *question answering*, RAG, model Qwen, preprocessing data, embedding teks, FAISS, retrieval dan mekanisme gating, chunking dokumen, konfigurasi representasi numerik model, kuantisasi 4-bit NF4, kebutuhan memori GPU, latency, serta metrik evaluasi yang digunakan dalam penelitian.

2.1 Penelitian Terdahulu

Penelitian ini berkaitan dengan beberapa kelompok studi sebelumnya, yaitu pengembangan *Multimodal Large Language Model* (MLLM), pengembangan model Qwen untuk pemrosesan citra dan audio, penerapan *Retrieval-Augmented Generation* (RAG) pada sistem *question answering*, serta penggunaan indeks vektor dalam pencarian konteks dokumen. Bagian ini merangkum penelitian terdahulu yang menjadi landasan dan pembandingan terhadap sistem yang dikembangkan.

2.1.1 Penelitian Yin et al. tentang Multimodal Large Language Model

Yin et al. melakukan kajian komprehensif mengenai perkembangan *Multimodal Large Language Model* (MLLM) [5]. Kajian tersebut menjelaskan bahwa MLLM memperluas kemampuan LLM tradisional dengan menambahkan pemrosesan lintas-modalitas, seperti teks, citra, audio, dan video. Secara umum, arsitektur MLLM terdiri atas *multimodal encoder*, *modality interface*, dan LLM utama sebagai komponen penalaran. Komponen-komponen tersebut memungkinkan masukan non-teks dikonversi menjadi representasi yang dapat dipahami oleh model bahasa.

Studi tersebut juga menjelaskan bahwa pengembangan MLLM tidak hanya bergantung pada ukuran model, tetapi juga pada proses penyesuaian antarmodalitas dan data instruksi multimodal. Tantangan yang dibahas meliputi *multimodal hallucination*, keterbatasan konteks panjang, kebutuhan komputasi yang tinggi, serta kesulitan mengevaluasi keluaran multimodal secara menyeluruh. Relevansi studi ini terhadap penelitian yang dilakukan terletak pada dasar konseptual pemrosesan multimodal, khususnya bagaimana citra dan audio dapat diubah menjadi konteks yang mendukung proses penjawaban berbasis teks.

2.1.2 Penelitian Wang et al. tentang Qwen2-VL

Wang et al. memperkenalkan Qwen2-VL sebagai model *vision-language* yang dirancang untuk memahami informasi visual pada berbagai resolusi citra [7]. Model ini memiliki kemampuan untuk melakukan *visual question answering*, memahami dokumen visual, membaca informasi dalam gambar, dan menangani struktur visual yang kompleks. Kemampuan tersebut menjadikan Qwen2-VL relevan untuk sistem yang memanfaatkan citra sebagai konteks tambahan dalam proses penjawaban.

Dalam penelitian ini, Qwen2-VL digunakan untuk memproses masukan citra pada skenario

image-only dan *text-image RAG*. Berbeda dari studi Wang et al. yang berfokus pada pengembangan dan evaluasi model secara umum, penelitian ini memanfaatkan Qwen2-VL sebagai komponen dalam sistem *question answering* berbasis dokumen. Dengan demikian, kontribusi penelitian ini terletak pada integrasi kemampuan visual Qwen2-VL dengan proses retrieval dokumen dan generasi jawaban berbasis RAG.

2.1.3 Penelitian Chu et al. tentang Qwen2-Audio

Chu et al. memperkenalkan Qwen2-Audio sebagai model bahasa audio besar yang mampu memproses masukan audio dan menghasilkan respons berbasis teks [8]. Model ini mendukung berbagai tugas, seperti pengenalan ucapan, analisis audio, dan interaksi berbasis suara. Qwen2-Audio menunjukkan bahwa informasi audio dapat dipahami secara langsung oleh model bahasa melalui representasi audio yang diselaraskan dengan kemampuan generatif LLM.

Penelitian ini menggunakan Qwen2-Audio untuk mendukung skenario *audio-only* dan *text-audio RAG*. Pada skenario tersebut, audio tidak diperlakukan sebagai basis pengetahuan utama, melainkan sebagai konteks tambahan yang membantu sistem memahami maksud pertanyaan atau informasi pendukung dari pengguna. Dengan demikian, penelitian ini memperluas pemanfaatan Qwen2-Audio ke dalam alur RAG berbasis dokumen dan mengevaluasi dampaknya terhadap kualitas jawaban, latency, serta kebutuhan memori GPU.

2.1.4 Penelitian Lewis et al. dan Li et al. tentang Retrieval-Augmented Generation

Lewis et al. memperkenalkan RAG sebagai pendekatan yang menggabungkan mekanisme retrieval dengan model generatif untuk menyelesaikan tugas NLP yang membutuhkan pengetahuan eksternal [3]. Dalam RAG, sistem terlebih dahulu mengambil dokumen atau pasase yang relevan dari basis pengetahuan, kemudian menggunakan konteks tersebut untuk menghasilkan jawaban. Pendekatan ini mengurangi ketergantungan model terhadap pengetahuan parametrik yang tersimpan selama pelatihan.

Li et al. membahas penerapan RAG pada konteks pendidikan melalui survei sistematis [9]. Studi tersebut menegaskan bahwa RAG dapat meningkatkan relevansi dan faktualitas jawaban karena model diberi konteks dari dokumen eksternal. Hal ini penting pada lingkungan pendidikan dan institusional karena jawaban yang diberikan perlu dapat diverifikasi terhadap sumber tertentu. Relevansi kedua penelitian tersebut terhadap penelitian ini terletak pada penggunaan RAG untuk membangun sistem *question answering* berbasis dokumen institusional.

2.1.5 Penelitian Gu dan Qin tentang Qwen dan RAG

Gu dan Qin mengembangkan sistem *question answering* untuk artikel arXiv dengan menggabungkan model Qwen dan RAG [10]. Sistem tersebut menggunakan mekanisme pemrosesan data, pemilihan rute, pencarian konten, dan generasi jawaban. Model Qwen digunakan sebagai komponen generatif, sedangkan retrieval digunakan untuk mengambil informasi akademik yang relevan sebelum proses penjawaban.

Hasil penelitian tersebut menunjukkan bahwa kombinasi Qwen dan RAG dapat meningkatkan relevansi serta koherensi jawaban dibandingkan penggunaan model bahasa tanpa retrieval. Penelitian tersebut memiliki kedekatan dengan penelitian ini karena sama-sama memanfaatkan Qwen dan RAG. Perbedaannya, penelitian ini tidak hanya berfokus pada dokumen akademik berbasis teks, tetapi juga mengevaluasi masukan multimodal berupa citra dan audio, membandingkan konfigurasi *chunking*, serta menganalisis konfigurasi representasi numerik model FP32, FP16, dan 4-bit NF4.

2.1.6 Penelitian tentang Indeks Vektor dan Efisiensi Retrieval

Indeks vektor merupakan komponen penting dalam sistem RAG karena menentukan kecepatan dan relevansi pengambilan konteks. FAISS digunakan secara luas untuk pencarian kemiripan vektor berskala besar karena menyediakan mekanisme pencarian *nearest neighbor* yang efisien [11]. Dalam sistem RAG, FAISS memungkinkan embedding query dibandingkan dengan embedding chunk dokumen untuk memperoleh konteks yang paling relevan.

Penelitian ini memanfaatkan FAISS sebagai indeks vektor untuk menyimpan embedding chunk dokumen. Berbeda dari studi yang hanya berfokus pada efisiensi pencarian vektor, penelitian ini menempatkan FAISS sebagai bagian dari alur sistem yang lebih luas, yaitu preprocessing dokumen, chunking, embedding, retrieval, reranking, dan generasi jawaban oleh model Qwen. Dengan demikian, performa sistem tidak hanya dipengaruhi oleh pencarian vektor, tetapi juga oleh kualitas chunk, konfigurasi model, dan kemampuan generatif model dalam memanfaatkan konteks hasil retrieval.

2.1.7 Posisi Penelitian

Berdasarkan penelitian terdahulu, dapat disimpulkan bahwa kajian MLLM telah memberikan dasar konseptual untuk pemrosesan multimodal, Qwen2-VL dan Qwen2-Audio telah menunjukkan kemampuan pemrosesan citra dan audio, RAG telah terbukti relevan untuk sistem *question answering* berbasis dokumen, dan FAISS mendukung retrieval berbasis embedding secara efisien. Namun, sebagian besar penelitian terdahulu belum secara bersamaan mengevaluasi sistem Qwen berbasis RAG pada masukan teks, citra, dan audio dengan konfigurasi *chunking* dan representasi numerik model yang berbeda.

Penelitian ini menempati posisi sebagai penelitian implementatif dan evaluatif yang mengintegrasikan Qwen2.5, Qwen2-VL, Qwen2-Audio, FAISS, dan RAG dalam satu sistem *question answering* multimodal berbasis dokumen. Selain itu, penelitian ini menganalisis *trade-off* antara kualitas keluaran, kualitas retrieval, latency, dan kebutuhan memori GPU pada konfigurasi FP32, FP16, dan 4-bit NF4.

Tabel 2.1: Ringkasan penelitian atau perancangan terdahulu

Penelitian	Fokus	Hasil atau Kontribusi	Relevansi dengan Penelitian Ini
Yin et al. [5]	Survei MLLM	Menjelaskan arsitektur, pelatihan, kemampuan, dan tantangan model multimodal	Menjadi dasar konseptual pemrosesan citra dan audio dalam sistem QA
Wang et al. [7]	Qwen2-VL	Mengembangkan model visi-bahasa untuk pemahaman citra, dokumen visual, dan VQA	Menjadi dasar penggunaan Qwen2-VL pada skenario <i>image-only</i> dan <i>text-image RAG</i>
Chu et al. [8]	Qwen2-Audio	Mengembangkan model audio-bahasa untuk pemrosesan audio dan respons tekstual	Menjadi dasar penggunaan Qwen2-Audio pada skenario <i>audio-only</i> dan <i>text-audio RAG</i>
Lewis et al. [3]	RAG	Menggabungkan retrieval dan model generatif untuk tugas berbasis pengetahuan eksternal	Menjadi dasar arsitektur RAG dalam sistem QA berbasis dokumen

Tabel 2.1 – lanjutan

Penelitian	Fokus	Hasil atau Kontribusi	Relevansi dengan Penelitian Ini
Li et al. [9]	RAG pada pendidikan	Menunjukkan relevansi RAG untuk meningkatkan faktualitas dan relevansi jawaban pada domain pendidikan	Mendukung penggunaan RAG pada konteks institusi pendidikan
Gu dan Qin [10]	Qwen dan RAG	Mengembangkan sistem QA artikel arXiv berbasis Qwen dan RAG	Menjadi pembanding terdekat untuk integrasi Qwen dan RAG, tetapi penelitian ini menambahkan masukan citra, audio, chunking, dan profiling GPU
Douze et al. [11]	FAISS	Menyediakan pustaka pencarian kemiripan vektor yang efisien	Menjadi dasar penggunaan FAISS sebagai indeks embedding dokumen

2.2 Dasar Teori

Bagian ini menjelaskan konsep-konsep teknis yang menjadi dasar perancangan, implementasi, dan evaluasi sistem. Konsep yang dibahas meliputi model bahasa besar, pemrosesan multimodal, RAG, model Qwen, preprocessing data, embedding, indeks vektor, chunking, representasi numerik model, kebutuhan memori GPU, latency, serta metrik evaluasi.

2.2.1 Large Language Model (LLM)

Large Language Model (LLM) merupakan model bahasa berukuran besar yang dilatih menggunakan korpus teks berskala besar untuk memahami dan menghasilkan bahasa alami. LLM umumnya dibangun menggunakan arsitektur *transformer*, yaitu arsitektur jaringan saraf yang memanfaatkan mekanisme *self-attention* untuk menangkap hubungan antar-token dalam suatu urutan teks [12]. Arsitektur ini menjadi dasar bagi banyak model bahasa modern karena mampu memproses konteks panjang dan mempelajari hubungan semantik secara lebih fleksibel dibandingkan pendekatan berbasis urutan tradisional.

Pada model *transformer*, setiap token dalam masukan diubah menjadi representasi vektor. Representasi tersebut kemudian diproses melalui mekanisme *attention* untuk menentukan seberapa besar suatu token perlu memperhatikan token lain dalam konteks yang sama. Secara umum, mekanisme *scaled dot-product attention* dapat dirumuskan sebagai berikut:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2.1)$$

Pada Persamaan 2.1, Q merupakan matriks *query*, K merupakan matriks *key*, V merupakan matriks *value*, dan d_k merupakan dimensi dari vektor *key*. Hasil perkalian QK^T menunjukkan skor kemiripan antara token yang sedang diproses dengan token lain dalam konteks. Nilai tersebut kemudian dinormalisasi menggunakan fungsi *softmax* dan digunakan untuk menghitung kombinasi berbobot dari V .

Selain *single-head attention*, arsitektur *transformer* juga menggunakan *multi-head attention*. Mekanisme ini memungkinkan model mempelajari beberapa jenis hubungan antar-token secara

paralel. Secara umum, *multi-head attention* dapat dituliskan sebagai berikut:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (2.2)$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (2.3)$$

Pada Persamaan 2.2 dan 2.3, h menunjukkan jumlah *attention head*, sedangkan W_i^Q , W_i^K , W_i^V , dan W^O merupakan matriks bobot yang dipelajari selama proses pelatihan.

LLM umumnya dilatih menggunakan objektif prediksi token berikutnya atau *next-token prediction*. Jika diberikan urutan token $x_1, x_2, \dots, x_t$, model mempelajari probabilitas token berikutnya x_{t+1} berdasarkan konteks sebelumnya. Secara sederhana, objektif tersebut dapat dituliskan sebagai berikut:

$$P(x_{t+1} \mid x_1, x_2, \dots, x_t) \quad (2.4)$$

Pada tahap inferensi, model menghasilkan token secara autoregresif, yaitu token yang dihasilkan pada langkah sebelumnya digunakan kembali sebagai bagian dari konteks untuk menghasilkan token berikutnya. Proses ini memungkinkan LLM menghasilkan jawaban dalam bentuk teks yang koheren.

Meskipun LLM memiliki kemampuan generatif yang kuat, model ini memiliki keterbatasan karena pengetahuan yang digunakan berasal dari parameter hasil pelatihan. Pengetahuan tersebut bersifat statis dan tidak selalu sesuai dengan informasi lokal, terbaru, atau spesifik terhadap dokumen tertentu. Akibatnya, model dapat menghasilkan jawaban yang tidak sesuai fakta, tidak relevan, atau tidak dapat diverifikasi. Fenomena ini dikenal sebagai *hallucination* [2]. Oleh karena itu, sistem *question answering* berbasis LLM membutuhkan mekanisme tambahan agar jawaban yang dihasilkan dapat diarahkan pada sumber pengetahuan yang lebih spesifik dan dapat ditelusuri.

2.2.2 Tokenisasi, Batas Konteks, dan Generasi Teks

Sebelum diproses oleh model bahasa, teks diubah menjadi unit-unit diskret yang disebut token. Token dapat berupa kata, bagian kata, karakter, atau simbol, bergantung pada tokenizer yang digunakan. Tokenizer memetakan teks menjadi deretan indeks numerik yang dapat diterima oleh model. Oleh karena itu, panjang teks dalam karakter atau kata tidak selalu sama dengan panjang teks dalam token.

Setiap model memiliki batas panjang masukan yang menentukan jumlah maksimum token yang dapat diproses dalam satu inferensi. Apabila gabungan instruksi, pertanyaan, dan konteks dokumen melebihi batas tersebut, masukan perlu dipotong melalui mekanisme *truncation*. Pemotongan memungkinkan masukan tetap sesuai dengan kapasitas model, tetapi berpotensi menghilangkan informasi pada bagian yang terpotong [13]. Dalam sistem RAG, ukuran chunk, jumlah konteks hasil retrieval, dan batas token masukan perlu disusun secara konsisten agar informasi penting tetap masuk ke dalam prompt.

Panjang keluaran dapat dikendalikan menggunakan parameter *maximum new tokens*. Parameter ini membatasi jumlah token baru yang dihasilkan di luar token pada prompt [14]. Karena model menghasilkan teks secara autoregresif, semakin banyak token keluaran, semakin banyak langkah inferensi yang diperlukan.

Pemilihan token berikutnya dapat dilakukan melalui beberapa strategi decoding. Pada

greedy decoding, model memilih token dengan probabilitas tertinggi pada setiap langkah:

$$\hat{x}_{t+1} = \arg \max_x P(x | x_1, \dots, x_t) \quad (2.5)$$

Strategi ini bersifat deterministik untuk kondisi inferensi yang sama. Sebaliknya, strategi sampling memilih token berdasarkan distribusi probabilitas dan dapat menghasilkan keluaran yang bervariasi. Dalam eksperimen komparatif, generasi deterministik membantu mengurangi variasi yang tidak berasal dari konfigurasi model yang dibandingkan.

Model autoregresif juga dapat menggunakan *Key-Value cache* atau KV cache. KV cache menyimpan representasi *key* dan *value* dari token yang telah diproses agar tidak dihitung ulang pada setiap langkah generasi [15]. Penggunaan cache dapat mempercepat generasi, tetapi menambah kebutuhan memori. Besarnya KV cache dipengaruhi oleh panjang konteks, jumlah token keluaran, jumlah layer, struktur attention, dimensi representasi, dan jenis data yang digunakan.

2.2.3 Multimodal Language Model

Multimodal Language Model merupakan pengembangan dari LLM yang tidak hanya memproses teks, tetapi juga mampu menerima dan memahami modalitas lain, seperti citra, audio, atau video. Model ini memungkinkan sistem memanfaatkan informasi lintas modalitas untuk menghasilkan respons yang lebih kontekstual. Dalam penelitian ini, modalitas yang digunakan mencakup teks, citra, dan audio.

Secara umum, model bahasa multimodal dibangun dengan menghubungkan encoder khusus modalitas tertentu dengan model bahasa. Pada model *vision-language*, citra diproses oleh encoder visual untuk menghasilkan representasi visual. Representasi tersebut kemudian disejajarkan dengan ruang representasi bahasa agar dapat dipahami oleh LLM. Pada model *audio-language*, sinyal audio diproses oleh encoder audio untuk menghasilkan representasi suara yang kemudian dihubungkan dengan model bahasa. Proses penyelarasan ini dikenal sebagai *multimodal alignment*, yaitu proses yang memungkinkan model memahami hubungan semantik antara teks dan modalitas lain [5].

Dalam konteks input citra, model *vision-language* dapat digunakan untuk mengenali objek, teks visual, tampilan antarmuka, aktivitas, atau struktur visual pada gambar. Keluaran model dapat berupa deskripsi gambar, jawaban atas pertanyaan terkait gambar, atau konteks visual dalam bentuk teks. Dalam konteks input audio, model *audio-language* dapat digunakan untuk memahami ucapan, menghasilkan transkripsi, merangkum isi audio, atau menjawab pertanyaan berdasarkan isi audio.

Pada penelitian ini, kemampuan multimodal digunakan untuk mendukung sistem *question answering* yang mampu menerima masukan berupa teks, citra, dan audio. Namun, integrasi multimodal dilakukan secara modular. Citra dan audio diproses oleh model khusus untuk menghasilkan konteks tekstual, kemudian konteks tersebut digabungkan dengan pertanyaan pengguna dan konteks dokumen hasil retrieval. Dengan pendekatan ini, sistem dapat mempertimbangkan informasi visual dan audio tanpa harus membangun indeks multimodal berbasis citra atau audio. Integrasi tersebut dapat dipandang sebagai *late fusion* pada tingkat prompt karena keluaran dari pemroses modalitas digabungkan setelah informasi visual atau audio direpresentasikan dalam bentuk bahasa. Pada skenario non-retrieval, model multimodal menghasilkan keluaran berdasarkan media yang diberikan. Pada skenario RAG, informasi multimodal digunakan bersama pertanyaan dan konteks dokumen untuk membentuk jawaban akhir.

2.2.4 Sistem Question Answering

Sistem *Question Answering* (QA) merupakan sistem yang dirancang untuk menjawab pertanyaan pengguna dalam bahasa alami. Berbeda dengan sistem pencarian dokumen yang biasanya mengembalikan daftar dokumen atau tautan, sistem QA berupaya memberikan jawaban langsung terhadap pertanyaan yang diberikan. Sistem QA dapat dibedakan menjadi *open-domain question answering* dan *closed-domain question answering*.

Pada *open-domain question answering*, sistem dapat menjawab pertanyaan dari berbagai topik dengan cakupan pengetahuan yang luas. Sebaliknya, pada *closed-domain question answering*, sistem menjawab pertanyaan berdasarkan cakupan sumber pengetahuan tertentu, seperti dokumen institusi, dokumen layanan, panduan teknis, atau korpus khusus. Penelitian ini menggunakan pendekatan *closed-domain question answering* karena jawaban diarahkan agar sesuai dengan dokumen sumber yang dimasukkan ke dalam basis pengetahuan.

Secara umum, sistem QA berbasis dokumen dapat dipandang sebagai fungsi yang memetakan pertanyaan pengguna x dan himpunan dokumen sumber D menjadi jawaban y . Hubungan tersebut dapat dituliskan sebagai berikut:

$$y = f(x, D) \quad (2.6)$$

Pada sistem QA berbasis LLM tanpa retrieval, model dapat menghasilkan jawaban hanya berdasarkan parameter internalnya. Namun, pendekatan tersebut kurang sesuai untuk informasi institusi yang spesifik, dinamis, dan harus mengacu pada dokumen tertentu. Oleh karena itu, pendekatan RAG digunakan agar fungsi penjawaban tidak hanya bergantung pada model generatif, tetapi juga pada konteks dokumen yang relevan.

Dalam penelitian ini, sistem QA dikembangkan untuk mendukung beberapa skenario masukan, yaitu teks, citra, audio, teks-citra, dan teks-audio. Skenario berbasis RAG digunakan untuk mengevaluasi kemampuan sistem menjawab pertanyaan berdasarkan konteks dokumen, sedangkan skenario non-retrieval multimodal digunakan untuk mengevaluasi kemampuan model dalam memahami citra dan audio secara terpisah.

2.2.5 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) merupakan pendekatan yang menggabungkan mekanisme pencarian informasi atau *retrieval* dengan kemampuan generatif model bahasa. RAG diperkenalkan sebagai pendekatan yang memungkinkan model generatif menggunakan dokumen eksternal sebagai sumber pengetahuan tambahan [3]. Dengan demikian, jawaban yang dihasilkan tidak hanya bergantung pada pengetahuan internal model, tetapi juga pada konteks dokumen yang diambil dari basis pengetahuan.

Secara umum, RAG terdiri dari dua tahap utama, yaitu tahap *retrieval* dan tahap *generation*. Pada tahap *retrieval*, pertanyaan pengguna diubah menjadi representasi vektor, kemudian dibandingkan dengan representasi vektor dari potongan dokumen. Sistem mengambil beberapa potongan dokumen yang memiliki kemiripan tertinggi terhadap pertanyaan. Pada tahap *generation*, potongan dokumen tersebut digabungkan dengan pertanyaan pengguna dan diberikan kepada LLM untuk menghasilkan jawaban akhir.

Jika x menyatakan pertanyaan pengguna dan $D = \{d_1, d_2, \dots, d_n\}$ menyatakan kumpulan potongan dokumen dalam basis pengetahuan, maka proses retrieval bertujuan memilih himpunan konteks C_k yang terdiri dari k potongan dokumen paling relevan:

$$C_k = \text{TopK}(x, D) \quad (2.7)$$

Setelah konteks diperoleh, jawaban dihasilkan oleh model generatif M_θ berdasarkan pertanyaan x dan konteks C_k :

$$\hat{y} = M_\theta(x, C_k) \quad (2.8)$$

Pada implementasi praktis, konteks C_k tidak diberikan sebagai variabel terpisah, tetapi disusun ke dalam sebuah prompt bersama instruksi dan pertanyaan pengguna. Jika I menyatakan instruksi sistem, m menyatakan konteks multimodal yang berasal dari citra atau audio, maka prompt dapat dirumuskan secara konseptual sebagai berikut:

$$P = \text{Concat}(I, C_k, m, x) \quad (2.9)$$

Kemudian jawaban akhir diperoleh melalui:

$$\hat{y} = M_\theta(P) \quad (2.10)$$

Pada penelitian ini, RAG digunakan pada skenario *text-only RAG*, *text-image RAG*, dan *text-audio RAG*. Pada skenario *text-only RAG*, sistem menggunakan pertanyaan teks dan konteks dokumen hasil retrieval. Pada skenario *text-image RAG*, konteks visual dari citra ditambahkan ke dalam prompt. Pada skenario *text-audio RAG*, konteks audio ditambahkan ke dalam prompt. Dengan demikian, RAG menjadi mekanisme utama untuk menjaga agar jawaban tetap terikat pada dokumen sumber, sedangkan input citra dan audio berperan sebagai konteks tambahan.

Top-K Retrieval dan Relevance Gating

Top-k retrieval memilih k konteks dengan skor tertinggi. Nilai k yang terlalu kecil dapat menyebabkan informasi penting tidak ikut diambil, sedangkan nilai k yang terlalu besar dapat memasukkan konteks yang kurang relevan dan menambah panjang prompt.

Selain *Top-k*, sistem dapat menggunakan ambang relevansi atau *similarity threshold*. Jika skor konteks terbaik berada di bawah ambang τ , konteks dapat dinilai tidak cukup meyakinkan untuk digunakan. Mekanisme tersebut dapat dituliskan sebagai:

$$g(x) = \begin{cases} 1, & \text{jika } s_{\max} \geq \tau \\ 0, & \text{jika } s_{\max} < \tau \end{cases} \quad (2.11)$$

Pada Persamaan 2.11, $s_{\max}$ merupakan skor konteks terbaik. Jika $g(x) = 1$, konteks dianggap cukup relevan. Jika $g(x) = 0$, sistem dapat menggunakan respons fallback atau jalur penjawaban lain. Gating mengurangi risiko penggunaan konteks yang tidak sesuai, tetapi ambang yang terlalu tinggi dapat menolak konteks yang sebenarnya bermanfaat. Oleh karena itu, nilai ambang merupakan parameter implementasi yang perlu ditetapkan dan digunakan secara konsisten.

2.2.6 Model Qwen

Qwen merupakan keluarga model bahasa yang dikembangkan untuk berbagai kebutuhan pemrosesan bahasa dan multimodal. Dalam penelitian ini, model Qwen digunakan sebagai komponen utama dalam sistem karena memiliki varian yang mendukung pemrosesan teks, citra, dan audio. Pendekatan ini sesuai dengan rancangan sistem yang menggunakan arsitektur

modular.

Qwen2.5 untuk Pemrosesan Teks

Qwen2.5 merupakan model bahasa yang dikembangkan untuk memahami instruksi dan menghasilkan jawaban berbasis teks. Model ini digunakan sebagai model generatif utama pada sistem yang dikembangkan. Pada skenario berbasis RAG, Qwen2.5 menerima prompt yang berisi instruksi, pertanyaan pengguna, konteks hasil retrieval, dan jika tersedia, konteks tambahan dari citra atau audio. Model kemudian menghasilkan jawaban akhir berdasarkan prompt tersebut [6].

Pemilihan Qwen2.5-1.5B-Instruct dalam penelitian ini berkaitan dengan kebutuhan sistem yang mempertimbangkan keterbatasan sumber daya komputasi. Model dengan ukuran yang lebih kecil dibandingkan model LLM berparameter besar dapat lebih memungkinkan untuk dijalankan pada lingkungan lokal atau terbatas, meskipun tetap perlu dievaluasi dari sisi kualitas jawaban.

Qwen2-VL untuk Pemrosesan Citra

Qwen2-VL merupakan varian *vision-language* dari keluarga Qwen yang dirancang untuk memproses input teks dan citra. Model ini mendukung pemahaman visual terhadap gambar dengan resolusi beragam dan dapat digunakan untuk menjawab pertanyaan berbasis citra, mendeskripsikan gambar, atau mengekstraksi informasi visual tertentu. Qwen2-VL juga menggunakan pendekatan *Multimodal Rotary Position Embedding* (MRoPE) untuk membantu model memahami posisi dan struktur visual pada gambar [7].

Dalam penelitian ini, Qwen2-VL digunakan pada skenario *image-only* dan *text-image RAG*. Pada skenario *image-only*, model digunakan untuk mengevaluasi kemampuan pemahaman citra tanpa retrieval dokumen. Pada skenario *text-image RAG*, hasil pemrosesan citra digunakan sebagai konteks visual yang digabungkan dengan pertanyaan pengguna dan konteks dokumen.

Qwen2-Audio untuk Pemrosesan Audio

Qwen2-Audio merupakan model audio-language yang dirancang untuk menerima input audio dan menghasilkan keluaran berbasis bahasa alami. Model ini dapat digunakan untuk memahami isi audio, melakukan transkripsi, merangkum informasi audio, atau menjawab pertanyaan berdasarkan sinyal suara [8].

Dalam penelitian ini, Qwen2-Audio digunakan pada skenario *audio-only* dan *text-audio RAG*. Pada skenario *audio-only*, model dievaluasi berdasarkan kedekatan semantik antara keluaran model dan referensi. Pada skenario *text-audio RAG*, keluaran dari model audio digunakan sebagai konteks tambahan dalam proses penjawaban berbasis RAG.

2.2.7 Preprocessing Data

Preprocessing merupakan tahap penyiapan data sebelum digunakan oleh model atau indeks vektor. Tujuannya adalah menyeragamkan format masukan, mengurangi artefak, dan memastikan setiap modalitas sesuai dengan kebutuhan komponen pemrosesnya. Preprocessing yang tidak tepat dapat menurunkan kualitas embedding, retrieval, dan keluaran model.

Preprocessing Dokumen Teks

Dokumen PDF, TXT, dan MD perlu diubah menjadi teks sebelum diindeks. Pada dokumen PDF, susunan visual halaman tidak selalu sama dengan urutan teks hasil ekstraksi. Artefak yang dapat muncul antara lain kata yang terpotong pada pergantian baris, spasi berlebih, karakter tidak terlihat, baris yang menempel, dan bagian dokumen yang tidak informatif.

Preprocessing teks dapat mencakup normalisasi karakter, penyatuan baris, perbaikan kata terpotong, normalisasi spasi, dan penyaringan bagian yang terlalu pendek atau berisi noise. Hasil preprocessing digunakan sebagai masukan proses chunking. Pembersihan perlu dilakukan secara hati-hati karena transformasi yang terlalu agresif dapat mengubah makna atau menggabungkan token yang seharusnya terpisah.

Preprocessing Citra

Model *vision-language* menggunakan processor untuk mengubah citra menjadi tensor visual. Tahap ini dapat mencakup pembacaan format, konversi ruang warna, penyesuaian ukuran, dan pembatasan jumlah piksel. Resolusi yang lebih tinggi dapat membantu pembacaan teks kecil, tetapi meningkatkan jumlah token visual, kebutuhan memori, dan waktu pemrosesan. Sebaliknya, resolusi yang terlalu rendah dapat menghilangkan detail penting.

Preprocessing Audio

Model audio memerlukan sinyal pada sampling rate tertentu. Oleh karena itu, audio dapat melalui decoding, konversi menjadi satu kanal, dan resampling. Apabila durasi dibatasi, bagian audio di luar batas akan dipotong. Pemotongan mengurangi kebutuhan komputasi, tetapi dapat menghilangkan informasi apabila bagian relevan berada setelah batas durasi. Kualitas rekaman, noise, volume, pelafalan, dan panjang audio juga dapat memengaruhi keluaran model.

2.2.8 Embedding Teks dan Representasi Semantik

Embedding teks merupakan representasi numerik dari teks dalam bentuk vektor berdimensi tetap. Dalam retrieval semantik, embedding digunakan untuk membandingkan pertanyaan dengan potongan dokumen berdasarkan kedekatan representasi, bukan hanya kecocokan kata secara literal.

Jika f_e menyatakan model embedding, teks t dipetakan menjadi vektor $\mathbf{e}$:

$$\mathbf{e} = f_e(t) \quad (2.12)$$

Penelitian ini menggunakan *all-MiniLM-L6-v2*. Model tersebut memetakan kalimat dan paragraf ke ruang vektor padat berdimensi 384 serta dapat digunakan untuk pencarian semantik [16], [17]. Embedding digunakan pada saat membangun indeks dokumen dan membentuk embedding query.

Kemiripan dua vektor dapat dihitung menggunakan *cosine similarity*:

$$\text{cosine}(\mathbf{q}, \mathbf{d}_i) = \frac{\mathbf{q} \cdot \mathbf{d}_i}{\|\mathbf{q}\|_2 \|\mathbf{d}_i\|_2} \quad (2.13)$$

Jika kedua vektor dinormalisasi sehingga memiliki norma satu:

$$\|\mathbf{q}\|_2 = \|\mathbf{d}_i\|_2 = 1 \quad (2.14)$$

maka cosine similarity sama dengan inner product:

$$\text{cosine}(\mathbf{q}, \mathbf{d}_i) = \mathbf{q}^T \mathbf{d}_i \quad (2.15)$$

Normalisasi tersebut memungkinkan indeks berbasis inner product digunakan untuk pencarian cosine similarity.

2.2.9 FAISS sebagai Indeks Vektor

FAISS atau *Facebook AI Similarity Search* merupakan pustaka untuk pencarian kemiripan dan pengelompokan vektor padat [11]. Dalam sistem RAG, FAISS menyimpan embedding chunk dokumen dan mencari vektor yang paling dekat dengan embedding query.

Jika basis pengetahuan terdiri atas embedding $\mathbf{D} = \{\mathbf{d}_1, \mathbf{d}_2, \dots, \mathbf{d}_n\}$ dan query direpresentasikan sebagai $\mathbf{q}$, retrieval memilih k dokumen dengan skor tertinggi:

$$\text{TopK}(\mathbf{q}) = \{d_i \mid i \in \text{argsort}_k(\text{sim}(\mathbf{q}, \mathbf{d}_i))\} \quad (2.16)$$

FAISS menyediakan beberapa jenis indeks. `IndexFlatIP` melakukan pencarian exact inner product tanpa kompresi indeks. Jika seluruh vektor telah dinormalisasi, inner product ekuivalen dengan cosine similarity [18]. Pendekatan exact search sesuai ketika ukuran basis pengetahuan masih dapat ditangani dalam memori dan hasil tanpa aproksimasi dibutuhkan.

Selain vektor, sistem menyimpan metadata setiap chunk, seperti nama sumber, nomor chunk, dan teks. Metadata menghubungkan hasil pencarian vektor dengan isi dokumen yang diberikan kepada model generatif dan mendukung keterlacakan sumber konteks.

2.2.10 Chunking Dokumen

Chunking merupakan proses membagi dokumen panjang menjadi potongan-potongan teks yang lebih kecil. Proses ini penting dalam sistem RAG karena dokumen sumber dapat memiliki panjang yang melebihi batas konteks model. Selain itu, retrieval terhadap seluruh dokumen kurang efisien dan dapat menghasilkan konteks yang terlalu luas. Dengan membagi dokumen menjadi chunk, sistem dapat melakukan retrieval pada unit informasi yang lebih spesifik.

Secara konseptual, dokumen D dapat dipotong menjadi sejumlah chunk sebagai berikut:

$$D = \{c_1, c_2, \dots, c_n\} \quad (2.17)$$

Setiap chunk c_i kemudian diubah menjadi embedding dan dimasukkan ke indeks vektor. Kualitas chunk berpengaruh langsung terhadap kualitas retrieval. Chunk yang terlalu pendek dapat kehilangan konteks penting, sedangkan chunk yang terlalu panjang dapat membawa informasi yang tidak relevan terhadap query. Oleh karena itu, strategi chunking perlu menjaga keseimbangan antara kelengkapan konteks dan ketepatan informasi [19].

Structure-Aware Fixed-Size Chunking with Overlap

Fixed-size chunking merupakan metode pemotongan dokumen berdasarkan ukuran tetap, misalnya jumlah karakter atau jumlah token tertentu. Pada metode ini, dokumen dipotong menjadi beberapa bagian dengan panjang maksimum L . Agar informasi pada batas antar-chunk tidak hilang, digunakan overlap sebesar O . Jika s_i menyatakan indeks awal chunk ke- i , maka indeks awal chunk berikutnya dapat dituliskan sebagai:

$$s_{i+1} = s_i + (L - O) \quad (2.18)$$

Pada Persamaan 2.18, L menyatakan ukuran maksimum chunk, sedangkan O menyatakan panjang overlap. Semakin besar overlap, semakin besar kemungkinan konteks pada batas pemotongan tetap terbawa ke chunk berikutnya. Namun, overlap yang terlalu besar dapat meningkatkan jumlah chunk dan memperbesar biaya embedding serta retrieval.

Pada pendekatan extitstructure-aware, batas paragraf dipertahankan apabila memungkinkan.

Paragraf yang melebihi ukuran maksimum dipotong lebih lanjut. Dengan demikian, metode tetap memiliki kontrol ukuran sekaligus berupaya mengurangi pemotongan pada batas struktur dokumen. Overlap yang lebih besar dapat menjaga kesinambungan konteks, tetapi meningkatkan redundansi, jumlah chunk, ukuran indeks, dan biaya retrieval.

Recursive Chunking

Recursive chunking merupakan metode pemotongan dokumen yang menggunakan separator secara bertingkat. Metode ini berusaha memotong teks berdasarkan struktur yang lebih besar terlebih dahulu, seperti paragraf atau baris baru. Jika potongan masih terlalu panjang, pemotongan dilanjutkan menggunakan separator yang lebih kecil, seperti kalimat, tanda baca, spasi, hingga karakter.

Secara konseptual, recursive chunking menggunakan daftar separator:

$$S = \{s_1, s_2, \dots, s_m\} \quad (2.19)$$

Pemotongan dimulai dari separator s_1 . Jika hasil potongan masih lebih panjang dari batas chunk, maka metode berlanjut ke separator s_2 , dan seterusnya. Pendekatan ini bertujuan menghasilkan chunk yang lebih menjaga struktur semantik dokumen dibandingkan pemotongan berbasis ukuran murni. Penelitian terkait menunjukkan bahwa perbedaan metode chunking dapat memengaruhi kualitas retrieval dan performa sistem RAG [20].

Dalam penelitian ini, perbandingan strategi chunking dilakukan pada skenario berbasis RAG. Skenario *image-only* dan *audio-only* tidak menggunakan chunking karena tidak melibatkan retrieval dokumen.

2.2.11 Konfigurasi Representasi Numerik Model

Representasi numerik menentukan jumlah bit dan karakteristik nilai yang digunakan untuk menyimpan atau menghitung parameter model. Penggunaan representasi berbeda memengaruhi kebutuhan memori, ketelitian numerik, dukungan kernel, dan performa inferensi.

Floating-Point 32-bit

FP32 menggunakan representasi floating-point 32-bit. Representasi ini menyediakan rentang dan presisi lebih tinggi dibandingkan FP16, tetapi membutuhkan memori parameter lebih besar. FP32 digunakan sebagai konfigurasi presisi penuh dalam pengujian.

Pada GPU NVIDIA tertentu, operasi yang menerima tensor FP32 dapat menggunakan TensorFloat-32 atau TF32 untuk mempercepat perkalian matriks dan konvolusi. TF32 tetap menerima dan menghasilkan tensor FP32, tetapi menggunakan ketelitian mantissa lebih rendah pada operasi internal [21]. Oleh karena itu, pengujian FP32 perlu membedakan operasi FP32 penuh dari operasi FP32 yang diakselerasi melalui TF32.

Floating-Point 16-bit

FP16 menggunakan representasi floating-point 16-bit. Secara teoritis, bobot FP16 membutuhkan setengah kapasitas penyimpanan bobot FP32. Pada GPU yang mendukung operasi half precision, FP16 dapat mengurangi perpindahan data dan memanfaatkan kernel yang dioptimalkan. Namun, performa aktual tetap dipengaruhi arsitektur GPU, bentuk tensor, panjang urutan, dan implementasi pustaka.

Kuantisasi 4-bit NormalFloat

Kuantisasi merupakan teknik representasi parameter menggunakan jumlah bit lebih rendah. Tujuan utamanya adalah mengurangi kebutuhan memori bobot dan memungkinkan model dimuat pada perangkat dengan kapasitas lebih terbatas. Kebutuhan memori teoritis bobot dapat diperkirakan sebagai:

$$M_{\text{weight}} \approx \frac{N \times b}{8} \quad (2.20)$$

Pada Persamaan 2.20, N menyatakan jumlah parameter dan b jumlah bit per parameter. Persamaan ini hanya memperkirakan penyimpanan bobot dan belum mencakup komponen memori lain.

Kuantisasi linier generik memetakan bobot w ke nilai kuantisasi q menggunakan skala s :

$$q = \text{clip}\left(\text{round}\left(\frac{w}{s}\right), q_{\min}, q_{\max}\right) \quad (2.21)$$

Bobot dapat didekuantisasi menjadi pendekatan nilai awal:

$$\hat{w} = s \cdot q \quad (2.22)$$

Persamaan tersebut menjelaskan kuantisasi linier secara umum. Konfigurasi 4-bit dalam penelitian ini menggunakan *NormalFloat 4* (NF4), bukan kuantisasi 4-bit linier murni. NF4 merupakan tipe data 4-bit dengan level kuantisasi yang dirancang untuk bobot dengan distribusi mendekati normal [22], [23].

Pada BitsAndBytes, perlu dibedakan antara *storage representation* dan *compute dtype*. Bobot dapat disimpan dalam NF4, sedangkan operasi komputasi dilakukan menggunakan tipe data lebih tinggi, seperti FP16. Sebelum digunakan dalam operasi matriks, bobot 4-bit didekuantisasi menuju jenis data komputasi. Dengan demikian, istilah model 4-bit tidak berarti seluruh operasi aritmetika dijalankan secara langsung dalam 4-bit.

Double quantization atau *nested quantization* mengkuantisasi kembali konstanta kuantisasi yang digunakan untuk merepresentasikan bobot terkuantisasi. Teknik ini mengurangi overhead penyimpanan informasi skala [22], [23]. Penghematan tersebut tidak menghapus kebutuhan aktivasi, KV cache, buffer, dan komponen runtime.

Hubungan Kuantisasi dan Latency

Dokumentasi pustaka kuantisasi menyatakan bahwa representasi berpresisi rendah dapat mengurangi jejak memori dan berpotensi mempercepat inferensi [23], [24]. Akan tetapi, percepatan tidak selalu terjadi pada setiap model dan perangkat.

Latency model terkuantisasi merupakan hasil interaksi pengurangan bandwidth memori, overhead dekuantisasi, jenis data komputasi, efisiensi kernel, ukuran model, panjang masukan, jumlah token keluaran, batch size, dan karakteristik GPU. Jika penghematan transfer memori lebih dominan, model terkuantisasi dapat lebih cepat. Jika overhead dekuantisasi dan pemanggilan kernel lebih dominan, NF4 dapat memiliki latency lebih tinggi daripada FP16 atau FP32. Oleh karena itu, manfaat yang dapat diasumsikan langsung dari penurunan bit adalah pengurangan penyimpanan bobot, sedangkan dampak terhadap latency harus diukur secara empiris.

2.2.12 Kebutuhan Memori GPU

Kebutuhan memori GPU pada inferensi tidak hanya berasal dari bobot model. Secara konseptual, total kebutuhan memori dapat ditulis sebagai:

$$M_{\text{GPU}} = M_{\text{weight}} + M_{\text{activation}} + M_{\text{KV}} + M_{\text{input}} + M_{\text{workspace}} + M_{\text{allocator}} + M_{\text{runtime}} \quad (2.23)$$

M_{weight} merupakan memori bobot, $M_{\text{activation}}$ aktivasi antara, M_{KV} KV cache, M_{input} tensor input, $M_{\text{workspace}}$ buffer kernel, $M_{\text{allocator}}$ memori yang dikelola caching allocator, dan M_{runtime} konteks CUDA serta pustaka pendukung. Karena itu, penurunan bobot dari 32-bit menjadi 4-bit tidak menjamin total VRAM berkurang tepat delapan kali.

PyTorch membedakan memori dialokasikan dan dicadangkan. *Memory allocated* adalah memori yang sedang ditempati tensor, sedangkan *memory reserved* merupakan memori yang dikelola caching allocator. *Peak memory allocated* mencatat puncak memori tensor, sedangkan *peak memory reserved* mencatat puncak memori yang dikelola allocator [25], [26].

Nilai pada alat pemantauan tingkat proses dapat berbeda dari statistik allocator PyTorch karena dapat mencakup konteks CUDA dan alokasi di luar allocator. Oleh karena itu, instrumen dan jenis metrik memori perlu disebutkan secara jelas. Kebutuhan memori GPU juga berbeda dari *GPU utilization*. Memori menunjukkan kapasitas yang digunakan, sedangkan utilization menunjukkan tingkat aktivitas komputasi dalam interval pemantauan.

2.2.13 Latency dan Pengukuran Waktu Inferensi

Latency merupakan waktu yang dibutuhkan sistem untuk memproses input hingga menghasilkan keluaran:

$$L = t_{\text{end}} - t_{\text{start}} \quad (2.24)$$

Rata-rata latency untuk n pengujian dihitung sebagai:

$$\bar{L} = \frac{1}{n} \sum_{i=1}^n L_i \quad (2.25)$$

Cakupan latency perlu didefinisikan. *Cold-start latency* mencakup pemuatan model dan inisialisasi sebelum inferensi. *Warm inference latency* diukur ketika model telah dimuat. *End-to-end latency* dapat mencakup preprocessing, embedding query, retrieval, pembentukan prompt, inferensi multimodal, generasi, dan decoding.

Operasi CUDA umumnya berjalan asinkron terhadap CPU. Pemanggilan fungsi dapat kembali sebelum seluruh pekerjaan GPU selesai. Oleh karena itu, pengukuran dengan waktu CPU perlu melakukan sinkronisasi agar t_{end} dicatat setelah pekerjaan GPU selesai [27], [28]. CUDA event juga dapat digunakan untuk mengukur durasi operasi GPU.

Inferensi pertama dapat mengandung overhead pemuatan model, alokasi awal, inisialisasi kernel, dan pembentukan cache. Warm-up digunakan untuk memisahkan overhead awal dari inferensi berulang. Perlakuan warm-up harus konsisten pada seluruh konfigurasi.

Latency dipengaruhi ukuran model, representasi numerik, panjang prompt, jumlah token keluaran, resolusi citra, durasi audio, batch size, penggunaan cache, retrieval, dan pemindahan model. Perbandingan latency memerlukan standardisasi input, batas token, perangkat, dan

prosedur pengukuran.

2.2.14 Metrik Evaluasi Sistem

Evaluasi sistem dilakukan untuk mengukur kualitas jawaban, kualitas retrieval, kemampuan pemrosesan multimodal, latency, dan kebutuhan memori GPU. Metrik disesuaikan dengan karakteristik skenario. Skenario berbasis RAG menggunakan BLEU, ROUGE-L, *context precision*, *context recall*, dan latency. Skenario *image-only* dan *audio-only* menggunakan cosine similarity dan latency. Profiling memori GPU digunakan untuk membandingkan kebutuhan sumber daya antarkonfigurasi representasi numerik.

BLEU

BLEU atau *Bilingual Evaluation Understudy* mengukur kemiripan n-gram antara kandidat dan referensi [29]:

$$\text{BLEU} = BP \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right) \quad (2.26)$$

p_n menyatakan presisi n-gram ke- n , w_n bobot, dan BP brevity penalty:

$$BP = \begin{cases} 1, & c > r \\ e^{(1-r/c)}, & c \leq r \end{cases} \quad (2.27)$$

BLEU sensitif terhadap kesamaan bentuk permukaan. Jawaban yang benar secara semantik tetapi menggunakan parafrasa dapat memperoleh skor lebih rendah. Sebaliknya, kesamaan n-gram tidak dengan sendirinya menjamin kebenaran faktual.

ROUGE-L

ROUGE-L menggunakan *Longest Common Subsequence* (LCS) untuk mengukur kemiripan urutan kandidat dan referensi [30]:

$$P_{\text{LCS}} = \frac{\text{LCS}(X, Y)}{|X|} \quad (2.28)$$

$$R_{\text{LCS}} = \frac{\text{LCS}(X, Y)}{|Y|} \quad (2.29)$$

$$F_{\text{LCS}} = \frac{(1 + \beta^2) R_{\text{LCS}} P_{\text{LCS}}}{R_{\text{LCS}} + \beta^2 P_{\text{LCS}}} \quad (2.30)$$

ROUGE-L menangkap kesamaan urutan lebih panjang dibandingkan kecocokan n-gram lokal. Namun, metrik ini tetap berorientasi pada kemiripan teks dan belum secara langsung mengukur grounding terhadap dokumen.

Context Precision

Context precision mengevaluasi kemampuan retriever menempatkan konteks relevan pada urutan lebih tinggi daripada konteks tidak relevan [31], [32]:

$$\text{ContextPrecision} = \frac{1}{|\mathcal{R}|} \sum_{i=1}^k P@i \cdot \text{rel}(c_i) \quad (2.31)$$

Pada varian berbasis LLM, penentuan relevansi dilakukan oleh model evaluator. Skor dapat dipengaruhi kemampuan evaluator, instruksi evaluasi, panjang konteks, dan konfigurasi generasi.

Context Recall

Context recall mengukur sejauh mana informasi penting pada jawaban referensi didukung oleh konteks yang berhasil diambil [31], [33]:

$$\text{ContextRecall} = \frac{|\mathcal{S}_{\text{supported}}|}{|\mathcal{S}_{\text{reference}}|} \quad (2.32)$$

Evaluasi berbasis LLM bergantung pada model evaluator dan tidak sepenuhnya sama dengan penghitungan deterministik berbasis label manual.

Cosine Similarity

Cosine similarity digunakan untuk mengukur kedekatan semantik embedding keluaran model **a** dan referensi **b**:

$$\text{CosineSimilarity}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|_2 \|\mathbf{b}\|_2} \quad (2.33)$$

Metrik ini dapat menangkap kedekatan semantik ketika susunan kata berbeda. Namun, hasilnya bergantung pada model embedding dan belum menjamin bahwa seluruh rincian faktual kandidat sama dengan referensi.

Latency

Latency digunakan untuk membandingkan waktu respons dalam cakupan pengukuran yang sama. Interpretasi perlu mempertimbangkan apakah pemuatan model, retrieval, preprocessing, dan inferensi multimodal termasuk dalam pengukuran. Latency bukan ukuran langsung penggunaan memori sehingga perlu dianalisis bersama kebutuhan memori GPU.

Kebutuhan Memori GPU

Kebutuhan memori GPU digunakan untuk mengukur jejak memori pemuatan model dan inferensi. Metrik dapat berupa puncak memori tensor, puncak memori allocator, atau penggunaan memori proses. Nilai hanya dapat dibandingkan secara adil apabila alat, titik reset, urutan pemuatan model, dan cakupan proses dibuat sama. Interpretasi tidak hanya didasarkan pada rasio bit bobot karena total memori juga mencakup aktivasi, input multimodal, cache, buffer, dan runtime.

2.2.15 Matriks Evaluasi Penelitian

Setiap skenario menggunakan metrik yang disesuaikan dengan tujuan pengujian. Matriks evaluasi ditunjukkan pada Tabel 2.2.

Tabel 2.2: Matriks evaluasi sistem pada penelitian

Skenario atau Aspek	Metrik Evaluasi	Tujuan Evaluasi	Keluaran
<i>Text-Only RAG</i>	BLEU, <i>context context latency</i> , ROUGE-L, <i>precision, recall</i> , dan	Menilai kualitas jawaban, kualitas konteks retrieval, dan waktu respons pada pertanyaan teks berbasis dokumen	Skor kualitas, skor retrieval, dan waktu respons

Tabel 2.2 – lanjutan

Skenario atau Aspek	Metrik Evaluasi	Tujuan Evaluasi	Keluaran
<i>Image-Only</i>	<i>Cosine similarity</i> dan <i>latency</i>	Menilai kesesuaian semantik keluaran terhadap referensi citra dan waktu respons	Skor kesamaan semantik dan waktu respons
<i>Audio-Only</i>	<i>Cosine similarity</i> dan <i>latency</i>	Menilai kesesuaian semantik keluaran terhadap referensi audio dan waktu respons	Skor kesamaan semantik dan waktu respons
<i>Text-Image RAG</i>	BLEU, ROUGE-L, <i>context precision</i> , <i>context recall</i> , dan <i>latency</i>	Menilai jawaban yang memanfaatkan pertanyaan, citra, dan konteks dokumen	Skor kualitas, skor retrieval, dan waktu respons
<i>Text-Audio RAG</i>	BLEU, ROUGE-L, <i>context precision</i> , <i>context recall</i> , dan <i>latency</i>	Menilai jawaban yang memanfaatkan pertanyaan, audio, dan konteks dokumen	Skor kualitas, skor retrieval, dan waktu respons
Perbandingan metode <i>chunking</i>	Metrik kualitas jawaban, retrieval, dan <i>latency</i>	Menilai pengaruh metode pemotongan dokumen terhadap performa skenario RAG	Perbandingan performa antar metode
Perbandingan konfigurasi representasi numerik model	Kualitas keluaran, <i>latency</i> , dan kebutuhan memori GPU	Menilai <i>trade-off</i> antara kualitas keluaran, waktu respons, dan kebutuhan memori	Perbandingan FP32, FP16, dan 4-bit NF4
Profiling memori GPU	Kebutuhan memori GPU pada konfigurasi representatif	Menilai perubahan jejak memori selama pemuatan dan inferensi model	Nilai penggunaan atau puncak memori sesuai instrumen pengujian

Berdasarkan matriks tersebut, evaluasi tidak menempatkan satu metrik sebagai satu-satunya indikator konfigurasi terbaik. Kualitas keluaran, latency, dan kebutuhan memori GPU merupakan dimensi berbeda. Konfigurasi yang lebih cepat belum tentu lebih hemat memori, sedangkan konfigurasi yang lebih hemat memori belum tentu memiliki latency terendah. Rekomendasi sistem disusun berdasarkan *trade-off* antara kualitas keluaran, latency, dan kebutuhan memori GPU.

[Halaman ini sengaja dikosongkan]

BAB III

DESAIN DAN IMPLEMENTASI

Bab ini menjelaskan desain dan implementasi sistem *question answering* multimodal berbasis model Qwen dan metode RAG. Pembahasan mencakup ruang lingkup pengembangan sistem, lingkungan implementasi dan pengujian, data penelitian, *preprocessing*, pembangunan indeks pengetahuan, perancangan sistem, implementasi *pipeline*, konfigurasi representasi numerik model, desain pengujian, serta metode analisis. Eksperimen digunakan untuk memvalidasi sistem yang dikembangkan dan untuk menilai pengaruh karakteristik *pipeline*, konfigurasi *chunking*, serta konfigurasi representasi numerik model terhadap kualitas keluaran, kualitas *retrieval*, *latency*, dan kebutuhan memori GPU.

3.1 Desain Penelitian

Penelitian ini menggunakan pendekatan eksperimental melalui pengembangan sistem *question answering* multimodal berbasis keluarga model Qwen dan metode *Retrieval-Augmented Generation* (RAG). Sistem dirancang untuk menerima pertanyaan teks serta masukan tambahan berupa citra atau audio. Dokumen tekstual digunakan sebagai basis pengetahuan, sedangkan citra dan audio diproses pada saat sistem menerima pertanyaan multimodal.

Penelitian tidak menggunakan satu model omnimodal. Sistem dibangun menggunakan arsitektur modular yang terdiri atas model teks, model *vision-language*, model *audio-language*, model *embedding*, dan indeks vektor FAISS. Pemisahan tersebut memungkinkan setiap modalitas diproses oleh model yang sesuai, kemudian representasi tekstual hasil pemrosesan citra atau audio digunakan bersama konteks dokumen untuk menghasilkan jawaban akhir.

3.1.1 Ruang Lingkup Pengembangan Sistem

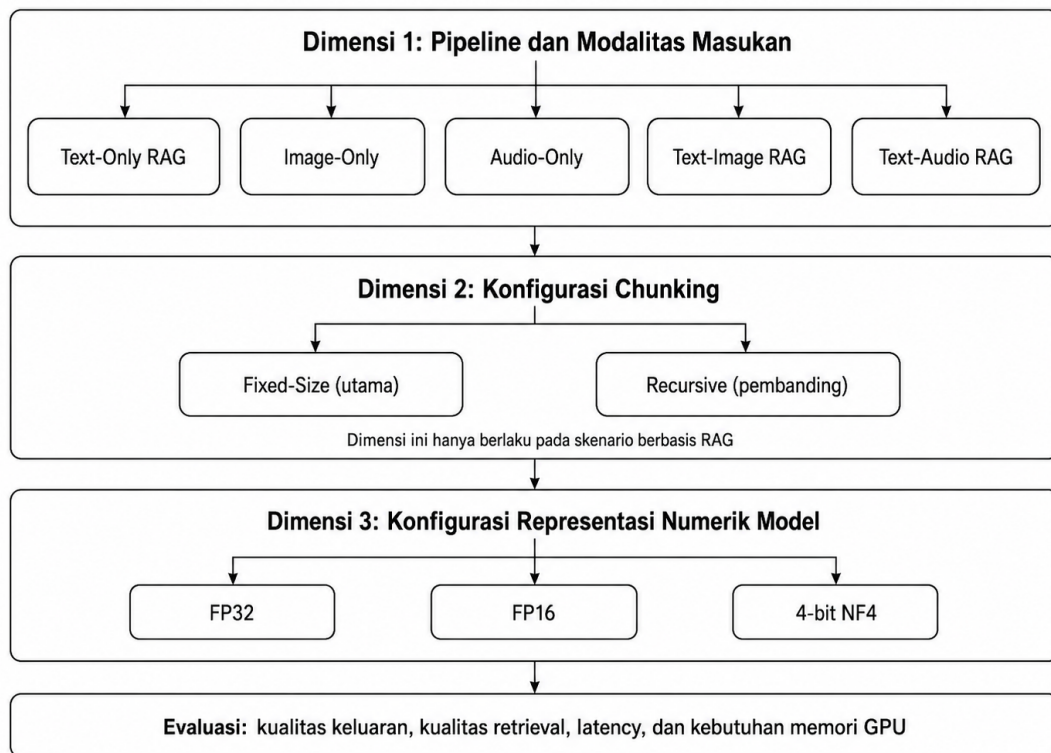
Pengembangan sistem pada penelitian ini tidak hanya mencakup pengujian performa model, tetapi juga perancangan alur *question answering* yang dapat memanfaatkan dokumen, citra, dan audio secara terintegrasi. Sistem dikembangkan untuk menerima pertanyaan pengguna, memvalidasi masukan, memilih *pipeline* sesuai modalitas, melakukan *retrieval* terhadap basis pengetahuan, menyusun *prompt*, memanggil model Qwen yang sesuai, dan mengembalikan jawaban beserta informasi sumber.

Ruang lingkup pengembangan sistem mencakup lima komponen utama. Komponen pertama adalah basis pengetahuan berbasis dokumen ITS yang diproses menjadi indeks vektor. Komponen kedua adalah antarmuka dan *backend* yang menangani permintaan pengguna. Komponen ketiga adalah *pipeline* RAG untuk pertanyaan teks. Komponen keempat adalah integrasi masukan citra dan audio melalui model multimodal. Komponen kelima adalah manajemen model dan memori GPU agar model teks, visual, dan audio dapat digunakan pada perangkat dengan kapasitas memori terbatas.

Eksperimen dalam penelitian ini ditempatkan sebagai tahap evaluasi terhadap sistem yang telah dikembangkan. Dengan demikian, hasil pengujian tidak hanya digunakan untuk membandingkan skor metrik, tetapi juga untuk menilai apakah rancangan sistem *question answering* multimodal dapat bekerja secara konsisten pada skenario teks, citra, audio, dan kombinasi modalitas dengan dokumen.

Eksperimen disusun dalam tiga dimensi analisis. Dimensi pertama adalah perbedaan karakteristik *pipeline* berdasarkan modalitas masukan, yaitu teks, citra, dan audio. Dimensi kedua adalah perbandingan konfigurasi *chunking* pada skenario berbasis RAG. Dimensi ketiga adalah perbandingan konfigurasi representasi numerik model, yaitu FP32, FP16, dan 4-bit *NormalFloat 4* (NF4). Evaluasi dilakukan berdasarkan kualitas keluaran, kualitas *retrieval*, *latency*, dan kebutuhan memori GPU. Seluruh pengujian ditempatkan sebagai evaluasi domain tertutup pada dokumen dan layanan ITS yang digunakan dalam penelitian.

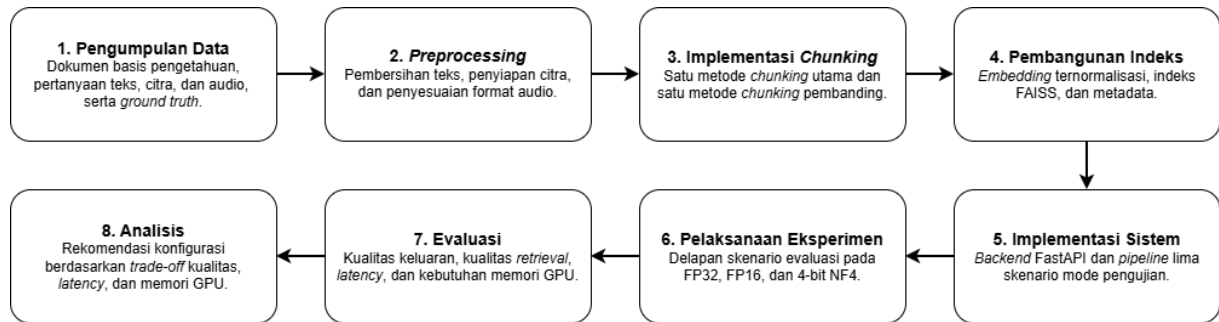
Tiga dimensi eksperimen tersebut ditunjukkan pada Gambar 3.1. Diagram ini memperlihatkan bahwa evaluasi tidak hanya membandingkan konfigurasi representasi numerik model, tetapi juga mempertimbangkan perbedaan modalitas masukan dan konfigurasi *chunking* pada skenario berbasis RAG.



Gambar 3.1: Tiga dimensi eksperimen penelitian

3.1.2 Tahapan Penelitian

Tahapan penelitian disusun untuk menggambarkan rangkaian proses mulai dari persiapan data, pembangunan sistem, pelaksanaan eksperimen, hingga analisis hasil. Rangkaian tahapan penelitian ditunjukkan pada Gambar 3.2.



Gambar 3.2: Tahapan penelitian

Tahap pertama adalah pengumpulan data yang mencakup dokumen basis pengetahuan, pertanyaan teks, citra, audio, dan *ground truth*. Dokumen basis pengetahuan digunakan sebagai sumber informasi pada skenario berbasis RAG, sedangkan pertanyaan, citra, audio, dan *ground truth* digunakan sebagai data uji.

Tahap kedua adalah *preprocessing* sesuai karakteristik modalitas. Teks dokumen diekstraksi dan dinormalisasi sebelum digunakan dalam pembangunan indeks. Citra divalidasi agar dapat diterima oleh model visual, sedangkan audio disiapkan sesuai format dan *sampling rate* yang dibutuhkan oleh model audio.

Tahap ketiga adalah implementasi konfigurasi *chunking*. *Structure-aware fixed-size chunking with overlap* digunakan sebagai konfigurasi utama, sedangkan *recursive chunking* digunakan sebagai konfigurasi pembanding yang menghasilkan potongan lebih granular. Perbandingan dilakukan pada dokumen yang sama tanpa mengasumsikan bahwa salah satu konfigurasi selalu lebih baik pada seluruh dataset.

Tahap keempat adalah pembangunan indeks pengetahuan. Setiap *chunk* diubah menjadi *embedding*, dinormalisasi menggunakan norma L_2 , dan dimasukkan ke indeks FAISS bersama metadata sumber. Proses tersebut menghasilkan indeks *fixed-size* sebagai indeks utama dan indeks *recursive* sebagai indeks pembanding.

Tahap kelima adalah implementasi sistem *question answering*. Aplikasi menyediakan tiga *pipeline* utama, yaitu *text-only RAG*, *text-image RAG*, dan *text-audio RAG*. Selain itu, *pipeline image-only* dan *audio-only* diimplementasikan khusus untuk pengujian kemampuan dasar model multimodal dan tidak tersedia sebagai mode pada aplikasi pengguna.

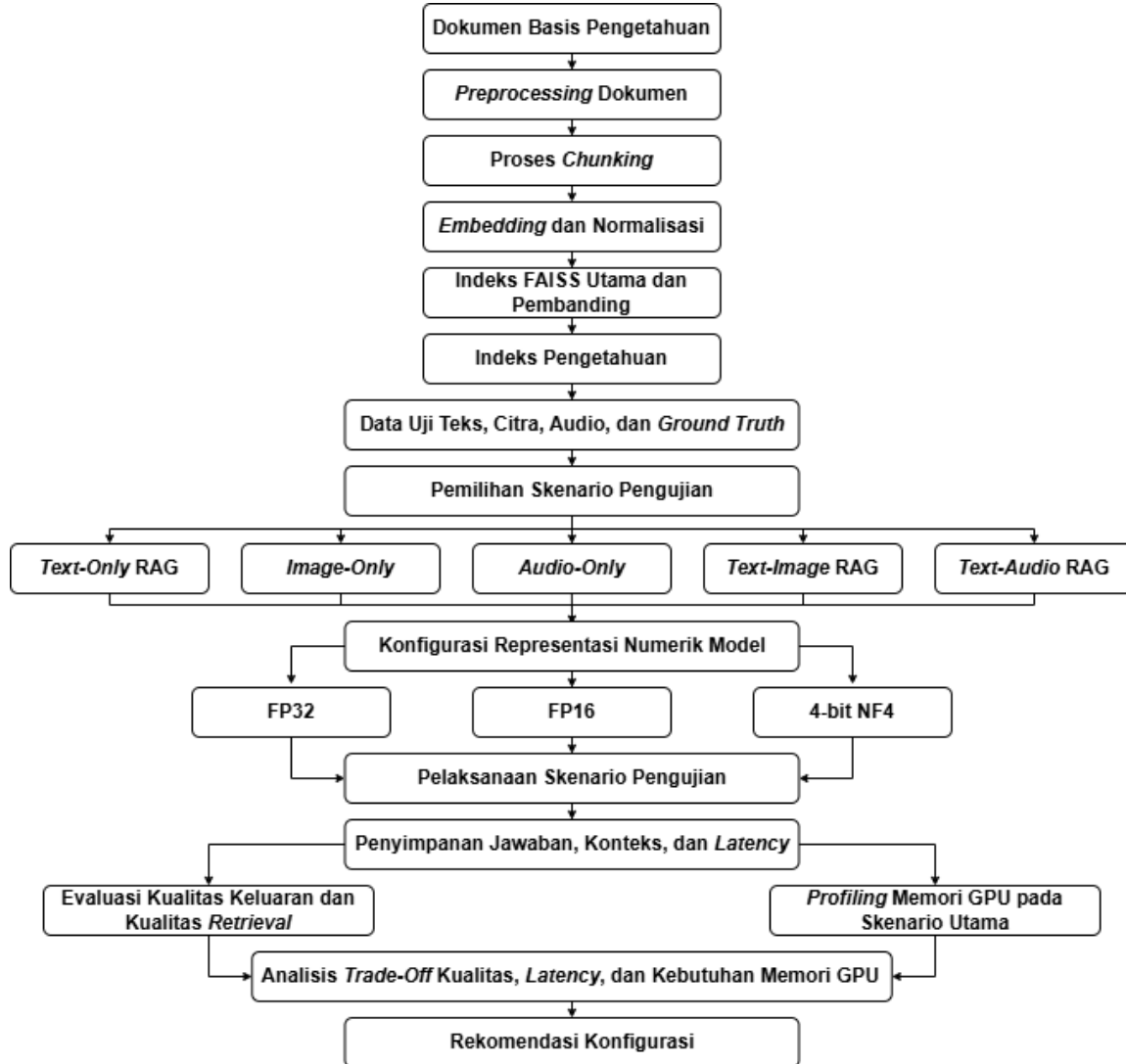
Tahap keenam adalah pelaksanaan eksperimen. Tiga kategori RAG dijalankan menggunakan indeks *fixed-size* dan *recursive*, sedangkan *image-only* dan *audio-only* tidak menggunakan indeks. Susunan tersebut menghasilkan delapan skenario evaluasi. Setiap skenario dijalankan pada FP32, FP16, dan 4-bit NF4 dengan data, batas token, parameter *retrieval*, serta perangkat keras yang dikendalikan.

Tahap ketujuh adalah evaluasi hasil. Kualitas keluaran, kualitas *retrieval*, dan *latency* dinilai sesuai karakteristik skenario. *Profiling* kebutuhan memori GPU dilakukan pada lima skenario representatif yang menggunakan konfigurasi utama *fixed-size*. Pemilihan tersebut memusatkan pengukuran memori pada pengaruh konfigurasi representasi numerik model dalam konfigurasi sistem utama.

Tahap terakhir adalah analisis hasil. Analisis membandingkan performa antarskenario, konfigurasi *chunking*, serta konfigurasi FP32, FP16, dan 4-bit NF4. Rekomendasi disusun berdasarkan *trade-off* antara kualitas keluaran, *latency*, dan kebutuhan memori GPU.

3.1.3 Diagram Metodologi Penelitian

Diagram metodologi penelitian merangkum hubungan antara pembangunan basis pengetahuan, pelaksanaan skenario pengujian, evaluasi, dan analisis. Alur metodologi ditunjukkan pada Gambar 3.3.



Gambar 3.3: Diagram metodologi penelitian

Metodologi diawali dengan *preprocessing* dokumen dan penerapan konfigurasi *chunking*. Konfigurasi utama menggunakan *structure-aware fixed-size chunking with overlap*, sedangkan *recursive chunking* digunakan sebagai pembanding. Hasil kedua konfigurasi diubah menjadi *embedding* menggunakan all-MiniLM-L6-v2, dinormalisasi menggunakan norma L_2 , dan disimpan pada indeks FAISS yang terpisah.

Data uji terdiri atas pertanyaan teks, citra, audio, dan *ground truth*. Tiga kategori RAG masing-masing diuji dengan indeks utama dan indeks pembanding, sedangkan *image-only* dan *audio-only* digunakan sebagai skenario non-*retrieval*. Delapan skenario tersebut dijalankan pada FP32, FP16, dan 4-bit NF4. Jawaban, konteks hasil *retrieval*, *latency*, serta status pengujian disimpan untuk proses evaluasi.

Evaluasi kualitas keluaran dan *retrieval* dilakukan pada skenario yang sesuai. *Profiling* kebutuhan memori GPU dibatasi pada lima skenario *fixed-size* sebagai konfigurasi utama. Seluruh

hasil selanjutnya dianalisis untuk memperoleh rekomendasi berdasarkan *trade-off* kualitas keluaran, *latency*, dan kebutuhan memori GPU.

3.2 Lingkungan Implementasi dan Pengujian

Implementasi dan pengujian dilakukan menggunakan satu konfigurasi perangkat keras dan lingkungan perangkat lunak yang sama. Pengendalian lingkungan tersebut ditujukan agar perbedaan hasil tidak berasal dari pergantian perangkat, versi pustaka, atau kapasitas memori yang digunakan.

3.2.1 Spesifikasi Perangkat Keras

Spesifikasi perangkat keras ditunjukkan pada Tabel 3.1.

Tabel 3.1: Spesifikasi perangkat keras implementasi dan pengujian

Komponen	Spesifikasi
Prosesor	Intel Xeon E5-2673 v4
GPU	NVIDIA GeForce RTX 5080
Kapasitas memori GPU	16 GB
RAM	32 GB
Kapasitas penyimpanan yang dialokasikan	NVMe 50 GB
CUDA yang tersedia pada lingkungan pengujian	13.0

GPU digunakan untuk inferensi model teks, visual, dan audio. Konfigurasi perangkat keras dipertahankan sama pada FP32, FP16, dan 4-bit NF4. Oleh karena itu, perbedaan performa di-analisis sebagai hasil interaksi antara konfigurasi representasi numerik dan karakteristik *pipeline*, bukan sebagai akibat penggunaan perangkat yang berbeda.

3.2.2 Spesifikasi Perangkat Lunak

Perangkat lunak utama ditunjukkan pada Tabel 3.2. Versi pustaka inti dipertahankan sama pada seluruh konfigurasi eksperimen.

Tabel 3.2: Perangkat lunak yang digunakan

Komponen	Perangkat Lunak	Versi atau Keterangan
Bahasa pemrograman	Python	3.11
Model dan <i>processor</i>	Transformers	4.48.3
<i>Embedding</i> teks	SentenceTransformers	5.2.3
Indeks vektor	FAISS	1.13.2
Kuantisasi	BitsAndBytes	0.49.2
Evaluasi RAG	RAGAS	0.4.3

Versi yang tidak tercatat pada berkas hasil eksperimen tidak ditambahkan ke tabel untuk menghindari pencantuman informasi yang tidak dapat diverifikasi kembali setelah lingkungan pengujian dihentikan.

3.2.3 Model yang Digunakan

Model dipilih berdasarkan fungsi setiap komponen. Rincian model ditunjukkan pada Tabel 3.3.

Tabel 3.3: Model yang digunakan dalam sistem

Jenis Model	Nama Model	Fungsi
Model teks	Qwen2.5-1.5B-Instruct	Menghasilkan jawaban pada <i>text-only RAG</i> dan jawaban akhir pada skenario RAG multimodal.
Model <i>vision-language</i>	Qwen2-VL-2B-Instruct	Memahami citra pada <i>image-only</i> dan mengekstraksi bukti visual pada <i>text-image RAG</i> .
Model <i>audio-language</i>	Qwen2-Audio-7B-Instruct	Memahami audio pada <i>audio-only</i> dan menghasilkan transkripsi atau konteks tekstual pada <i>text-audio RAG</i> .
Model <i>embedding</i>	all-MiniLM-L6-v2	Mengubah kueri dan <i>chunk</i> dokumen menjadi vektor berdimensi 384.

3.3 Data Penelitian

Data penelitian dibagi menjadi dokumen basis pengetahuan, data uji, dan *ground truth*. Dokumen basis pengetahuan digunakan sebagai sumber pada proses *retrieval*, sedangkan data uji dan *ground truth* digunakan untuk mengevaluasi keluaran sistem. Basis pengetahuan terdiri atas 15 dokumen tekstual. Data uji terdiri atas lima kelompok yang masing-masing berjumlah 10 data, yaitu *text-only RAG*, *image-only*, *audio-only*, *text-image RAG*, dan *text-audio RAG*. Seluruh data ditempatkan dalam ruang lingkup domain layanan dan dokumen ITS sehingga hasil penelitian merepresentasikan performa sistem pada domain tertutup tersebut.

3.3.1 Dokumen Basis Pengetahuan

Basis pengetahuan terdiri atas 15 dokumen tekstual yang memuat layanan digital akademik, administrasi akademik, penelitian, pengabdian masyarakat, kolaborasi internasional, beasiswa, dan dukungan publikasi ITS. Seluruh dokumen diperoleh dari website resmi ITS dan unit resmi terkait, kemudian disimpan secara lokal sebagai basis pengetahuan penelitian. Penyimpanan lokal dilakukan agar isi dokumen yang digunakan dalam eksperimen tidak berubah selama proses pengujian. Seluruh dokumen digunakan sebagai sumber informasi yang sama pada setiap konfigurasi eksperimen berbasis RAG. Dokumen tidak mengalami perubahan isi selama perbandingan FP32, FP16, 4-bit NF4, maupun konfigurasi *chunking*.

Daftar dokumen yang digunakan ditunjukkan pada Tabel 3.4.

Tabel 3.4: Dokumen sumber pada basis pengetahuan

Nama Dokumen	Jenis Informasi	Ringkasan Isi
Panduan Mahasiswa Baru DPTSI 2025	Layanan digital akademik	Panduan myITS Portal, email ITS, SIAKAD, presensi, MFA, dan layanan teknologi informasi mahasiswa.

Tabel 3.4 – lanjutan

Nama Dokumen	Jenis Informasi	Ringkasan Isi
Pengumuman Kelengkapan Seminar Pra Proposal TA Gasal 2025/2026	Administrasi akademik	Persyaratan dokumen, ketentuan berpakaian, kesiapan perangkat, dan antisipasi teknis seminar.
Panduan Penelitian Dana ITS Tahun 2026	Penelitian internal	Skema, persyaratan, luaran, pendanaan, dan pelaksanaan penelitian dana ITS.
Panduan Abmas ITS Tahun 2026	Pengabdian masyarakat	Skema pendanaan, sasaran, indikator luaran, dan ketentuan pelaksanaan pengabdian masyarakat.
Panduan Penelitian dan Abmas Tahun 2026	Penelitian dan pengabdian	Arah strategis, luaran penelitian, inovasi, HKI, dan dampak sosial.
Panduan PMKI ITS 2026	Kolaborasi dan pengabdian	Program pengabdian dan kolaborasi internasional, target IKU, SDGs, dan dampak masyarakat.
Panduan RKI ITS 2026	Riset kolaborasi internasional	Target publikasi, kerja sama global, indikator kinerja, dan strategi reputasi riset.
Panduan Riset Jatim Melaju 2026	Riset daerah dan inovasi	Program riset untuk inovasi, kolaborasi, dampak riset, dan pembangunan daerah.
Panduan Penelitian Frontiers Dana HETI	Riset frontiers	Publikasi bereputasi, kolaborasi internasional, SDGs, dan reputasi global ITS.
Panduan FAST-D, Beasiswa Unggulan, dan GES	Beasiswa dan pendidikan doctoral	Program pendidikan doctoral, beasiswa, publikasi, dan kolaborasi internasional.
Panduan Post-Doctoral 2026	Program post-doctoral	Peningkatan publikasi, mitra internasional, reputasi global, dan riset berdampak.
Panduan Insentif Publikasi Terindeks Dana ITS Tahun 2026	Insentif publikasi	Ketentuan dukungan dan insentif publikasi ilmiah internasional terindeks.
Panduan Bantuan Biaya APC Dana ITS Tahun 2026	Bantuan biaya publikasi	Persyaratan dan mekanisme bantuan biaya <i>Article Processing Charge</i> .
Panduan Bantuan Biaya APC Dana EQUITY WCU Tahun 2026	Bantuan publikasi WCU	Bantuan APC untuk publikasi internasional dan peningkatan reputasi ITS.
Panduan Insentif Publikasi Top Tiers Dana EQUITY WCU Tahun 2026	Insentif publikasi Top Tier	Insentif jurnal Top Tier, peningkatan sitasi, reputasi akademik, dan pencapaian WCU.

Dokumen pada Tabel 3.4 menjadi sumber *retrieval* pada skenario *text-only RAG*, *text-image RAG*, dan *text-audio RAG*. Citra dan audio tidak dimasukkan ke indeks FAISS, tetapi digunakan sebagai masukan pada saat inferensi.

3.3.2 Ringkasan Data Uji

Ringkasan jumlah data uji ditunjukkan pada Tabel 3.5. Media yang sama digunakan kembali pada skenario non-*retrieval* dan skenario RAG multimodal agar kemampuan dasar model dapat dibandingkan secara deskriptif dengan kinerja *pipeline* multimodal. IMG01–IMG10 digunakan pada *image-only* dan dipasangkan dengan kueri TI01–TI10 pada *text-image RAG*. AUD01–AUD10 digunakan pada *audio-only* dan dipasangkan dengan kueri TA01–TA10 pada *text-audio RAG*.

Tabel 3.5: Ringkasan data uji penelitian

Kelompok Data	Jumlah	Identitas Data	Penggunaan
<i>Text-Only RAG</i>	10	Q01–Q10	Dijalankan pada indeks <i>fixed-size</i> dan <i>recursive</i> .
<i>Image-Only</i>	10	IMG01–IMG10	Pemahaman citra tanpa <i>retrieval</i> .
<i>Audio-Only</i>	10	AUD01–AUD10	Pemahaman audio tanpa <i>retrieval</i> .
<i>Text-Image RAG</i>	10	TI01–TI10 dengan IMG01–IMG10	Dijalankan pada indeks <i>fixed-size</i> dan <i>recursive</i> .
<i>Text-Audio RAG</i>	10	TA01–TA10 dengan AUD01–AUD10	Dijalankan pada indeks <i>fixed-size</i> dan <i>recursive</i> .

3.3.3 Data Uji Text-Only RAG

Data uji *text-only RAG* terdiri atas 10 pertanyaan layanan digital ITS. Daftar pertanyaan ditunjukkan pada Tabel 3.6.

Tabel 3.6: Data uji *text-only RAG*

ID	Kueri	Tujuan Pengujian
Q01	Apa itu myITS Portal?	Menilai kemampuan menjelaskan definisi layanan utama myITS Portal.
Q02	Aplikasi penting apa saja yang dapat diakses melalui myITS Portal?	Menilai kemampuan mengambil daftar layanan yang tersedia pada portal.
Q03	Apa format email resmi ITS untuk mahasiswa?	Menilai jawaban spesifik mengenai format akun email mahasiswa.
Q04	Untuk apa mahasiswa diminta mengganti password setelah login?	Menilai pemahaman alasan keamanan penggantian password.
Q05	Berapa minimal karakter password baru?	Menilai kemampuan mengambil informasi numerik dari dokumen.
Q06	Apa contoh password yang benar?	Menilai kemampuan memberikan contoh sesuai ketentuan dokumen.

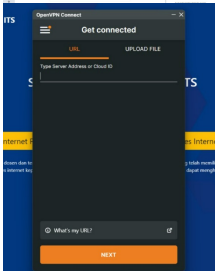
Tabel 3.6 – lanjutan

ID	Kueri	Tujuan Pengujian
Q07	Apa itu SIAKAD ITS?	Menilai kemampuan menjelaskan layanan akademik SIAKAD.
Q08	Apa itu myITS Presensi?	Menilai kemampuan menjelaskan layanan presensi mahasiswa.
Q09	Apa aplikasi yang digunakan untuk aktivasi MFA?	Menilai kemampuan mengambil nama aplikasi pendukung keamanan akun.
Q10	Mengapa Microsoft Authenticator tidak boleh diinstall?	Menilai kemampuan menjelaskan kebutuhan penggunaan berkelanjutan aplikasi MFA.

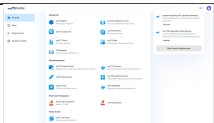
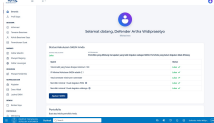
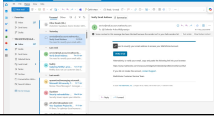
3.3.4 Data Uji Image-Only

Data *image-only* terdiri atas 10 citra aktivitas dan antarmuka layanan digital. Data dikumpulkan secara mandiri oleh peneliti. sebagian besar berupa tangkapan layar aplikasi atau layanan digital, sedangkan satu data merepresentasikan aktivitas pembelajaran. Informasi pribadi yang tidak diperlukan tidak digunakan sebagai target evaluasi. Citra yang sama digunakan kembali pada skenario *text-image RAG*. Daftar citra ditunjukkan pada Tabel 3.7.

Tabel 3.7: Data uji *image-only*

ID	Citra	Input Citra	Tujuan Pengujian
IMG01		Aktivitas kelas	Mengenali aktivitas pembelajaran di kelas.
IMG02		Zoom Workplace	Mengenali aplikasi rapat daring.
IMG03		OpenVPN Connect	Mengenali aplikasi VPN dan elemen antarmukanya.
IMG04		Halaman akses internet ITS	Memahami halaman layanan akses internet kampus.
IMG05		Halaman ubah password myITS	Mengenali formulir penggantian password.

Tabel 3.7 – lanjutan

ID	Citra	Input Citra	Tujuan Pengujian
IMG06		Dashboard myITS Portal	Mengenali daftar aplikasi pada dashboard.
IMG07		Halaman awal my-ITS Portal	Mengenali halaman utama portal.
IMG08		Halaman myITS Presensi	Memahami tampilan layanan presensi.
IMG09		Halaman myITS Student Connect	Memahami informasi SKEM dan portofolio mahasiswa.
IMG10		Microsoft Outlook	Mengenali aplikasi email dan elemen kotak masuk.

3.3.5 Data Uji Audio-Only

Data *audio-only* terdiri atas 10 rekaman singkat mengenai layanan digital ITS. Seluruh audio direkam secara mandiri menggunakan suara peneliti dalam bahasa Indonesia, melibatkan satu penutur, tidak menggunakan *text-to-speech*, dan direkam pada lingkungan yang relatif tenang tanpa penambahan kebisingan latar. Perekaman dilakukan menggunakan mikrofon bawaan telepon pintar. Durasi setiap audio kurang dari tujuh detik sehingga tidak melewati batas maksimum 30 detik yang diterapkan oleh *engine*. Audio yang sama digunakan kembali pada skenario *text-audio RAG*. Daftar audio ditunjukkan pada Tabel 3.8.

Tabel 3.8: Data uji *audio-only*

ID	Input Audio	Durasi (s)	Tujuan Pengujian
AUD01	myITS Portal	5,120	Memahami fungsi myITS Portal dari audio.
AUD02	Zoom ITS	5,696	Memahami kegunaan Zoom ITS dari audio.
AUD03	VPN ITS	4,779	Memahami fungsi VPN ITS dari audio.
AUD04	Akses internet ITS	5,397	Memahami layanan internet kampus dari audio.
AUD05	Ubah password myITS	4,565	Memahami fungsi halaman ubah password.
AUD06	myITS Presensi	5,056	Memahami fungsi layanan presensi.
AUD07	Outlook atau email ITS	5,995	Memahami fungsi email ITS melalui Outlook.
AUD08	myITS Student Connect	4,992	Memahami informasi SKEM dan portofolio.
AUD09	SIAKAD	6,251	Memahami informasi akademik pada SIAKAD.

Tabel 3.8 – lanjutan

ID	Input Audio	Durasi (s)	Tujuan Pengujian
AUD10	Office 365	5,909	Memahami fasilitas Office 365.

3.3.6 Data Uji Text-Image RAG

Setiap data *text-image RAG* terdiri atas query teks dan citra. Pasangan data ditunjukkan pada Tabel 3.9.

Tabel 3.9: Data uji *text-image RAG*

ID	Citra	Kueri	Tujuan Pengujian
TI01	IMG01	Berdasarkan gambar kegiatan kelas dan konteks layanan pembelajaran ITS seperti myITS Classroom, aktivitas pembelajaran apa yang terlihat pada gambar?	Mengintegrasikan aktivitas visual dengan konteks layanan pembelajaran.
TI02	IMG02	Berdasarkan gambar dan panduan Zoom ITS, aplikasi apa yang terlihat dan untuk apa digunakan?	Mengaitkan aplikasi visual dengan dokumen Zoom ITS.
TI03	IMG03	Berdasarkan gambar dan panduan VPN ITS, aplikasi apa yang terlihat dan apa fungsinya?	Menghubungkan tampilan aplikasi dengan fungsi VPN ITS.
TI04	IMG04	Berdasarkan gambar dan panduan akses internet ITS, layanan apa yang ditampilkan pada halaman ini?	Mengenali halaman layanan internet dan menjawab berdasarkan dokumen.
TI05	IMG05	Berdasarkan gambar dan panduan myITS Account, apa fungsi halaman Ubah Password?	Menghubungkan tampilan formulir dengan fungsi penggantian password.
TI06	IMG06	Berdasarkan gambar dashboard myITS Portal dan panduan mahasiswa baru ITS, aplikasi utama apa saja yang dapat diakses mahasiswa melalui portal tersebut?	Mengambil daftar aplikasi dari visual dan dokumen.
TI07	IMG07	Berdasarkan gambar dan panduan myITS Portal, apa fungsi utama myITS Portal bagi mahasiswa ITS?	Menjelaskan fungsi portal dengan dukungan citra halaman awal.
TI08	IMG08	Berdasarkan gambar dan panduan myITS Presensi, apa fungsi myITS Presensi?	Menghubungkan tampilan presensi dengan fungsi layanan.
TI09	IMG09	Berdasarkan gambar myITS Student Connect dan konteks layanan kemahasiswaan ITS, informasi SKEM dan portofolio apa yang ditampilkan pada halaman tersebut?	Memahami informasi visual rinci pada layanan kemahasiswaan.
TI10	IMG10	Berdasarkan gambar Outlook dan panduan email ITS, aplikasi email apa yang terlihat dan apa fungsi email ITS?	Mengenali aplikasi email dan fungsi email ITS berdasarkan dokumen.

3.3.7 Data Uji Text-Audio RAG

Setiap data *text-audio RAG* terdiri atas query teks dan audio. Pasangan data ditunjukkan pada Tabel 3.10.

Tabel 3.10: Data uji *text-audio RAG*

ID	Audio	Kueri	Tujuan Pengujian
TA01	AUD01	Berdasarkan audio, apa fungsi myITS Portal?	Memanfaatkan konteks audio tentang myITS Portal.
TA02	AUD02	Berdasarkan audio, apa kegunaan Zoom ITS?	Memanfaatkan konteks audio tentang Zoom ITS.
TA03	AUD03	Berdasarkan audio, apa fungsi VPN ITS?	Memanfaatkan konteks audio tentang VPN ITS.
TA04	AUD04	Berdasarkan audio, layanan internet kampus apa yang dapat digunakan mahasiswa ITS?	Memanfaatkan konteks audio tentang akses internet ITS.
TA05	AUD05	Berdasarkan audio, apa fungsi halaman ubah password myITS?	Memanfaatkan konteks audio tentang penggantian password.
TA06	AUD06	Berdasarkan audio, apa fungsi myITS Presensi?	Memanfaatkan konteks audio tentang presensi.
TA07	AUD07	Berdasarkan audio, apa fungsi email ITS melalui Microsoft Outlook?	Memanfaatkan konteks audio tentang email ITS.
TA08	AUD08	Berdasarkan audio, informasi apa yang tersedia pada myITS Student Connect?	Memanfaatkan konteks audio tentang layanan kemahasiswaan.
TA09	AUD09	Berdasarkan audio, informasi apa yang tersedia pada SIAKAD?	Memanfaatkan konteks audio tentang informasi akademik.
TA10	AUD10	Berdasarkan audio, fasilitas apa yang disediakan Office 365?	Memanfaatkan konteks audio tentang Office 365.

3.3.8 Ground Truth

Setiap data pada lima kelompok pengujian dilengkapi dengan *ground truth*. *Ground truth* berfungsi sebagai referensi jawaban untuk menilai kesesuaian keluaran sistem. Referensi tidak disusun sebagai jawaban panjang, tetapi sebagai jawaban target yang ringkas, faktual, dan sesuai dengan informasi yang tersedia pada dokumen, citra, atau audio.

Penyusunan *ground truth* dilakukan melalui beberapa tahap. Tahap pertama adalah mengidentifikasi sumber informasi utama yang relevan dengan setiap data uji. Pada *text-only RAG*, sumber utama berasal dari dokumen basis pengetahuan. Pada *image-only*, sumber utama berasal dari elemen visual yang tampak pada citra. Pada *audio-only*, sumber utama berasal dari isi ucapan pada audio. Pada *text-image RAG* dan *text-audio RAG*, sumber utama berasal dari gabungan pertanyaan, media, dan dokumen pendukung. Tahap kedua adalah menyusun jawaban referensi awal berdasarkan sumber tersebut. Tahap ketiga adalah memeriksa kembali jawaban referensi terhadap dokumen, citra, atau audio agar tidak memuat informasi di luar data uji. Tahap terakhir adalah merapikan referensi menjadi kalimat evaluasi yang konsisten.

Pada *text-only RAG*, sebagian pertanyaan memiliki beberapa referensi yang berbeda dalam

susunan kata tetapi ekuivalen secara faktual. Skenario ini digunakan sebagai baseline tekstual berbasis dokumen sehingga beberapa formulasi referensi disediakan untuk mengurangi penalti metrik leksikal akibat variasi redaksi yang tidak mengubah makna. BLEU-1 dan ROUGE-L F1 dihitung terhadap seluruh referensi yang tersedia, kemudian nilai tertinggi digunakan sebagai skor data tersebut.

Pada skenario *text-image RAG* dan *text-audio RAG*, setiap pasangan media dan pertanyaan menggunakan satu referensi kanonik yang merangkum keluaran yang diharapkan. Referensi kanonik dipilih karena setiap data multimodal telah memiliki pasangan citra atau audio yang spesifik. Oleh karena itu, perbandingan utama skor leksikal dilakukan dalam skenario yang sama, yaitu antara konfigurasi representasi numerik atau konfigurasi *chunking*. Perbandingan absolut antarskenario dibaca secara deskriptif karena jumlah referensi *text-only RAG* berbeda dari skenario multimodal.

Contoh bentuk *ground truth* yang digunakan pada setiap kelompok data ditunjukkan pada Tabel 3.11. Referensi lengkap disimpan bersama dataset evaluasi dan digunakan secara identik pada seluruh konfigurasi FP32, FP16, dan 4-bit NF4.

Tabel 3.11: Contoh *ground truth* pada data uji

Kelompok Data	ID	Contoh <i>Ground Truth</i>
<i>Text-Only RAG</i>	Q01	myITS Portal adalah portal <i>single sign-on</i> yang digunakan mahasiswa untuk mengakses berbagai layanan digital ITS.
<i>Image-Only</i>	IMG02	Citra menunjukkan aplikasi Zoom Workplace yang digunakan untuk rapat atau pertemuan daring.
<i>Audio-Only</i>	AUD01	Audio menjelaskan bahwa myITS Portal digunakan sebagai portal <i>single sign-on</i> untuk mengakses berbagai layanan ITS.
<i>Text-Image RAG</i>	TI02	Aplikasi yang terlihat adalah Zoom Workplace atau Zoom ITS, yang digunakan untuk pertemuan daring atau rapat online.
<i>Text-Audio RAG</i>	TA01	myITS Portal digunakan sebagai portal <i>single sign-on</i> untuk mengakses berbagai layanan ITS.

Pertanyaan, citra, audio, dan *ground truth* dipertahankan identik pada FP32, FP16, dan 4-bit NF4. Dengan demikian, perubahan konfigurasi representasi numerik tidak mengubah target evaluasi.

3.4 Preprocessing Data

Preprocessing dilakukan sesuai karakteristik setiap modalitas untuk menghasilkan masukan yang konsisten dengan kebutuhan model. Tahap ini dibedakan dari penyaringan hasil *chunking*. *Preprocessing* membersihkan data mentah, sedangkan validasi panjang, jumlah kata, dan *noise* dilakukan setelah *chunk* terbentuk.

Waktu pelaksanaan *preprocessing* berbeda untuk setiap jenis data. *Preprocessing* dokumen dilakukan sebelum pembangunan indeks pengetahuan karena hasilnya digunakan sebagai bahan *chunking* dan *embedding*. *Preprocessing* citra dan audio dilakukan pada saat inferensi atau evaluasi karena kedua media tersebut tidak disimpan ke indeks FAISS. Dengan demikian, dokumen diproses pada tahap persiapan basis pengetahuan, sedangkan citra dan audio diproses

ketika *pipeline* menerima masukan multimodal.

3.4.1 Preprocessing Dokumen

Dokumen berformat PDF, TXT, dan MD dibaca menggunakan *loader* sesuai jenis berkas. Untuk PDF, teks diekstraksi per halaman menggunakan pustaka pembaca PDF, kemudian seluruh halaman digabungkan. Normalisasi mencakup penyamaan karakter baris baru, penghapusan karakter tidak terlihat yang umum, penggabungan kata yang terpotong akibat pemenggalan baris, perbaikan spasi, pemisahan token yang menempel akibat ekstraksi PDF, dan normalisasi URL.

Hasil tahap ini berupa teks bersih yang masih mempertahankan urutan isi dokumen. Aturan minimum 120 karakter, minimum 20 kata, dan penyaringan pola *noise* tidak diterapkan pada dokumen mentah, tetapi pada hasil *chunking* sebagaimana dijelaskan pada Subbab 3.5.

Rincian proses *preprocessing* dokumen ditunjukkan pada Pseudocode 3.1.

Listing 3.1: Pseudocode preprocessing dokumen

```
1 Input : dokumen d dalam format PDF, TXT, atau MD
2 Output: teks bersih t
3
4 Jika d adalah PDF:
5     baca teks setiap halaman menggunakan pembaca PDF
6     gabungkan teks seluruh halaman
7 Jika d adalah TXT atau MD:
8     baca isi berkas sebagai teks
9
10 samakan karakter baris baru
11 hapus karakter tidak terlihat umum
12 hapus pemenggalan kata akibat hyphen dan baris baru
13 gabungkan baris tunggal menjadi spasi
14 rapikan paragraf dan spasi berlebih
15 perbaiki token yang pecah atau menempel
16 normalisasi URL
17 kembalikan teks bersih t
```

3.4.2 Preprocessing Citra

Citra divalidasi berdasarkan format yang didukung, yaitu PNG, JPG, JPEG, WEBP, dan BMP. Citra selanjutnya diproses oleh *processor* Qwen2-VL dengan pembatasan jumlah piksel untuk menjaga keterbacaan teks pada tangkapan layar sekaligus mengendalikan kebutuhan memori GPU. Citra tidak diubah menjadi *embedding* dan tidak dimasukkan ke indeks pengetahuan.

3.4.3 Preprocessing Audio

Audio divalidasi berdasarkan format WAV, MP3, M4A, FLAC, OGG, dan AAC. Berkas dimuat sebagai sinyal mono dan di-*resampling* menuju *sampling rate* yang dibutuhkan oleh *processor* Qwen2-Audio. Durasi audio dibatasi maksimal 30 detik. Audio yang melebihi batas tersebut dipotong untuk menjaga konsistensi masukan dan mengendalikan kebutuhan komputasi.

3.5 Pembangunan Indeks Pengetahuan

Pembangunan indeks pengetahuan dilakukan setelah *preprocessing* dokumen. *Structure-aware fixed-size chunking with overlap* dengan ukuran maksimum 1000 karakter dan *overlap* 200 karakter digunakan sebagai konfigurasi utama sistem. *Recursive chunking* dengan ukuran maksimum 512 karakter dan *overlap* 128 karakter digunakan sebagai konfigurasi pembanding yang merepresentasikan pemotongan lebih granular dengan separator hierarkis. Kedua konfigurasi

urasi *chunking* diimplementasikan menggunakan fungsi kustom Python dan tidak menggunakan pustaka pemotongan dokumen eksternal seperti *text splitter* siap pakai.

Pemilihan konfigurasi pembandingan tidak dimaksudkan untuk menyatakan bahwa *recursive chunking* merupakan konfigurasi terbaik secara universal. Literatur menunjukkan bahwa *recursive chunking* merupakan salah satu strategi yang umum digunakan sebagai pembandingan dalam sistem RAG, sementara keuntungan strategi pemotongan yang lebih kompleks tidak selalu konsisten pada seluruh dataset dan tugas [34], [35]. Oleh karena itu, efektivitas kedua konfigurasi diuji secara empiris pada dokumen, model *embedding*, mekanisme *retrieval*, dan nilai Top-*k* yang sama.

Perbandingan pada penelitian ini disebut perbandingan konfigurasi *chunking*, bukan pengaruh algoritma pemotongan secara terisolasi. Selain strategi pemisahan, ukuran maksimum dan *overlap* juga berbeda. Hasil pengujian dengan demikian merepresentasikan pengaruh gabungan dari strategi pemisahan, granularitas, *overlap*, jumlah *chunk*, dan distribusi informasi dalam indeks.

3.5.1 Konfigurasi Utama: *Structure-Aware Fixed-Size Chunking*

Konfigurasi utama mempertahankan batas paragraf apabila memungkinkan. Dokumen terlebih dahulu dipisahkan menjadi paragraf, kemudian paragraf digabungkan selama panjang gabungan tidak melebihi 1000 karakter. Paragraf yang lebih panjang dari batas tersebut dipotong menggunakan jendela bergeser. Bagian akhir *chunk* sebelumnya sepanjang 200 karakter digunakan sebagai *overlap* untuk menjaga kesinambungan informasi di sekitar batas pemotongan.

Setelah *chunk* terbentuk, sistem menerapkan panjang minimum 120 karakter dan minimum 20 kata. Hasil yang memenuhi pola daftar isi, templat kosong, deretan titik, kumpulan kata kunci SDG, atau daftar padat tertentu ditandai sebagai *noise* dan tidak dimasukkan ke indeks. Setiap *chunk* yang dipertahankan disimpan bersama nama dokumen, lokasi sumber, nomor *chunk*, jumlah karakter, jumlah kata, dan isi teks.

Rincian konfigurasi utama ditunjukkan pada Pseudocode 3.2.

Listing 3.2: Pseudocode structure-aware fixed-size chunking

```
1 Input : teks t, chunk_size = 1000, overlap = 200,
2       min_length = 120
3 Output: daftar chunk C
4
5 pisahkan t menjadi daftar paragraf
6 C <- daftar chunk kosong
7 current <- teks kosong
8
9 Untuk setiap paragraf p:
10     rapikan spasi pada p
11
12     Jika panjang p > chunk_size:
13         Jika current tidak kosong:
14             tambahkan current ke C
15             kosongkan current
16
17         potong p menggunakan sliding window
18         dengan langkah chunk_size - overlap
19         tambahkan potongan yang memenuhi min_length ke C
20         lanjut ke paragraf berikutnya
21
22     candidate <- gabungkan current dan p
```

```

23
24     Jika panjang candidate <= chunk_size:
25         current <- candidate
26     Jika tidak:
27         tambahkan current ke C
28         ambil overlap terakhir dari chunk sebelumnya
29         current <- gabungkan overlap dan p
30
31 Jika current tidak kosong:
32     tambahkan current ke C
33
34 bersihkan setiap chunk
35 hapus chunk dengan panjang kurang dari min_length
36 hapus chunk dengan jumlah kata kurang dari 20
37 kembalikan C

```

3.5.2 Konfigurasi Pembandingan: *Recursive Chunking*

Konfigurasi pembandingan menggunakan ukuran maksimum 512 karakter, *overlap* 128 karakter, dan panjang minimum 120 karakter. Teks dipisahkan menggunakan separator bertingkat, yaitu paragraf, baris baru, kalimat, titik koma, koma, spasi, dan karakter. Apabila hasil pemisahan masih melebihi ukuran maksimum, proses dilanjutkan menggunakan separator berikutnya.

Potongan yang terbentuk digabungkan kembali hingga mendekati batas 512 karakter. Bagian akhir *chunk* sebelumnya digunakan sebagai *overlap*. Kriteria minimum 120 karakter, minimum 20 kata, dan penyaringan *noise* dibuat sama dengan konfigurasi utama agar perbedaan hasil tidak berasal dari kriteria validasi yang berbeda. Indeks ini hanya digunakan pada skenario pembandingan *text-only RAG*, *text-image RAG*, dan *text-audio RAG*.

Rincian konfigurasi pembandingan ditunjukkan pada Pseudocode 3.3.

Listing 3.3: Pseudocode recursive chunking

```

1 Input : teks t, chunk_size = 512, overlap = 128,
2         daftar separator S, min_length = 120
3 Output: daftar chunk C
4
5 normalisasi teks t
6
7 Fungsi split_recursive(teks x, separator S):
8     Jika panjang x <= chunk_size:
9         kembalikan x sebagai satu potongan
10
11     Jika S kosong:
12         potong x per chunk_size karakter
13         kembalikan hasil potongan
14
15     ambil separator pertama s dari S
16     pisahkan x menggunakan s
17
18     Jika s tidak ditemukan:
19         panggil split_recursive(x, sisa separator)
20
21     Untuk setiap bagian hasil pemisahan:
22         gabungkan bagian selama panjangnya <= chunk_size
23         Jika bagian masih terlalu panjang:
24             panggil split_recursive(bagian, sisa separator)
25

```

```

26 splits <- split_recursive(t, S)
27 C <- gabungan splits dengan overlap antar chunk
28 bersihkan setiap chunk
29 hapus chunk dengan panjang kurang dari min_length
30 hapus chunk dengan jumlah kata kurang dari 20
31 kembalikan C

```

3.5.3 Statistik Hasil *Chunking*

Statistik metadata setelah proses penyaringan ditunjukkan pada Tabel 3.12. Kedua indeks dibangun dari 15 dokumen yang sama. Konfigurasi *recursive* menghasilkan jumlah *chunk* lebih besar dan panjang rata-rata lebih pendek daripada konfigurasi utama.

Tabel 3.12: Statistik hasil konfigurasi *chunking* setelah penyaringan

Statistik	<i>Fixed-Size 1000/200</i>	<i>Recursive 512/128</i>
Jumlah dokumen	15	15
Jumlah <i>chunk</i>	609	1.859
Rata-rata karakter per <i>chunk</i>	990,24	399,46
Median karakter per <i>chunk</i>	1.000	485
Rentang karakter	208–1.000	129–512
Rata-rata kata per <i>chunk</i>	131,50	53,48
Median kata per <i>chunk</i>	132	61
Rentang kata	27–188	20–103

Perbedaan statistik tersebut menjadi bagian dari interpretasi hasil. Konfigurasi *recursive* tidak hanya menggunakan mekanisme pemisahan yang berbeda, tetapi juga menghasilkan granularitas dan jumlah kandidat indeks yang berbeda.

3.5.4 *Embedding* dan Normalisasi Vektor

Setiap *chunk* diubah menjadi *embedding* menggunakan all-MiniLM-L6-v2. Model menghasilkan vektor berdimensi 384. *Embedding* dokumen dan kueri dinormalisasi menggunakan norma L_2 sebagaimana ditunjukkan pada Persamaan 3.1.

$$\hat{\mathbf{e}} = \frac{\mathbf{e}}{\|\mathbf{e}\|_2} \quad (3.1)$$

Vektor disimpan dalam tipe data `float32` sebelum dimasukkan ke FAISS. Setelah normalisasi, *inner product* antara dua vektor ekuivalen dengan *cosine similarity*.

3.5.5 FAISS dan Metadata

Indeks menggunakan IndexFlatIP yang melakukan pencarian *exact inner product* terhadap seluruh vektor. Hasil kedua konfigurasi *chunking* disimpan pada indeks dan metadata yang terpisah. Metadata memuat identitas dokumen sumber, nomor *chunk*, jumlah karakter, jumlah kata, dan teks. Pemisahan tersebut memungkinkan konfigurasi indeks dipilih sesuai skenario tanpa membangun ulang dokumen pada setiap inferensi.

Proses pembangunan indeks dirumuskan pada Persamaan 3.2.

$$I_m = \text{IndexFlatIP}(\text{Normalize}(\text{Embed}(\text{Chunk}_m(\text{Preprocess}(D)))))) \quad (3.2)$$

Pada Persamaan 3.2, D merupakan kumpulan dokumen dan m merupakan konfigurasi *chunking*. Rincian implementasi ditunjukkan pada Pseudocode 3.4.

Listing 3.4: Pseudocode pembangunan indeks pengetahuan

```
1 Input : kumpulan dokumen D dan konfigurasi chunking m
2 Output: indeks FAISS I dan metadata M
3
4 C <- daftar chunk kosong
5 M <- daftar metadata kosong
6
7 Untuk setiap dokumen d pada D:
8     t <- preprocessing dokumen d
9     C_d <- bentuk chunk dari t menggunakan konfigurasi m
10
11     Untuk setiap chunk c pada C_d:
12         Jika c memenuhi batas panjang, jumlah kata, dan filter noise:
13             tambahkan c ke C
14             tambahkan metadata c ke M
15
16 E <- embedding seluruh chunk pada C
17 E <- normalisasi L2 setiap vektor pada E
18 I <- bangun FAISS IndexFlatIP menggunakan E
19 simpan I dan M pada direktori konfigurasi m
20 kembalikan I dan M
```

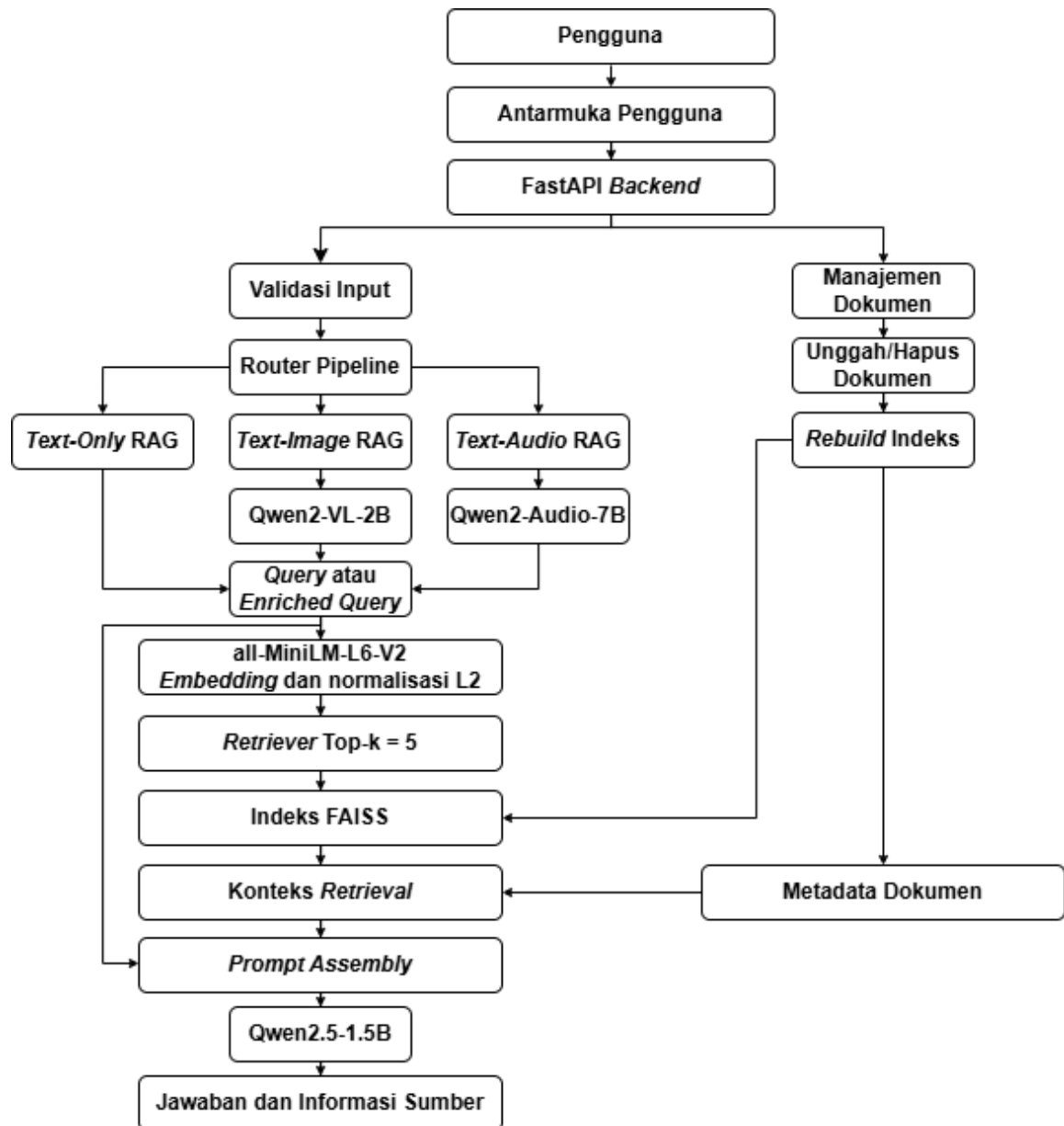
3.6 Perancangan Sistem

Sistem dirancang menggunakan arsitektur *client-server*. Pengguna mengirimkan pertanyaan teks serta berkas citra atau audio melalui antarmuka aplikasi. Permintaan diteruskan ke *backend* FastAPI yang menangani validasi masukan, pemilihan *pipeline*, pemrosesan modalitas, *retrieval*, penyusunan *prompt*, pemanggilan model, dan pengembalian jawaban beserta informasi sumber.

Sistem ditempatkan sebagai sistem *question answering* domain tertutup. Basis pengetahuan, perluasan kueri, dan mekanisme pemeringkatan ulang diarahkan pada istilah layanan dan dokumen ITS. Selain melayani pertanyaan, *backend* menyediakan manajemen dokumen untuk menampilkan, mengunggah, dan menghapus dokumen. Perubahan dokumen diikuti pembangunan ulang indeks utama beserta metadata.

3.6.1 Arsitektur Sistem

Arsitektur sistem terdiri atas pengguna, antarmuka pengguna, *backend* FastAPI, validasi masukan, *router pipeline*, model multimodal, model *embedding*, *retriever*, indeks FAISS, metadata dokumen, Qwen2.5, dan manajemen dokumen. Hubungan antarkomponen ditunjukkan pada Gambar 3.4.



Gambar 3.4: Arsitektur sistem *question answering* multimodal

Permintaan diterima oleh FastAPI dan diperiksa berdasarkan keberadaan pertanyaan, mode yang dipilih, serta format berkas media. Masukan yang valid diarahkan menuju *text-only RAG*, *text-image RAG*, atau *text-audio RAG*. *Image-only* dan *audio-only* tidak ditampilkan pada arsitektur aplikasi karena hanya digunakan sebagai skenario pengujian.

Pada *text-only RAG*, pertanyaan digunakan sebagai kueri awal. Pada *text-image RAG*, Qwen2-VL-2B-Instruct mengekstraksi bukti visual dari citra. Pada *text-audio RAG*, Qwen2-Audio-7B-Instruct menghasilkan transkripsi atau konteks tekstual. Hasil pemrosesan media digabungkan dengan pertanyaan untuk membentuk *enriched query*.

Kueri diubah menjadi vektor menggunakan all-MiniLM-L6-v2 dan dinormalisasi menggunakan norma L_2 . *Retriever* menggunakan vektor tersebut untuk mencari konteks pada indeks FAISS. Identitas hasil pencarian dipasangkan dengan metadata agar sistem memperoleh isi *chunk* dan informasi sumber. Pertanyaan, informasi media, dan konteks *retrieval* kemudian disusun pada tahap *prompt assembly*. Qwen2.5-1.5B-Instruct menghasilkan jawaban akhir yang

dikembalikan ke antarmuka pengguna.

Modul manajemen dokumen berada pada *backend*. Ketika dokumen ditambah atau dihapus, sistem membangun ulang indeks FAISS utama berbasis *fixed-size* dan metadata terkait. Indeks *recursive* dipertahankan sebagai indeks pembandingan eksperimen dan tidak menjadi indeks operasional utama aplikasi.

3.6.2 Mekanisme Retrieval dan Reranking Berbasis Domain

Mekanisme *retrieval* menerapkan perluasan kueri dan *heuristic reranking* berbasis domain. Sebelum pembentukan *embedding*, kueri diperluas menggunakan istilah layanan, maksud pertanyaan, dan kata kunci yang berkaitan dengan domain ITS. FAISS mengambil kandidat awal sebanyak maksimum delapan kali Top- k . Untuk Top- $k = 5$, jumlah kandidat awal paling banyak 40 atau dibatasi oleh jumlah vektor dalam indeks.

Setiap kandidat dinilai kembali menggunakan skor *inner product*, kecocokan leksikal, penguatan kata kunci, dan penalti domain. Skor gabungan kandidat ke- i dirumuskan pada Persamaan 3.3.

$$s_i = s_{IP,i} + b_{keyword,i} + 0,15 s_{lexical,i} - p_{domain,i} \quad (3.3)$$

Pada Persamaan 3.3, $s_{IP,i}$ merupakan skor *inner product* dari FAISS, $b_{keyword,i}$ merupakan penguatan kata kunci, $s_{lexical,i}$ merupakan kecocokan token, dan $p_{domain,i}$ merupakan penalti untuk kandidat yang tidak sesuai dengan domain pertanyaan. Kandidat diurutkan berdasarkan s_i , kemudian lima konteks dengan skor tertinggi dipilih.

Ambang relevansi $\tau = 0,38$ diterapkan terhadap skor gabungan hasil *reranking*, bukan terhadap *cosine similarity* atau *inner product* murni. Jika skor terbaik berada di bawah ambang, sistem mengembalikan respons bahwa informasi belum ditemukan secara meyakinkan. Seluruh aturan perluasan kueri, penguatan kata kunci, penalti domain, dan ambang relevansi dipertahankan sama pada FP32, FP16, 4-bit NF4, serta kedua konfigurasi *chunking*. Oleh karena mekanisme tersebut dikembangkan untuk domain ITS, hasil penelitian tidak dimaksudkan sebagai ukuran generalisasi pada pertanyaan di luar domain tersebut.

3.6.3 Backend dan Endpoint API

Backend dibangun menggunakan FastAPI. *Endpoint* utama `/api/ask` menerima mode, pertanyaan, dan berkas media apabila diperlukan. Mode aplikasi terdiri atas `text`, `text-image`, dan `text-audio`. *Backend* juga menyediakan *endpoint* pemeriksaan kesehatan, daftar dokumen, unggah dokumen, hapus dokumen, dan pembangunan ulang indeks.

Ketika dokumen ditambah atau dihapus, indeks *fixed-size* dibangun ulang agar isi indeks dan metadata tetap sesuai dengan dokumen yang tersedia. Indeks *recursive* tidak menggantikan indeks aplikasi dan hanya digunakan saat skenario pembandingan dijalankan.

3.6.4 Manajemen Model dan Memori GPU

Model dimuat sesuai *pipeline* agar seluruh model besar tidak berada di memori GPU secara bersamaan. Pada *text-image RAG*, model visual digunakan untuk mengekstraksi bukti visual sebelum model teks menghasilkan jawaban akhir. Pada *text-audio RAG*, model audio digunakan untuk menghasilkan transkripsi atau konteks tekstual, kemudian dilepas sebelum Qwen2.5 dimuat.

Pelepasan model dilakukan melalui penghapusan objek model, *garbage collection*, pengosongan *cache* CUDA, dan sinkronisasi apabila diperlukan. *Backend* juga menggunakan

asynchronous lock agar operasi GPU berat tidak dijalankan secara bersamaan. Pada penggunaan aplikasi, proses muat dan lepas model dapat terjadi kembali pada permintaan berikutnya sehingga waktu operasional dapat mencakup *startup latency*. Eksperimen menggunakan *warm-up* dan strategi eksekusi yang dijelaskan pada Subbab 3.9.7 agar pengukuran utama tidak didominasi pemuatan awal.

Konfigurasi operasional aplikasi menggunakan mode campuran, yaitu model teks FP32, model visual FP32, dan model audio 4-bit NF4. Konfigurasi operasional tersebut tidak digunakan sebagai dasar perbandingan Bab IV. seluruh eksperimen terkontrol menggunakan konfigurasi yang seragam pada model-model dalam satu *pipeline*.

3.7 Implementasi *Pipeline* Sistem dan Pengujian

Implementasi dibedakan menjadi *pipeline* utama aplikasi dan *pipeline* khusus pengujian. Aplikasi menyediakan *text-only RAG*, *text-image RAG*, dan *text-audio RAG*. Ketiganya menggunakan *retrieval* dengan $\text{Top-}k = 5$ dan Qwen2.5-1.5B-Instruct sebagai model pembentuk jawaban akhir. *Image-only* dan *audio-only* digunakan khusus untuk mengisolasi kemampuan dasar Qwen2-VL dan Qwen2-Audio tanpa FAISS dan tanpa Qwen2.5.

Dalam perbandingan konfigurasi representasi numerik, dokumen, indeks, mekanisme *retrieval*, $\text{Top-}k$, ambang relevansi, pertanyaan, media, dan *ground truth* dipertahankan sama. Indeks I pada persamaan dan pseudocode mengacu pada indeks *fixed-size* atau *recursive* sesuai skenario.

3.7.1 *Text-Only RAG*

Pipeline text-only RAG menerima pertanyaan tanpa media tambahan. Pertanyaan digunakan untuk melakukan *retrieval*, kemudian lima konteks dengan skor tertinggi dipertahankan apabila skor terbaik memenuhi ambang relevansi. Pertanyaan dan konteks disusun menjadi *prompt* untuk Qwen2.5-1.5B-Instruct. Keluaran dibersihkan dan diperiksa. keluaran yang tidak memenuhi pemeriksaan dapat digenerasi ulang atau diganti dengan jawaban ekstraktif dari konteks.

$$C_5 = \text{TopK}_5 \left(\{s_i(q, I)\}_{i=1}^N \right) \quad (3.4)$$

$$g(q) = \begin{cases} 1, & \text{score}(C_1) \geq \tau, \\ 0, & \text{score}(C_1) < \tau. \end{cases} \quad (3.5)$$

Pada Persamaan 3.4, q merupakan pertanyaan, I merupakan indeks aktif, dan C_5 merupakan lima konteks dengan skor gabungan tertinggi. Persamaan 3.5 menunjukkan pemeriksaan ambang relevansi. Rincian implementasi ditunjukkan pada Pseudocode 3.5.

Listing 3.5: Pseudocode pipeline text-only RAG

```

1 Input : pertanyaan q, indeks I, Top-k = 5, ambang tau
2 Output: jawaban a dan konteks C
3
4 C <- retrieve(q, I, 5)
5
6 Jika C kosong atau score(C[1]) < tau:
7     kembalikan respons informasi tidak ditemukan
8
9 P <- susun prompt dari q dan C
10 a <- hasil generasi Qwen2.5 menggunakan P

```

```

11 a <- bersihkan keluaran a
12
13 Jika a terdeteksi tidak layak:
14     a <- generasi ulang menggunakan prompt lebih ketat
15
16 Jika a masih tidak layak:
17     a <- bentuk jawaban ekstraktif dari C
18
19 kembalikan a dan C

```

3.7.2 Text-Image RAG

Pipeline text-image RAG menerima pertanyaan dan citra. Qwen2-VL-2B-Instruct terlebih dahulu mengekstraksi bukti visual, termasuk objek, nama aplikasi, elemen antarmuka, dan teks yang terlihat. Bukti visual digabungkan dengan pertanyaan untuk membentuk *enriched query*. Kueri tersebut digunakan untuk mengambil konteks dokumen, kemudian Qwen2.5 menghasilkan jawaban akhir dari pertanyaan, bukti visual, dan konteks yang relevan.

$$q_{vi} = \text{Concat}(q, e_v), \quad C_5 = \text{TopK}_5 \left(\{s_i(q_{vi}, I)\}_{i=1}^N \right) \quad (3.6)$$

Pada Persamaan 3.6, e_v merupakan bukti visual yang dihasilkan Qwen2-VL dan q_{vi} merupakan *enriched query*. Rincian implementasi ditunjukkan pada Pseudocode 3.6.

Listing 3.6: Pseudocode pipeline text-image RAG

```

1 Input : pertanyaan q, citra v, indeks I, Top-k = 5
2 Output: jawaban a dan konteks C
3
4 e_v <- ekstraksi bukti visual dari q dan v menggunakan Qwen2-VL
5
6 Jika e_v terdeteksi lemah:
7     e_v <- ulangi ekstraksi dengan instruksi visual lebih ketat
8
9 Jika e_v tetap tidak memadai:
10     kembalikan respons bahwa citra belum dikenali dengan andal
11
12 q_vi <- gabungkan q dan e_v
13 C_raw <- retrieve(q_vi, I, 5)
14 C <- saring C_raw berdasarkan ambang dan selisih skor
15
16 P <- susun prompt dari q, e_v, dan C
17 a <- hasil generasi Qwen2.5 menggunakan P
18 a <- bersihkan keluaran a
19
20 Jika a terdeteksi tidak layak:
21     a <- generasi ulang berbasis bukti visual
22
23 kembalikan a dan C

```

3.7.3 Text-Audio RAG

Pipeline text-audio RAG menerima pertanyaan dan berkas audio. Qwen2-Audio-7B-Instruct menghasilkan transkripsi atau konteks tekstual, kemudian hasil tersebut digabungkan dengan pertanyaan untuk membentuk *enriched query*. Kueri digunakan untuk mengambil lima konteks. Setelah pemrosesan audio selesai, model audio dilepas dari memori GPU sebelum Qwen2.5

menghasilkan jawaban akhir.

$$q_{au} = \text{Concat}(q, t_u), \quad C_5 = \text{TopK}_5 \left(\{s_i(q_{au}, I)\}_{i=1}^N \right) \quad (3.7)$$

Pada Persamaan 3.7, t_u merupakan konteks tekstual dari audio u dan q_{au} merupakan *enriched query*. Pseudocode 3.7 menggambarkan alur logis satu permintaan pada aplikasi. Strategi dua fase yang digunakan dalam pengujian dijelaskan terpisah pada Subbab 3.9.7.

Listing 3.7: Pseudocode pipeline text-audio RAG pada aplikasi

```

1 Input : pertanyaan q, audio u, indeks I, Top-k = 5
2 Output: jawaban a dan konteks C
3
4 t_u <- ekstraksi konteks tekstual dari u menggunakan Qwen2-Audio
5
6 Jika t_u terdeteksi tidak layak:
7     t_u <- ulangi pemrosesan audio
8
9 Jika t_u tetap tidak layak:
10     kembalikan respons bahwa audio belum dipahami dengan andal
11
12 q_au <- gabungkan q dan t_u
13 C <- retrieve(q_au, I, 5)
14
15 lepaskan model audio dari memori GPU
16 P <- susun prompt dari q, t_u, dan C
17 a <- hasil generasi Qwen2.5 menggunakan P
18 a <- bersihkan keluaran a
19
20 Jika a terdeteksi tidak layak:
21     a <- generasi ulang atau respons alternatif
22
23 kembalikan a dan C

```

3.7.4 Pipeline Pengujian Image-Only

Image-only tidak tersedia sebagai mode aplikasi. Skenario ini mengevaluasi kemampuan dasar Qwen2-VL-2B-Instruct dalam memahami citra tanpa *retrieval*, FAISS, dan Qwen2.5. Model menerima citra serta instruksi deskripsi, kemudian menghasilkan keluaran tekstual yang dibandingkan dengan *ground truth* menggunakan *cosine similarity*. Waktu pemrosesan dicatat sebagai *latency*.

Skenario ini digunakan sebagai acuan diagnostik untuk *text-image RAG*. Keluaran *image-only* yang rendah dapat menunjukkan keterbatasan pada pemahaman visual, sedangkan keluaran *image-only* yang baik tetapi hasil RAG multimodal menurun mengarahkan analisis pada pembentukan *enriched query*, *retrieval*, atau generasi jawaban akhir.

3.7.5 Pipeline Pengujian Audio-Only

Audio-only tidak tersedia sebagai mode aplikasi. Skenario ini mengevaluasi kemampuan dasar Qwen2-Audio-7B-Instruct dalam memahami informasi utama pada audio tanpa *retrieval*, FAISS, dan Qwen2.5. Keluaran tekstual dibandingkan dengan *ground truth* menggunakan *cosine similarity*, sedangkan waktu pemrosesan dicatat sebagai *latency*.

Pengujian tidak diposisikan sebagai transkripsi kata demi kata sehingga *Word Error Rate* tidak digunakan. Hasil *audio-only* menjadi acuan diagnostik untuk membedakan keterbatasan

pemahaman audio dari keterbatasan pembentukan *enriched query*, *retrieval*, dan generasi jawaban pada *text-audio RAG*.

3.8 Konfigurasi Representasi Numerik Model

Eksperimen membandingkan FP32, FP16, dan 4-bit NF4. Seluruh model yang terlibat dalam satu *pipeline* menggunakan konfigurasi representasi numerik yang sama. Pada *text-image RAG* FP16, Qwen2-VL dan Qwen2.5 sama-sama menggunakan FP16. Pada *text-audio RAG* 4-bit NF4, Qwen2-Audio dan Qwen2.5 sama-sama menggunakan 4-bit NF4. Pengendalian tersebut memastikan bahwa label konfigurasi merepresentasikan keseluruhan *pipeline*, bukan hanya salah satu model.

3.8.1 FP32

Model dimuat menggunakan `torch.float32`. TF32 dinonaktifkan pada *backend* CUDA agar pengujian FP32 tidak menggunakan jalur akselerasi TF32. Konfigurasi ini digunakan sebagai representasi numerik penuh dalam eksperimen.

3.8.2 FP16

Model dimuat menggunakan `torch.float16`. Konfigurasi ini menggunakan bobot dan operasi *half precision* pada GPU. FP16 mengurangi kebutuhan penyimpanan bobot dibandingkan FP32, tetapi pengaruhnya terhadap *latency* tetap ditentukan melalui hasil empiris.

3.8.3 4-bit NF4

Konfigurasi 4-bit diterapkan menggunakan BitsAndBytes dengan bobot NF4, tipe data komputasi FP16, dan *double quantization*. Parameter implementasi ditunjukkan sebagai berikut.

Listing 3.8: Konfigurasi kuantisasi 4-bit NF4

```
load_in_4bit=True
bnb_4bit_quant_type="nf4"
bnb_4bit_compute_dtype=torch.float16
bnb_4bit_use_double_quant=True
```

Konfigurasi tersebut menunjukkan bahwa bobot disimpan menggunakan representasi NF4, sedangkan komputasi dijalankan menggunakan FP16. Oleh karena itu, konfigurasi ini disebut 4-bit NF4 dan tidak disamakan dengan kuantisasi integer 4-bit linier. Penggunaan 4-bit NF4 ditujukan terutama untuk mengurangi kebutuhan memori. Konfigurasi ini tidak diasumsikan selalu menghasilkan *latency* paling rendah.

3.9 Desain Pengujian

Pengujian dirancang agar perubahan hasil dapat ditelusuri terhadap variabel yang dibandingkan. Pada perbandingan representasi numerik, data, *ground truth*, indeks, mekanisme *retrieval*, batas token, dan perangkat keras dipertahankan sama. Pada perbandingan konfigurasi *chunking*, data dan konfigurasi model dipertahankan sama, sedangkan indeks *fixed-size* dan *recursive* digunakan secara bergantian. Skrip evaluasi melakukan validasi konfigurasi sebelum inferensi utama dijalankan.

3.9.1 Skenario Evaluasi

Delapan skenario evaluasi ditunjukkan pada Tabel 3.13. Lima skenario konfigurasi utama ditempatkan terlebih dahulu, kemudian tiga skenario RAG *recursive* ditempatkan sebagai pem-

banding.

Tabel 3.13: Skenario evaluasi sistem

No.	Skenario	Masukan	Model yang Terlibat	Indeks
1	<i>Text-Only RAG Fixed-Size</i>	Teks	Qwen2.5-1.5B-Instruct	<i>Fixed-size</i>
2	<i>Image-Only</i>	Citra	Qwen2-VL-2B-Instruct	Tidak menggunakan indeks
3	<i>Audio-Only</i>	Audio	Qwen2-Audio-7B-Instruct	Tidak menggunakan indeks
4	<i>Text-Image RAG Fixed-Size</i>	Teks dan citra	Qwen2-VL-2B-Instruct dan Qwen2.5-1.5B-Instruct	<i>Fixed-size</i>
5	<i>Text-Audio RAG Fixed-Size</i>	Teks dan audio	Qwen2-Audio-7B-Instruct dan Qwen2.5-1.5B-Instruct	<i>Fixed-size</i>
6	<i>Text-Only RAG Recursive</i>	Teks	Qwen2.5-1.5B-Instruct	<i>Recursive</i>
7	<i>Text-Image RAG Recursive</i>	Teks dan citra	Qwen2-VL-2B-Instruct dan Qwen2.5-1.5B-Instruct	<i>Recursive</i>
8	<i>Text-Audio RAG Recursive</i>	Teks dan audio	Qwen2-Audio-7B-Instruct dan Qwen2.5-1.5B-Instruct	<i>Recursive</i>

Setiap skenario dijalankan menggunakan FP32, FP16, dan 4-bit NF4. Setiap konfigurasi representasi numerik mencakup delapan skenario dengan 10 data uji, sehingga menghasilkan 80 inferensi utama. Tiga konfigurasi menghasilkan total 240 inferensi utama. Jumlah tersebut tidak mencakup *warm-up*, percobaan ulang internal, evaluasi RAGAS, dan *profiling* kebutuhan memori GPU.

3.9.2 Konfigurasi Eksperimen

Konfigurasi eksperimen ditunjukkan pada Tabel 3.14.

Tabel 3.14: Konfigurasi eksperimen

Parameter	Nilai
Model <i>embedding</i>	all-MiniLM-L6-v2
Dimensi <i>embedding</i>	384
Indeks vektor	FAISS IndexFlatIP
Top- <i>k retrieval</i>	5 konteks
Ambang relevansi	0,38 pada skor gabungan hasil <i>reranking</i>
Batas masukan Qwen2.5	2048 token
Batas keluaran Qwen2.5	384 token
Batas keluaran Qwen2-VL	256 token
Batas keluaran Qwen2-Audio	128 token
Batas piksel citra	200704–1003520 piksel
Durasi audio maksimum	30 detik

Tabel 3.14 – lanjutan

Parameter	Nilai
Konfigurasi representasi numerik	FP32, FP16, dan 4-bit NF4
Metode generasi	Deterministik, <code>do_sample=False</code>
Ukuran <i>batch</i>	Satu data per inferensi

Batas keluaran Qwen2-VL dan Qwen2-Audio digunakan pada tahap ekstraksi informasi media. Batas 384 token digunakan oleh Qwen2.5 untuk generasi teks, sedangkan implementasi *text-image RAG* dapat menerapkan batas yang lebih kecil pada jawaban akhir melalui fungsi `min` untuk menjaga keluaran tetap ringkas.

3.9.3 Kontrol Eksperimen

Dokumen, pertanyaan, citra, audio, *ground truth*, $\text{Top-}k = 5$, ambang relevansi, batas token, perangkat keras, dan mekanisme *retrieval* dipertahankan sama pada konfigurasi yang dibandingkan. Pada skenario multimodal, seluruh model dalam satu *pipeline* menggunakan representasi numerik yang sama. Perubahan FP32, FP16, atau 4-bit NF4 tidak mengubah indeks dan aturan *reranking*.

Generasi dijalankan dengan satu data per inferensi. *Warm-up* dilakukan sebelum pengujian utama dan tidak dimasukkan ke hasil. Jumlah konteks, skor konteks terbaik, waktu komponen, jawaban, dan status kesalahan disimpan apabila tersedia. Rata-rata metrik dihitung dari data dengan inferensi berhasil dan keluaran tidak kosong. Jumlah data berhasil, jumlah data gagal, dan pesan kesalahan dipertahankan agar rata-rata dapat ditafsirkan bersama tingkat keberhasilan sistem.

3.9.4 Validasi Konfigurasi Sebelum Pengujian

Sebelum pengujian dijalankan, skrip memeriksa label eksperimen, mode model, indeks FAISS, parameter generasi, dan data uji. Pemeriksaan diterapkan pada FP32, FP16, dan 4-bit NF4 dengan menyesuaikan model teks, visual, atau audio yang digunakan oleh setiap *pipeline*.

Validasi juga mencakup $\text{Top-}k$, batas token, keberadaan indeks dan metadata, kelengkapan kolom dataset, serta ketersediaan berkas media. Setelah konfigurasi dinyatakan sesuai, satu masukan awal dijalankan sebagai *warm-up*. Ketidaksesuaian konfigurasi menghentikan proses atau menyebabkan data terkait dicatat sebagai gagal sehingga tidak diperlakukan sebagai hasil valid.

3.9.5 Evaluasi Kualitas Keluaran

Skenario RAG dievaluasi menggunakan BLEU-1 dan ROUGE-L F1. BLEU-1 pada implementasi menghitung presisi unigram dengan *brevity penalty*, sedangkan ROUGE-L F1 menggunakan *longest common subsequence* dan rata-rata harmonik antara *precision* dan *recall*. Penamaan tersebut digunakan secara konsisten pada Bab IV.

Jawaban tidak langsung dinilai dalam bentuk keluaran mentah. Skrip mengekstraksi bagian jawaban akhir, menghapus label keluaran dan bagian sumber apabila ada, menormalisasi Unicode, huruf, tanda baca, serta spasi, kemudian melakukan tokenisasi. Pada *text-image RAG*, teks evaluasi dibatasi maksimal 70 kata untuk mencegah keluaran berulang yang terlalu panjang mendominasi metrik leksikal. Aturan normalisasi mengikuti skrip masing-masing skenario, tetapi dipertahankan sama pada FP32, FP16, 4-bit NF4, indeks *fixed-size*, dan indeks *recursive* dalam skenario yang sama.

Pada *text-only RAG*, skor dihitung terhadap seluruh referensi ekuivalen dan nilai tertinggi

dipilih. Pada *text-image RAG* dan *text-audio RAG*, skor dihitung terhadap satu referensi kanonik per data. Oleh karena itu, perbandingan utama dilakukan dalam skenario yang sama. Perbandingan antarskenario digunakan secara deskriptif dan tidak diperlakukan sebagai perbandingan yang sepenuhnya setara.

Skenario *image-only* dan *audio-only* menggunakan *cosine similarity* antara *embedding* keluaran dan referensi. Kedua vektor dibentuk menggunakan all-MiniLM-L6-v2 dan dinormalisasi sebelum skor kesamaan dihitung. Rincian evaluasi leksikal ditunjukkan pada Pseudocode 3.9.

Listing 3.9: Pseudocode evaluasi kualitas keluaran skenario RAG

```
1 Input : jawaban model a, daftar referensi R, aturan skenario z
2 Output: skor BLEU-1 b dan ROUGE-L F1 r
3
4 a_eval <- ekstrak bagian jawaban akhir dari a
5 a_eval <- hapus label keluaran dan informasi sumber
6 a_eval <- normalisasi sesuai aturan skenario z
7
8 Jika z adalah text-image RAG dan a_eval melebihi 70 kata:
9     a_eval <- ambil 70 kata pertama
10
11 B <- daftar skor BLEU-1 kosong
12 G <- daftar skor ROUGE-L F1 kosong
13
14 Untuk setiap referensi r_i pada R:
15     r_eval <- normalisasi r_i sesuai aturan skenario z
16     tambahkan BLEU-1(a_eval, r_eval) ke B
17     tambahkan ROUGE-L F1(a_eval, r_eval) ke G
18
19 b <- nilai maksimum B
20 r <- nilai maksimum G
21 kembalikan b dan r
```

3.9.6 Evaluasi Kualitas *Retrieval*

Kualitas *retrieval* pada enam skenario RAG dievaluasi menggunakan *context precision* dan *context recall*. Evaluasi dilakukan melalui RAGAS menggunakan Llama-3.1-8B-Instant sebagai model evaluator. Lima konteks dipertahankan, sedangkan setiap konteks dibatasi maksimal 800 karakter pada payload evaluator. Batas keluaran evaluator ditetapkan 4096 token dan batas waktu evaluasi 240 detik.

Sebelum evaluasi RAGAS, spasi konteks dinormalisasi dan angka yang berdiri sendiri dihapus sebagai bagian dari penyaringan nomor halaman pada skrip evaluasi. Tahap tersebut hanya diterapkan pada salinan konteks yang dikirim kepada evaluator dan tidak mengubah konteks yang digunakan untuk menghasilkan jawaban. Konsekuensi aturan tersebut terhadap konteks yang memuat informasi numerik dipertimbangkan sebagai keterbatasan interpretasi *context precision* dan *context recall*.

Pada pengujian final FP32, *context precision* dan *context recall* dihitung untuk enam skenario RAG. Hasil FP16 dan 4-bit NF4 yang telah diperoleh sebelumnya dipertahankan karena dokumen, pertanyaan, *ground truth*, Top-k, indeks, konteks *retrieval*, model evaluator, dan konfigurasi evaluasi tidak diubah.

3.9.7 Pengukuran *Latency*

Warm-up dijalankan sebelum pengukuran utama dan dikeluarkan dari hasil. Pada *text-only RAG*, nilai utama menggunakan *total_s* dari *pipeline*. jika tidak tersedia, skrip menggunakan pengukuran *wall-clock*. Pada *image-only*, nilai diambil dari *total_s*, *vision_total_s*, atau *wall-clock*. Pada *audio-only*, nilai diambil dari *total_s*, *audio_total_s*, atau *wall-clock* sesuai nilai yang tersedia.

Pada *text-image RAG*, *latency* mencakup pemrosesan citra, *retrieval*, dan generasi jawaban. Pada aplikasi, *text-audio RAG* memproses satu permintaan secara berurutan dan melepas model audio sebelum memuat model teks. Pemuatan ulang model pada permintaan berikutnya dapat menambah *startup latency* operasional.

Pengujian *text-audio RAG* menggunakan strategi dua fase. Seluruh audio diproses terlebih dahulu dengan model audio tetap dimuat, kemudian model audio dilepas satu kali. Setelah itu, seluruh *text RAG* dijalankan. *Latency* per data dihitung sebagai agregasi dua komponen:

$$L_{\text{text-audio}} = L_{\text{audio}} + L_{\text{text-RAG}} \quad (3.8)$$

Nilai pada Persamaan 3.8 merupakan *latency* komponen teragregasi, bukan satu pengukuran *wall-clock* utuh dari awal hingga akhir permintaan tunggal. *Warm-up* audio dan *warm-up text RAG* tidak dimasukkan ke hasil. Komponen *wall-clock*, waktu *retrieval*, waktu generasi, dan waktu pemrosesan media disimpan apabila tersedia untuk kebutuhan audit.

3.9.8 *Profiling* Kebutuhan Memori GPU

Profiling dilakukan pada lima skenario konfigurasi utama, yaitu *text-only RAG*, *image-only*, *audio-only*, *text-image RAG*, dan *text-audio RAG* dengan indeks *fixed-size* pada skenario RAG. Setiap skenario menggunakan 10 data dan dijalankan pada FP32, FP16, serta 4-bit NF4. Dengan demikian, terdapat 15 kombinasi skenario–representasi numerik dan 150 pengukuran pada tingkat data.

Pengukuran menyimpan memori PyTorch sebelum dan setelah inferensi, *peak allocated memory*, *peak reserved memory*, penggunaan memori GPU sebelum, setelah, dan pada kondisi puncak, serta selisih penggunaan puncak. Pengambilan sampel tingkat perangkat dilakukan setiap 0,10 detik menggunakan informasi dari GPU. Utilisasi GPU, daya, temperatur, jumlah sampel, *latency pipeline*, *wall-clock latency*, status keberhasilan, dan pesan kesalahan juga disimpan. Pada *text-audio RAG*, puncak fase audio dan fase *text RAG* dipisahkan apabila tersedia.

Torch peak allocated dan *torch peak reserved* menjelaskan alokasi yang dikelola PyTorch. *GPU memory used peak* menggambarkan puncak penggunaan memori pada perangkat GPU selama inferensi dan dapat mencakup alokasi selain tensor yang dikelola PyTorch. Utilisasi GPU dianalisis sebagai metrik terpisah dan tidak digunakan sebagai sinonim kebutuhan VRAM.

Profiling dibatasi pada konfigurasi utama *fixed-size* karena tujuan pengukuran adalah membandingkan pengaruh FP32, FP16, dan 4-bit NF4 terhadap kebutuhan memori pada konfigurasi sistem yang representatif. *Recursive chunking* tetap dibandingkan melalui kualitas keluaran, kualitas *retrieval*, dan *latency*. Kegagalan inferensi, termasuk kehabisan memori, dicatat secara transparan dan tidak diubah menjadi nilai valid.

3.10 Metode Analisis

Analisis dilakukan pada empat tingkat. Kualitas keluaran dianalisis menggunakan BLEU-1, ROUGE-L F1, atau *cosine similarity* sesuai skenario. Kualitas *retrieval* dianalisis menggunakan

context precision dan *context recall*. *Latency* dianalisis berdasarkan waktu utama dan komponen yang tersedia. Kebutuhan memori GPU dianalisis pada 15 kombinasi *profiling*.

Perbandingan representasi numerik dilakukan tanpa mengasumsikan bahwa jumlah bit yang lebih rendah selalu menghasilkan *latency* lebih kecil. FP32, FP16, dan 4-bit NF4 dibandingkan berdasarkan hasil empiris. Konfigurasi yang mengurangi kebutuhan memori tetapi memiliki *latency* lebih tinggi diperlakukan sebagai perbedaan antara efisiensi memori dan kecepatan inferensi, bukan sebagai kegagalan pengujian.

Analisis konfigurasi *chunking* dilakukan pada tiga kategori RAG dengan membandingkan BLEU-1, ROUGE-L F1, *context precision*, *context recall*, dan *latency*. Interpretasi tidak mengisolasi pengaruh algoritma pemotongan secara tunggal karena konfigurasi utama dan pembanding juga memiliki ukuran *chunk* serta *overlap* yang berbeda. Perbedaan hasil dikaitkan dengan strategi pemisahan, granularitas, jumlah *chunk*, panjang konteks, dan distribusi informasi dalam indeks.

Perbandingan absolut antarskenario digunakan secara deskriptif. Analisis utama dilakukan dalam skenario yang sama karena karakteristik masukan, model, cakupan *latency*, dan jumlah referensi dapat berbeda. Hasil juga dibaca bersama jumlah data berhasil dan status kegagalan agar rata-rata tidak menutupi keterbatasan inferensi.

Rekomendasi akhir tidak ditentukan oleh satu metrik. Konfigurasi dipilih berdasarkan *trade-off* antara kualitas keluaran, *latency*, dan kebutuhan memori GPU. Rekomendasi dibatasi pada domain dokumen dan layanan ITS yang digunakan dalam penelitian. Hasil numerik serta interpretasi seluruh pengujian disajikan pada Bab IV.

[Halaman ini sengaja dikosongkan]

BAB IV

PENGUJIAN DAN ANALISIS

Bab ini menyajikan hasil pengujian dan analisis terhadap sistem *question answering* multi-modal berbasis model Qwen dengan metode *Retrieval-Augmented Generation* (RAG). Pengujian dilakukan untuk menilai kemampuan sistem dalam memanfaatkan dokumen, citra, dan audio sebagai dasar penjawaban. Selain itu, pengujian juga digunakan untuk mengevaluasi pengaruh konfigurasi *chunking*, konfigurasi representasi numerik model, *latency*, serta kebutuhan memori GPU.

Berbeda dengan Bab III yang menjelaskan rancangan dan implementasi sistem, Bab IV berfokus pada hasil eksperimen. Analisis dilakukan secara deskriptif-komparatif berdasarkan nilai metrik per data, nilai rata-rata, grafik perbandingan, dan pembacaan *trade-off*. Dengan demikian, pembahasan tidak hanya mencari konfigurasi dengan nilai rata-rata tertinggi, tetapi juga mempertimbangkan kestabilan kualitas keluaran, kualitas *retrieval*, waktu respons, dan kebutuhan memori GPU.

4.1 Gambaran Umum Pengujian

Pengujian utama dilakukan terhadap delapan skenario. Skenario tersebut terdiri atas *text-only RAG* dengan konfigurasi *fixed-size*, *text-only RAG* dengan konfigurasi *recursive*, *image-only*, *audio-only*, *text-image RAG* dengan konfigurasi *fixed-size*, *text-image RAG* dengan konfigurasi *recursive*, *text-audio RAG* dengan konfigurasi *fixed-size*, dan *text-audio RAG* dengan konfigurasi *recursive*. Pengujian utama menggunakan konfigurasi representasi numerik FP32, FP16, dan 4-bit NF4. Selain itu, dilakukan pengujian tambahan pada konfigurasi *fixed-size* untuk membaca pengaruh INT8 WO A16W8 dan FP32 *strict* terhadap *latency*.

Pada skenario berbasis RAG, sistem mengambil lima konteks teratas dari indeks FAISS sesuai nilai $Top-k = 5$. Konfigurasi *fixed-size* digunakan sebagai konfigurasi *chunking* utama, sedangkan konfigurasi *recursive* digunakan sebagai pembanding. Pada skenario *image-only* dan *audio-only*, sistem tidak menggunakan proses *retrieval*, sehingga evaluasi dilakukan terhadap pemahaman dasar model visual dan audio.

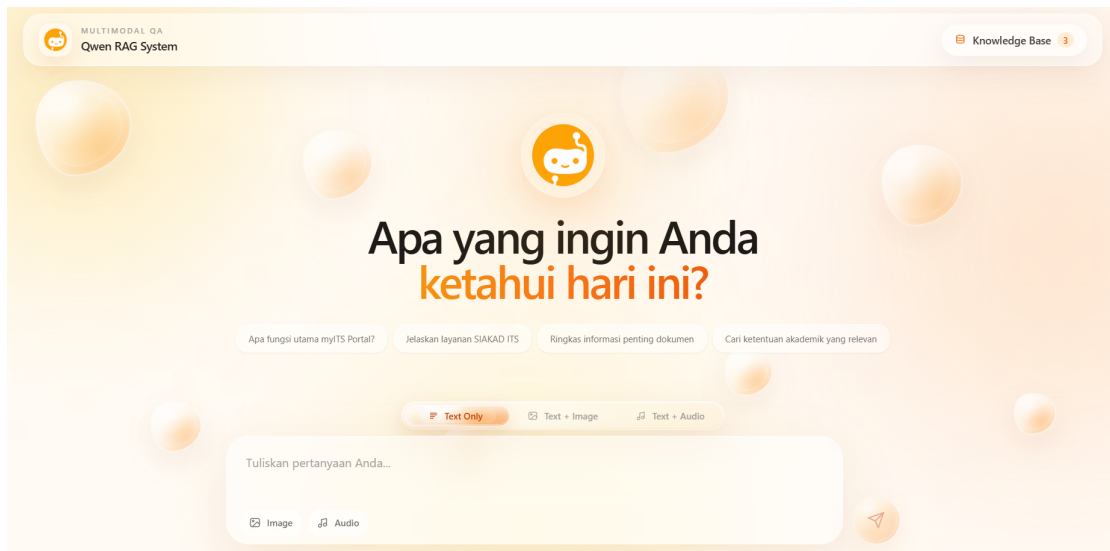
Profiling kebutuhan memori GPU dilakukan pada lima skenario representatif berbasis konfigurasi utama *fixed-size*, yaitu *text-only RAG*, *image-only*, *audio-only*, *text-image RAG*, dan *text-audio RAG*. Pembatasan tersebut dilakukan karena profiling difokuskan pada konfigurasi utama sistem dan perbandingan FP32, FP16, serta 4-bit NF4 terhadap kebutuhan memori GPU. Pengujian tambahan INT8 tidak dimasukkan ke dalam profiling memori GPU karena ruang lingkupnya difokuskan pada *latency fixed-size*.

4.2 Hasil Implementasi Sistem Question Answering

Bagian ini menunjukkan hasil implementasi antarmuka dan keluaran sistem *question answering* multimodal. Penyajian ini diperlukan untuk menunjukkan bahwa pengujian tidak hanya dilakukan pada skrip evaluasi, tetapi juga berangkat dari sistem yang memiliki alur masukan pengguna, pemilihan *pipeline*, proses *retrieval*, penyusunan *prompt*, dan pengembalian jawaban beserta informasi sumber.

4.2.1 Tampilan Antarmuka Sistem

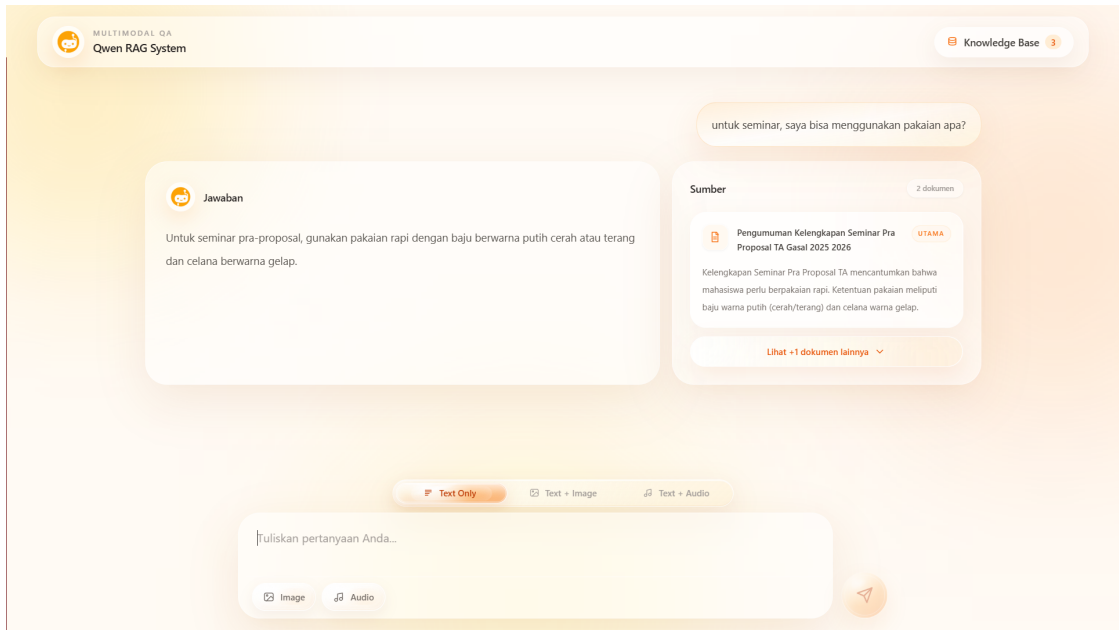
Antarmuka pengguna dirancang sebagai media interaksi utama antara pengguna dan sistem *question answering*. Melalui antarmuka ini, pengguna dapat mengirimkan pertanyaan dalam bentuk teks, melampirkan citra, atau melampirkan audio sesuai kebutuhan pengujian. Antarmuka juga menampilkan mode masukan yang aktif secara otomatis, area jawaban, sumber dokumen hasil *retrieval*, serta fitur pengelolaan basis pengetahuan. Permintaan dari antarmuka diteruskan ke *backend* FastAPI untuk diproses sesuai modalitas masukan yang digunakan.



Gambar 4.1: Tampilan antarmuka utama sistem *question answering*.

4.2.2 Contoh Respons Text-Only RAG

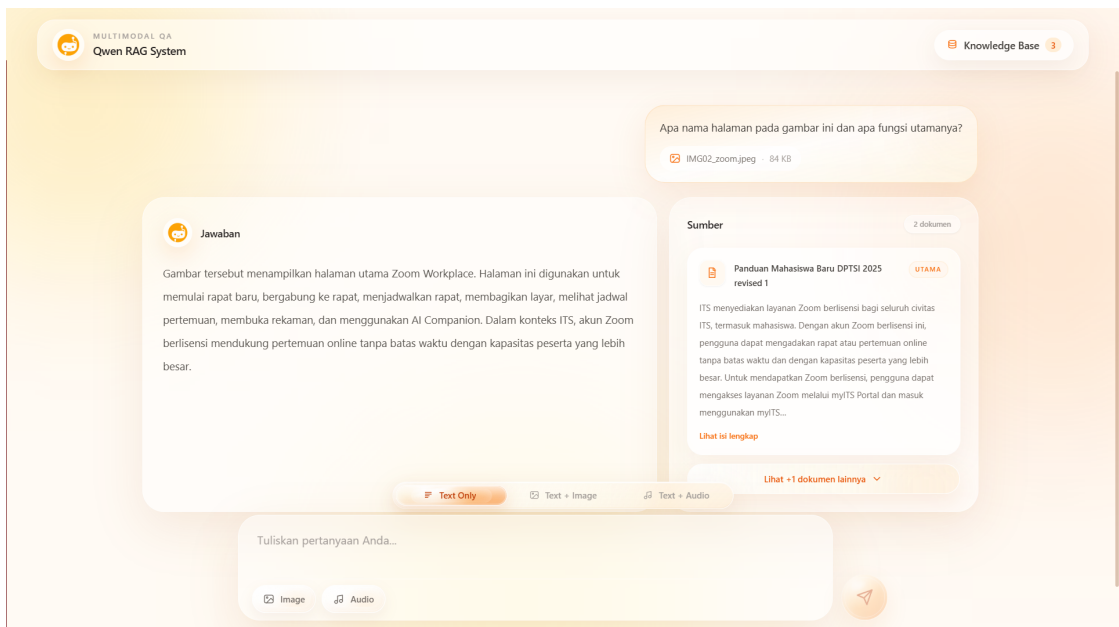
Contoh respons *text-only RAG* menunjukkan alur penjawaban ketika pengguna hanya mengirimkan pertanyaan teks. Pada mode ini, sistem menerima pertanyaan pengguna, melakukan *retrieval* terhadap basis pengetahuan dokumen, menyusun konteks yang relevan, kemudian menghasilkan jawaban akhir menggunakan model Qwen2.5. Tampilan respons memperlihatkan bahwa jawaban sistem disajikan pada panel utama, sedangkan dokumen sumber yang digunakan ditampilkan pada panel terpisah.



Gambar 4.2: Contoh respons sistem pada mode *text-only* RAG.

4.2.3 Contoh Respons Text-Image RAG

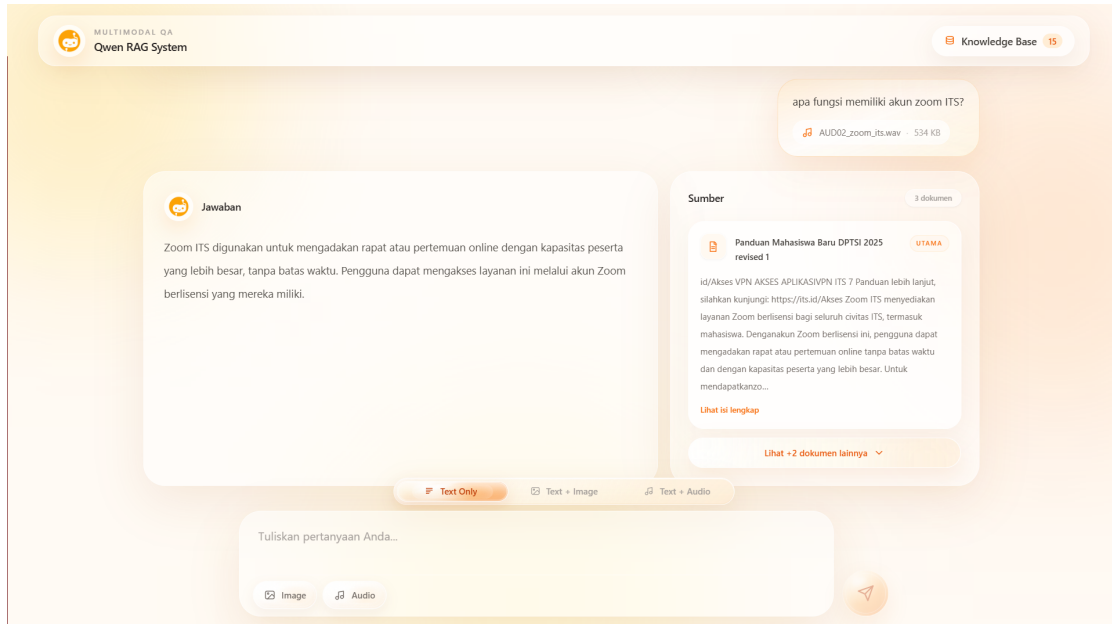
Contoh respons *text-image* RAG menunjukkan alur penjawaban ketika pengguna mengirimkan pertanyaan beserta citra. Pada mode ini, informasi visual diekstraksi terlebih dahulu dari citra masukan, kemudian digabungkan dengan pertanyaan pengguna untuk memperoleh konteks dokumen yang relevan. Jawaban akhir dibentuk berdasarkan gabungan informasi visual dan hasil *retrieval*, sehingga sistem dapat memberikan respons yang sesuai dengan isi citra dan konteks dokumen.



Gambar 4.3: Contoh respons sistem pada mode *text-image* RAG.

4.2.4 Contoh Respons Text-Audio RAG

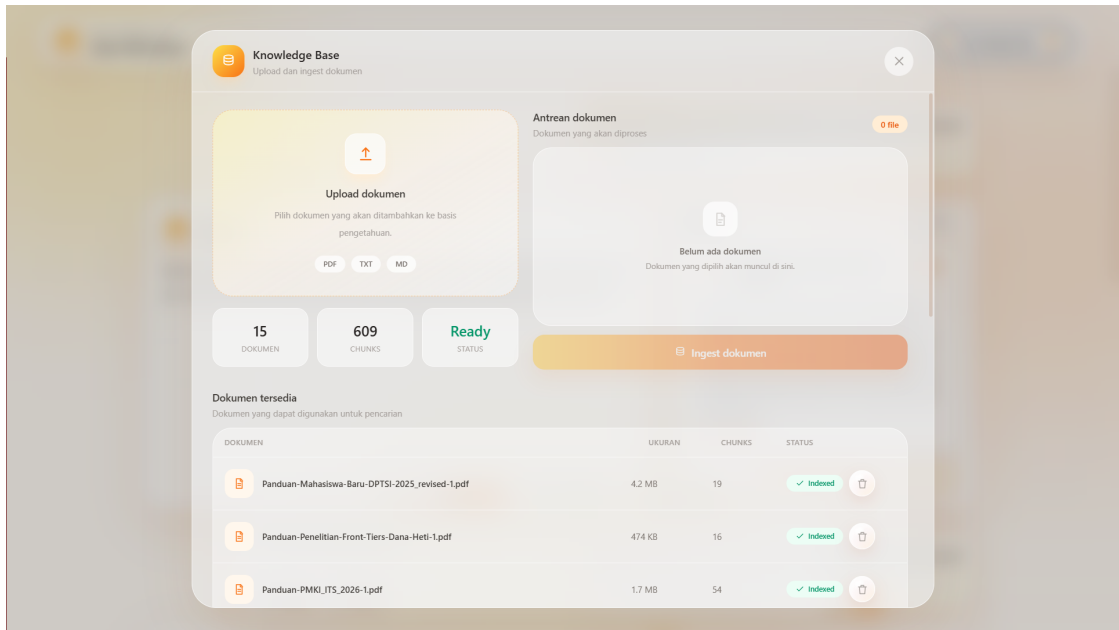
Contoh respons *text-audio RAG* menunjukkan alur penjawaban ketika pengguna mengirimkan pertanyaan dan audio. Audio diproses menjadi konteks tekstual terlebih dahulu, kemudian informasi tersebut digunakan bersama pertanyaan pengguna untuk melakukan *retrieval* terhadap dokumen yang relevan. Hasil *retrieval* selanjutnya digunakan untuk membentuk jawaban akhir yang ditampilkan bersama sumber dokumen pendukung.



Gambar 4.4: Contoh respons sistem pada mode *text-audio RAG*.

4.2.5 Fitur Manajemen Dokumen

Selain fungsi penjawaban, sistem menyediakan fitur manajemen dokumen sebagai bagian dari pengelolaan basis pengetahuan. Fitur ini memungkinkan pengguna menampilkan dokumen yang telah tersedia, menambahkan dokumen baru, menghapus dokumen yang tidak lagi relevan, serta membangun ulang indeks utama sistem. Dengan adanya fitur ini, basis pengetahuan dapat diperbarui secara lebih fleksibel sesuai kebutuhan penggunaan sistem.



Gambar 4.5: Tampilan fitur manajemen dokumen pada sistem.

4.3 Konfigurasi dan Ruang Lingkup Hasil Pengujian

Bagian ini merangkum cakupan hasil pengujian yang dianalisis pada Bab IV. Detail rancangan pengujian, data uji, dan metode evaluasi telah dijelaskan pada Bab III. Oleh karena itu, bagian ini hanya menyajikan ringkasan agar pembaca dapat mengikuti struktur hasil yang disajikan.

Tabel 4.1: Ruang lingkup hasil pengujian pada Bab IV

Kelompok Pengujian	Pengu-	Skenario	Konfigurasi	Metrik Utama
Non-RAG	multi-modal	<i>image-only</i> dan <i>audio-only</i>	FP32, FP16, 4-bit NF4	<i>cosine similarity</i> dan <i>latency</i>
RAG utama	konfigurasi	<i>text-only RAG</i> , <i>text-image RAG</i> , dan <i>text-audio RAG</i> dengan <i>fixed-size</i>	FP32, FP16, 4-bit NF4	BLEU-1, ROUGE-L F1, <i>context precision</i> , <i>context recall</i> , dan <i>latency</i>
RAG pembandingan	konfigurasi	<i>text-only RAG</i> , <i>text-image RAG</i> , dan <i>text-audio RAG</i> dengan <i>recursive</i>	FP32, FP16, 4-bit NF4	BLEU-1, ROUGE-L F1, <i>context precision</i> , <i>context recall</i> , dan <i>latency</i>
Profiling GPU	memori	Lima skenario <i>fixed-size</i> representatif	FP32, FP16, 4-bit NF4	<i>peak allocated memory</i> , <i>peak reserved memory</i> , puncak penggunaan memori GPU, dan <i>latency</i>

Seluruh hasil dianalisis pada dataset pengujian yang sama untuk setiap skenario. Pada bagian yang membandingkan konfigurasi representasi numerik model pada pengujian utama, perbedaan hasil dibaca sebagai pengaruh konfigurasi FP32, FP16, dan 4-bit NF4 terhadap kualitas keluaran,

kualitas *retrieval*, *latency*, atau kebutuhan memori GPU. Pada bagian pengujian tambahan, hasil INT8 WO A16W8 dan FP32 *strict* dibaca sebagai pelengkap analisis *latency* pada konfigurasi *fixed-size*. Pada bagian yang membandingkan konfigurasi *chunking*, perbedaan hasil dibaca sebagai pengaruh gabungan dari strategi pemotongan dokumen, ukuran *chunk*, dan nilai *overlap*.

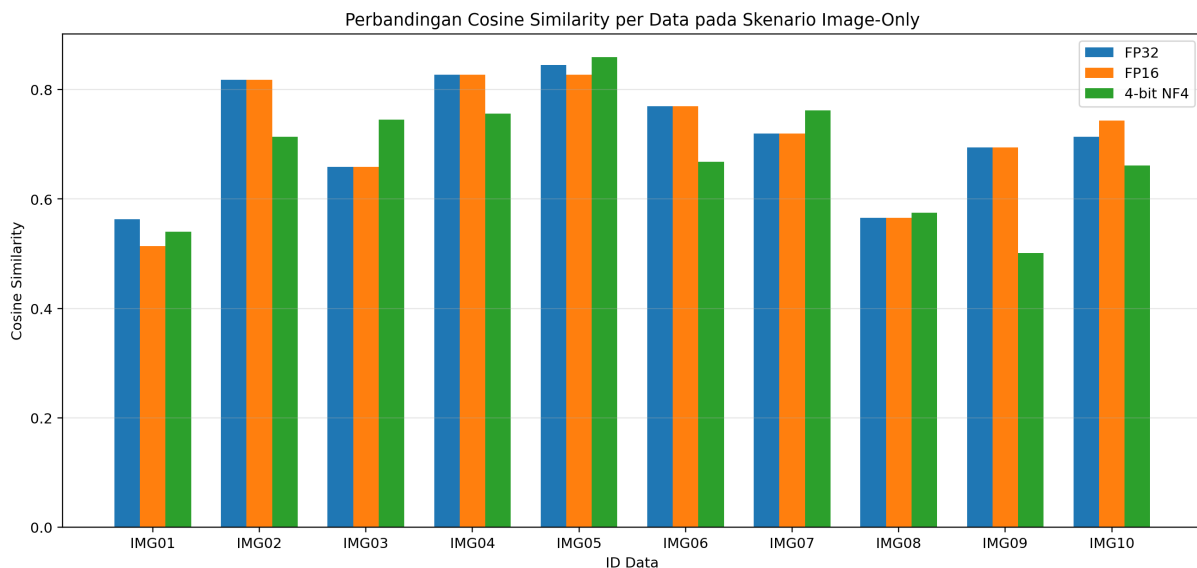
Selain penyajian metrik dan grafik pada bagian utama Bab IV, hasil jawaban sistem pada setiap data uji Q1–Q10 juga disiapkan sebagai data pendukung. Jawaban tersebut mencakup keluaran sistem pada skenario *image-only*, *audio-only*, *text-only RAG*, *text-image RAG*, dan *text-audio RAG*, baik pada konfigurasi *fixed-size* maupun *recursive*. Karena jumlah kombinasi pengujian cukup besar, jawaban lengkap per data uji disajikan pada lampiran agar pembahasan utama Bab IV tetap berfokus pada analisis metrik, perbandingan konfigurasi, dan interpretasi *trade-off*.

4.4 Hasil Pengujian Non-Retrieval Multimodal

Bagian ini membahas hasil pengujian *image-only* dan *audio-only*. Kedua skenario ini tidak menggunakan indeks FAISS dan tidak menggunakan proses *retrieval*. Oleh karena itu, hasilnya digunakan sebagai acuan kemampuan dasar model visual dan audio sebelum informasi media digabungkan ke dalam *pipeline RAG* multimodal.

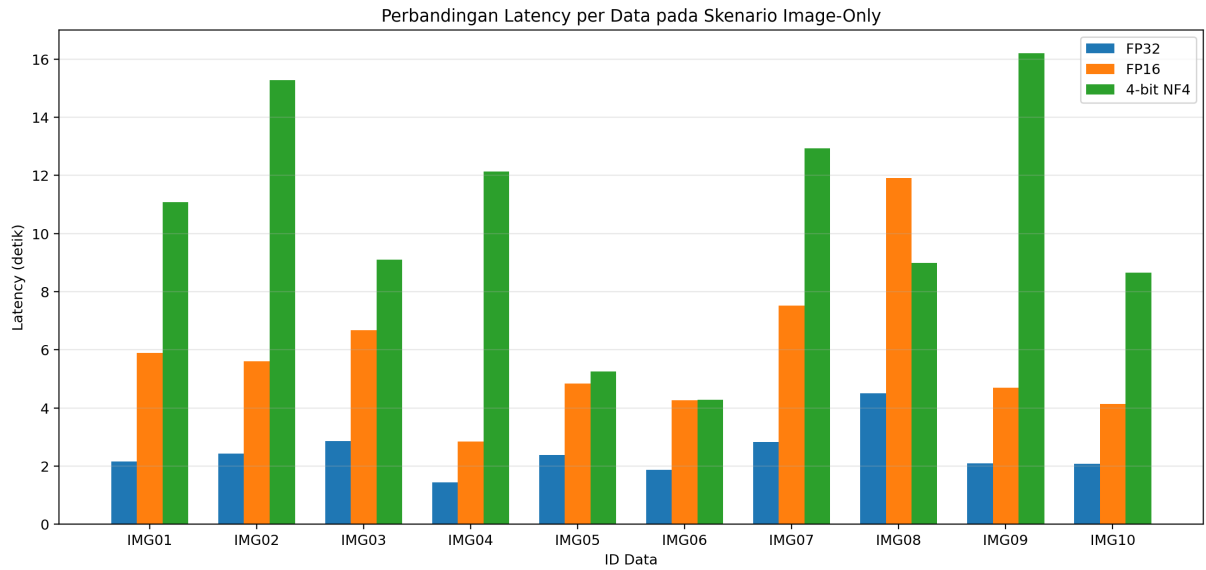
4.4.1 Image-Only

Skenario *image-only* digunakan untuk mengevaluasi kemampuan dasar model visual dalam memahami citra tanpa proses *retrieval* dokumen. Pada skenario ini, model menerima citra dan menghasilkan keluaran tekstual yang dibandingkan dengan *ground truth* menggunakan *cosine similarity*. Selain kualitas semantik, pengujian juga mencatat *latency* untuk melihat waktu pemrosesan pada konfigurasi FP32, FP16, dan 4-bit NF4.



Gambar 4.6: Perbandingan *cosine similarity* per data pada skenario *image-only*.

Gambar 4.6 digunakan untuk melihat kedekatan semantik keluaran model terhadap referensi pada setiap data uji. Nilai yang lebih tinggi menunjukkan bahwa keluaran model semakin dekat secara makna dengan *ground truth*. Pada skenario ini, seluruh konfigurasi berhasil memproses 10 citra yang sama sehingga perbandingan dapat dilakukan pada data uji yang lengkap.

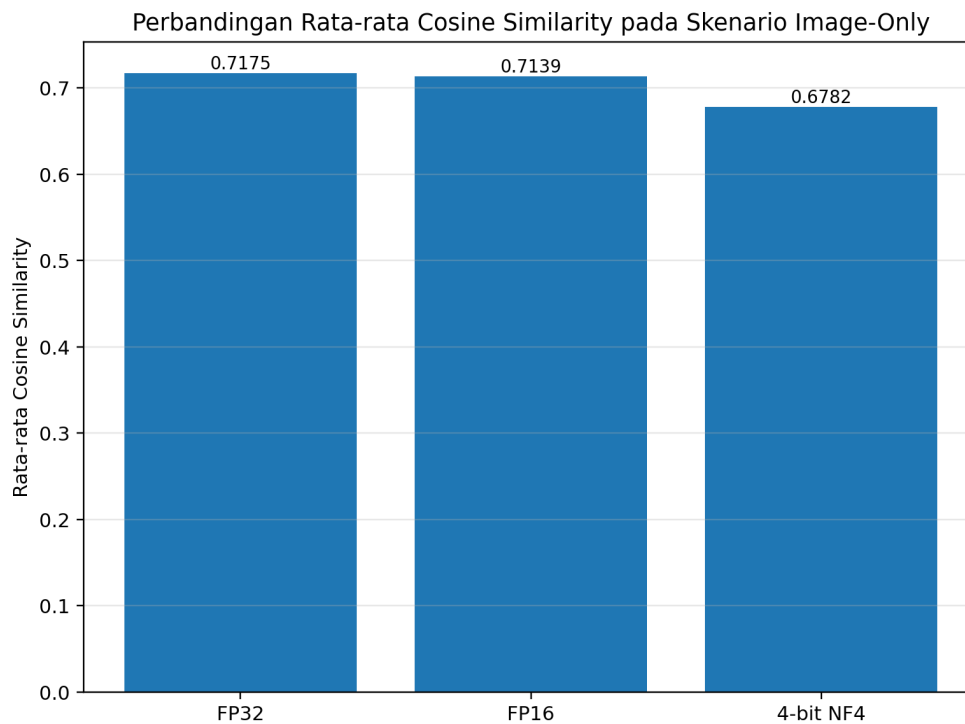


Gambar 4.7: Perbandingan *latency* per data pada skenario *image-only*.

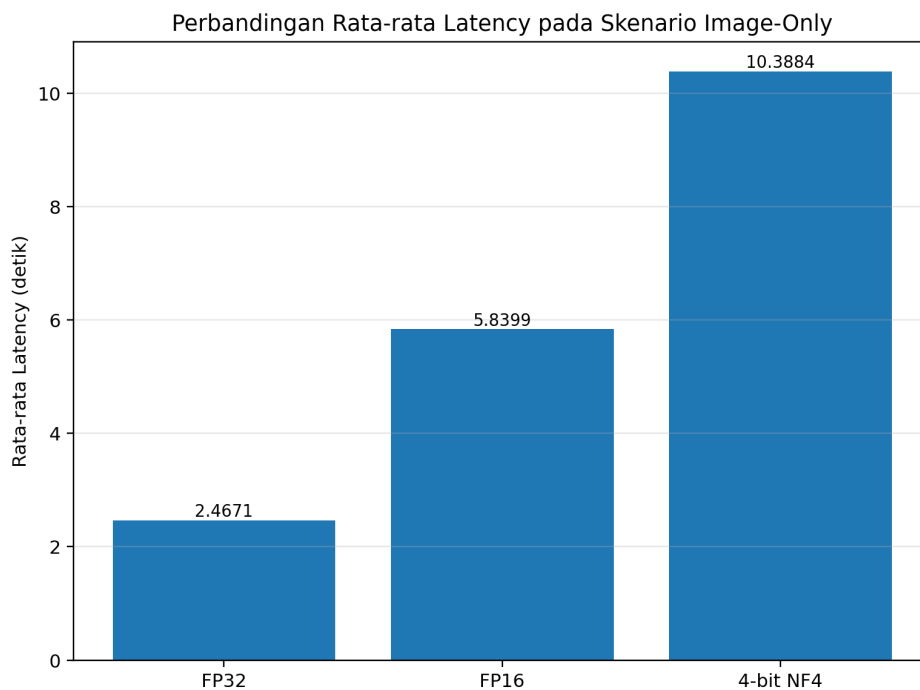
Gambar 4.7 digunakan untuk melihat waktu pemrosesan pada setiap data uji. Pada skenario non-RAG, *latency* terutama dipengaruhi oleh pemrosesan citra dan generasi keluaran oleh model visual. Perbandingan ini digunakan untuk melihat apakah konfigurasi representasi numerik dengan jumlah bit lebih rendah selalu menghasilkan waktu respons yang lebih rendah.

Tabel 4.2: Rata-rata hasil pengujian *image-only*

Konfigurasi	Data Valid	Cosine Similarity	Latency (s)
FP32	10/10	0.7175	2.4671
FP16	10/10	0.7139	5.8399
4-bit NF4	10/10	0.6782	10.3884



Gambar 4.8: Perbandingan rata-rata *cosine similarity* pada skenario *image-only*.



Gambar 4.9: Perbandingan rata-rata *latency* pada skenario *image-only*.

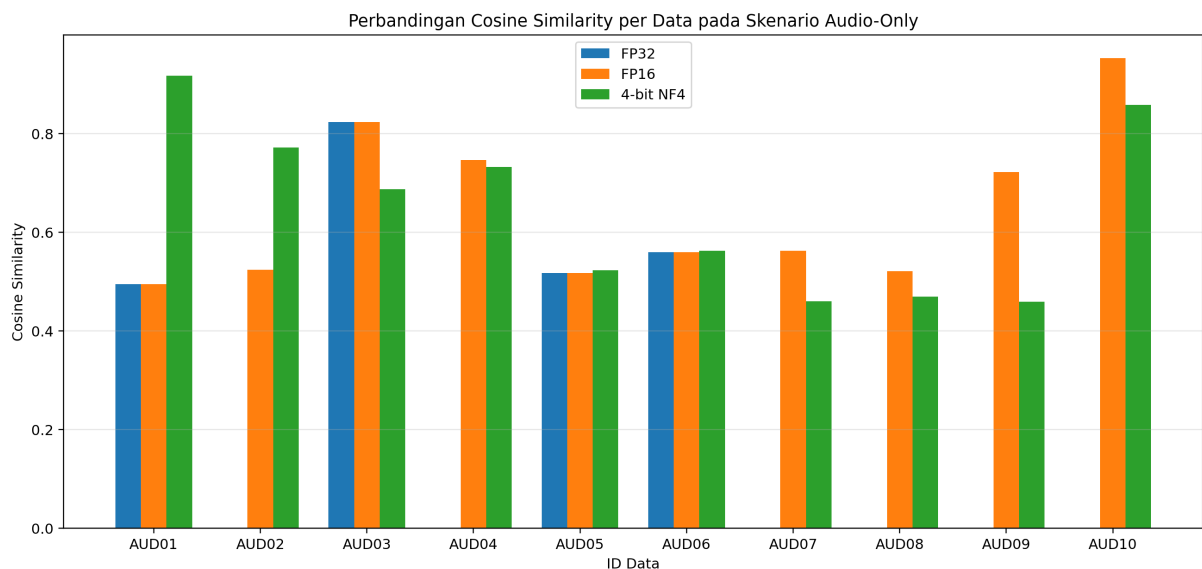
Berdasarkan Tabel 4.2, konfigurasi FP32 memperoleh rata-rata *cosine similarity* tertinggi, yaitu 0.7175. Nilai tersebut sedikit lebih tinggi dibandingkan FP16 sebesar 0.7139 dan 4-bit NF4 sebesar 0.6782. Hasil ini menunjukkan bahwa pada skenario *image-only*, konfigurasi FP32 dan FP16 menghasilkan kualitas semantik yang relatif berdekatan, sedangkan 4-bit NF4 menghasilkan nilai yang lebih rendah.

Dari sisi *latency*, FP32 memperoleh waktu rata-rata paling rendah, yaitu 2.4671 detik. FP16 membutuhkan waktu rata-rata 5.8399 detik, sedangkan 4-bit NF4 membutuhkan waktu rata-rata 10.3884 detik. Hasil ini menunjukkan bahwa pada pengujian citra, konfigurasi 4-bit NF4 tidak menghasilkan waktu respons paling rendah. Dengan demikian, pada skenario *image-only*, konfigurasi FP32 menjadi yang paling unggul berdasarkan kombinasi rata-rata *cosine similarity* dan *latency*.

Analisis pada subbagian ini digunakan sebagai acuan untuk memahami kemampuan dasar model terhadap modalitas citra. Hasilnya akan dibandingkan secara konseptual dengan skenario RAG multimodal untuk melihat apakah keterbatasan jawaban akhir lebih banyak berasal dari pemahaman media, proses *retrieval*, atau tahap generasi jawaban.

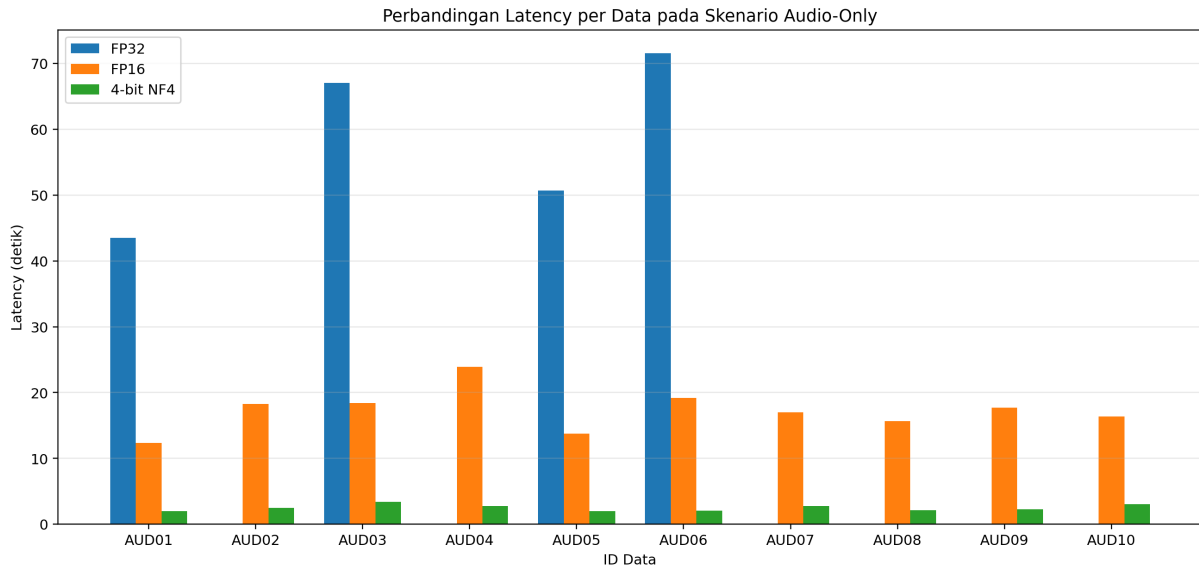
4.4.2 Audio-Only

Skenario *audio-only* digunakan untuk mengevaluasi kemampuan dasar model audio dalam memahami masukan audio tanpa proses *retrieval* dokumen. Pada skenario ini, model menerima audio dan menghasilkan keluaran tekstual yang dibandingkan dengan *ground truth* menggunakan *cosine similarity*. Selain kualitas semantik, pengujian juga mencatat *latency* untuk melihat waktu pemrosesan pada konfigurasi FP32, FP16, dan 4-bit NF4.



Gambar 4.10: Perbandingan *cosine similarity* per data pada skenario *audio-only*.

Gambar 4.10 digunakan untuk melihat kedekatan semantik keluaran model terhadap referensi pada setiap data uji. Nilai yang lebih tinggi menunjukkan bahwa keluaran model semakin dekat secara makna dengan *ground truth*. Pada konfigurasi FP32, sebagian data tidak memiliki nilai valid karena proses inferensi mengalami *OutOfMemoryError*. Oleh karena itu, batang FP32 hanya muncul pada data yang berhasil diproses.

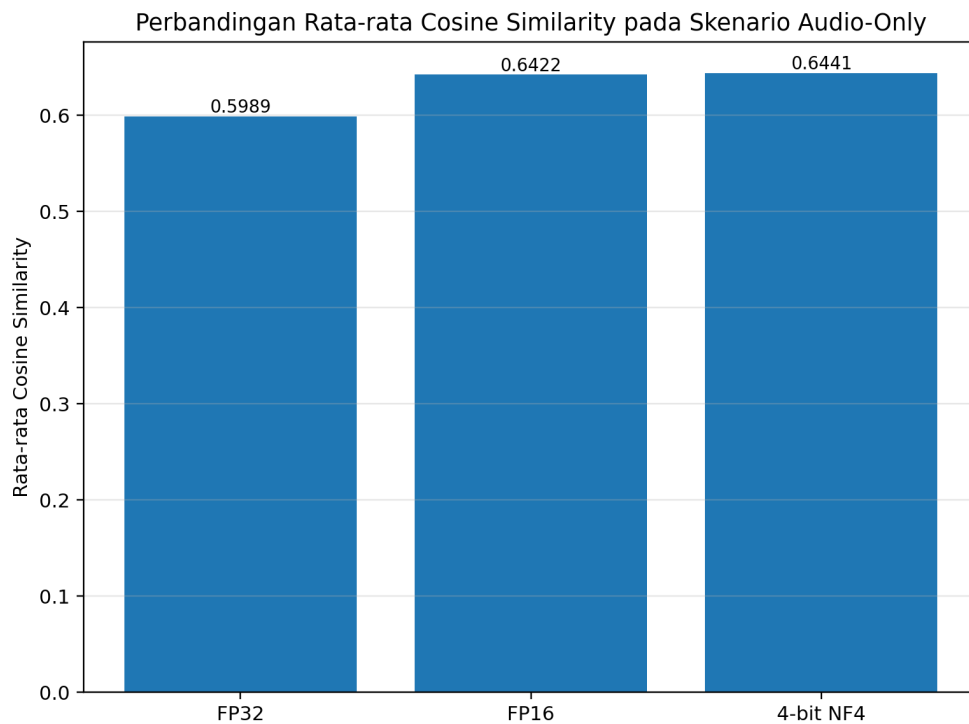


Gambar 4.11: Perbandingan *latency* per data pada skenario *audio-only*.

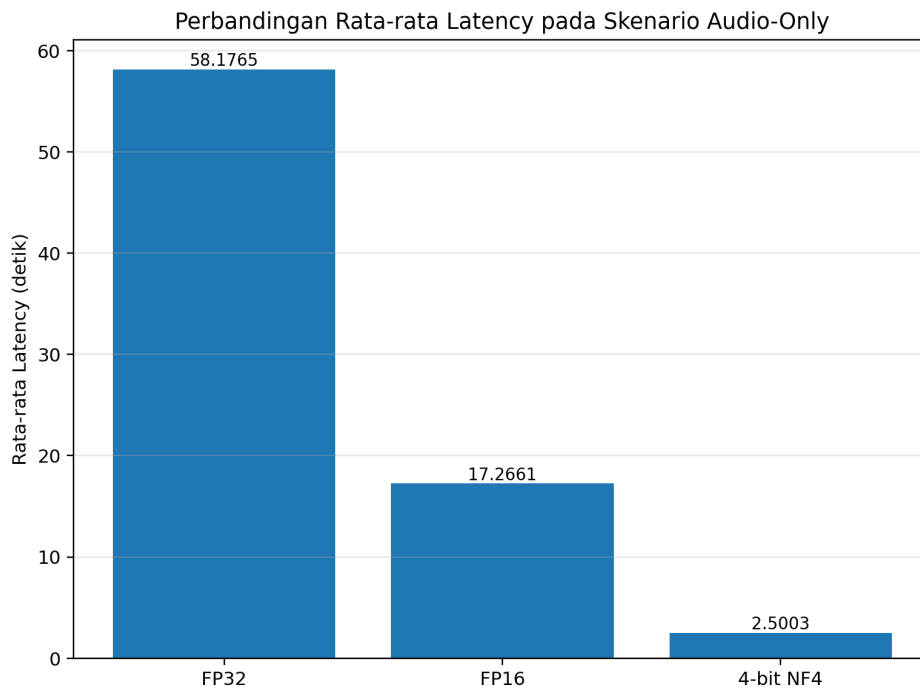
Gambar 4.11 digunakan untuk melihat waktu pemrosesan pada setiap data uji. Pada skenario non-RAG, *latency* terutama dipengaruhi oleh pemrosesan audio dan generasi keluaran oleh model audio. Pada konfigurasi FP32, nilai *latency* hanya tersedia untuk data yang berhasil diproses, sedangkan data yang gagal tidak dimasukkan ke dalam perhitungan rata-rata.

Tabel 4.3: Rata-rata hasil pengujian *audio-only*

Konfigurasi	Data Valid	Cosine Similarity	Latency (s)
FP32	4/10	0.5989	58.1765
FP16	10/10	0.6422	17.2661
4-bit NF4	10/10	0.6441	2.5003



Gambar 4.12: Perbandingan rata-rata *cosine similarity* pada skenario *audio-only*.



Gambar 4.13: Perbandingan rata-rata *latency* pada skenario *audio-only*.

Berdasarkan Tabel 4.3, konfigurasi FP32 hanya berhasil memproses 4 dari 10 data audio. Enam data lainnya mengalami *OutOfMemoryError*, sehingga rata-rata FP32 pada tabel dihitung hanya dari data yang berhasil. Kondisi ini menunjukkan bahwa penggunaan FP32 pada model audio memiliki kebutuhan memori yang lebih besar dan tidak stabil pada perangkat pengujian dengan VRAM 16 GB.

Dari sisi kualitas semantik, FP16 memperoleh rata-rata *cosine similarity* sebesar 0.6422, sedangkan 4-bit NF4 memperoleh nilai sebesar 0.6441. Kedua nilai tersebut sangat berdekatan, sehingga kualitas pemahaman audio pada FP16 dan 4-bit NF4 dapat dianggap relatif sebanding pada skenario ini. FP32 memperoleh rata-rata *cosine similarity* sebesar 0.5989 pada data yang berhasil diproses, tetapi nilai tersebut tidak dapat dibandingkan secara setara dengan FP16 dan 4-bit NF4 karena jumlah data berhasilnya lebih sedikit.

Dari sisi *latency*, 4-bit NF4 memperoleh waktu rata-rata paling rendah, yaitu 2.5003 detik. FP16 membutuhkan waktu rata-rata 17.2661 detik, sedangkan FP32 membutuhkan waktu rata-rata 58.1765 detik pada data yang berhasil. Hasil ini menunjukkan bahwa pada skenario *audio-only*, konfigurasi 4-bit NF4 memberikan keuntungan paling jelas dari sisi waktu respons dan keberhasilan pemrosesan. Dengan demikian, konfigurasi 4-bit NF4 lebih sesuai untuk pemrosesan audio pada lingkungan GPU terbatas dibandingkan FP32 dan FP16.

Analisis pada subbagian ini digunakan sebagai acuan untuk memahami kemampuan dasar model terhadap modalitas audio. Hasilnya akan dibandingkan secara konseptual dengan skenario RAG multimodal untuk melihat apakah keterbatasan jawaban akhir lebih banyak berasal dari pemahaman media, proses *retrieval*, atau tahap generasi jawaban.

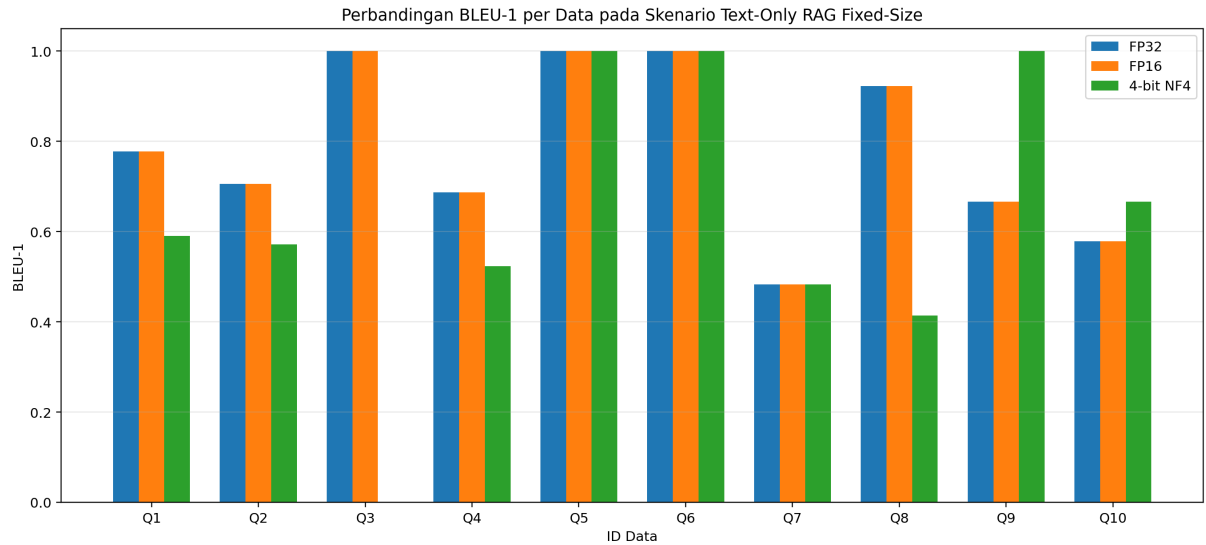
4.5 Hasil Pengujian RAG dengan Konfigurasi Chunking Utama Fixed-Size

Bagian ini membahas hasil pengujian pada skenario berbasis RAG yang menggunakan konfigurasi *structure-aware fixed-size chunking with overlap*. Konfigurasi ini merupakan konfigurasi *chunking* utama dalam penelitian dan menjadi dasar dalam pengujian kualitas jawaban, kualitas *retrieval*, *latency*, serta profiling kebutuhan memori GPU. Skenario yang dibahas pada bagian ini terdiri atas *text-only RAG fixed-size*, *text-image RAG fixed-size*, dan *text-audio RAG fixed-size*.

Pada setiap skenario, pengujian dilakukan terhadap tiga konfigurasi representasi numerik model, yaitu FP32, FP16, dan 4-bit NF4. Metrik yang digunakan meliputi BLEU-1, ROUGE-L F1, *context precision*, *context recall*, dan *latency*. BLEU-1 dan ROUGE-L F1 digunakan untuk mengevaluasi kualitas jawaban akhir terhadap *ground truth*, sedangkan *context precision* dan *context recall* digunakan untuk mengevaluasi kualitas konteks hasil *retrieval*. Metrik *latency* digunakan untuk melihat waktu respons sistem pada setiap konfigurasi.

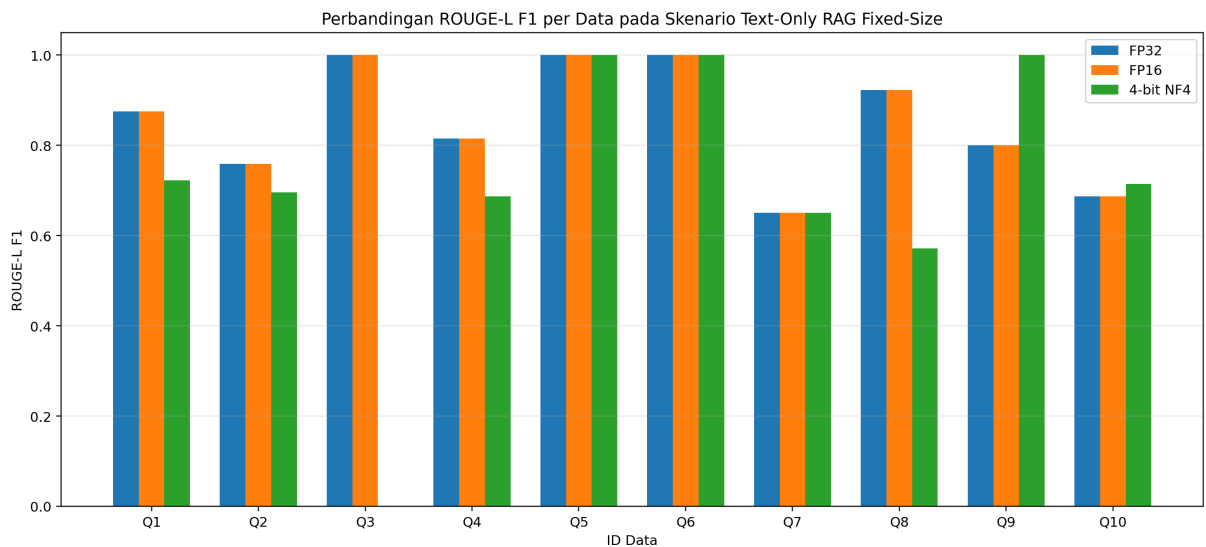
4.5.1 Text-Only RAG Fixed-Size

Skenario *text-only RAG fixed-size* digunakan untuk mengevaluasi kemampuan sistem dalam menghasilkan jawaban berdasarkan pertanyaan teks dan konteks dokumen hasil *retrieval*. Pada skenario ini, sistem tidak menerima masukan citra atau audio. Dengan demikian, kualitas jawaban akhir terutama dipengaruhi oleh kualitas pertanyaan, relevansi konteks hasil *retrieval*, dan kemampuan model teks dalam menghasilkan jawaban berdasarkan konteks tersebut.



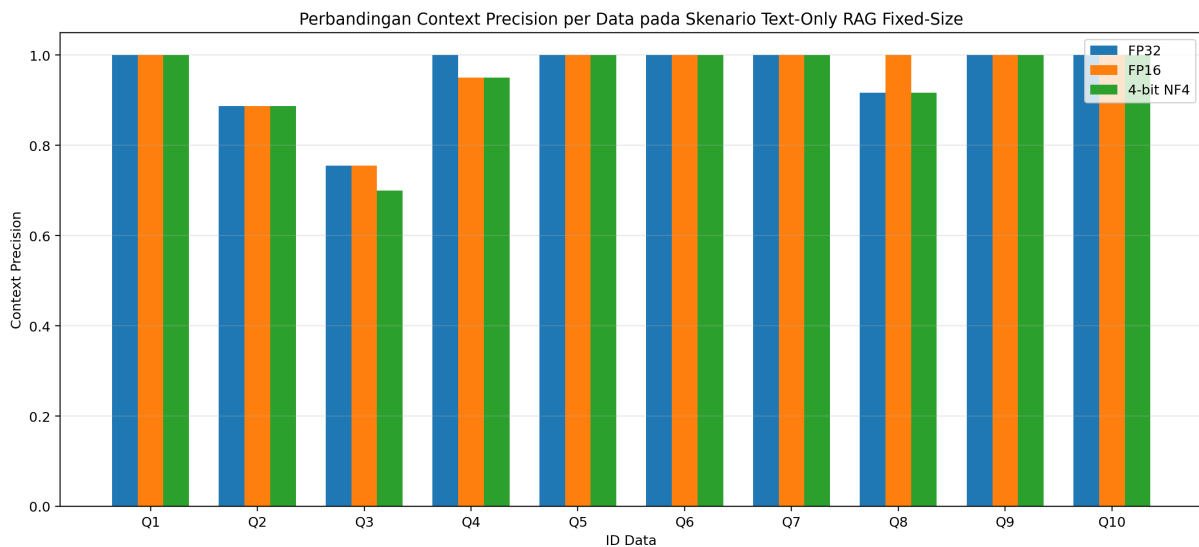
Gambar 4.14: Perbandingan BLEU-1 per data pada skenario *text-only RAG fixed-size*.

Gambar 4.14 menunjukkan nilai BLEU-1 pada setiap data uji. Secara umum, FP32 dan FP16 memiliki pola nilai yang sama pada seluruh data uji, sedangkan 4-bit NF4 menunjukkan penurunan pada beberapa data. Penurunan paling terlihat terjadi pada data yang membutuhkan ketepatan formulasi jawaban, karena BLEU-1 sensitif terhadap kecocokan kata antara jawaban sistem dan jawaban referensi.



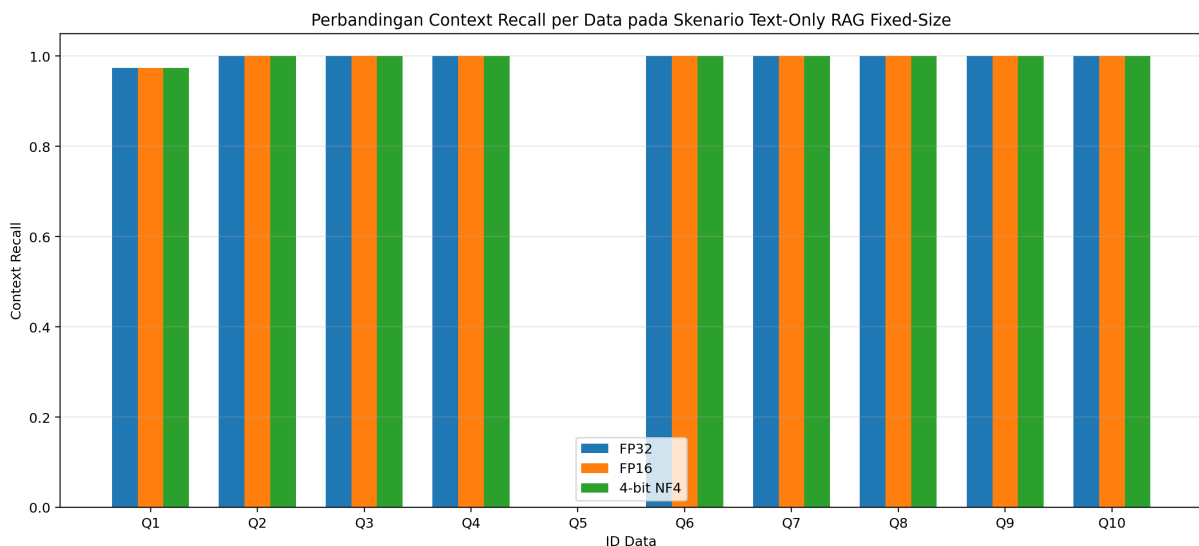
Gambar 4.15: Perbandingan ROUGE-L F1 per data pada skenario *text-only RAG fixed-size*.

Gambar 4.15 menunjukkan perbandingan ROUGE-L F1 pada setiap data uji. Pola ROUGE-L F1 sejalan dengan BLEU-1, yaitu FP32 dan FP16 menghasilkan nilai yang identik, sedangkan 4-bit NF4 memiliki nilai lebih rendah pada beberapa data. Hal ini menunjukkan bahwa konfigurasi 4-bit NF4 tidak hanya memengaruhi kecocokan kata, tetapi juga dapat memengaruhi kesesuaian urutan informasi dalam jawaban akhir.



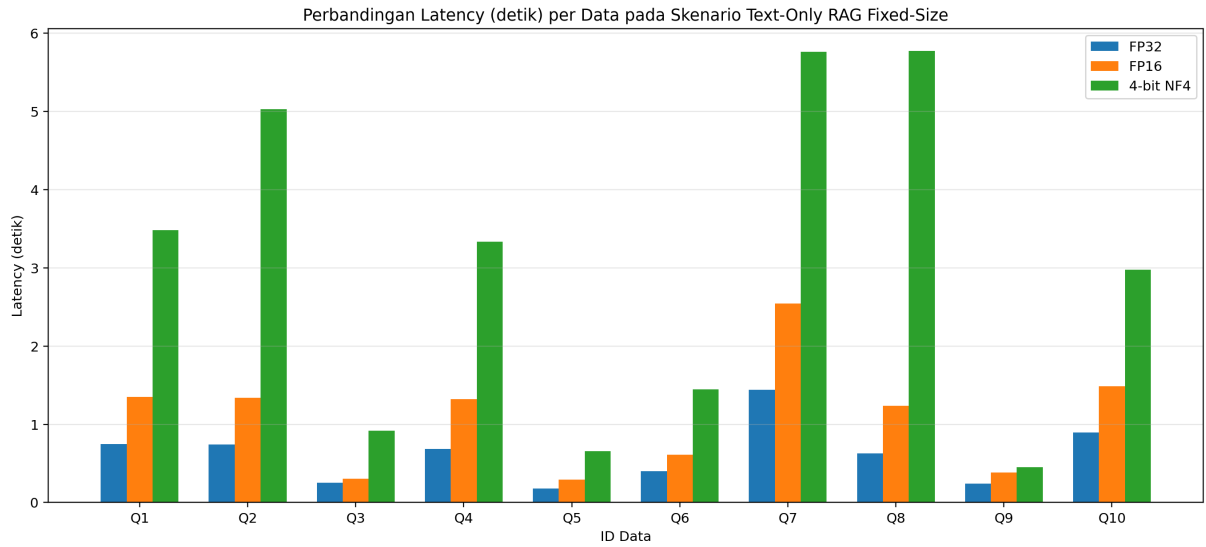
Gambar 4.16: Perbandingan *context precision* per data pada skenario *text-only RAG fixed-size*.

Gambar 4.16 menunjukkan nilai *context precision* pada setiap data uji. Nilai *context precision* pada ketiga konfigurasi berada pada rentang tinggi. Hal ini menunjukkan bahwa konteks yang diambil oleh sistem pada skenario *text-only RAG fixed-size* umumnya relevan terhadap pertanyaan. Perbedaan kecil antar konfigurasi dapat muncul karena proses evaluasi RAGAS dilakukan terhadap jawaban akhir dan konteks yang digunakan pada masing-masing hasil pengujian.



Gambar 4.17: Perbandingan *context recall* per data pada skenario *text-only RAG fixed-size*.

Gambar 4.17 menunjukkan nilai *context recall* pada setiap data uji. Nilai *context recall* relatif stabil pada ketiga konfigurasi. Hal ini menunjukkan bahwa informasi penting yang dibutuhkan untuk menjawab pertanyaan umumnya berhasil tercakup dalam konteks hasil *retrieval*. Dengan demikian, konfigurasi *fixed-size* mampu menyediakan cakupan konteks yang cukup baik untuk pertanyaan berbasis teks.

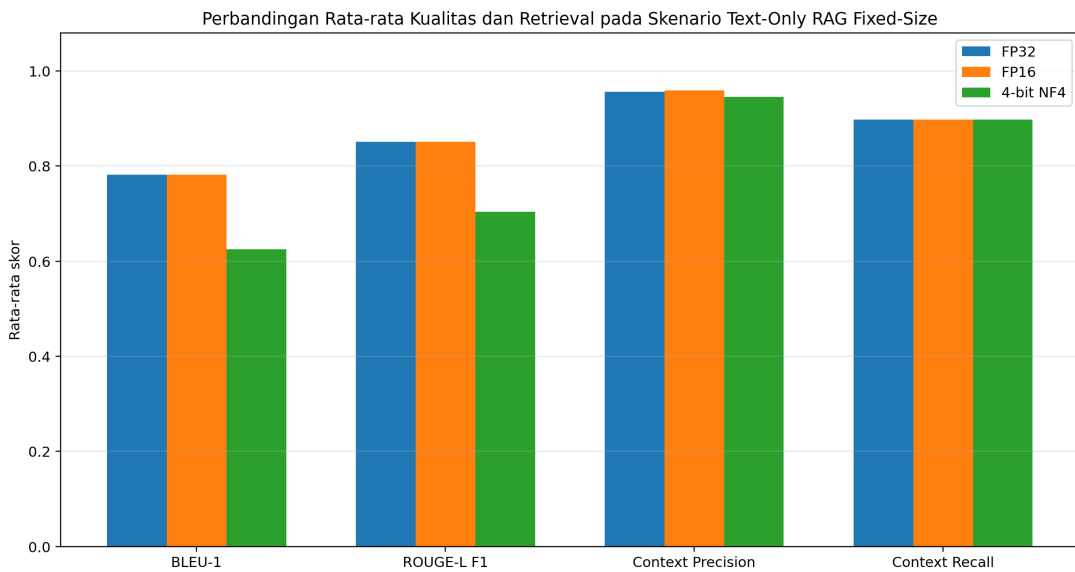


Gambar 4.18: Perbandingan *latency* per data pada skenario *text-only RAG fixed-size*.

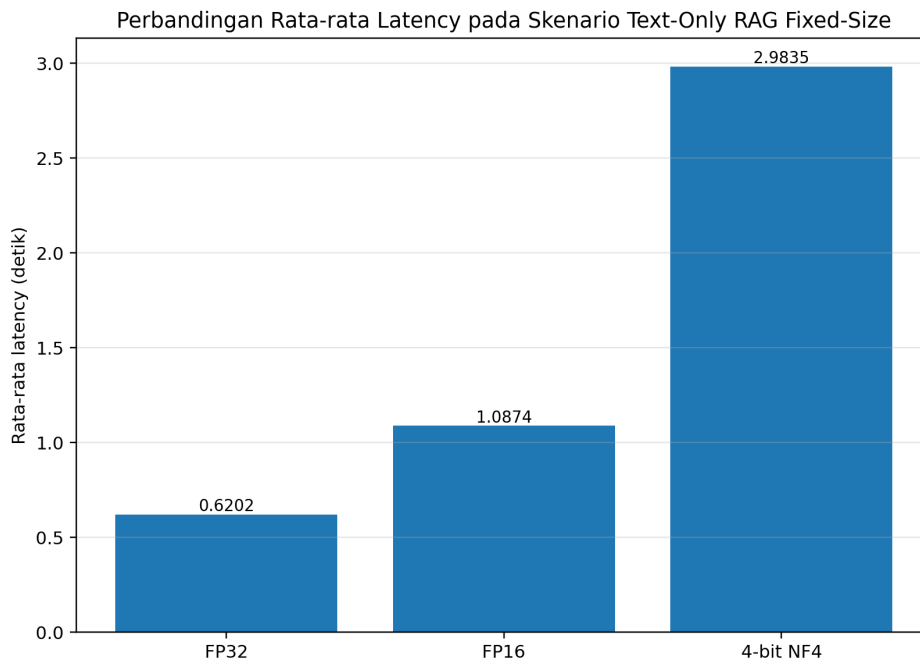
Gambar 4.18 menunjukkan *latency* pada setiap data uji. FP32 menghasilkan waktu respons paling rendah pada skenario ini, diikuti oleh FP16 dan 4-bit NF4. Hasil ini menunjukkan bahwa konfigurasi dengan jumlah bit lebih rendah tidak selalu menghasilkan waktu respons yang lebih cepat. Pada skenario teks, 4-bit NF4 justru memiliki *latency* lebih tinggi, yang dapat dipengaruhi oleh overhead kuantisasi dan dekuantisasi saat inferensi.

Tabel 4.4: Rata-rata hasil pengujian *text-only RAG fixed-size*

Konfigurasi	Data Valid	BLEU-1	ROUGE-L F1	CP	CR	Latency (s)
FP32	10/10	0.7823	0.8510	0.9560	0.8974	0.6202
FP16	10/10	0.7823	0.8510	0.9593	0.8974	1.0874
4-bit NF4	10/10	0.6249	0.7042	0.9454	0.8974	2.9835



Gambar 4.19: Perbandingan rata-rata BLEU-1, ROUGE-L F1, *context precision*, dan *context recall* pada skenario *text-only RAG fixed-size*.



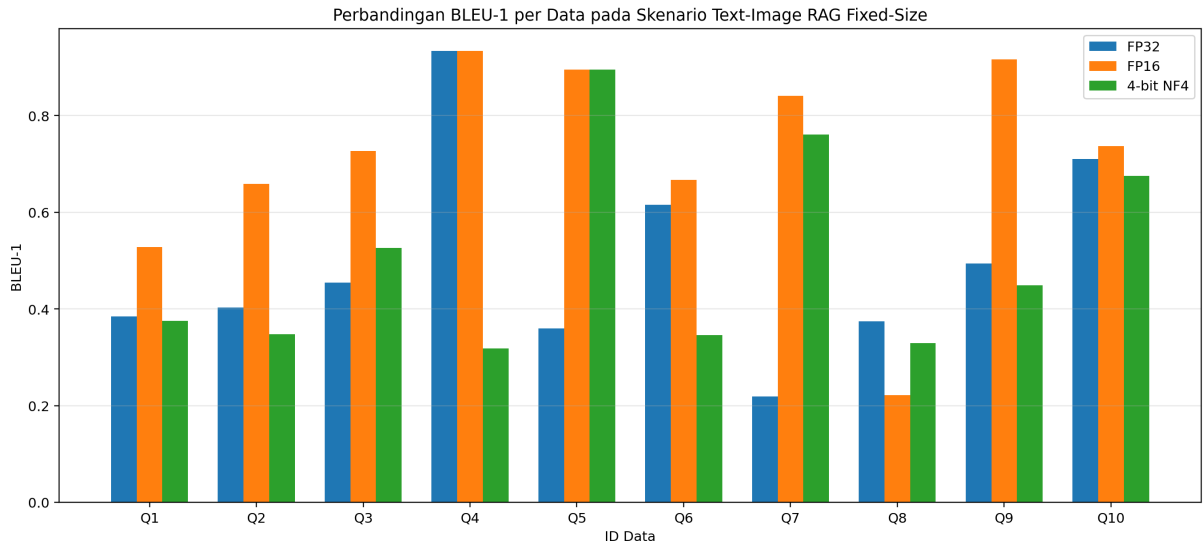
Gambar 4.20: Perbandingan rata-rata *latency* pada skenario *text-only RAG fixed-size*.

Berdasarkan Tabel 4.4, FP32 dan FP16 menghasilkan nilai BLEU-1 dan ROUGE-L F1 yang sama, yaitu BLEU-1 sebesar 0.7823 dan ROUGE-L F1 sebesar 0.8510. Nilai tersebut lebih tinggi dibandingkan 4-bit NF4 yang memperoleh BLEU-1 sebesar 0.6249 dan ROUGE-L F1 sebesar 0.7042. Dari sisi kualitas *retrieval*, ketiga konfigurasi memiliki nilai yang relatif tinggi. FP16 memperoleh *context precision* tertinggi sebesar 0.9593, sedangkan FP32 memperoleh 0.9560 dan 4-bit NF4 memperoleh 0.9454. Nilai *context recall* pada ketiga konfigurasi sama, yaitu 0.8974.

Dari sisi *latency*, FP32 memperoleh waktu respons rata-rata paling rendah, yaitu 0.6202 detik. FP16 membutuhkan 1.0874 detik, sedangkan 4-bit NF4 membutuhkan 2.9835 detik. Dengan demikian, pada skenario *text-only RAG fixed-size*, FP32 menjadi konfigurasi paling efisien dari sisi waktu, sementara FP32 dan FP16 sama-sama unggul dari sisi kualitas jawaban. Hasil ini menunjukkan bahwa pada skenario teks, 4-bit NF4 tidak memberikan keuntungan kualitas maupun *latency* dibandingkan FP32 dan FP16.

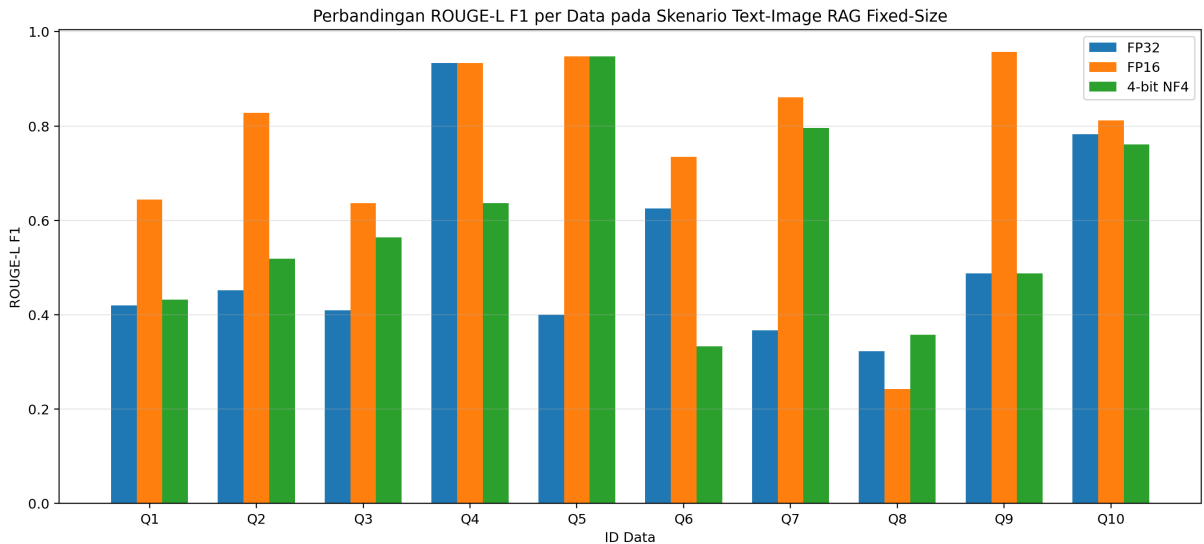
4.5.2 Text-Image RAG Fixed-Size

Skenario *text-image RAG fixed-size* digunakan untuk mengevaluasi kemampuan sistem dalam menghasilkan jawaban berdasarkan pertanyaan teks, konteks visual dari citra, dan konteks dokumen hasil *retrieval*. Pada skenario ini, citra digunakan sebagai konteks tambahan yang membantu sistem memahami informasi visual sebelum jawaban akhir dibentuk dengan dukungan dokumen hasil *retrieval*. Berbeda dari skenario *image-only*, skenario ini tidak hanya menguji pemahaman citra secara mandiri, tetapi juga menguji integrasi antara informasi visual, pertanyaan teks, konteks dokumen, dan proses generasi jawaban akhir.



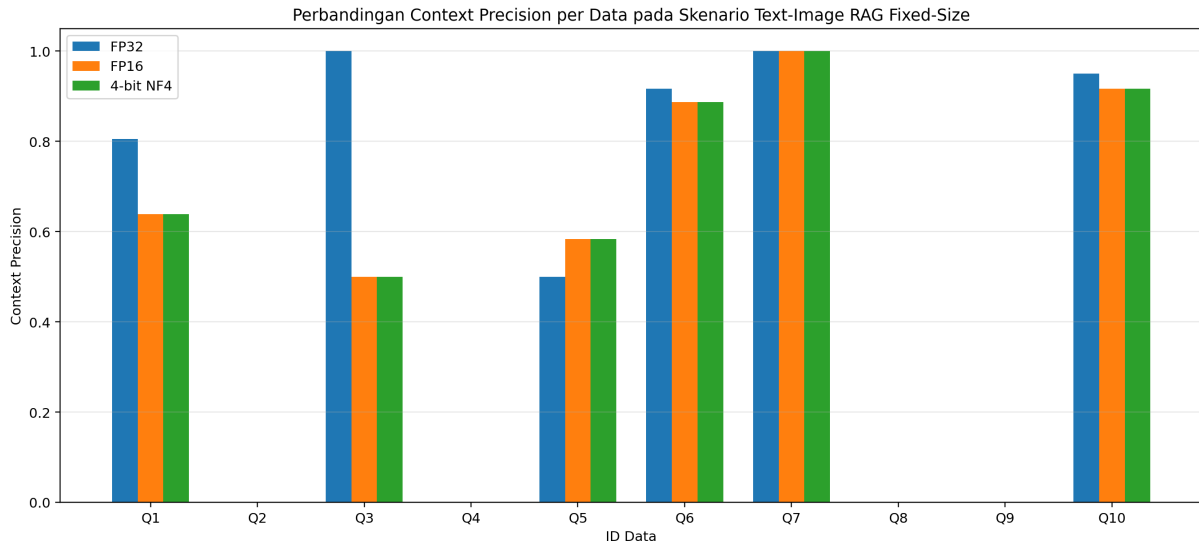
Gambar 4.21: Perbandingan BLEU-1 per data pada skenario *text-image RAG fixed-size*.

Gambar 4.21 menunjukkan nilai BLEU-1 pada skenario *text-image RAG fixed-size*. FP16 menunjukkan nilai yang lebih tinggi pada sebagian besar data uji dibandingkan FP32 dan 4-bit NF4. Hasil ini menunjukkan bahwa berdasarkan metrik kecocokan unigram, jawaban akhir pada konfigurasi FP16 lebih dekat dengan *ground truth*. Namun, hasil ini perlu dibaca sebagai performa pada *pipeline* RAG multimodal secara utuh, bukan sebagai ukuran tunggal kemampuan model dalam memahami citra.



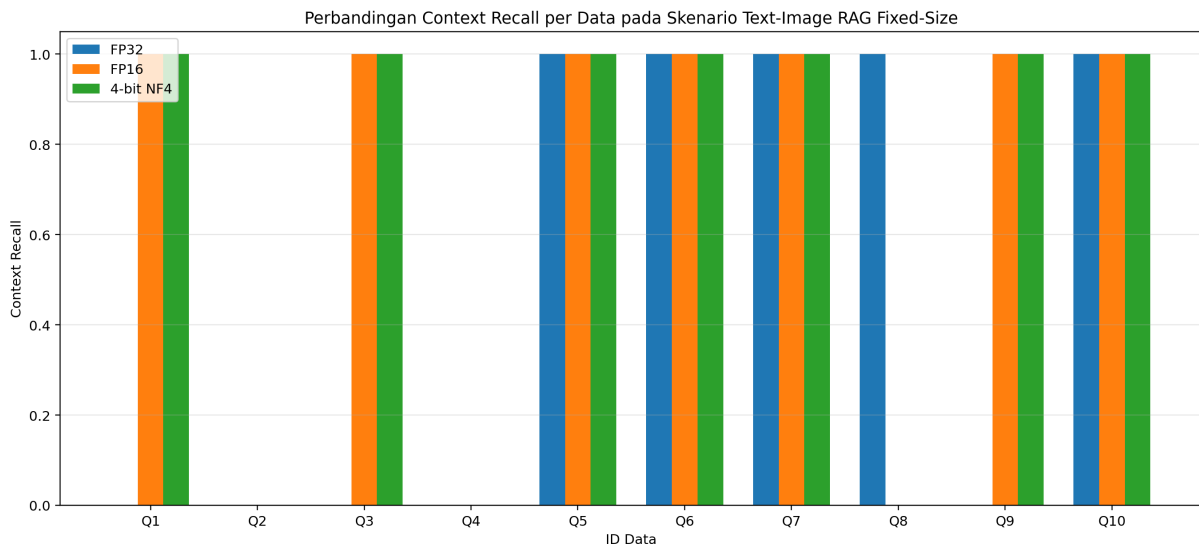
Gambar 4.22: Perbandingan ROUGE-L F1 per data pada skenario *text-image RAG fixed-size*.

Gambar 4.22 menunjukkan perbandingan ROUGE-L F1 pada setiap data uji. FP16 kembali menunjukkan performa paling kuat secara umum. Nilai ROUGE-L F1 yang lebih tinggi menunjukkan bahwa jawaban FP16 lebih mampu mempertahankan urutan informasi yang sesuai dengan referensi, khususnya pada data yang membutuhkan penggabungan informasi visual dan konteks dokumen.



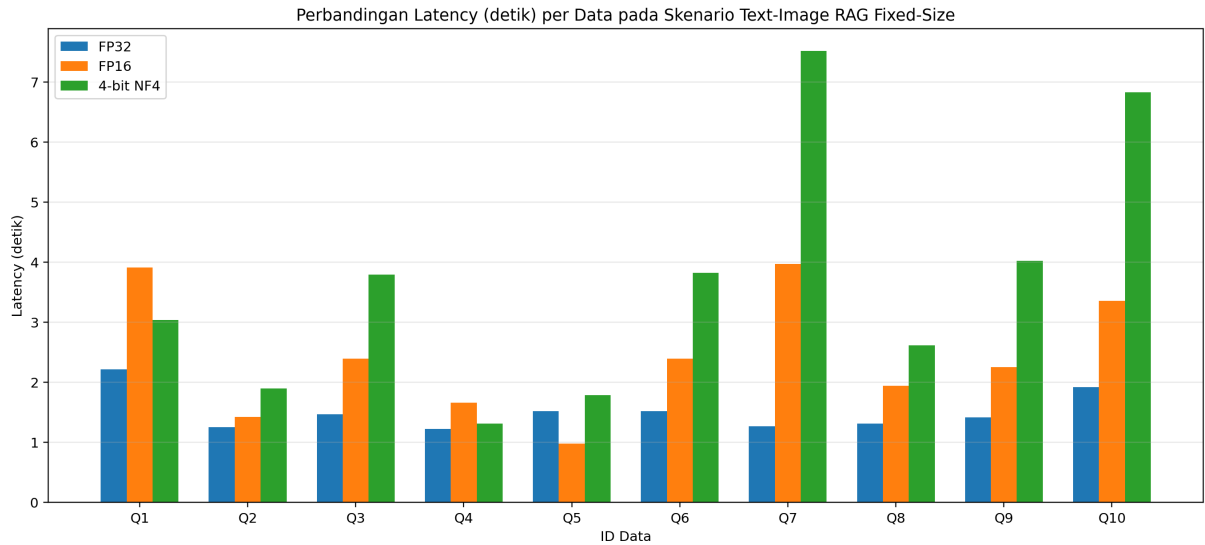
Gambar 4.23: Perbandingan *context precision* per data pada skenario *text-image RAG fixed-size*.

Gambar 4.23 menunjukkan nilai *context precision* pada setiap data uji. Nilai *context precision* pada skenario ini lebih bervariasi dibandingkan *text-only RAG*. Hal ini menunjukkan bahwa pertanyaan yang melibatkan citra tidak selalu mengambil konteks dokumen yang sepenuhnya presisi, terutama ketika informasi visual memiliki kontribusi besar terhadap jawaban akhir.



Gambar 4.24: Perbandingan *context recall* per data pada skenario *text-image RAG fixed-size*.

Gambar 4.24 menunjukkan nilai *context recall* pada setiap data uji. FP16 dan 4-bit NF4 memiliki nilai *context recall* rata-rata lebih tinggi dibandingkan FP32. Hal ini menunjukkan bahwa konteks yang digunakan pada FP16 dan 4-bit NF4 lebih banyak mencakup informasi penting yang dibutuhkan untuk menjawab pertanyaan, meskipun nilai *context precision* tidak selalu tinggi pada seluruh data.

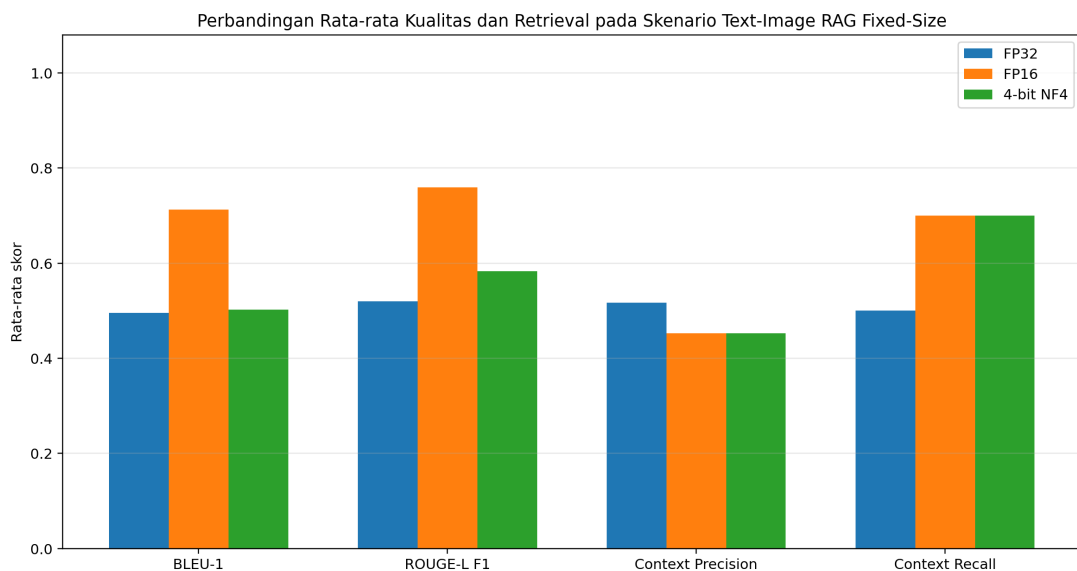


Gambar 4.25: Perbandingan *latency* per data pada skenario *text-image RAG fixed-size*.

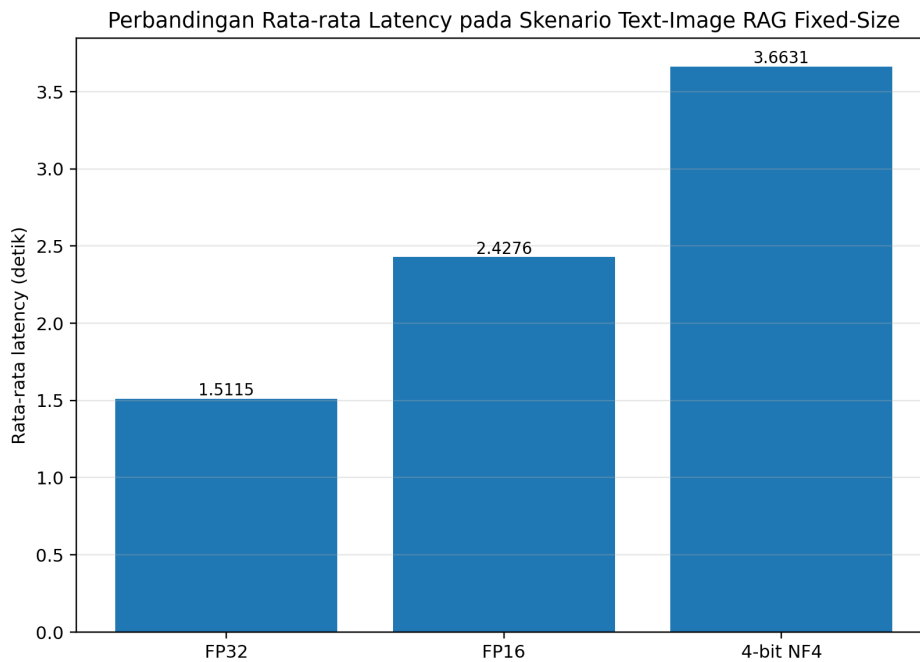
Gambar 4.25 menunjukkan *latency* pada skenario *text-image RAG fixed-size*. FP32 menghasilkan waktu respons rata-rata paling rendah, sedangkan 4-bit NF4 menghasilkan waktu respons paling tinggi. Hal ini menunjukkan bahwa pada skenario multimodal berbasis citra, konfigurasi 4-bit NF4 tidak otomatis mempercepat inferensi.

Tabel 4.5: Rata-rata hasil pengujian *text-image RAG fixed-size*

Konfigurasi	Data Valid	BLEU-1	ROUGE-L F1	CP	CR	Latency (s)
FP32	10/10	0.4951	0.5198	0.5172	0.5000	1.5115
FP16	10/10	0.7126	0.7594	0.4526	0.7000	2.4276
4-bit NF4	10/10	0.5024	0.5833	0.4526	0.7000	3.6631



Gambar 4.26: Perbandingan rata-rata BLEU-1, ROUGE-L F1, *context precision*, dan *context recall* pada skenario *text-image RAG fixed-size*.



Gambar 4.27: Perbandingan rata-rata *latency* pada skenario *text-image RAG fixed-size*.

Berdasarkan Tabel 4.5, FP16 memperoleh kualitas jawaban tertinggi dengan BLEU-1 sebesar 0.7126 dan ROUGE-L F1 sebesar 0.7594. Nilai tersebut lebih tinggi dibandingkan FP32 yang memperoleh BLEU-1 sebesar 0.4951 dan ROUGE-L F1 sebesar 0.5198, serta 4-bit NF4 yang memperoleh BLEU-1 sebesar 0.5024 dan ROUGE-L F1 sebesar 0.5833. Hasil ini menunjukkan bahwa pada skenario *text-image RAG fixed-size*, FP16 menghasilkan jawaban akhir yang paling dekat dengan *ground truth* berdasarkan metrik BLEU-1 dan ROUGE-L F1.

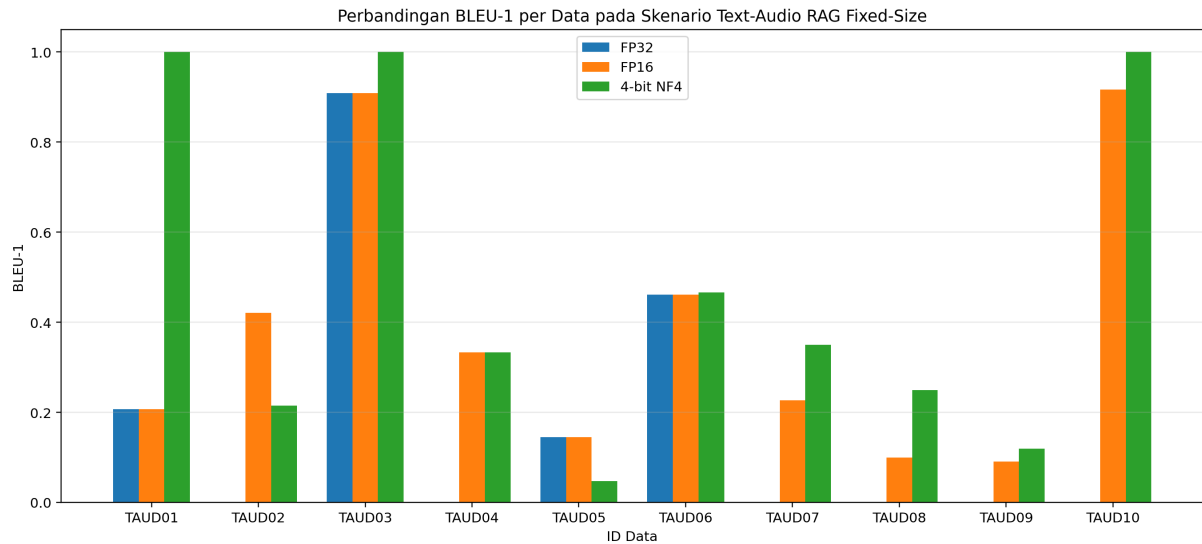
Hasil tersebut berbeda dengan skenario *image-only*, ketika FP32 memperoleh rata-rata *cosine similarity* tertinggi. Perbedaan ini menunjukkan bahwa kualitas pemahaman citra secara mandiri tidak selalu berbanding lurus dengan kualitas jawaban akhir pada *pipeline* RAG multi-modal. Pada skenario *text-image RAG*, jawaban akhir tidak hanya ditentukan oleh kemampuan model visual dalam memahami citra, tetapi juga oleh konteks dokumen hasil *retrieval*, proses penggabungan konteks visual dan tekstual, serta kesesuaian formulasi jawaban terhadap *ground truth*. Dengan demikian, performa FP32 yang lebih tinggi pada *image-only* tidak otomatis menjamin nilai BLEU-1 dan ROUGE-L F1 yang lebih tinggi pada skenario *text-image RAG*.

Dari sisi kualitas *retrieval*, FP32 memperoleh *context precision* tertinggi sebesar 0.5172, sedangkan FP16 dan 4-bit NF4 memperoleh nilai yang sama, yaitu 0.4526. Namun, FP16 dan 4-bit NF4 memiliki *context recall* lebih tinggi, yaitu 0.7000, dibandingkan FP32 sebesar 0.5000. Hal ini menunjukkan bahwa FP32 cenderung lebih presisi dalam konteks yang dinilai relevan, sedangkan FP16 dan 4-bit NF4 memiliki cakupan konteks yang lebih tinggi. Perbedaan antara *context precision* dan *context recall* ini juga menjelaskan bahwa nilai kualitas jawaban akhir tidak hanya dipengaruhi oleh presisi konteks, tetapi juga oleh cakupan informasi yang tersedia dalam konteks tersebut.

Dari sisi *latency*, FP32 menjadi konfigurasi paling cepat dengan rata-rata 1.5115 detik. FP16 membutuhkan 2.4276 detik, sedangkan 4-bit NF4 membutuhkan 3.6631 detik. Dengan demikian, pada skenario *text-image RAG fixed-size*, FP16 unggul dari sisi kualitas jawaban, tetapi FP32 unggul dari sisi waktu respons. Konfigurasi 4-bit NF4 tidak menjadi pilihan terbaik pada skenario ini karena kualitas jawabannya masih di bawah FP16 dan *latency*-nya paling tinggi.

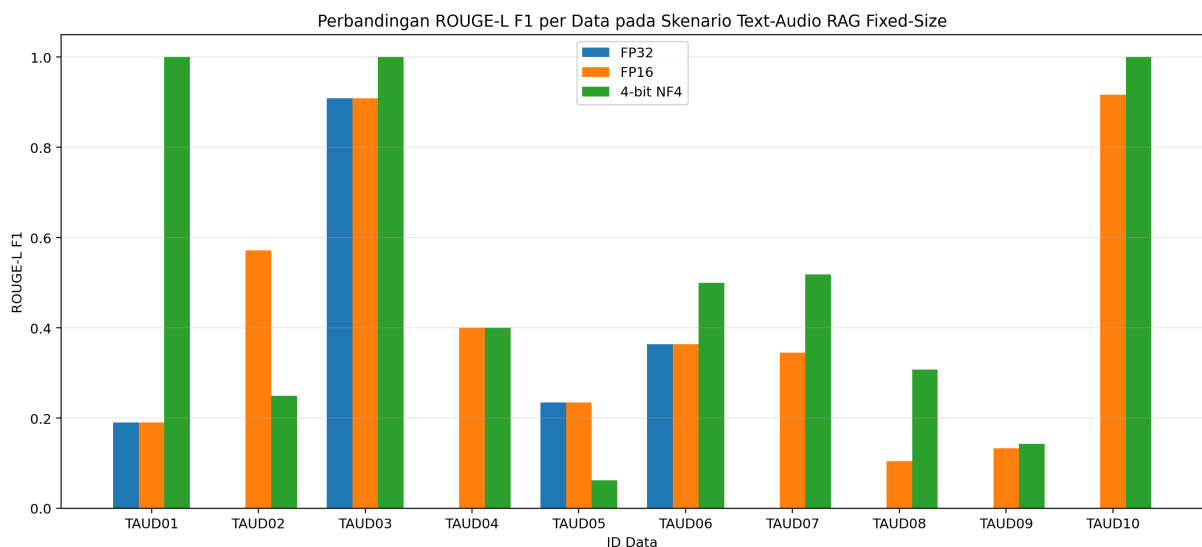
4.5.3 Text-Audio RAG Fixed-Size

Skenario *text-audio RAG fixed-size* digunakan untuk mengevaluasi kemampuan sistem dalam menghasilkan jawaban berdasarkan pertanyaan teks, konteks audio, dan konteks dokumen hasil *retrieval*. Pada skenario ini, audio terlebih dahulu diproses menjadi konteks tekstual, kemudian konteks tersebut digabungkan dengan pertanyaan dan dokumen hasil *retrieval* untuk membentuk jawaban akhir. Berbeda dari skenario teks dan citra, skenario audio memiliki kebutuhan komputasi yang lebih besar karena melibatkan pemrosesan model audio.



Gambar 4.28: Perbandingan BLEU-1 per data pada skenario *text-audio RAG fixed-size*.

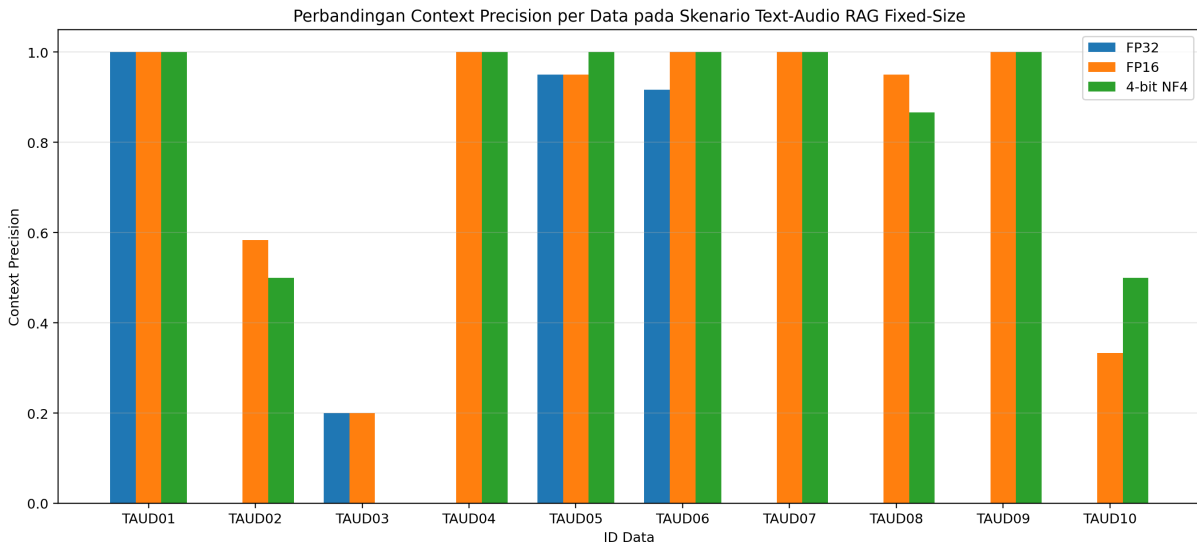
Gambar 4.28 menunjukkan nilai BLEU-1 pada skenario *text-audio RAG fixed-size*. Pada konfigurasi FP32, hanya sebagian data yang berhasil diproses karena beberapa data mengalami *OutOfMemoryError*. Oleh karena itu, batang FP32 hanya muncul pada data yang valid. Pada data yang berhasil diproses, nilai BLEU-1 FP32 cukup bervariasi, sedangkan 4-bit NF4 menunjukkan nilai rata-rata lebih tinggi dibandingkan FP16.



Gambar 4.29: Perbandingan ROUGE-L F1 per data pada skenario *text-audio RAG fixed-size*.

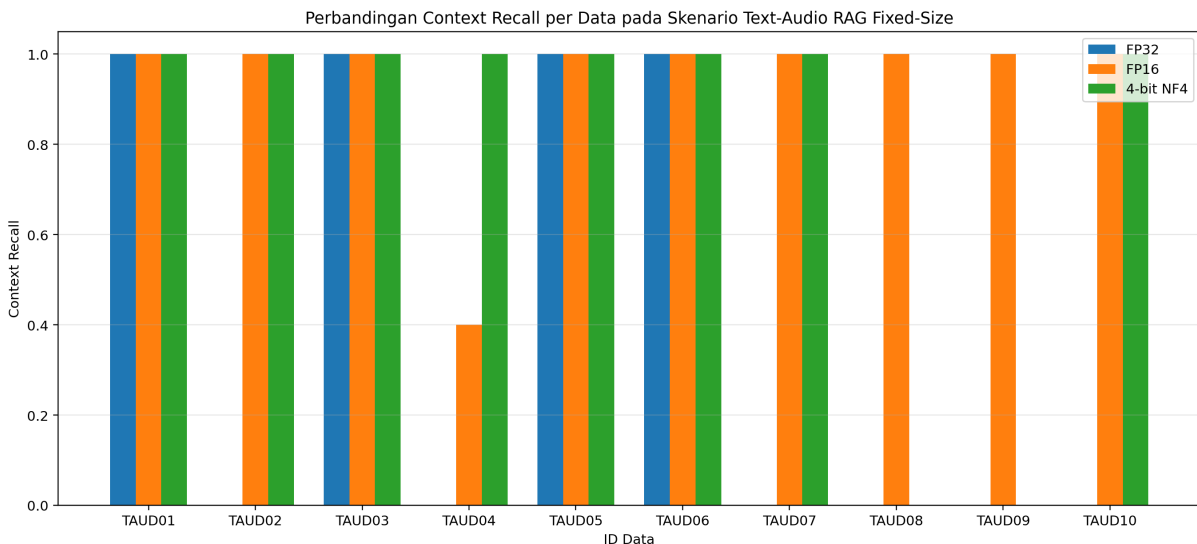
Gambar 4.29 menunjukkan perbandingan ROUGE-L F1 pada setiap data uji. Pola yang

muncul relatif sejalan dengan BLEU-1. Konfigurasi 4-bit NF4 menghasilkan nilai rata-rata ROUGE-L F1 lebih tinggi dibandingkan FP16, sedangkan FP32 hanya dapat dianalisis pada data yang berhasil diproses.



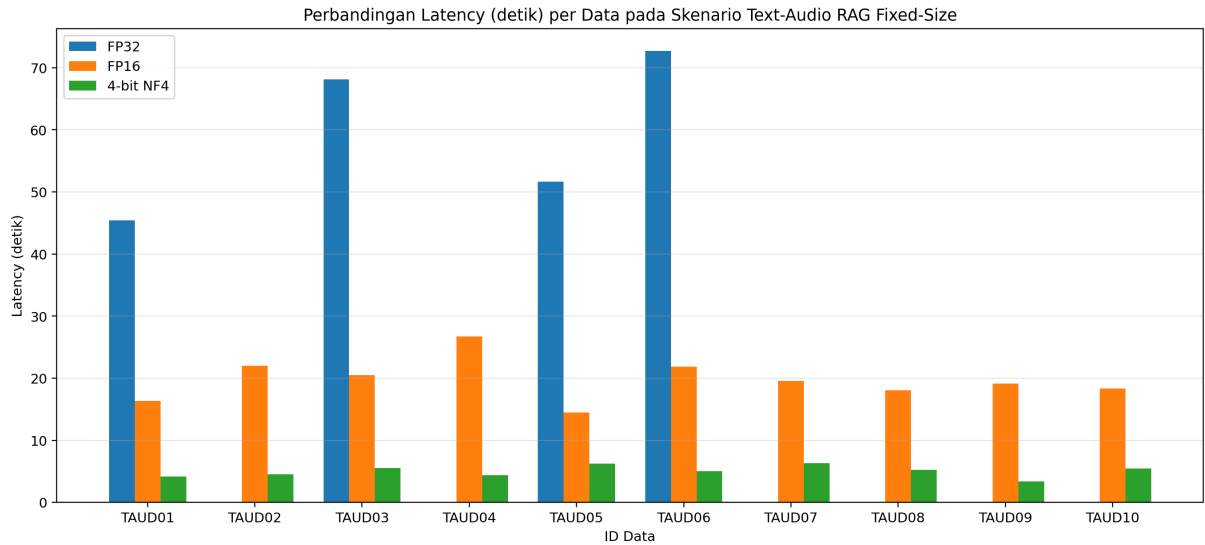
Gambar 4.30: Perbandingan *context precision* per data pada skenario *text-audio RAG fixed-size*.

Gambar 4.30 menunjukkan nilai *context precision* pada setiap data uji. FP16 memperoleh nilai rata-rata *context precision* tertinggi, diikuti oleh 4-bit NF4 dan FP32. Hal ini menunjukkan bahwa pada skenario *text-audio RAG fixed-size*, konteks hasil *retrieval* pada FP16 cenderung lebih presisi terhadap kebutuhan pertanyaan.



Gambar 4.31: Perbandingan *context recall* per data pada skenario *text-audio RAG fixed-size*.

Gambar 4.31 menunjukkan nilai *context recall* pada setiap data uji. Pada data yang berhasil diproses, FP32 memperoleh nilai *context recall* rata-rata tertinggi. Namun, interpretasi FP32 perlu dilakukan secara hati-hati karena hanya 4 dari 10 data yang berhasil diproses. FP16 memperoleh *context recall* sebesar 0.9400, sedangkan 4-bit NF4 memperoleh 0.8000.

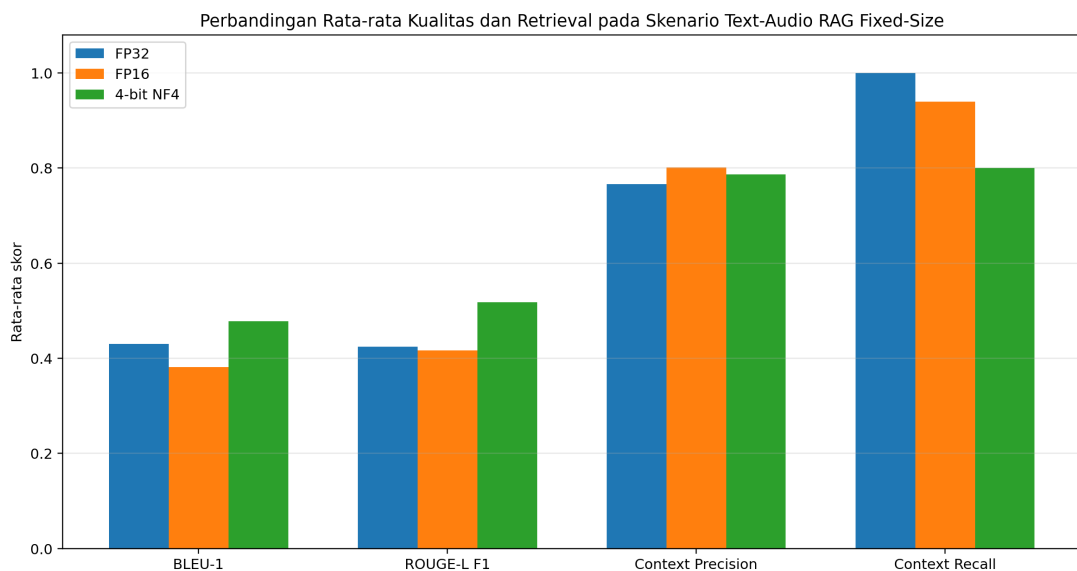


Gambar 4.32: Perbandingan *latency* per data pada skenario *text-audio RAG fixed-size*.

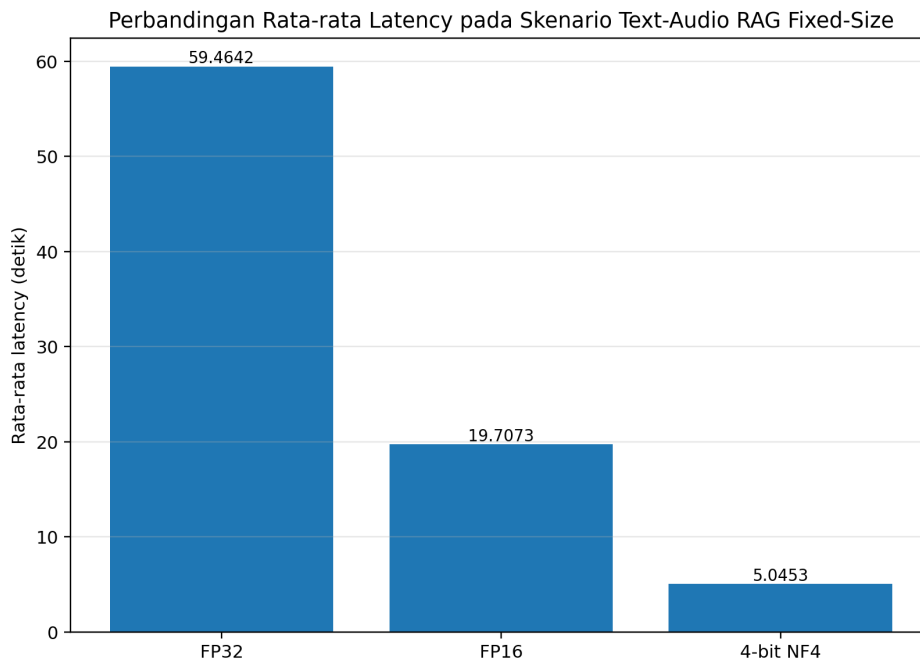
Gambar 4.32 menunjukkan *latency* pada skenario *text-audio RAG fixed-size*. FP32 memiliki waktu respons paling tinggi dan hanya berhasil pada sebagian data. FP16 juga memiliki *latency* yang relatif tinggi karena pemrosesan audio membutuhkan komputasi besar. Sebaliknya, 4-bit NF4 menghasilkan *latency* jauh lebih rendah pada seluruh data uji yang berhasil diproses.

Tabel 4.6: Rata-rata hasil pengujian *text-audio RAG fixed-size*

Konfigurasi	Data Valid	BLEU-1	ROUGE-L F1	CP	CR	Latency (s)
FP32	4/10	0.4306	0.4246	0.7667	1.0000	59.4643
FP16	10/10	0.3811	0.4170	0.8017	0.9400	19.7073
4-bit NF4	10/10	0.4782	0.5182	0.7867	0.8000	5.0453



Gambar 4.33: Perbandingan rata-rata BLEU-1, ROUGE-L F1, *context precision*, dan *context recall* pada skenario *text-audio RAG fixed-size*.



Gambar 4.34: Perbandingan rata-rata *latency* pada skenario *text-audio RAG fixed-size*.

Berdasarkan Tabel 4.6, FP32 hanya berhasil memproses 4 dari 10 data. Enam data lainnya mengalami *OutOfMemoryError*, sehingga nilai rata-rata FP32 dihitung hanya berdasarkan data yang berhasil. Kondisi ini menunjukkan bahwa penggunaan FP32 pada skenario *text-audio RAG fixed-size* tidak stabil pada perangkat pengujian dengan keterbatasan VRAM.

Dari sisi kualitas jawaban, 4-bit NF4 memperoleh BLEU-1 tertinggi sebesar 0.4782 dan ROUGE-L F1 tertinggi sebesar 0.5182. FP32 memperoleh BLEU-1 sebesar 0.4306 dan ROUGE-L F1 sebesar 0.4246 pada data yang berhasil, sedangkan FP16 memperoleh BLEU-1 sebesar 0.3811 dan ROUGE-L F1 sebesar 0.4170. Hasil ini menunjukkan bahwa pada skenario audio, 4-bit NF4 tidak hanya lebih efisien, tetapi juga menghasilkan kualitas jawaban rata-rata yang lebih baik dibandingkan FP16.

Dari sisi kualitas *retrieval*, FP16 memperoleh *context precision* tertinggi sebesar 0.8017, diikuti oleh 4-bit NF4 sebesar 0.7867 dan FP32 sebesar 0.7667. Sementara itu, FP32 memperoleh *context recall* sebesar 1.0000 pada data yang berhasil, FP16 memperoleh 0.9400, dan 4-bit NF4 memperoleh 0.8000. Namun, nilai FP32 perlu dibaca dengan hati-hati karena hanya dihitung dari data yang berhasil diproses.

Dari sisi *latency*, 4-bit NF4 memperoleh waktu respons paling rendah, yaitu 5.0453 detik. FP16 membutuhkan 19.7073 detik, sedangkan FP32 membutuhkan 59.4643 detik pada data yang berhasil. Dengan demikian, pada skenario *text-audio RAG fixed-size*, 4-bit NF4 menjadi konfigurasi paling layak karena berhasil memproses seluruh data, menghasilkan *latency* paling rendah, dan tetap memberikan kualitas jawaban yang lebih baik dibandingkan FP16.

Secara keseluruhan, hasil pengujian RAG dengan konfigurasi *fixed-size* menunjukkan bahwa konfigurasi terbaik berbeda pada setiap modalitas. Pada *text-only RAG*, FP32 dan FP16 unggul dari sisi kualitas jawaban, dengan FP32 memiliki *latency* paling rendah. Pada *text-image RAG*, FP16 menghasilkan kualitas jawaban terbaik berdasarkan BLEU-1 dan ROUGE-L F1, sedangkan FP32 memiliki waktu respons paling rendah. Hasil ini tidak bertentangan dengan pengujian *image-only*, karena *text-image RAG* menguji *pipeline* multimodal yang lebih kompleks, bukan hanya pemahaman citra secara mandiri. Pada *text-audio RAG*, 4-bit NF4 menjadi konfigurasi

paling efektif karena mampu menurunkan *latency* secara signifikan dan tetap menghasilkan kualitas jawaban yang baik. Hasil ini menunjukkan bahwa pemilihan konfigurasi representasi numerik model perlu mempertimbangkan karakteristik modalitas, tahapan *pipeline*, dan tujuan penggunaan, bukan hanya asumsi bahwa presisi yang lebih rendah selalu lebih cepat atau presisi yang lebih tinggi selalu lebih baik.

4.6 Pengujian Tambahan Latency pada Konfigurasi Fixed-Size

Bagian ini menyajikan pengujian tambahan yang difokuskan pada pembacaan *latency* pada konfigurasi *fixed-size*. Pengujian tambahan dilakukan karena konfigurasi *fixed-size* merupakan konfigurasi utama sistem, sedangkan konfigurasi *recursive* pada penelitian ini digunakan sebagai pembandingan strategi *chunking*. Dengan demikian, pengujian tambahan tidak diarahkan untuk mengganti seluruh hasil evaluasi utama, melainkan untuk memperjelas pengaruh konfigurasi representasi numerik model terhadap waktu respons pada konfigurasi utama sistem.

Konfigurasi tambahan yang diuji adalah INT8 *Weight-Only* A16W8 dengan TorchAO dan TF32 aktif. Konfigurasi ini dipilih karena lebih dekat dengan skenario inferensi presisi rendah berbasis dukungan komputasi integer dibandingkan 4-bit NF4 yang pada penelitian ini digunakan terutama untuk efisiensi bobot dan memori. Selain itu, pengujian FP32 *strict* dengan TF32 dinonaktifkan juga dilakukan pada beberapa skenario untuk memisahkan hasil FP32 penuh dari kemungkinan optimasi backend berbasis TF32. Oleh karena itu, istilah FP32 *strict* pada bagian ini merujuk pada konfigurasi FP32 dengan `torch.backends.cuda.matmul.allow_tf32 = False`, `torch.backends.cudnn.allow_tf32 = False`, dan presisi perkalian matriks diatur pada mode tertinggi.

Pengujian tambahan ini tidak menghitung ulang *context precision* dan *context recall*. Kedua metrik tersebut tetap dibaca dari pengujian utama karena CP dan CR berkaitan dengan kualitas *retrieval*, sedangkan pengujian tambahan ini difokuskan pada waktu respons model pada konfigurasi *fixed-size*. Dengan demikian, hasil INT8 pada bagian ini digunakan sebagai pembacaan tambahan terhadap efisiensi inferensi, bukan sebagai pengganti evaluasi kualitas *retrieval*.

Tabel 4.7: Ruang lingkup pengujian tambahan latency pada konfigurasi *fixed-size*

Kelompok	Konfigurasi	Metrik yang Dibaca
Non-RAG multimodal	INT8 WO A16W8 + TF32 ON; pembandingan FP16, FP32/FP32 strict, dan 4-bit NF4	<i>Cosine similarity</i> sebagai kualitas pendukung dan <i>latency</i> sebagai fokus utama
RAG <i>fixed-size</i>	INT8 WO A16W8 + TF32 ON; pembandingan FP16, FP32/FP32 strict, dan 4-bit NF4	BLEU-1 dan ROUGE-L F1 sebagai kualitas pendukung; <i>latency</i> sebagai fokus utama
FP32 <i>strict</i>	TF32 dinonaktifkan pada operasi FP32	Pembandingan untuk membedakan FP32 penuh dari FP32 yang berpotensi memperoleh percepatan backend

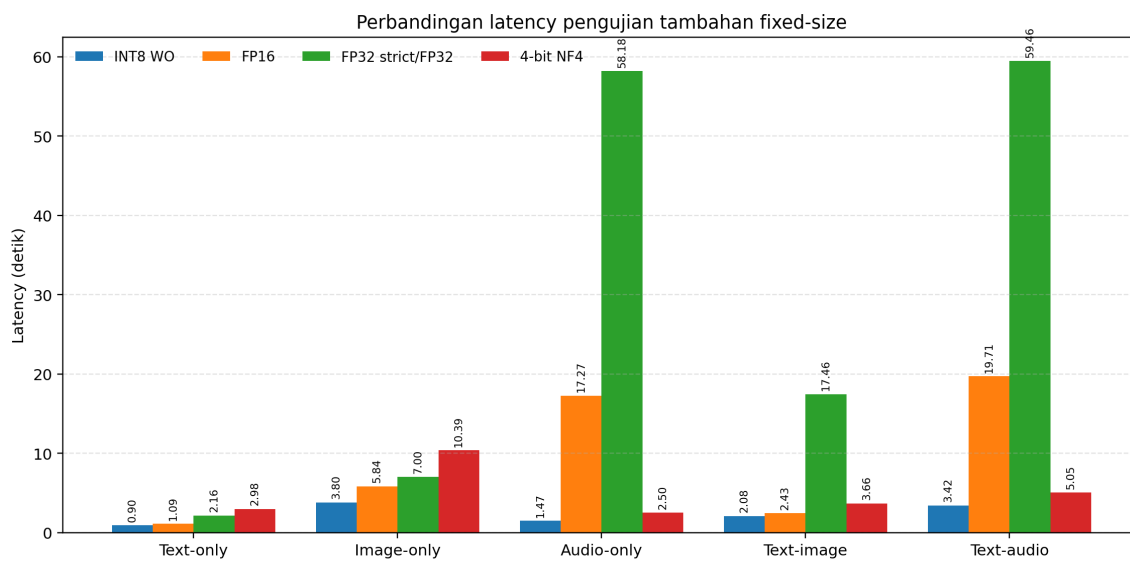
4.6.1 Ringkasan Hasil Pengujian Tambahan Latency

Tabel 4.8 merangkum hasil *latency* pengujian tambahan pada skenario *fixed-size*. Pada skenario audio, pembandingan FP32 tetap menggunakan hasil FP32 utama karena hasil tersebut sudah menunjukkan karakteristik penting berupa keterbatasan pemrosesan dan waktu respons

yang tinggi. Sementara itu, pada skenario teks dan citra yang memiliki hasil FP32 awal sangat cepat, pengujian FP32 *strict* digunakan untuk membaca pengaruh penonaktifan TF32.

Tabel 4.8: Ringkasan latency pengujian tambahan pada konfigurasi *fixed-size*

Skenario	INT8 WO	FP16	FP32/FP32 Strict	4-bit NF4	Urutan Latency
<i>Text-only RAG fixed-size</i>	0.9000	1.0874	2.1554	2.9835	INT8 WO < FP16 < FP32 strict < 4-bit NF4
<i>Image-only</i>	3.8000	5.8399	7.0000	10.3884	INT8 WO < FP16 < FP32 strict < 4-bit NF4
<i>Audio-only</i>	1.4719	17.2661	58.1765	2.5003	INT8 WO < 4-bit NF4 < FP16 < FP32
<i>Text-image RAG fixed-size</i>	2.0850	2.4276	17.4636	3.6631	INT8 WO < FP16 < 4-bit NF4 < FP32 strict
<i>Text-audio RAG fixed-size</i>	3.4230	19.7073	59.4643	5.0453	INT8 WO < 4-bit NF4 < FP16 < FP32



Gambar 4.35: Perbandingan *latency* pengujian tambahan pada konfigurasi *fixed-size*.

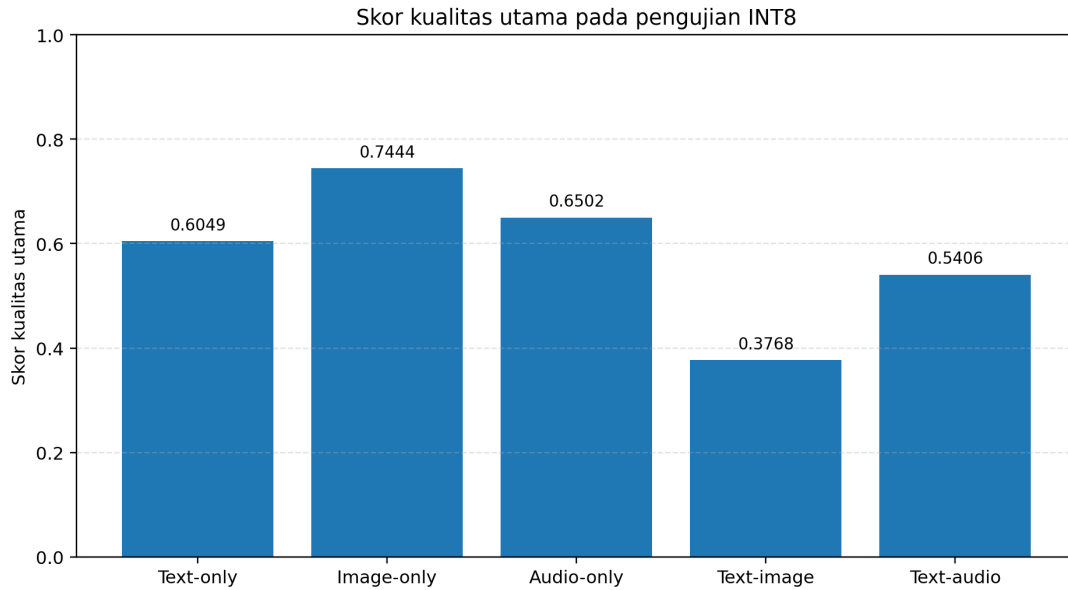
Berdasarkan Tabel 4.8 dan Gambar 4.35, konfigurasi INT8 WO A16W8 + TF32 ON menghasilkan *latency* terendah pada seluruh skenario tambahan yang diuji. Pada *text-only RAG fixed-size*, INT8 memperoleh *latency* 0.9000 detik. Pada *image-only*, INT8 memperoleh *latency* 3.8000 detik. Pada *audio-only*, INT8 memperoleh *latency* 1.4719 detik. Pada *text-image RAG fixed-size*, INT8 memperoleh *latency* 2.0850 detik. Pada *text-audio RAG fixed-size*, INT8 memperoleh *latency* 3.4230 detik. Hasil ini menunjukkan bahwa INT8 *weight-only* dapat memberikan efisiensi waktu respons yang konsisten pada konfigurasi utama sistem.

4.6.2 Hasil INT8 pada Setiap Skenario Fixed-Size

Tabel 4.9 menunjukkan skor kualitas pendukung pada pengujian INT8. Pada skenario RAG, kualitas dibaca menggunakan BLEU-1 dan ROUGE-L F1. Pada skenario non-RAG multimodal, kualitas dibaca menggunakan *cosine similarity*. Nilai CP dan CR tidak ditampilkan karena tidak dihitung ulang pada pengujian tambahan.

Tabel 4.9: Ringkasan kualitas pendukung dan latency pada pengujian INT8 fixed-size

Skenario	BLEU-1 / Cosine	ROUGE-L F1	Latency (s)
<i>Text-only RAG fixed-size</i>	0.5002	0.6049	0.9000
<i>Image-only</i>	0.7444	–	3.8000
<i>Audio-only</i>	0.6502	–	1.4719
<i>Text-image RAG fixed-size</i>	0.3693	0.3768	2.0850
<i>Text-audio RAG fixed-size</i>	0.4758	0.5406	3.4230

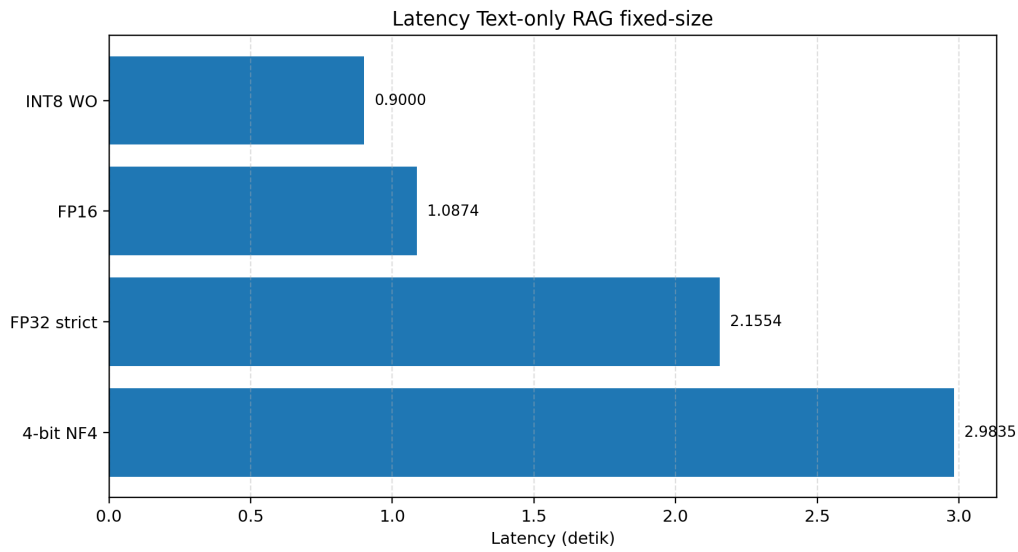


Gambar 4.36: Ringkasan skor kualitas utama pada pengujian INT8.

Hasil pada Tabel 4.9 menunjukkan bahwa INT8 tidak selalu menghasilkan kualitas keluaran tertinggi, tetapi mampu memberikan waktu respons yang rendah. Pada *text-only RAG fixed-size*, kualitas INT8 berada di bawah FP16 pada pengujian utama, tetapi *latency*-nya lebih rendah. Pada *image-only* dan *audio-only*, INT8 menghasilkan skor semantik yang kompetitif dan bahkan menjadi konfigurasi dengan skor rata-rata tertinggi pada pengujian tambahan. Pada *text-image RAG fixed-size*, INT8 menurunkan *latency*, tetapi kualitas jawaban masih lebih rendah dibanding FP16. Pada *text-audio RAG fixed-size*, INT8 menjadi konfigurasi tercepat sekaligus mempertahankan kualitas keluaran yang kompetitif dibanding 4-bit NF4.

4.6.3 Analisis Latency Text-Only RAG Fixed-Size

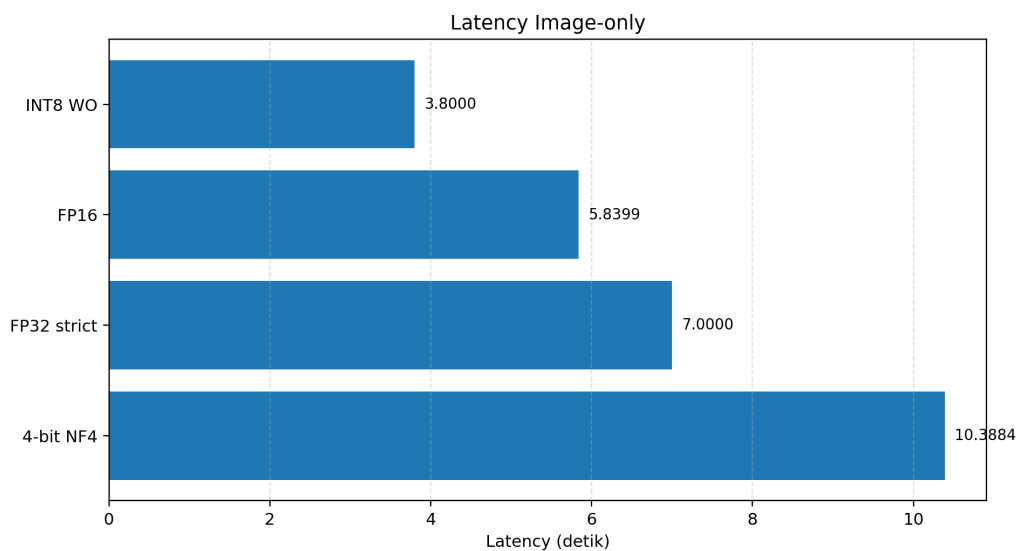
Pada *text-only RAG fixed-size*, INT8 WO A16W8 + TF32 ON memperoleh *latency* 0.9000 detik. Nilai ini lebih rendah dibanding FP16 sebesar 1.0874 detik, FP32 *strict* sebesar 2.1554 detik, dan 4-bit NF4 sebesar 2.9835 detik. Dengan demikian, pada skenario teks, INT8 menjadi konfigurasi paling efisien dari sisi waktu respons. Namun, hasil kualitas utama tetap menunjukkan bahwa FP16 lebih kuat untuk kualitas jawaban karena memperoleh BLEU-1 dan ROUGE-L F1 lebih tinggi pada pengujian utama.



Gambar 4.37: Perbandingan *latency* tambahan pada *text-only RAG fixed-size*.

4.6.4 Analisis Latency Image-Only

Pada skenario *image-only*, INT8 WO A16W8 + TF32 ON memperoleh *latency* 3.8000 detik. Nilai ini lebih rendah dibanding FP16 sebesar 5.8399 detik, FP32 *strict* sebesar 7.0000 detik, dan 4-bit NF4 sebesar 10.3884 detik. Selain itu, INT8 memperoleh *cosine similarity* sebesar 0.7444, lebih tinggi dibanding konfigurasi lain pada pengujian ini. Hasil tersebut menunjukkan bahwa pada pemahaman citra secara mandiri, INT8 memberikan kombinasi yang baik antara kualitas semantik dan waktu respons.

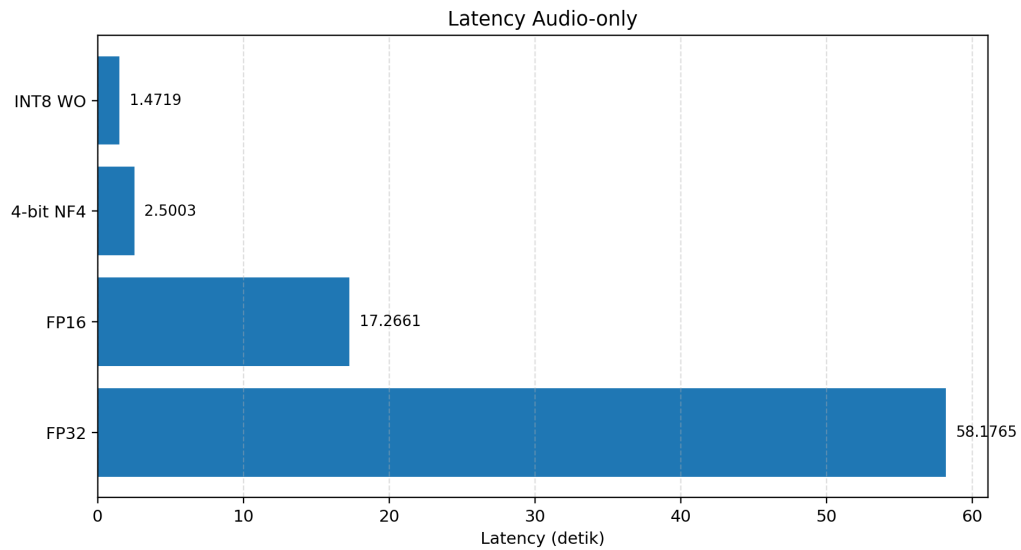


Gambar 4.38: Perbandingan *latency* tambahan pada skenario *image-only*.

4.6.5 Analisis Latency Audio-Only

Pada skenario *audio-only*, INT8 WO A16W8 + TF32 ON memperoleh *latency* 1.4719 detik dan *cosine similarity* sebesar 0.6502. Nilai *latency* ini lebih rendah dibanding 4-bit NF4 sebesar 2.5003 detik, FP16 sebesar 17.2661 detik, dan FP32 sebesar 58.1765 detik pada hasil utama. Hasil ini menunjukkan bahwa INT8 sangat efektif pada model audio karena mampu menurunkan

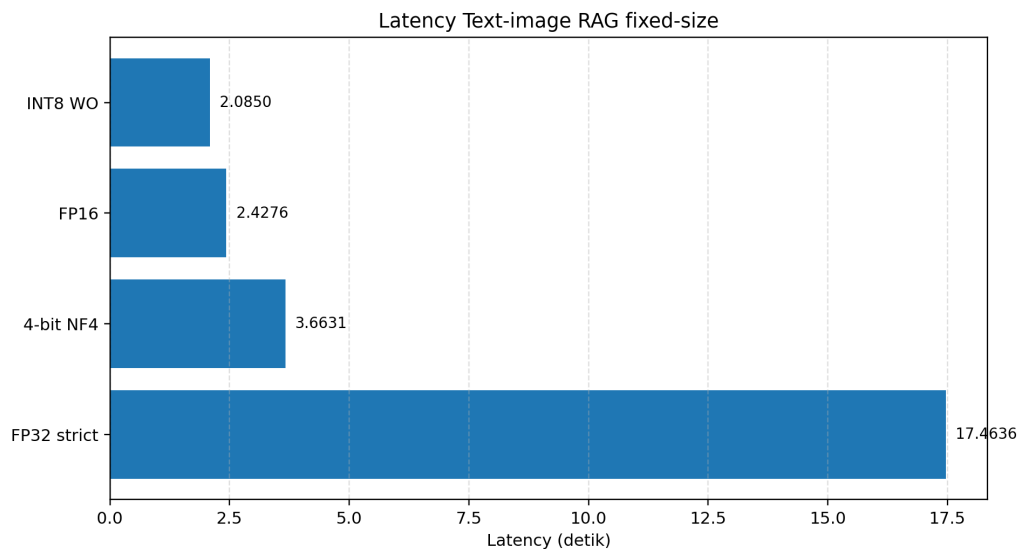
waktu respons tanpa mengorbankan kualitas semantik keluaran secara signifikan.



Gambar 4.39: Perbandingan *latency* tambahan pada skenario *audio-only*.

4.6.6 Analisis Latency Text-Image RAG Fixed-Size

Pada *text-image RAG fixed-size*, INT8 WO A16W8 + TF32 ON memperoleh *latency* 2.0850 detik. Nilai ini lebih rendah dibanding FP16 sebesar 2.4276 detik, 4-bit NF4 sebesar 3.6631 detik, dan FP32 *strict* sebesar 17.4636 detik. Akan tetapi, dari sisi kualitas jawaban, FP16 tetap menjadi konfigurasi yang lebih kuat karena memperoleh BLEU-1 dan ROUGE-L F1 tertinggi pada pengujian utama. Dengan demikian, INT8 lebih sesuai ketika prioritasnya adalah waktu respons, sedangkan FP16 lebih sesuai ketika prioritasnya adalah kualitas jawaban.

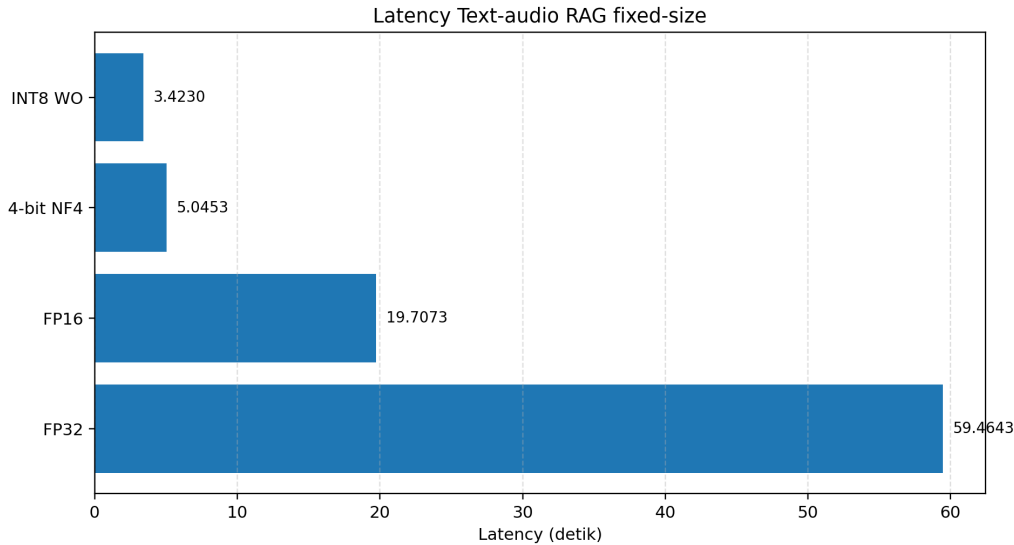


Gambar 4.40: Perbandingan *latency* tambahan pada *text-image RAG fixed-size*.

4.6.7 Analisis Latency Text-Audio RAG Fixed-Size

Pada *text-audio RAG fixed-size*, INT8 WO A16W8 + TF32 ON memperoleh *latency* 3.4230 detik. Nilai ini lebih rendah dibanding 4-bit NF4 sebesar 5.0453 detik, FP16 sebesar 19.7073 detik, dan FP32 sebesar 59.4643 detik pada hasil utama. Dari sisi kualitas, INT8 memperoleh

BLEU-1 sebesar 0.4758 dan ROUGE-L F1 sebesar 0.5406. Nilai ini kompetitif terhadap 4-bit NF4 dan lebih tinggi dibanding FP16 pada pengujian utama. Dengan demikian, pada skenario *text-audio RAG fixed-size*, INT8 menjadi konfigurasi paling efisien dari sisi waktu respons sekaligus tetap memberikan kualitas keluaran yang kompetitif.



Gambar 4.41: Perbandingan *latency* tambahan pada *text-audio RAG fixed-size*.

4.6.8 Pembacaan FP32 Awal dan FP32 Strict

Hasil pengujian tambahan menunjukkan bahwa FP32 awal tidak boleh langsung ditafsirkan sebagai FP32 penuh yang bebas dari optimasi backend. Pada beberapa skenario, FP32 awal menghasilkan *latency* yang sangat rendah, bahkan lebih rendah dibanding FP16. Setelah FP32 diuji ulang dalam mode *strict* dengan TF32 dinonaktifkan, *latency* meningkat pada skenario yang diuji ulang. Hal ini menunjukkan bahwa hasil FP32 awal memiliki indikasi kuat dipengaruhi oleh optimasi backend seperti TF32 atau pengaturan presisi perkalian matriks pada CUDA/PyTorch.

Namun, kesimpulan tersebut tetap perlu dibaca secara hati-hati. Tanpa log konfigurasi backend pada setiap pengujian awal, hasil FP32 awal tidak dapat dinyatakan secara mutlak sebagai TF32. Oleh karena itu, penelitian ini menggunakan istilah FP32 awal untuk hasil pengujian utama sebelumnya dan FP32 *strict* untuk hasil pengujian tambahan dengan TF32 dinonaktifkan. Perbedaan ini digunakan agar pembacaan *latency* lebih transparan dan tidak mencampur FP32 penuh dengan FP32 yang berpotensi memperoleh akselerasi Tensor Core berbasis TF32.

Fenomena FP32 awal yang lebih cepat dibanding FP16 juga dapat terjadi karena *latency* sistem merupakan pengukuran *end-to-end*. Nilai *latency* tidak hanya dipengaruhi oleh format representasi numerik, tetapi juga oleh panjang keluaran, pemilihan kernel, proses *preprocessing*, pemindahan data CPU–GPU, penggunaan *cache*, konfigurasi *attention*, dan kemungkinan optimasi backend. Dengan demikian, hasil pengujian tambahan memperjelas bahwa perbandingan konfigurasi representasi numerik perlu dibaca berdasarkan implementasi pipeline yang digunakan, bukan hanya berdasarkan asumsi teoritis bahwa satu format presisi selalu lebih cepat dari format lainnya.

4.7 Hasil Pengujian RAG dengan Konfigurasi Chunking Pembanding Re-

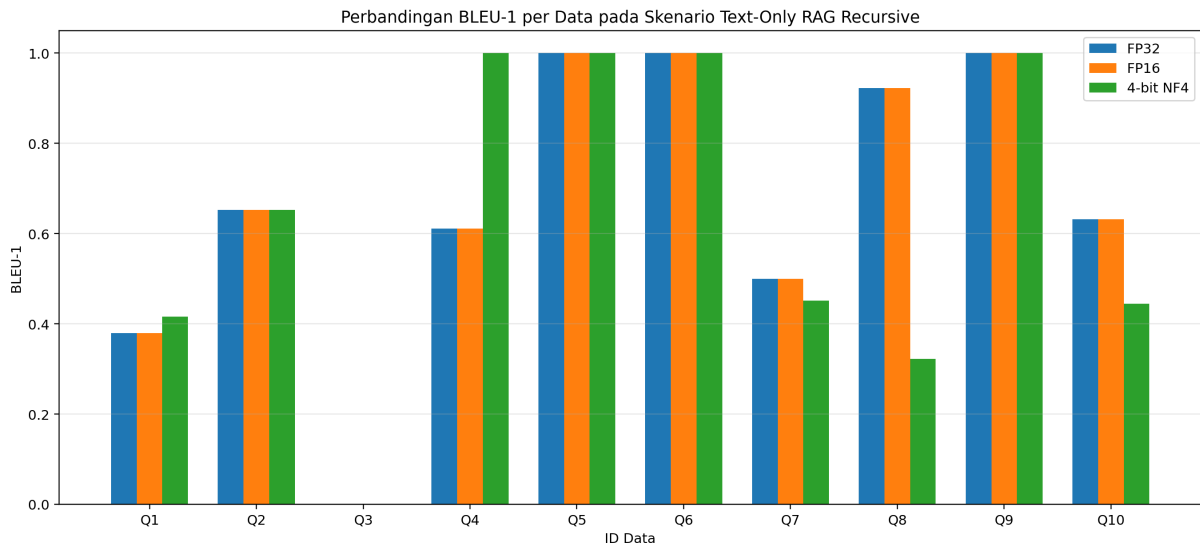
cursive

Bagian ini membahas hasil pengujian RAG dengan konfigurasi *recursive chunking*. Konfigurasi ini digunakan sebagai pembandingan terhadap konfigurasi utama *fixed-size*. Berbeda dari konfigurasi *fixed-size* yang membagi dokumen berdasarkan ukuran karakter tetap dengan mempertahankan struktur paragraf, konfigurasi *recursive* membagi teks secara hierarkis berdasarkan separator tertentu sehingga menghasilkan potongan konteks yang lebih granular. Hasil pada bagian ini digunakan untuk melihat apakah konfigurasi *recursive* memberikan perubahan terhadap kualitas *retrieval*, kualitas jawaban, dan *latency*.

Pengujian pada bagian ini dilakukan terhadap tiga skenario RAG, yaitu *text-only RAG recursive*, *text-image RAG recursive*, dan *text-audio RAG recursive*. Setiap skenario diuji menggunakan tiga konfigurasi representasi numerik model, yaitu FP32, FP16, dan 4-bit NF4. Metrik yang digunakan sama dengan pengujian *fixed-size*, yaitu BLEU-1, ROUGE-L F1, *context precision*, *context recall*, dan *latency*. Dengan demikian, hasil pada bagian ini dapat dibandingkan secara konseptual dengan hasil pengujian pada konfigurasi *fixed-size*.

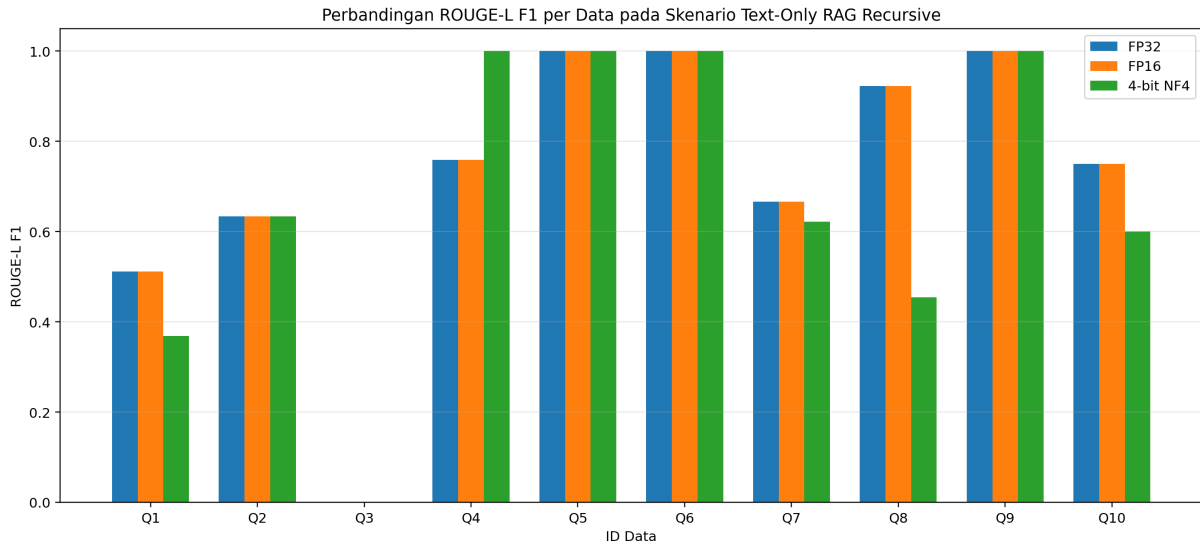
4.7.1 Text-Only RAG Recursive

Skenario *text-only RAG recursive* digunakan untuk mengevaluasi kemampuan sistem dalam menghasilkan jawaban berdasarkan pertanyaan teks dan konteks dokumen hasil *retrieval* pada indeks *recursive*. Data uji, *ground truth*, nilai Top-*k*, dan ambang relevansi dipertahankan sama, sedangkan indeks yang digunakan adalah indeks hasil *recursive chunking*. Dengan demikian, perbedaan hasil pada skenario ini terutama merepresentasikan pengaruh konfigurasi *chunking* dan konfigurasi representasi numerik model terhadap kualitas jawaban, kualitas konteks, dan waktu respons.



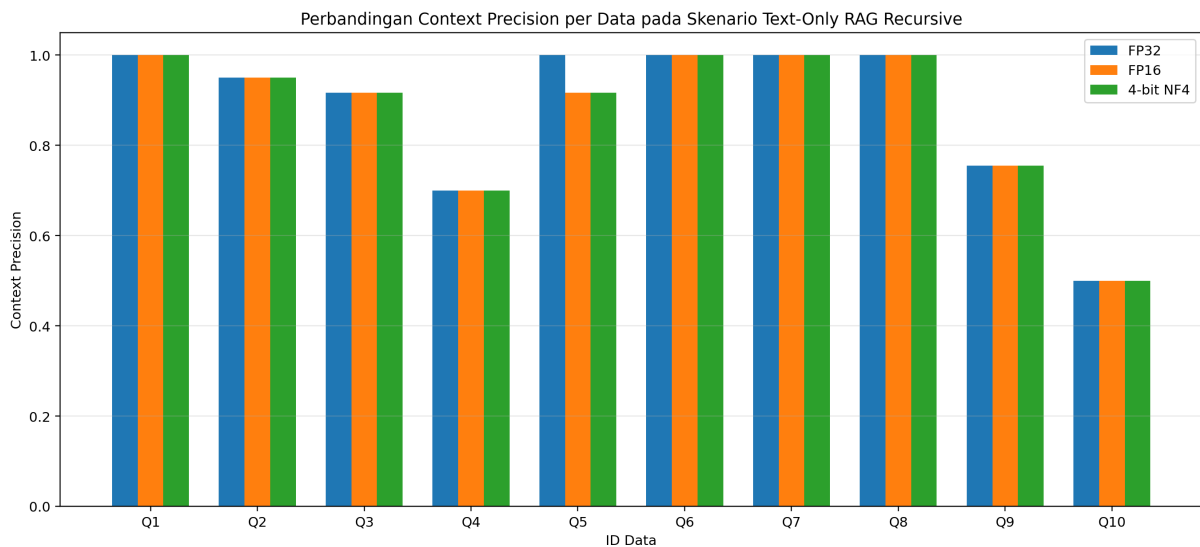
Gambar 4.42: Perbandingan BLEU-1 per data pada skenario *text-only RAG recursive*.

Gambar 4.42 menunjukkan nilai BLEU-1 pada setiap data uji. FP32 dan FP16 menghasilkan pola nilai yang sama pada seluruh data, sedangkan 4-bit NF4 menunjukkan nilai yang sedikit lebih rendah pada beberapa data. Hal ini menunjukkan bahwa pada skenario *text-only RAG recursive*, FP32 dan FP16 menghasilkan jawaban dengan kecocokan unigram yang setara terhadap jawaban referensi.



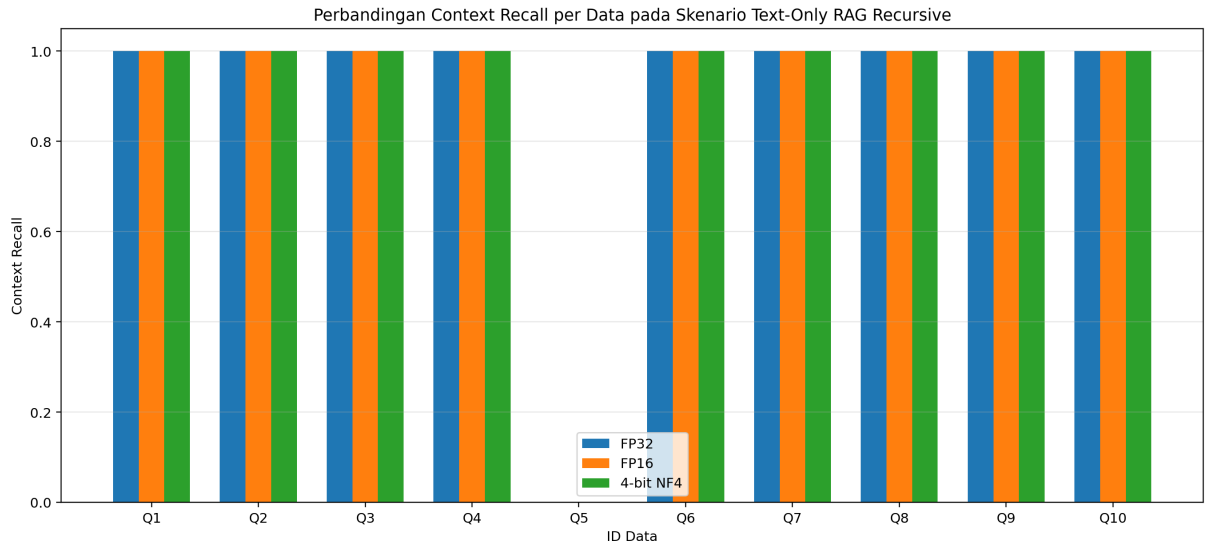
Gambar 4.43: Perbandingan ROUGE-L F1 per data pada skenario *text-only RAG recursive*.

Gambar 4.43 menunjukkan nilai ROUGE-L F1 pada setiap data uji. Pola ROUGE-L F1 sejalan dengan BLEU-1, yaitu FP32 dan FP16 memiliki nilai yang sama, sedangkan 4-bit NF4 cenderung lebih rendah. Hal ini menunjukkan bahwa konfigurasi 4-bit NF4 tidak hanya menghasilkan kecocokan kata yang sedikit lebih rendah, tetapi juga memengaruhi kesesuaian urutan informasi dalam jawaban akhir.



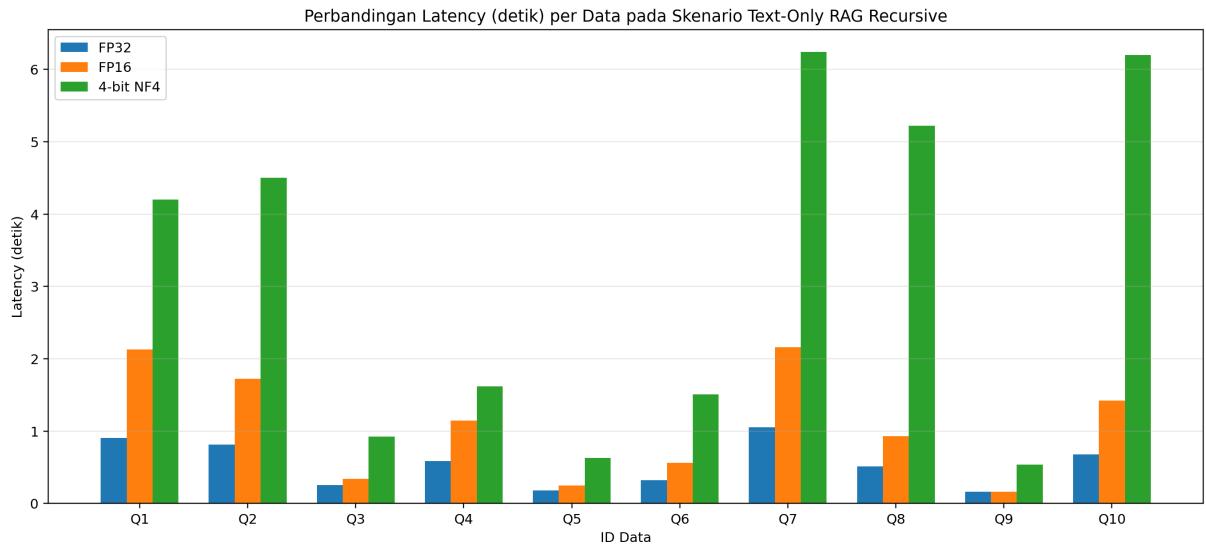
Gambar 4.44: Perbandingan *context precision* per data pada skenario *text-only RAG recursive*.

Gambar 4.44 menunjukkan nilai *context precision* pada setiap data uji. Nilai *context precision* pada ketiga konfigurasi berada pada rentang tinggi, meskipun FP32 memiliki rata-rata sedikit lebih tinggi dibandingkan FP16 dan 4-bit NF4. Hal ini menunjukkan bahwa konteks hasil *retrieval* pada indeks recursive umumnya tetap relevan terhadap pertanyaan.



Gambar 4.45: Perbandingan *context recall* per data pada skenario *text-only RAG recursive*.

Gambar 4.45 menunjukkan nilai *context recall* pada setiap data uji. Ketiga konfigurasi memperoleh rata-rata *context recall* yang sama. Hal ini menunjukkan bahwa pada skenario *text-only RAG recursive*, cakupan informasi penting dalam konteks hasil *retrieval* relatif stabil terhadap perubahan konfigurasi representasi numerik model.

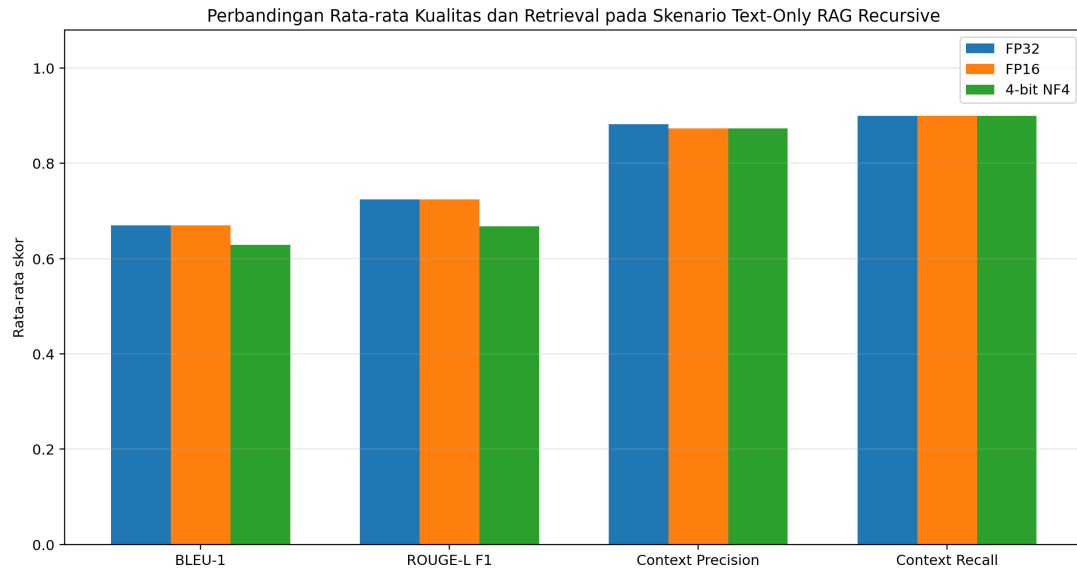


Gambar 4.46: Perbandingan *latency* per data pada skenario *text-only RAG recursive*.

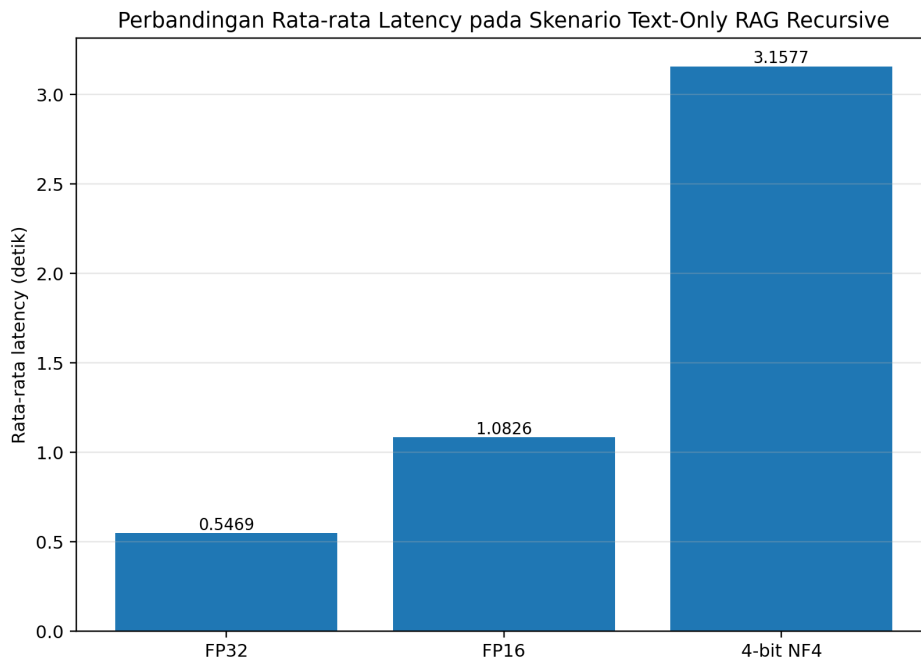
Gambar 4.46 menunjukkan *latency* pada setiap data uji. FP32 menghasilkan waktu respons paling rendah, diikuti oleh FP16 dan 4-bit NF4. Pola ini menunjukkan bahwa pada skenario teks, penggunaan 4-bit NF4 tidak otomatis menghasilkan inferensi yang lebih cepat karena adanya overhead kuantisasi dan dekuantisasi.

Tabel 4.10: Rata-rata hasil pengujian *text-only RAG recursive*

Konfigurasi	Data Valid	BLEU-1	ROUGE-L F1	CP	CR	Latency (s)
FP32	10/10	0.6697	0.7244	0.8822	0.9000	0.5469
FP16	10/10	0.6697	0.7244	0.8739	0.9000	1.0826
4-bit NF4	10/10	0.6287	0.6679	0.8739	0.9000	3.1577



Gambar 4.47: Perbandingan rata-rata BLEU-1, ROUGE-L F1, *context precision*, dan *context recall* pada skenario *text-only RAG recursive*.



Gambar 4.48: Perbandingan rata-rata *latency* pada skenario *text-only RAG recursive*.

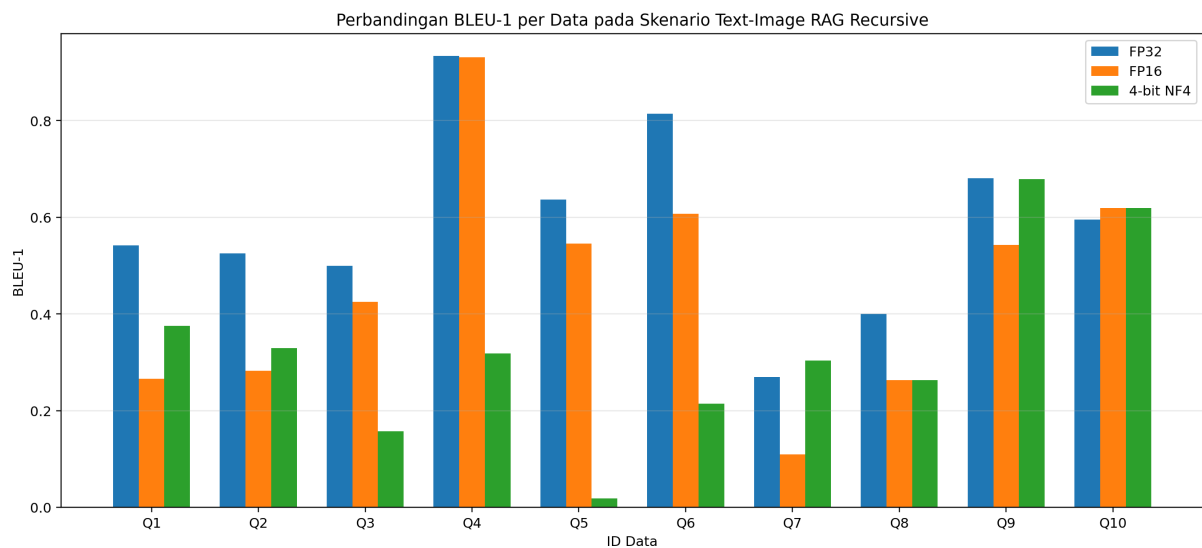
Berdasarkan Tabel 4.10, FP32 dan FP16 memperoleh nilai BLEU-1 dan ROUGE-L F1 yang sama, yaitu BLEU-1 sebesar 0.6697 dan ROUGE-L F1 sebesar 0.7244. Nilai tersebut lebih tinggi dibandingkan 4-bit NF4 yang memperoleh BLEU-1 sebesar 0.6287 dan ROUGE-L F1 sebesar 0.6679. Dari sisi kualitas *retrieval*, FP32 memperoleh *context precision* tertinggi sebesar 0.8822, sedangkan FP16 dan 4-bit NF4 memperoleh nilai yang sama, yaitu 0.8739. Nilai *context recall* pada ketiga konfigurasi sama, yaitu 0.9000.

Dari sisi *latency*, FP32 memperoleh waktu respons paling rendah, yaitu 0.5469 detik. FP16 membutuhkan 1.0826 detik, sedangkan 4-bit NF4 membutuhkan 3.1577 detik. Dengan demikian, pada skenario *text-only RAG recursive*, FP32 menjadi konfigurasi paling efisien dari

sisi waktu respons, sementara FP32 dan FP16 menjadi konfigurasi terbaik dari sisi kualitas jawaban. Konfigurasi 4-bit NF4 tetap menghasilkan nilai *context recall* yang sama, tetapi memiliki kualitas jawaban dan *latency* yang lebih rendah dibandingkan FP32 dan FP16.

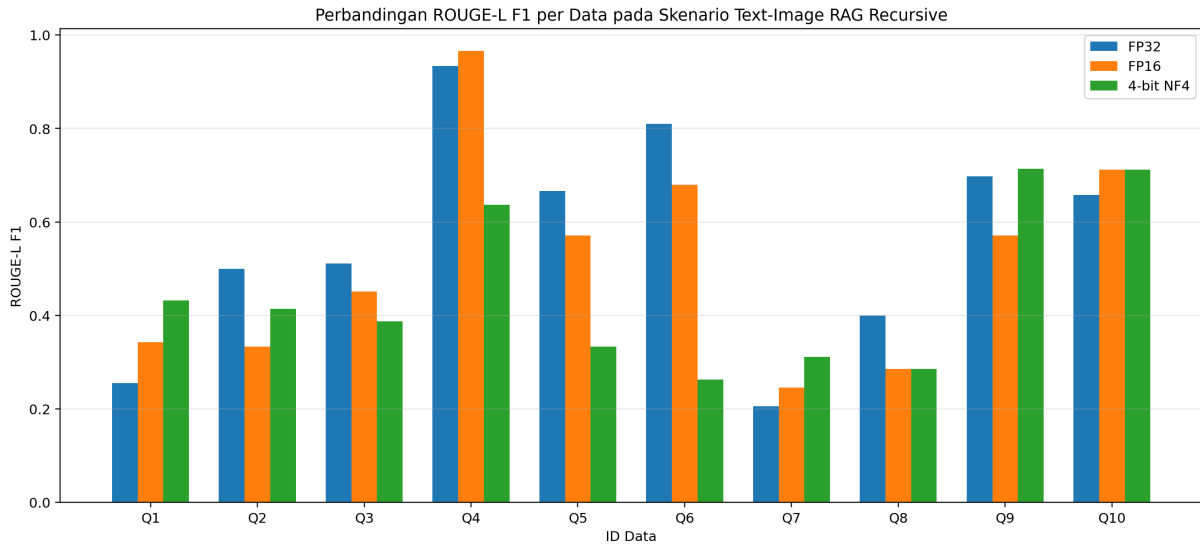
4.7.2 Text-Image RAG Recursive

Skenario *text-image RAG recursive* digunakan untuk mengevaluasi kemampuan sistem dalam menghasilkan jawaban berdasarkan pertanyaan teks, informasi visual dari citra, dan konteks dokumen hasil *retrieval* pada indeks recursive. Pada skenario ini, jawaban akhir tidak hanya dipengaruhi oleh pemahaman citra, tetapi juga oleh kualitas konteks hasil *retrieval* dan kemampuan model dalam menggabungkan informasi visual dengan informasi tekstual.



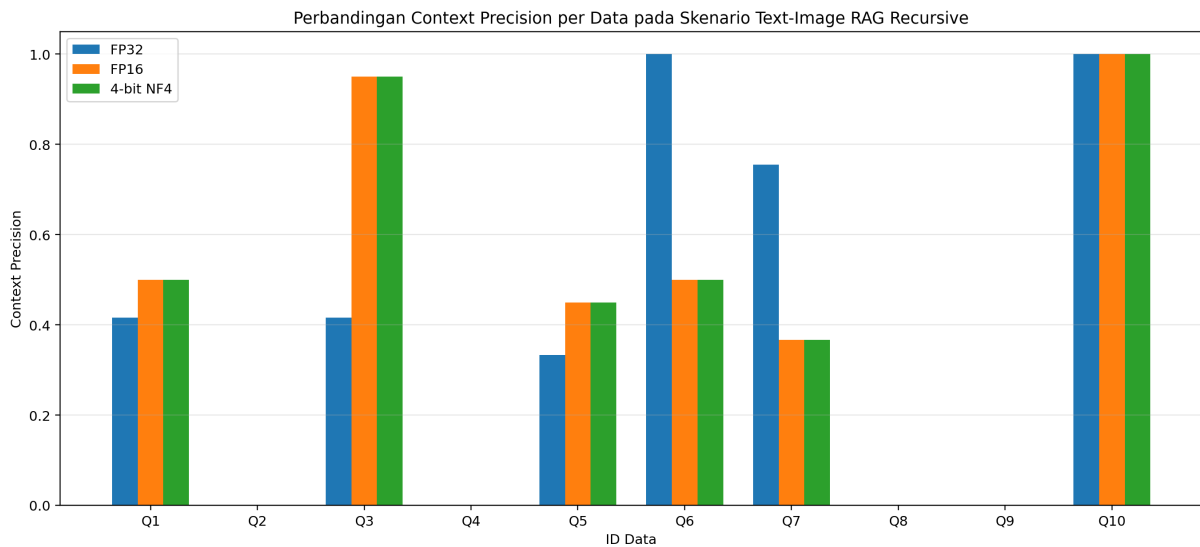
Gambar 4.49: Perbandingan BLEU-1 per data pada skenario *text-image RAG recursive*.

Gambar 4.49 menunjukkan nilai BLEU-1 pada setiap data uji. FP32 menghasilkan nilai rata-rata BLEU-1 tertinggi, sedangkan FP16 dan 4-bit NF4 memiliki nilai yang lebih rendah. Hasil ini menunjukkan bahwa pada skenario *text-image RAG recursive*, konfigurasi FP32 menghasilkan jawaban dengan kecocokan unigram yang lebih dekat terhadap *ground truth* dibandingkan konfigurasi lainnya.



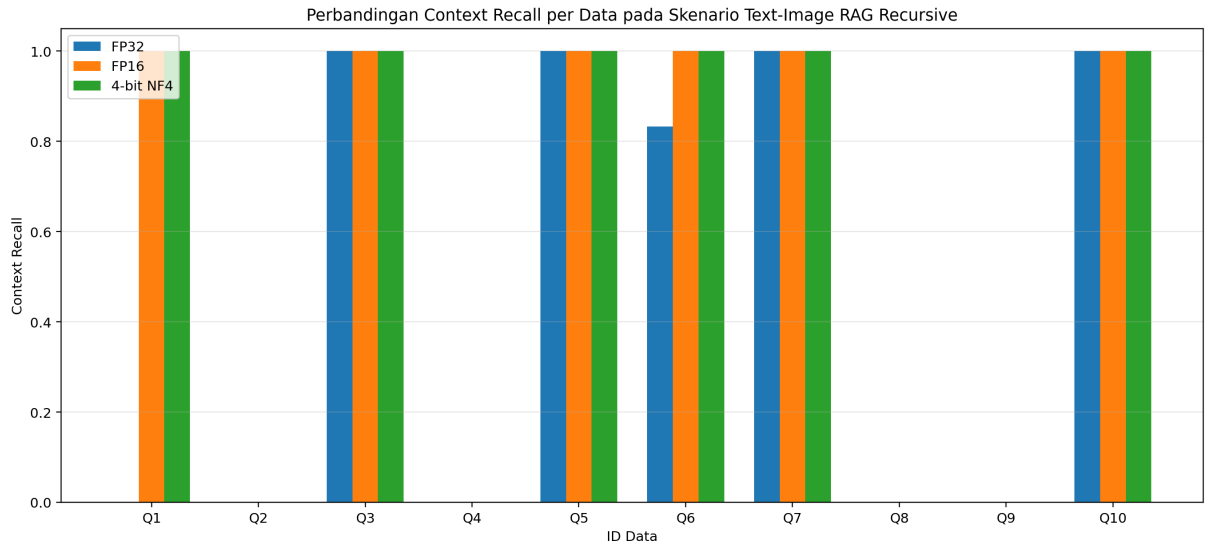
Gambar 4.50: Perbandingan ROUGE-L F1 per data pada skenario *text-image RAG recursive*.

Gambar 4.50 menunjukkan nilai ROUGE-L F1 pada setiap data uji. FP32 kembali memperoleh rata-rata tertinggi, diikuti oleh FP16 dan 4-bit NF4. Hal ini menunjukkan bahwa pada konfigurasi recursive, FP32 lebih mampu mempertahankan urutan informasi jawaban yang sesuai dengan jawaban referensi pada skenario *text-image RAG*.



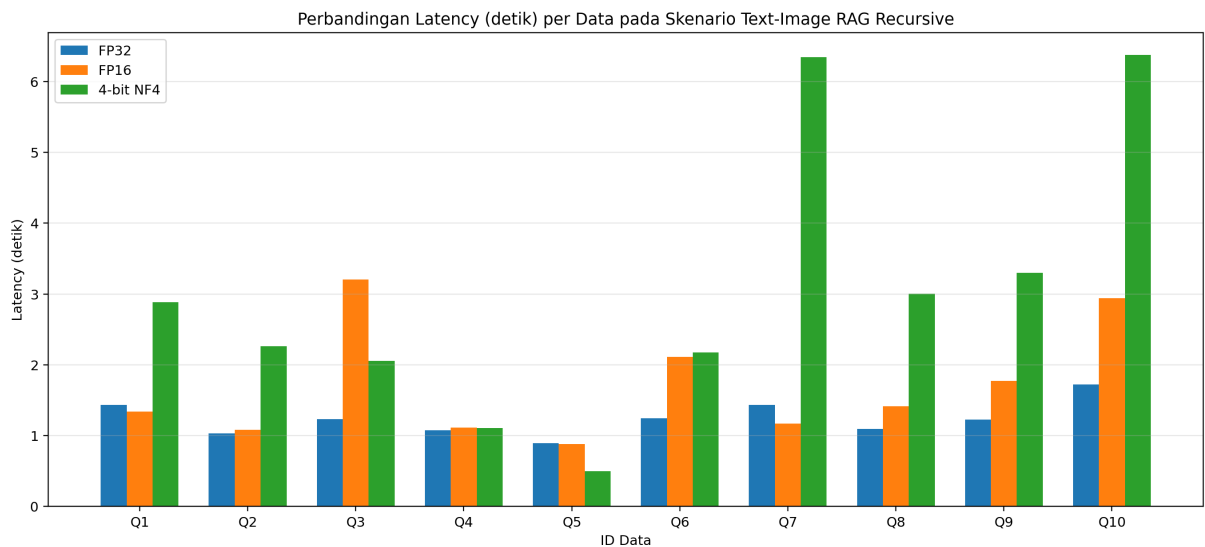
Gambar 4.51: Perbandingan *context precision* per data pada skenario *text-image RAG recursive*.

Gambar 4.51 menunjukkan nilai *context precision* pada setiap data uji. Nilai *context precision* pada skenario ini relatif lebih rendah dibandingkan skenario *text-only RAG recursive*. Hal ini menunjukkan bahwa pencarian konteks pada pertanyaan yang melibatkan citra lebih kompleks, karena sistem perlu menghubungkan informasi visual dengan informasi dokumen yang relevan.



Gambar 4.52: Perbandingan *context recall* per data pada skenario *text-image RAG recursive*.

Gambar 4.52 menunjukkan nilai *context recall* pada setiap data uji. FP16 dan 4-bit NF4 memperoleh nilai *context recall* rata-rata lebih tinggi dibandingkan FP32. Hal ini menunjukkan bahwa pada konfigurasi FP16 dan 4-bit NF4, konteks yang digunakan lebih banyak mencakup informasi penting yang dibutuhkan untuk menjawab pertanyaan, meskipun presisi konteksnya tidak lebih tinggi dibandingkan FP32.

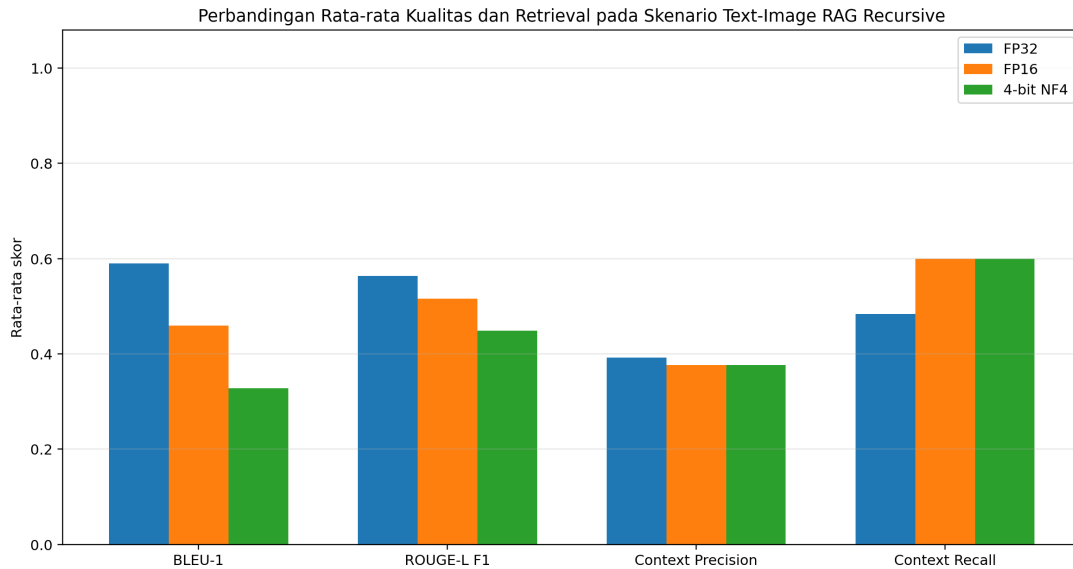


Gambar 4.53: Perbandingan *latency* per data pada skenario *text-image RAG recursive*.

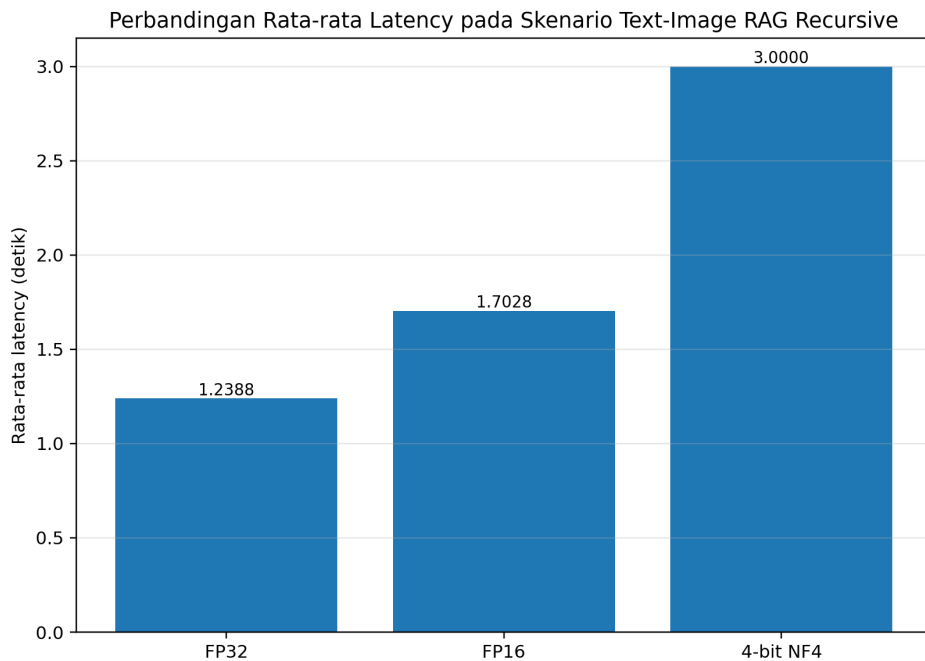
Gambar 4.53 menunjukkan *latency* pada setiap data uji. FP32 menghasilkan waktu respons paling rendah, diikuti oleh FP16 dan 4-bit NF4. Hasil ini kembali menunjukkan bahwa konfigurasi 4-bit NF4 tidak otomatis menghasilkan waktu respons lebih rendah pada skenario berbasis citra.

Tabel 4.11: Rata-rata hasil pengujian *text-image RAG recursive*

Konfigurasi	Data Valid	BLEU-1	ROUGE-L F1	CP	CR	Latency (s)
FP32	10/10	0.5897	0.5638	0.3922	0.4833	1.2388
FP16	10/10	0.4594	0.5160	0.3767	0.6000	1.7028
4-bit NF4	10/10	0.3280	0.4490	0.3767	0.6000	3.0000



Gambar 4.54: Perbandingan rata-rata BLEU-1, ROUGE-L F1, *context precision*, dan *context recall* pada skenario *text-image RAG recursive*.



Gambar 4.55: Perbandingan rata-rata *latency* pada skenario *text-image RAG recursive*.

Berdasarkan Tabel 4.11, FP32 memperoleh nilai BLEU-1 tertinggi sebesar 0.5897 dan ROUGE-L F1 tertinggi sebesar 0.5638. FP16 memperoleh BLEU-1 sebesar 0.4594 dan ROUGE-L F1 sebesar 0.5160, sedangkan 4-bit NF4 memperoleh BLEU-1 sebesar 0.3280 dan ROUGE-L

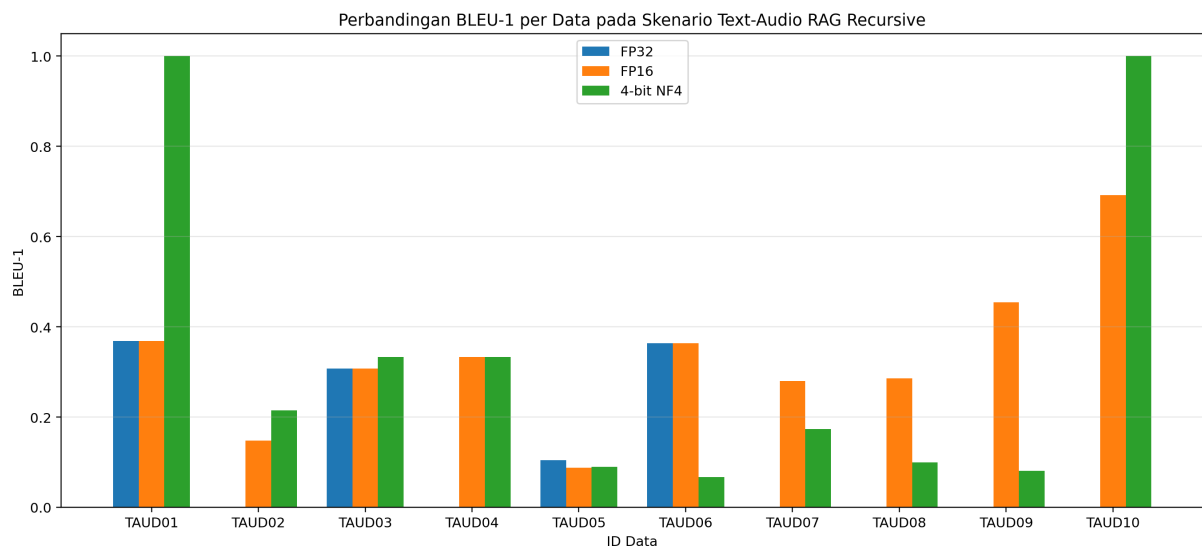
F1 sebesar 0.4490. Hasil ini menunjukkan bahwa pada skenario *text-image RAG recursive*, FP32 menghasilkan kualitas jawaban tertinggi berdasarkan metrik BLEU-1 dan ROUGE-L F1.

Dari sisi kualitas *retrieval*, FP32 memperoleh *context precision* tertinggi sebesar 0.3922, sedangkan FP16 dan 4-bit NF4 memperoleh nilai yang sama, yaitu 0.3767. Namun, FP16 dan 4-bit NF4 memperoleh *context recall* lebih tinggi, yaitu 0.6000, dibandingkan FP32 sebesar 0.4833. Pola ini menunjukkan bahwa FP32 lebih unggul dari sisi presisi konteks, sedangkan FP16 dan 4-bit NF4 memiliki cakupan konteks yang lebih tinggi.

Dari sisi *latency*, FP32 kembali menjadi konfigurasi tercepat dengan rata-rata 1.2388 detik. FP16 membutuhkan 1.7028 detik, sedangkan 4-bit NF4 membutuhkan 3.0000 detik. Dengan demikian, pada skenario *text-image RAG recursive*, FP32 menjadi konfigurasi paling baik dari sisi kualitas jawaban dan waktu respons, sedangkan FP16 dan 4-bit NF4 lebih unggul pada aspek *context recall*.

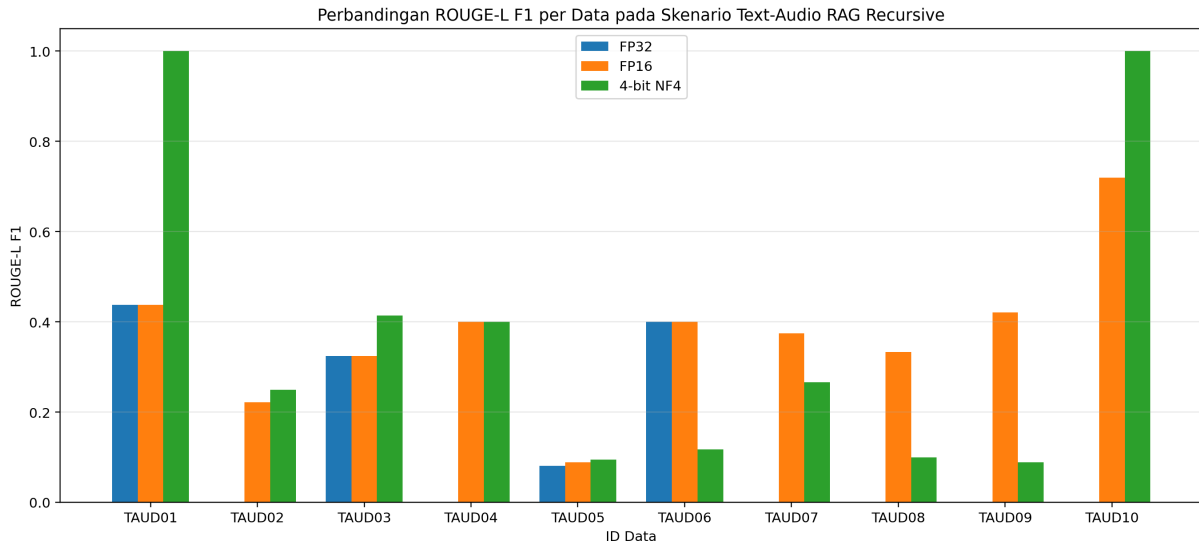
4.7.3 Text-Audio RAG Recursive

Skenario *text-audio RAG recursive* digunakan untuk mengevaluasi kemampuan sistem dalam menghasilkan jawaban berdasarkan pertanyaan teks, konteks audio, dan konteks dokumen hasil *retrieval* pada indeks recursive. Pada skenario ini, audio terlebih dahulu diproses menjadi konteks tekstual, kemudian konteks tersebut digunakan bersama pertanyaan dan konteks dokumen hasil *retrieval* untuk menghasilkan jawaban akhir. Skenario ini memiliki kebutuhan komputasi yang lebih besar dibandingkan skenario teks dan citra karena melibatkan pemrosesan model audio.



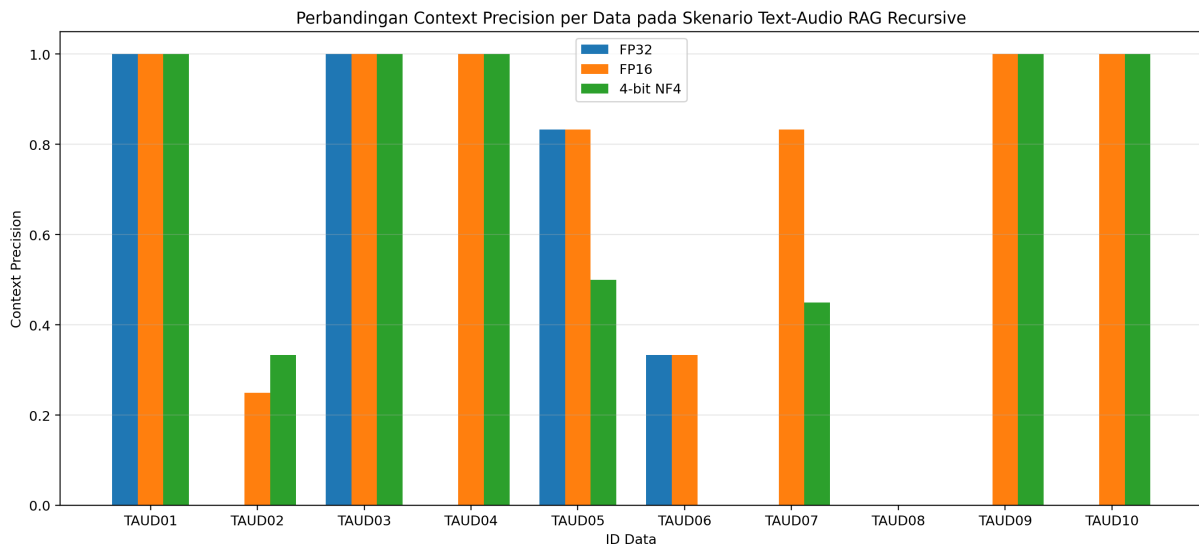
Gambar 4.56: Perbandingan BLEU-1 per data pada skenario *text-audio RAG recursive*.

Gambar 4.56 menunjukkan nilai BLEU-1 pada setiap data uji. Pada konfigurasi FP32, hanya sebagian data yang berhasil diproses karena beberapa data mengalami *OutOfMemoryError*. Oleh karena itu, nilai FP32 hanya dapat dianalisis pada data yang valid. Pada konfigurasi yang berhasil memproses seluruh data, 4-bit NF4 memperoleh rata-rata BLEU-1 sedikit lebih tinggi dibandingkan FP16.



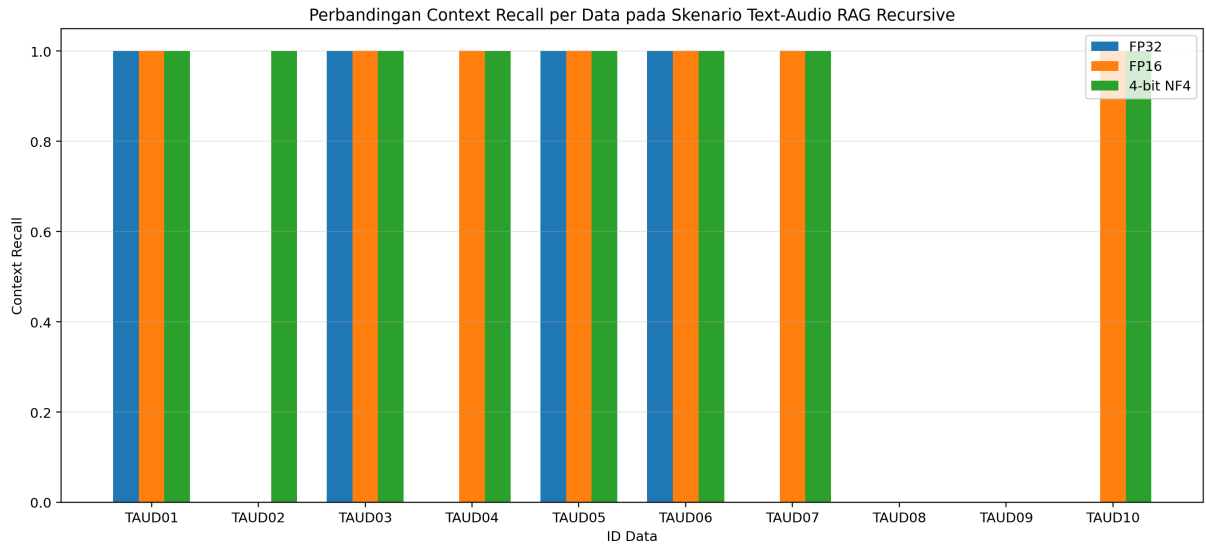
Gambar 4.57: Perbandingan ROUGE-L F1 per data pada skenario *text-audio RAG recursive*.

Gambar 4.57 menunjukkan nilai ROUGE-L F1 pada setiap data uji. Pola yang muncul relatif sejalan dengan BLEU-1. Konfigurasi 4-bit NF4 memperoleh rata-rata ROUGE-L F1 sedikit lebih tinggi dibandingkan FP16, sedangkan FP32 hanya dapat dibandingkan secara terbatas karena jumlah data validnya lebih sedikit.



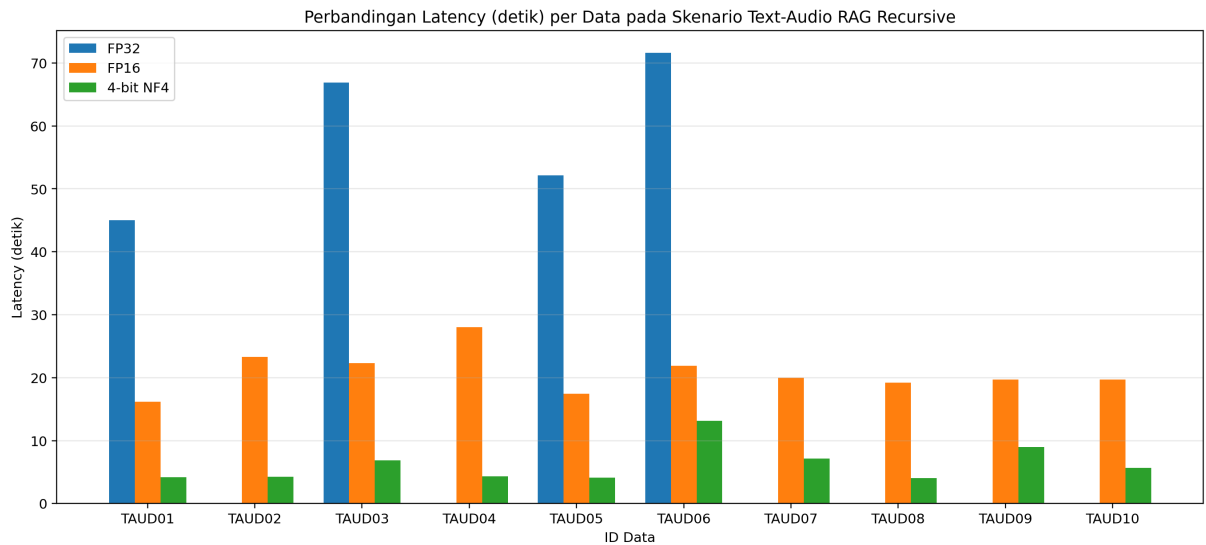
Gambar 4.58: Perbandingan *context precision* per data pada skenario *text-audio RAG recursive*.

Gambar 4.58 menunjukkan nilai *context precision* pada setiap data uji. FP32 memperoleh nilai rata-rata *context precision* tertinggi pada data yang berhasil diproses. Namun, hasil FP32 perlu dibaca secara hati-hati karena hanya dihitung dari data yang valid. Pada konfigurasi yang berhasil memproses seluruh data, FP16 memiliki *context precision* lebih tinggi dibandingkan 4-bit NF4.



Gambar 4.59: Perbandingan *context recall* per data pada skenario *text-audio RAG recursive*.

Gambar 4.59 menunjukkan nilai *context recall* pada setiap data uji. FP32 memperoleh nilai *context recall* rata-rata sebesar 1.0000 pada data yang berhasil diproses. Pada konfigurasi yang berhasil memproses seluruh data, 4-bit NF4 memperoleh *context recall* lebih tinggi dibandingkan FP16.

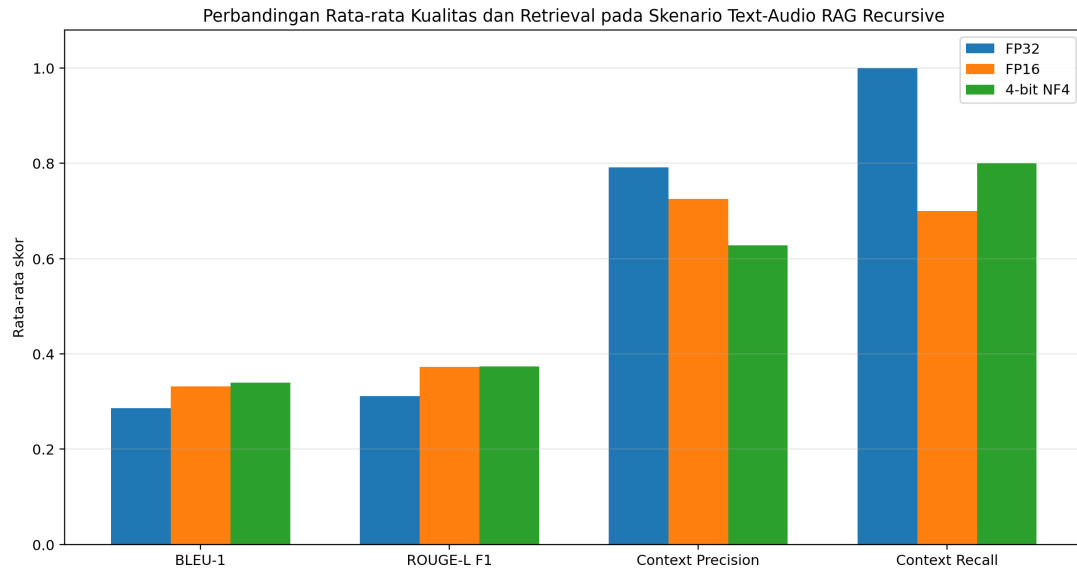


Gambar 4.60: Perbandingan *latency* per data pada skenario *text-audio RAG recursive*.

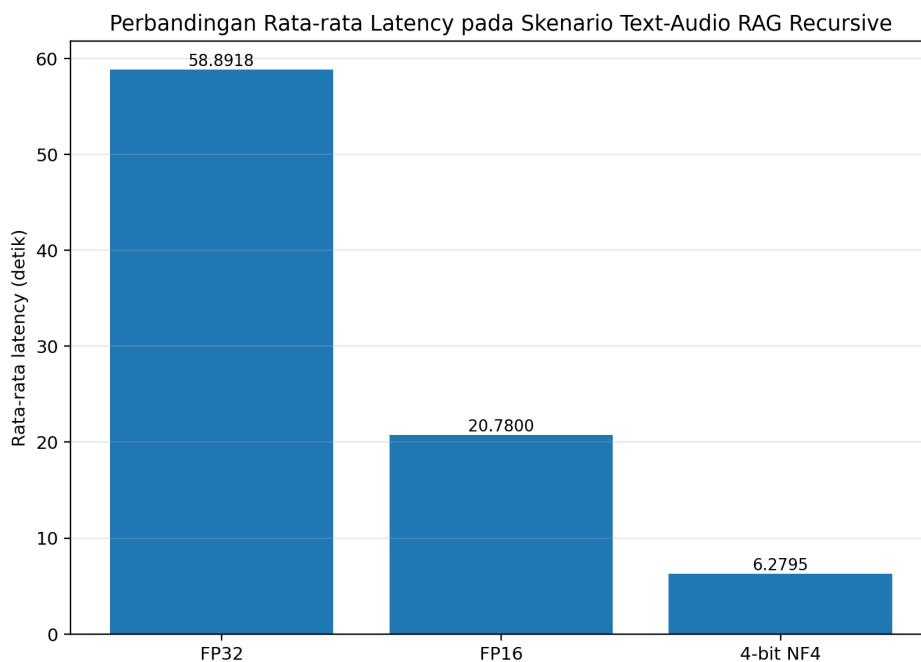
Gambar 4.60 menunjukkan *latency* pada setiap data uji. FP32 memiliki waktu respons paling tinggi dan hanya berhasil pada sebagian data. FP16 juga memiliki *latency* yang relatif tinggi karena proses audio membutuhkan komputasi besar. Sebaliknya, 4-bit NF4 menghasilkan *latency* paling rendah pada seluruh data uji yang berhasil diproses.

Tabel 4.12: Rata-rata hasil pengujian *text-audio RAG recursive*

Konfigurasi	Data Valid	BLEU-1	ROUGE-L F1	CP	CR	Latency (s)
FP32	4/10	0.2863	0.3109	0.7917	1.0000	58.8918
FP16	10/10	0.3322	0.3722	0.7250	0.7000	20.7800
4-bit NF4	10/10	0.3395	0.3732	0.6283	0.8000	6.2795



Gambar 4.61: Perbandingan rata-rata BLEU-1, ROUGE-L F1, *context precision*, dan *context recall* pada skenario *text-audio RAG recursive*.



Gambar 4.62: Perbandingan rata-rata *latency* pada skenario *text-audio RAG recursive*.

Berdasarkan Tabel 4.12, FP32 hanya berhasil memproses 4 dari 10 data. Enam data lainnya mengalami *OutOfMemoryError*, sehingga rata-rata FP32 hanya dihitung dari data yang berhasil diproses. Kondisi ini menunjukkan bahwa penggunaan FP32 pada skenario *text-audio RAG recursive* tidak stabil pada perangkat pengujian dengan keterbatasan VRAM.

Dari sisi kualitas jawaban, 4-bit NF4 memperoleh BLEU-1 tertinggi sebesar 0.3395 dan ROUGE-L F1 tertinggi sebesar 0.3732. FP16 memperoleh BLEU-1 sebesar 0.3322 dan ROUGE-L F1 sebesar 0.3722. Perbedaan antara FP16 dan 4-bit NF4 relatif kecil, tetapi 4-bit NF4 tetap menghasilkan nilai rata-rata yang sedikit lebih tinggi. FP32 memperoleh BLEU-1 sebesar 0.2863 dan ROUGE-L F1 sebesar 0.3109 pada data yang berhasil, tetapi hasil tersebut tidak

dapat dibandingkan secara setara dengan FP16 dan 4-bit NF4 karena jumlah data validnya lebih sedikit.

Dari sisi kualitas *retrieval*, FP32 memperoleh *context precision* sebesar 0.7917 dan *context recall* sebesar 1.0000 pada data yang valid. Namun, interpretasi FP32 perlu dilakukan secara terbatas karena hanya berhasil pada 4 data. Pada konfigurasi yang berhasil memproses seluruh data, FP16 memiliki *context precision* lebih tinggi, yaitu 0.7250, dibandingkan 4-bit NF4 sebesar 0.6283. Sebaliknya, 4-bit NF4 memiliki *context recall* lebih tinggi, yaitu 0.8000, dibandingkan FP16 sebesar 0.7000.

Dari sisi *latency*, 4-bit NF4 memperoleh waktu respons paling rendah, yaitu 6.2795 detik. FP16 membutuhkan 20.7800 detik, sedangkan FP32 membutuhkan 58.8918 detik pada data yang berhasil. Dengan demikian, pada skenario *text-audio RAG recursive*, 4-bit NF4 menjadi konfigurasi yang paling layak dari sisi efisiensi waktu dan keberhasilan pemrosesan, meskipun kualitas *retrieval*-nya perlu dilihat bersama nilai *context precision* dan *context recall*.

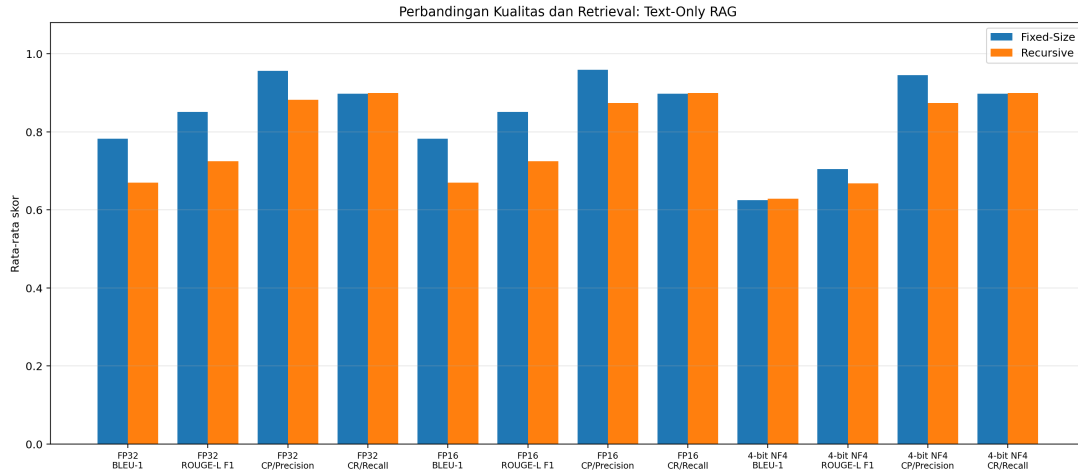
Secara keseluruhan, hasil pengujian dengan konfigurasi *recursive* menunjukkan bahwa konfigurasi terbaik berbeda pada setiap modalitas. Pada *text-only RAG recursive*, FP32 dan FP16 menghasilkan kualitas jawaban yang sama, tetapi FP32 memiliki *latency* paling rendah. Pada *text-image RAG recursive*, FP32 menghasilkan kualitas jawaban dan *latency* terbaik, sementara FP16 dan 4-bit NF4 memiliki *context recall* lebih tinggi. Pada *text-audio RAG recursive*, 4-bit NF4 menjadi konfigurasi paling efisien karena mampu memproses seluruh data dengan *latency* paling rendah. Hasil ini menunjukkan bahwa konfigurasi *recursive* tidak memberikan keuntungan yang seragam pada semua skenario, sehingga interpretasinya perlu dilakukan berdasarkan karakteristik modalitas dan metrik evaluasi yang digunakan.

4.8 Perbandingan Konfigurasi Chunking

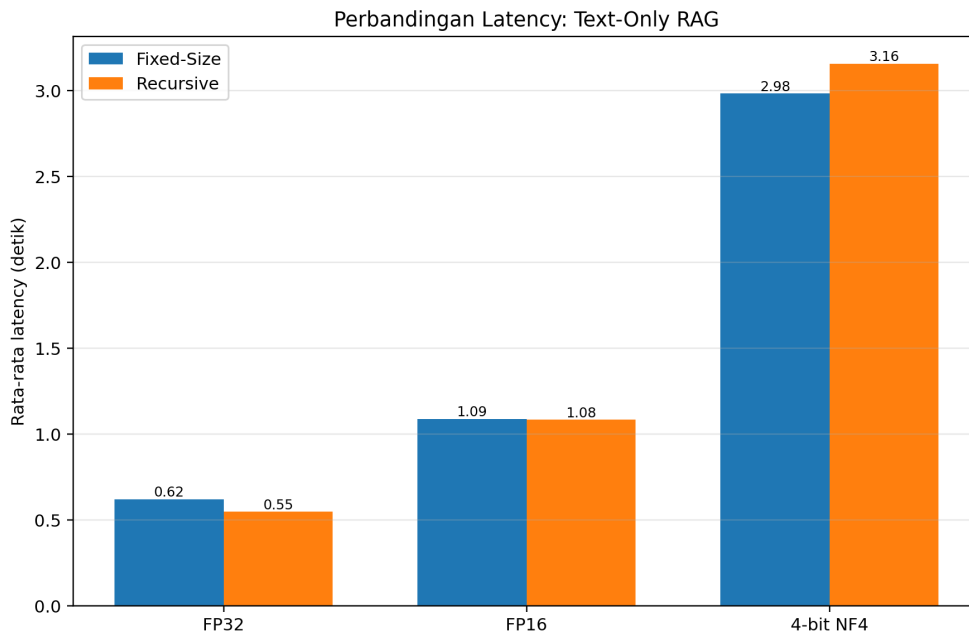
Bagian ini membandingkan konfigurasi *fixed-size* sebagai konfigurasi utama dan *recursive* sebagai konfigurasi pembanding. Perbandingan hanya dilakukan pada skenario berbasis RAG karena skenario *image-only* dan *audio-only* tidak menggunakan proses *chunking* dokumen. Skenario yang dibandingkan meliputi *text-only RAG*, *text-image RAG*, dan *text-audio RAG*. Metrik yang digunakan dalam perbandingan ini mencakup BLEU-1, ROUGE-L F1, *context precision*, *context recall*, dan *latency*.

Perbandingan ini tidak dimaksudkan untuk menyatakan bahwa satu konfigurasi *chunking* selalu lebih unggul dalam seluruh kondisi, tetapi untuk melihat kecenderungan performa pada setiap modalitas. Hal ini penting karena konfigurasi *fixed-size* dan *recursive* memiliki karakteristik berbeda. Konfigurasi *fixed-size* menghasilkan potongan konteks yang lebih panjang dan stabil, sedangkan konfigurasi *recursive* menghasilkan potongan yang lebih granular berdasarkan separator teks. Perbedaan karakteristik tersebut dapat memengaruhi kualitas konteks hasil *retrieval*, kecukupan informasi dalam konteks, serta waktu respons sistem.

4.8.1 Perbandingan Chunking pada Text-Only RAG



Gambar 4.63: Perbandingan kualitas dan *retrieval* antara konfigurasi *fixed-size* dan *recursive* pada skenario *text-only* RAG.



Gambar 4.64: Perbandingan *latency* antara konfigurasi *fixed-size* dan *recursive* pada skenario *text-only* RAG.

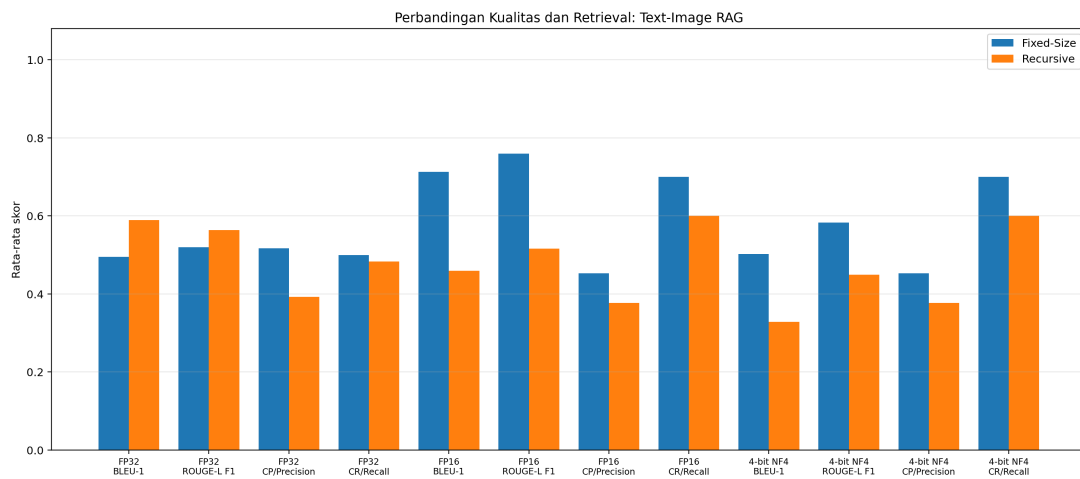
Gambar 4.63 menunjukkan perbandingan kualitas jawaban dan kualitas *retrieval* antara konfigurasi *fixed-size* dan *recursive* pada skenario *text-only* RAG. Pada konfigurasi *fixed-size*, FP32 dan FP16 memperoleh BLEU-1 sebesar 0.7823 dan ROUGE-L F1 sebesar 0.8510. Nilai tersebut lebih tinggi dibandingkan konfigurasi *recursive*, yang memperoleh BLEU-1 sebesar 0.6697 dan ROUGE-L F1 sebesar 0.7244 pada FP32 dan FP16. Pada 4-bit NF4, *fixed-size* memperoleh BLEU-1 sebesar 0.6249 dan ROUGE-L F1 sebesar 0.7042, sedangkan *recursive* memperoleh BLEU-1 sebesar 0.6287 dan ROUGE-L F1 sebesar 0.6679.

Dari sisi kualitas *retrieval*, konfigurasi *fixed-size* memperoleh *context precision* yang lebih tinggi pada seluruh konfigurasi representasi numerik model. Pada FP32, *fixed-size* memperoleh

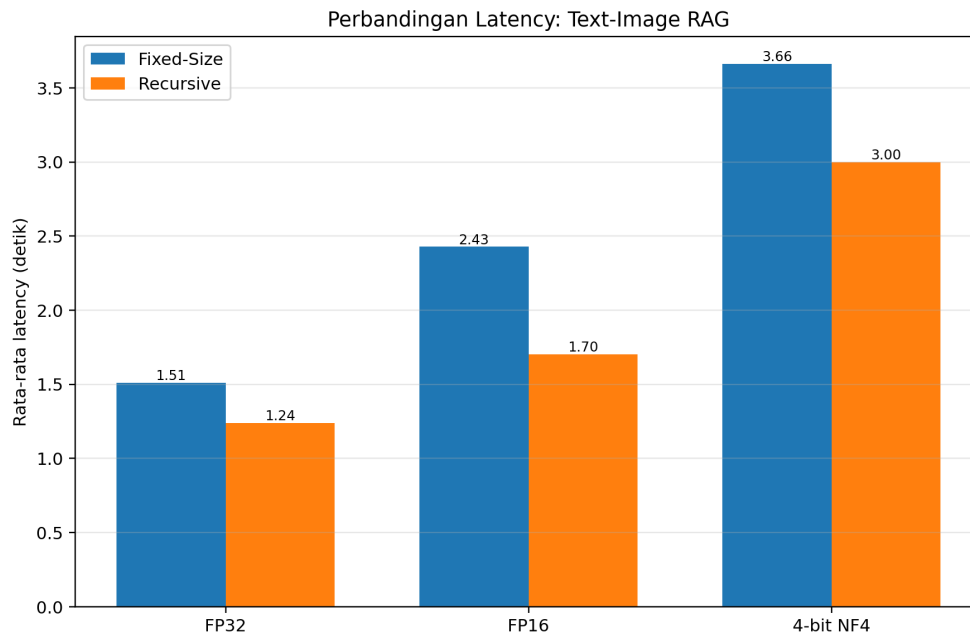
context precision sebesar 0.9560, sedangkan *recursive* memperoleh 0.8822. Pada FP16, *fixed-size* memperoleh 0.9593, sedangkan *recursive* memperoleh 0.8739. Pada 4-bit NF4, *fixed-size* memperoleh 0.9454, sedangkan *recursive* memperoleh 0.8739. Sementara itu, *context recall* pada *recursive* sedikit lebih tinggi, yaitu 0.9000, dibandingkan *fixed-size* sebesar 0.8974 pada seluruh konfigurasi. Perbedaan *context recall* tersebut sangat kecil, sehingga keuntungan utama *recursive* pada skenario ini tidak terlihat signifikan.

Gambar 4.64 menunjukkan perbandingan *latency* pada skenario *text-only RAG*. Pada FP32, *recursive* memperoleh *latency* 0.5469 detik, sedikit lebih rendah dibandingkan *fixed-size* sebesar 0.6202 detik. Pada FP16, *fixed-size* dan *recursive* hampir sama, yaitu 1.0874 detik dan 1.0826 detik. Pada 4-bit NF4, *fixed-size* lebih rendah dengan 2.9835 detik dibandingkan *recursive* sebesar 3.1577 detik. Dengan demikian, pada skenario *text-only RAG*, konfigurasi *fixed-size* tetap lebih kuat dari sisi kualitas jawaban dan *context precision*, sedangkan *recursive* hanya menunjukkan sedikit keuntungan pada *latency* FP32 dan *context recall*.

4.8.2 Perbandingan Chunking pada Text-Image RAG



Gambar 4.65: Perbandingan kualitas dan *retrieval* antara konfigurasi *fixed-size* dan *recursive* pada skenario *text-image RAG*.



Gambar 4.66: Perbandingan *latency* antara konfigurasi *fixed-size* dan *recursive* pada skenario *text-image RAG*.

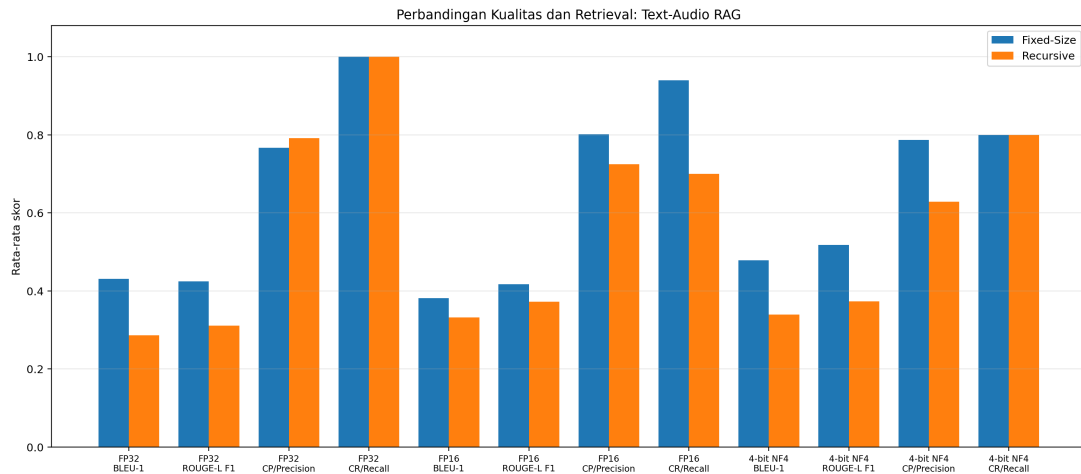
Gambar 4.65 menunjukkan perbandingan kualitas jawaban dan kualitas *retrieval* pada skenario *text-image RAG*. Pada konfigurasi *fixed-size*, FP16 memperoleh kualitas jawaban tertinggi dengan BLEU-1 sebesar 0.7126 dan ROUGE-L F1 sebesar 0.7594. Nilai ini lebih tinggi dibandingkan seluruh hasil pada konfigurasi *recursive*. Pada *recursive*, FP32 menjadi konfigurasi terbaik dengan BLEU-1 sebesar 0.5897 dan ROUGE-L F1 sebesar 0.5638.

Jika dilihat berdasarkan rata-rata lintas konfigurasi representasi numerik model, *fixed-size* memperoleh BLEU-1 sebesar 0.5700 dan ROUGE-L F1 sebesar 0.6209, sedangkan *recursive* memperoleh BLEU-1 sebesar 0.4590 dan ROUGE-L F1 sebesar 0.5096. Hal ini menunjukkan bahwa pada skenario *text-image RAG*, konfigurasi *fixed-size* lebih kuat dari sisi kualitas jawaban akhir.

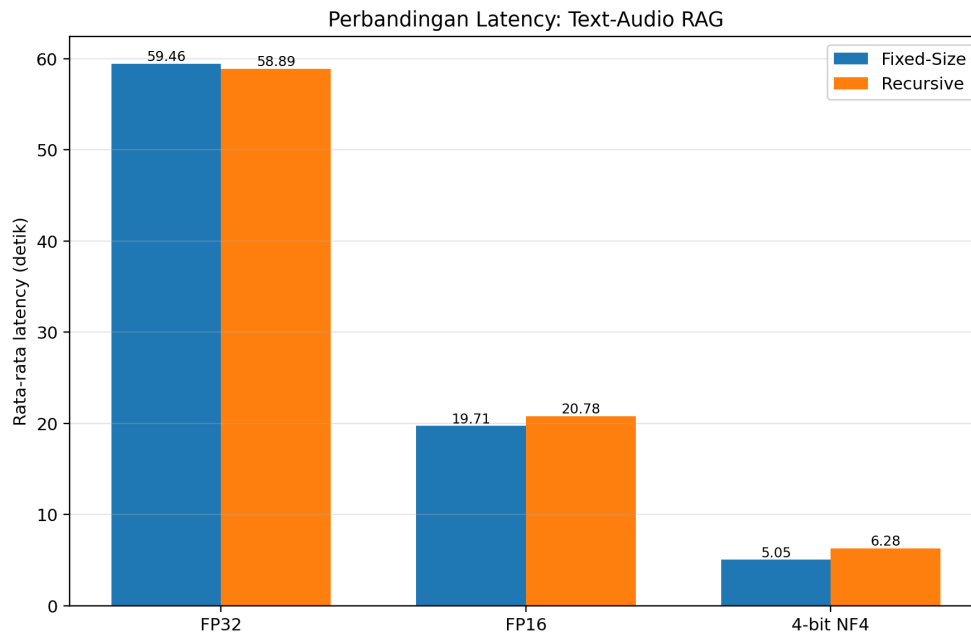
Dari sisi kualitas *retrieval*, *fixed-size* juga memperoleh rata-rata *context precision* dan *context recall* yang lebih tinggi. Rata-rata *context precision fixed-size* adalah 0.4742, sedangkan *recursive* adalah 0.3819. Rata-rata *context recall fixed-size* adalah 0.6333, sedangkan *recursive* adalah 0.5611. Hal ini menunjukkan bahwa pada skenario yang melibatkan citra, potongan konteks yang lebih panjang pada konfigurasi *fixed-size* lebih membantu sistem menyediakan informasi dokumen yang relevan dan cukup untuk mendukung jawaban akhir.

Gambar 4.66 menunjukkan bahwa konfigurasi *recursive* memiliki *latency* lebih rendah pada beberapa konfigurasi. Pada FP32, *recursive* memperoleh 1.2388 detik, lebih rendah dibandingkan *fixed-size* sebesar 1.5115 detik. Pada FP16, *recursive* memperoleh 1.7028 detik, lebih rendah dibandingkan *fixed-size* sebesar 2.4276 detik. Pada 4-bit NF4, *recursive* memperoleh 3.0000 detik, lebih rendah dibandingkan *fixed-size* sebesar 3.6631 detik. Dengan demikian, pada skenario *text-image RAG*, *recursive* lebih baik dari sisi *latency*, tetapi *fixed-size* tetap lebih unggul dari sisi kualitas jawaban dan kualitas *retrieval*.

4.8.3 Perbandingan Chunking pada Text-Audio RAG



Gambar 4.67: Perbandingan kualitas dan *retrieval* antara konfigurasi *fixed-size* dan *recursive* pada skenario *text-audio RAG*.



Gambar 4.68: Perbandingan *latency* antara konfigurasi *fixed-size* dan *recursive* pada skenario *text-audio RAG*.

Gambar 4.67 menunjukkan perbandingan kualitas jawaban dan kualitas *retrieval* pada skenario *text-audio RAG*. Pada skenario ini, FP32 pada kedua konfigurasi *chunking* hanya berhasil memproses 4 dari 10 data karena sebagian data mengalami *OutOfMemoryError*. Oleh karena itu, nilai FP32 perlu dibaca secara hati-hati karena rata-ratanya hanya dihitung dari data yang berhasil diproses.

Secara umum, konfigurasi *fixed-size* menghasilkan kualitas jawaban yang lebih tinggi dibandingkan *recursive*. Rata-rata BLEU-1 lintas konfigurasi representasi numerik model pada *fixed-size* adalah 0.4300, sedangkan pada *recursive* adalah 0.3193. Rata-rata ROUGE-L F1 pada *fixed-size* adalah 0.4533, sedangkan pada *recursive* adalah 0.3521. Pada konfigurasi 4-bit NF4,

fixed-size memperoleh BLEU-1 sebesar 0.4782 dan ROUGE-L F1 sebesar 0.5182, sedangkan *recursive* memperoleh BLEU-1 sebesar 0.3395 dan ROUGE-L F1 sebesar 0.3732. Hal ini menunjukkan bahwa pada skenario audio, konfigurasi *fixed-size* lebih mendukung pembentukan jawaban akhir yang dekat dengan referensi.

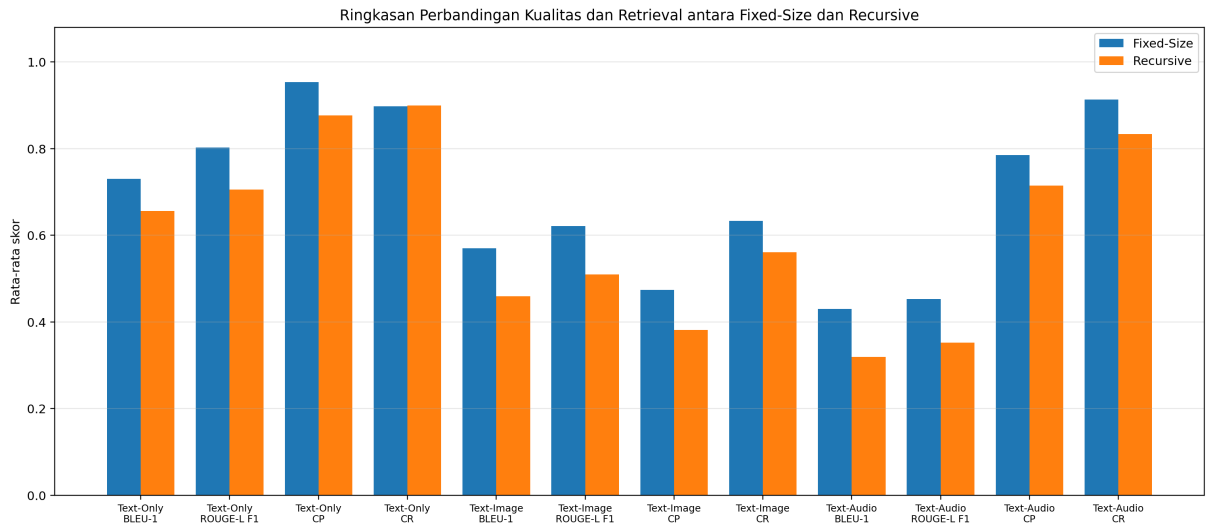
Dari sisi kualitas *retrieval*, *fixed-size* juga memperoleh rata-rata yang lebih tinggi. Rata-rata *context precision fixed-size* adalah 0.7850, sedangkan *recursive* adalah 0.7150. Rata-rata *context recall fixed-size* adalah 0.9133, sedangkan *recursive* adalah 0.8333. Hasil ini menunjukkan bahwa pada skenario *text-audio RAG*, konfigurasi *fixed-size* lebih mampu menyediakan konteks yang presisi dan mencakup informasi penting dibandingkan konfigurasi *recursive*.

Gambar 4.68 menunjukkan bahwa perbedaan *latency* antara *fixed-size* dan *recursive* pada skenario audio tidak sebesar perbedaan kualitas jawaban. Pada FP32, *recursive* memperoleh 58.8918 detik, sedikit lebih rendah dibandingkan *fixed-size* sebesar 59.4643 detik. Namun, keduanya hanya valid pada 4 data. Pada FP16, *fixed-size* memperoleh 19.7073 detik, sedangkan *recursive* memperoleh 20.7800 detik. Pada 4-bit NF4, *fixed-size* memperoleh 5.0453 detik, sedangkan *recursive* memperoleh 6.2795 detik. Dengan demikian, pada skenario *text-audio RAG*, konfigurasi *fixed-size* lebih unggul dari sisi kualitas jawaban, kualitas *retrieval*, dan *latency* pada konfigurasi FP16 dan 4-bit NF4.

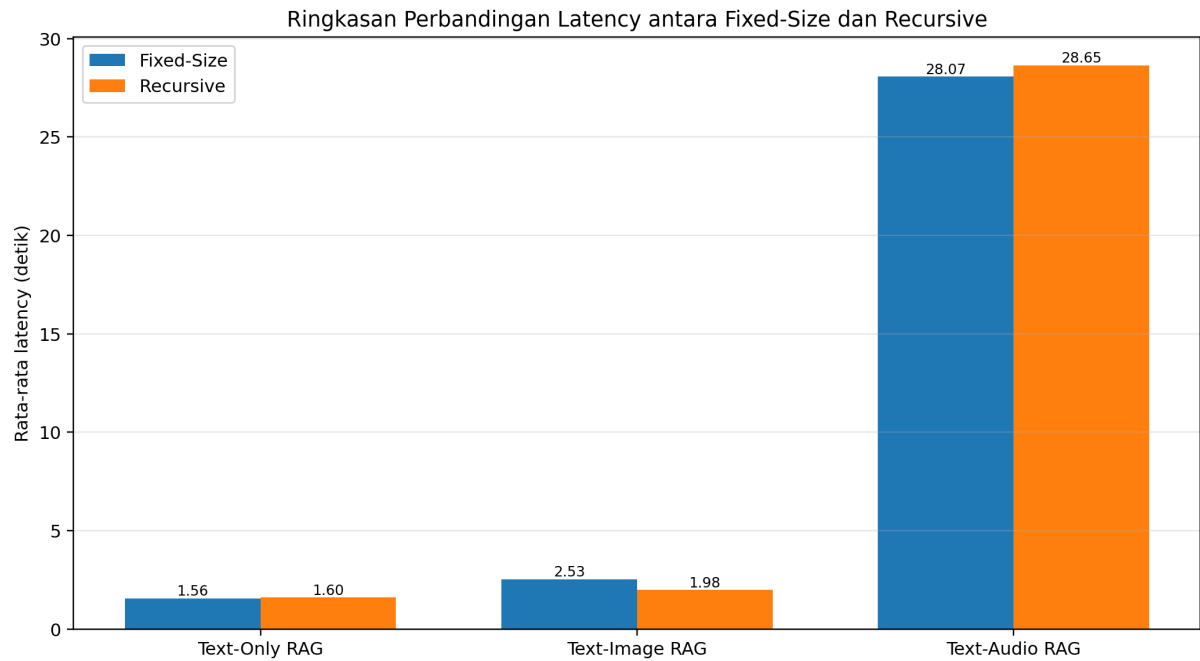
4.8.4 Ringkasan Perbandingan Konfigurasi Chunking

Tabel 4.13: Ringkasan perbandingan konfigurasi chunking pada seluruh skenario RAG

Skenario	Chunking	Konfigurasi	BLEU-1	ROUGE-L F1	CP	CR	Latency (s)
Text-only RAG	Fixed-size	FP32	0.7823	0.8510	0.9560	0.8974	0.6202
Text-only RAG	Fixed-size	FP16	0.7823	0.8510	0.9593	0.8974	1.0874
Text-only RAG	Fixed-size	4-bit NF4	0.6249	0.7042	0.9454	0.8974	2.9835
Text-only RAG	Recursive	FP32	0.6697	0.7244	0.8822	0.9000	0.5469
Text-only RAG	Recursive	FP16	0.6697	0.7244	0.8739	0.9000	1.0826
Text-only RAG	Recursive	4-bit NF4	0.6287	0.6679	0.8739	0.9000	3.1577
Text-image RAG	Fixed-size	FP32	0.4951	0.5198	0.5172	0.5000	1.5115
Text-image RAG	Fixed-size	FP16	0.7126	0.7594	0.4526	0.7000	2.4276
Text-image RAG	Fixed-size	4-bit NF4	0.5024	0.5833	0.4526	0.7000	3.6631
Text-image RAG	Recursive	FP32	0.5897	0.5638	0.3922	0.4833	1.2388
Text-image RAG	Recursive	FP16	0.4594	0.5160	0.3767	0.6000	1.7028
Text-image RAG	Recursive	4-bit NF4	0.3280	0.4490	0.3767	0.6000	3.0000
Text-audio RAG	Fixed-size	FP32	0.4306	0.4246	0.7667	1.0000	59.4643
Text-audio RAG	Fixed-size	FP16	0.3811	0.4170	0.8017	0.9400	19.7073
Text-audio RAG	Fixed-size	4-bit NF4	0.4782	0.5182	0.7867	0.8000	5.0453
Text-audio RAG	Recursive	FP32	0.2863	0.3109	0.7917	1.0000	58.8918
Text-audio RAG	Recursive	FP16	0.3322	0.3722	0.7250	0.7000	20.7800
Text-audio RAG	Recursive	4-bit NF4	0.3395	0.3732	0.6283	0.8000	6.2795



Gambar 4.69: Ringkasan perbandingan kualitas dan *retrieval* antara konfigurasi *fixed-size* dan *recursive*.



Gambar 4.70: Ringkasan perbandingan *latency* antara konfigurasi *fixed-size* dan *recursive*.

Gambar 4.69 menunjukkan ringkasan perbandingan kualitas jawaban dan kualitas *retrieval* antara konfigurasi *fixed-size* dan *recursive*. Secara umum, konfigurasi *fixed-size* lebih unggul pada sebagian besar metrik kualitas. Pada skenario *text-only RAG*, *fixed-size* menghasilkan BLEU-1, ROUGE-L F1, dan *context precision* yang lebih tinggi, sedangkan *recursive* hanya sedikit lebih tinggi pada *context recall*. Pada skenario *text-image RAG*, *fixed-size* menghasilkan rata-rata BLEU-1, ROUGE-L F1, *context precision*, dan *context recall* yang lebih tinggi dibandingkan *recursive*. Pada skenario *text-audio RAG*, *fixed-size* juga lebih unggul pada rata-rata kualitas jawaban dan kualitas *retrieval*.

Gambar 4.70 menunjukkan ringkasan perbandingan *latency*. Pada skenario *text-only RAG*, rata-rata *latency fixed-size* adalah 1.5637 detik, sedangkan *recursive* adalah 1.5957 detik. Perbedaannya relatif kecil, tetapi *fixed-size* tetap sedikit lebih rendah secara rata-rata. Pada skenario

text-image RAG, recursive memiliki rata-rata *latency* lebih rendah, yaitu 1.9805 detik, dibandingkan *fixed-size* sebesar 2.5341 detik. Namun, keuntungan waktu ini disertai penurunan kualitas jawaban dan kualitas *retrieval*. Pada skenario *text-audio RAG, fixed-size* memiliki rata-rata *latency* 28.0723 detik, sedikit lebih rendah dibandingkan *recursive* sebesar 28.6504 detik.

Berdasarkan keseluruhan hasil, konfigurasi *recursive* tidak memberikan keuntungan yang konsisten pada seluruh skenario. Pada *text-only RAG, recursive* hanya sedikit lebih baik pada *context recall* dan *latency* FP32, tetapi kualitas jawaban dan *context precision* lebih rendah dibandingkan *fixed-size*. Pada *text-image RAG, recursive* memberikan keuntungan dari sisi *latency*, tetapi kualitas jawaban dan kualitas *retrieval* secara umum lebih rendah. Pada *text-audio RAG, fixed-size* lebih unggul baik dari sisi kualitas jawaban, kualitas *retrieval*, maupun *latency* pada konfigurasi FP16 dan 4-bit NF4.

Dengan demikian, konfigurasi *fixed-size* tetap layak dipertahankan sebagai konfigurasi utama sistem. Keputusan ini didasarkan pada performa yang lebih stabil pada skenario RAG, terutama dari sisi kualitas jawaban dan kualitas konteks hasil *retrieval*. Konfigurasi *recursive* tetap berguna sebagai pembanding karena menunjukkan bahwa potongan konteks yang lebih granular tidak selalu menghasilkan peningkatan kualitas jawaban. Dalam konteks penelitian ini, konfigurasi *fixed-size* memberikan keseimbangan yang lebih baik antara kualitas jawaban, kualitas *retrieval*, dan *latency*.

4.9 Hasil Profiling Kebutuhan Memori GPU

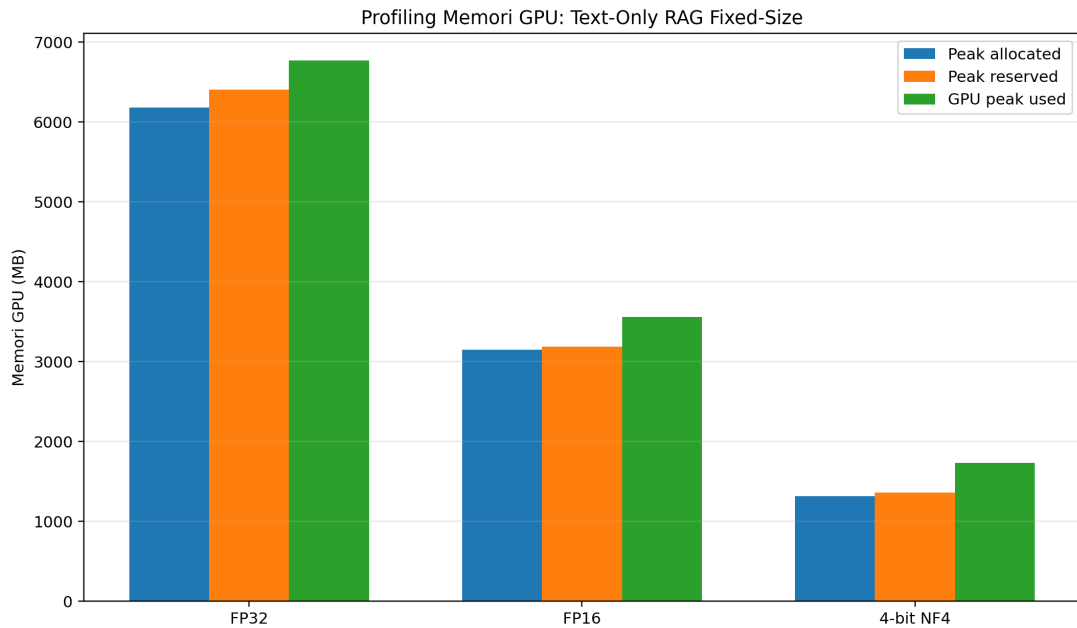
Bagian ini membahas hasil profiling kebutuhan memori GPU pada lima skenario representatif berbasis konfigurasi utama *fixed-size*. Lima skenario tersebut meliputi *text-only RAG fixed-size*, *image-only*, *audio-only*, *text-image RAG fixed-size*, dan *text-audio RAG fixed-size*. Profiling dilakukan untuk membedakan efisiensi memori dari *latency*, sehingga konfigurasi FP32, FP16, dan 4-bit NF4 tidak hanya dibandingkan berdasarkan waktu respons, tetapi juga berdasarkan jejak penggunaan memori GPU.

Nilai *latency profiling* pada bagian ini tidak digunakan untuk menggantikan nilai *latency* evaluasi kualitas pada bagian sebelumnya. *Latency profiling* dihitung dalam alur pengukuran memori GPU, sedangkan *latency* pada pengujian kualitas dihitung pada alur evaluasi masing-masing skenario. Oleh karena itu, perbedaan nilai antara kedua bagian dibaca sebagai konsekuensi perbedaan tujuan pengukuran, bukan sebagai pertentangan hasil.

Metrik profiling yang digunakan meliputi *peak allocated memory*, *peak reserved memory*, puncak penggunaan memori GPU, dan *latency profiling*. *Peak allocated memory* menunjukkan puncak memori yang dialokasikan oleh PyTorch selama proses inferensi, sedangkan *peak reserved memory* menunjukkan puncak memori yang dicadangkan oleh PyTorch. Puncak penggunaan memori GPU menunjukkan penggunaan memori GPU maksimum yang terpantau selama proses profiling. Sementara itu, *latency profiling* dihitung berdasarkan waktu eksekusi profiling pada setiap data uji. Perhitungan rata-rata pada bagian ini hanya menggunakan data yang berhasil diproses. Pada skenario *audio-only* dan *text-audio RAG fixed-size*, konfigurasi FP32 hanya berhasil pada 4 dari 10 data karena sebagian data mengalami *OutOfMemoryError*.

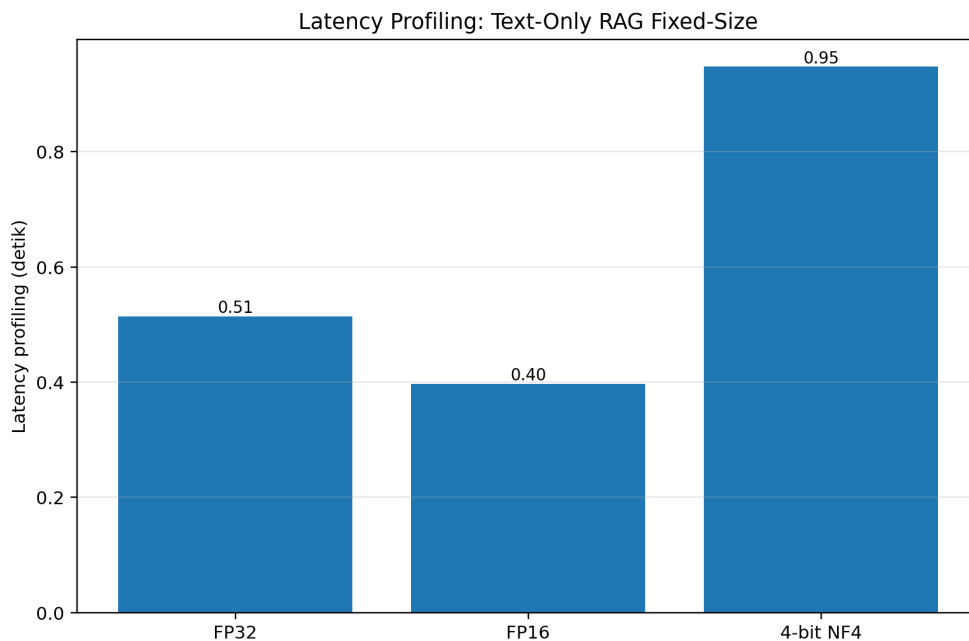
4.9.1 Profiling Text-Only RAG Fixed-Size

Profiling pada skenario *text-only RAG fixed-size* dilakukan untuk melihat kebutuhan memori GPU pada proses question answering berbasis teks dan dokumen hasil *retrieval*. Skenario ini menjadi skenario RAG paling ringan karena tidak melibatkan pemrosesan citra atau audio.



Gambar 4.71: Profiling kebutuhan memori GPU pada skenario *text-only RAG fixed-size*.

Gambar 4.71 menunjukkan bahwa kebutuhan memori GPU pada skenario *text-only RAG fixed-size* menurun seiring penurunan presisi representasi numerik model. FP32 memiliki rata-rata *GPU peak used* sebesar 6771.0 MB, FP16 sebesar 3557.0 MB, dan 4-bit NF4 sebesar 1725.8 MB. Pola yang sama juga terlihat pada *peak allocated memory* dan *peak reserved memory*. Hal ini menunjukkan bahwa pada skenario teks, 4-bit NF4 memberikan penghematan memori GPU yang paling besar.



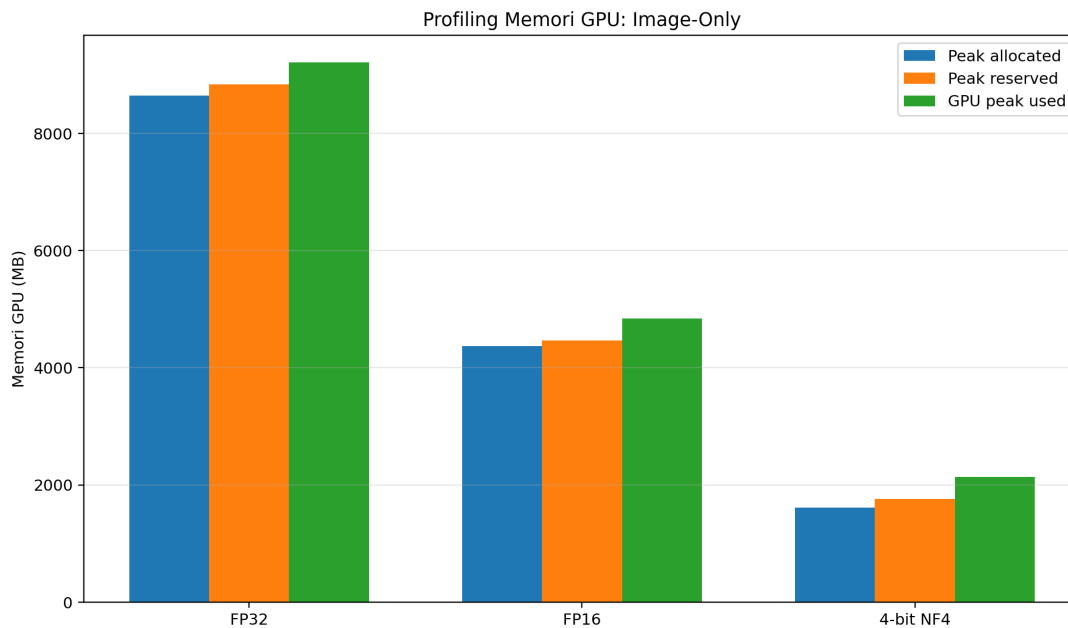
Gambar 4.72: Perbandingan *latency profiling* pada skenario *text-only RAG fixed-size*.

Gambar 4.72 menunjukkan bahwa *latency profiling* terendah pada skenario ini diperoleh oleh FP16, yaitu 0.3973 detik. FP32 memperoleh 0.5143 detik, sedangkan 4-bit NF4 memperoleh 0.9475 detik. Hasil ini menunjukkan bahwa meskipun 4-bit NF4 paling hemat memori,

konfigurasi tersebut tidak menghasilkan *latency* paling rendah pada skenario teks. Dengan demikian, penghematan memori tidak selalu berbanding lurus dengan percepatan waktu respons.

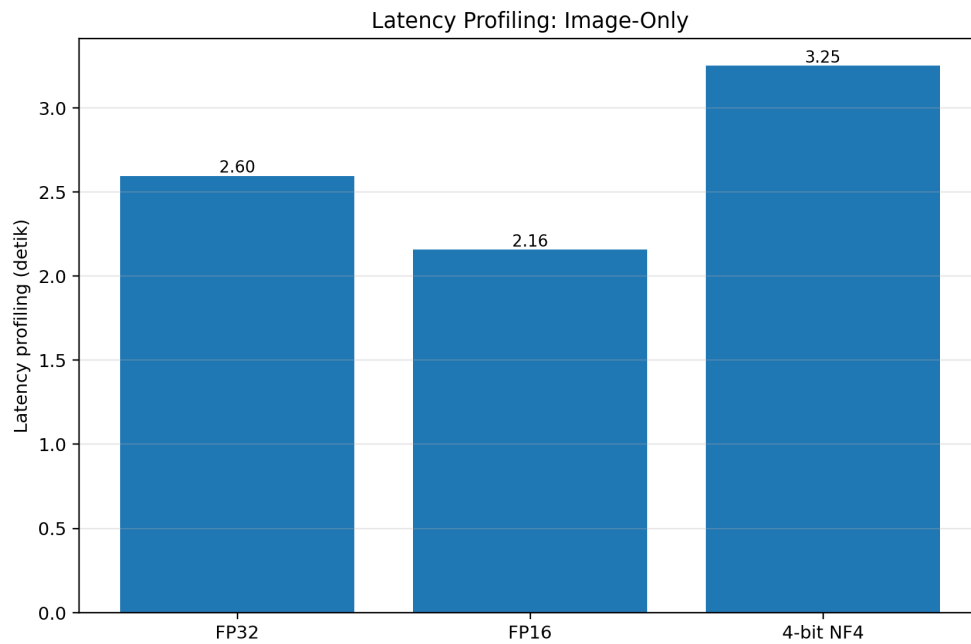
4.9.2 Profiling Image-Only

Profiling pada skenario *image-only* dilakukan untuk melihat kebutuhan memori GPU pada pemrosesan citra tanpa proses *retrieval* dokumen. Skenario ini digunakan untuk mengamati jejak memori model visual ketika menghasilkan keluaran berdasarkan masukan citra.



Gambar 4.73: Profiling kebutuhan memori GPU pada skenario *image-only*.

Gambar 4.73 menunjukkan bahwa FP32 memiliki kebutuhan memori GPU paling tinggi pada skenario *image-only*. Rata-rata *GPU peak used* pada FP32 adalah 9218.0 MB, sedangkan FP16 sebesar 4845.8 MB dan 4-bit NF4 sebesar 2138.0 MB. Hasil ini menunjukkan bahwa 4-bit NF4 memberikan pengurangan jejak memori GPU yang signifikan dibandingkan FP32 dan FP16.

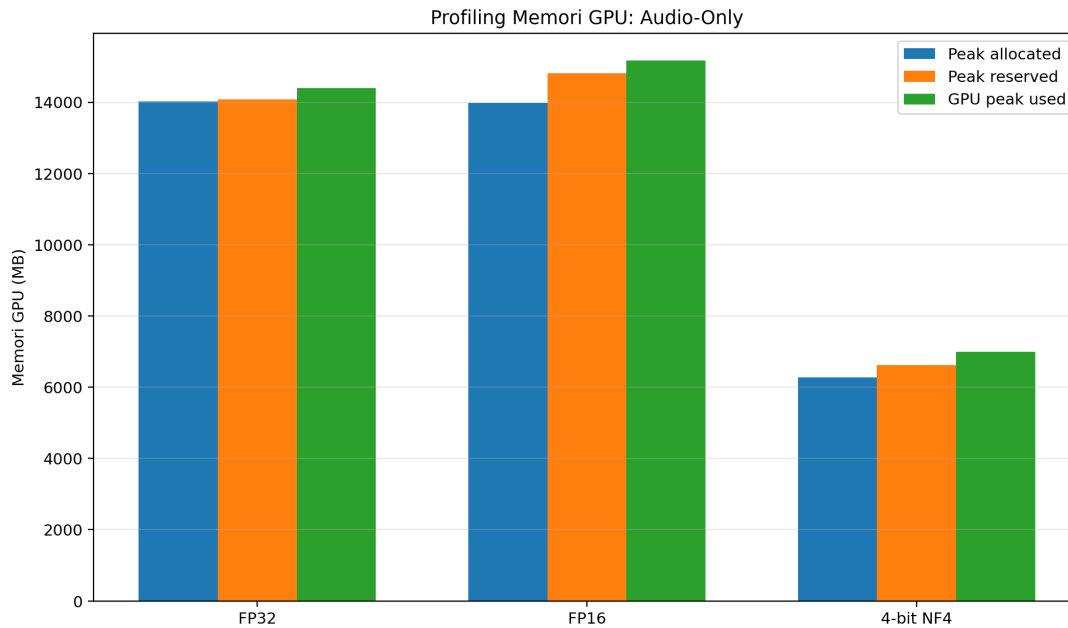


Gambar 4.74: Perbandingan *latency profiling* pada skenario *image-only*.

Gambar 4.74 menunjukkan bahwa FP16 memperoleh *latency profiling* terendah pada skenario *image-only*, yaitu 2.1569 detik. FP32 memperoleh 2.5962 detik, sedangkan 4-bit NF4 memperoleh 3.2506 detik. Hasil ini menunjukkan bahwa pada pemrosesan citra, 4-bit NF4 memang paling hemat memori, tetapi tidak menjadi konfigurasi tercepat. Hal ini menguatkan bahwa konfigurasi representasi numerik dengan jumlah bit lebih rendah dapat menurunkan kebutuhan memori, tetapi tetap memiliki overhead komputasi yang dapat meningkatkan *latency*.

4.9.3 Profiling Audio-Only

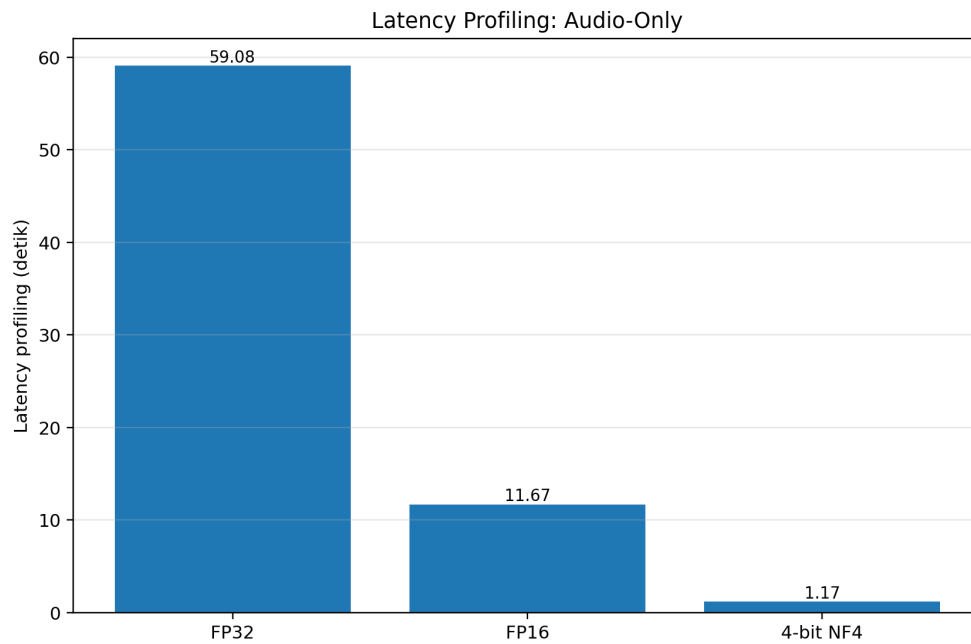
Profiling pada skenario *audio-only* dilakukan untuk melihat kebutuhan memori GPU pada pemrosesan audio tanpa proses *retrieval* dokumen. Skenario ini memiliki kebutuhan komputasi lebih besar dibandingkan *text-only* dan *image-only* karena model audio yang digunakan memiliki ukuran lebih besar dan memerlukan proses pemahaman sinyal audio.



Gambar 4.75: Profiling kebutuhan memori GPU pada skenario *audio-only*.

Gambar 4.75 menunjukkan bahwa skenario *audio-only* memiliki kebutuhan memori GPU yang tinggi. FP32 hanya berhasil memproses 4 dari 10 data karena sebagian data mengalami *OutOfMemoryError*. Pada data yang berhasil, FP32 memiliki rata-rata *GPU peak used* sebesar 14392.5 MB. FP16 berhasil memproses seluruh data dengan rata-rata *GPU peak used* sebesar 15175.4 MB, sedangkan 4-bit NF4 berhasil memproses seluruh data dengan rata-rata *GPU peak used* sebesar 6993.6 MB.

Nilai FP32 pada skenario ini perlu dibaca secara hati-hati karena hanya dihitung dari data yang berhasil. Kegagalan pada 6 data menunjukkan bahwa FP32 tidak stabil untuk skenario audio pada perangkat pengujian. Sebaliknya, 4-bit NF4 mampu menurunkan kebutuhan memori GPU secara signifikan dan tetap berhasil memproses seluruh data uji.

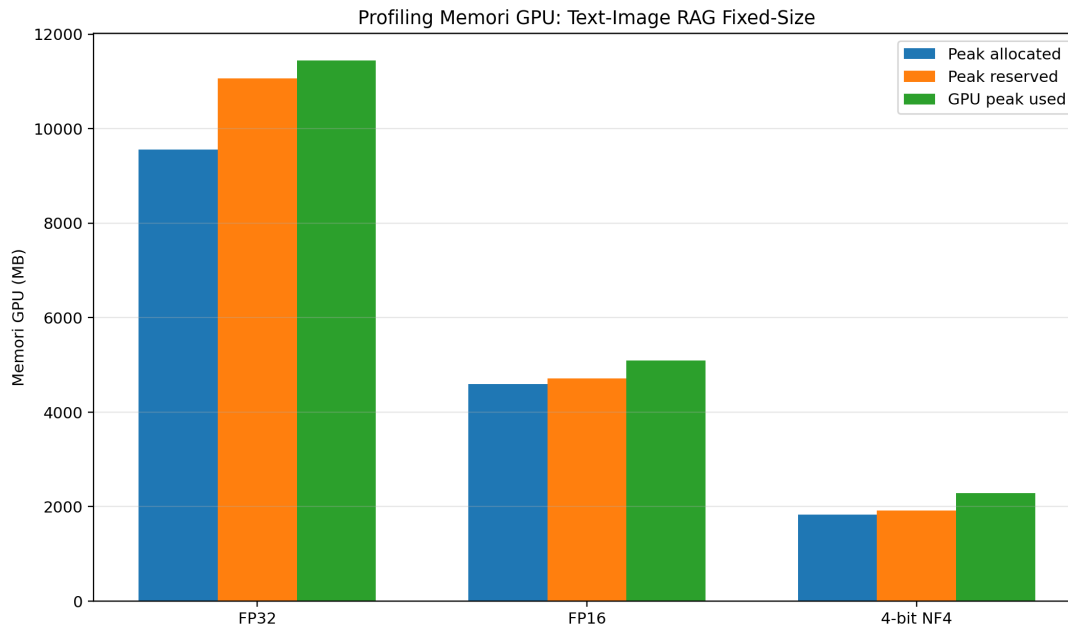


Gambar 4.76: Perbandingan *latency profiling* pada skenario *audio-only*.

Gambar 4.76 menunjukkan bahwa 4-bit NF4 memperoleh *latency profiling* paling rendah pada skenario *audio-only*, yaitu 1.1678 detik. FP16 memperoleh 11.6730 detik, sedangkan FP32 memperoleh 59.0846 detik pada data yang berhasil. Hasil ini menunjukkan bahwa pada skenario audio, 4-bit NF4 memberikan keuntungan ganda, yaitu pengurangan kebutuhan memori GPU dan penurunan *latency*. Berbeda dari skenario teks dan citra, keuntungan 4-bit NF4 pada audio terlihat lebih jelas karena ukuran dan kebutuhan komputasi model audio lebih besar.

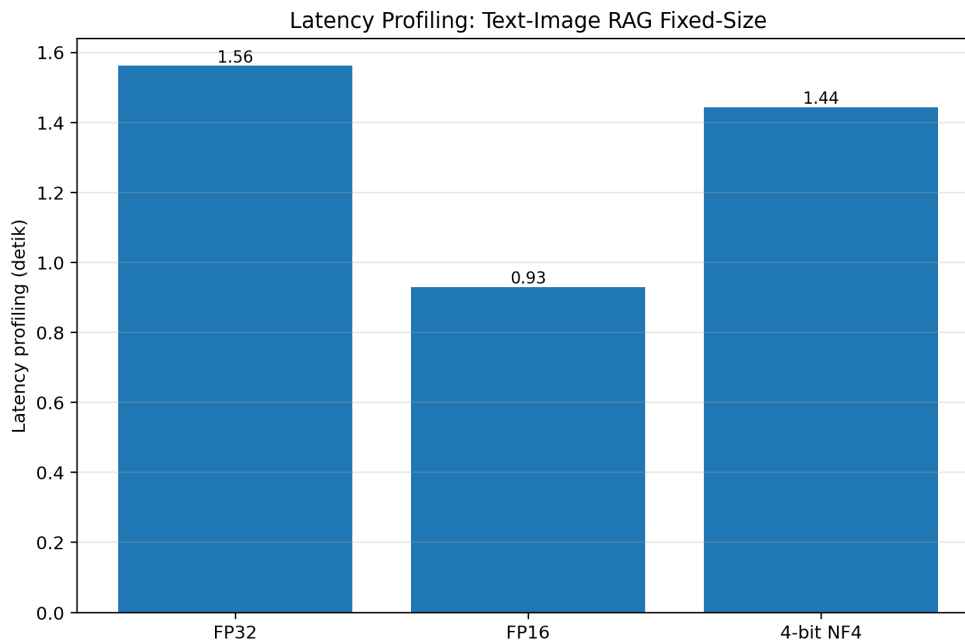
4.9.4 Profiling Text-Image RAG Fixed-Size

Profiling pada skenario *text-image RAG fixed-size* dilakukan untuk melihat kebutuhan memori GPU ketika sistem menggabungkan pemrosesan citra, pertanyaan teks, dan konteks dokumen hasil *retrieval*. Skenario ini lebih kompleks dibandingkan *image-only* karena jawaban akhir dibentuk melalui integrasi konteks visual dan konteks tekstual.



Gambar 4.77: Profiling kebutuhan memori GPU pada skenario *text-image RAG fixed-size*.

Gambar 4.77 menunjukkan bahwa FP32 memiliki kebutuhan memori GPU paling tinggi pada skenario *text-image RAG fixed-size*. Rata-rata *GPU peak used* pada FP32 adalah 11445.6 MB, sedangkan FP16 sebesar 5090.0 MB dan 4-bit NF4 sebesar 2288.0 MB. Penurunan memori dari FP32 ke FP16 dan 4-bit NF4 menunjukkan bahwa konfigurasi representasi numerik yang lebih rendah efektif mengurangi jejak memori pada skenario multimodal berbasis citra.



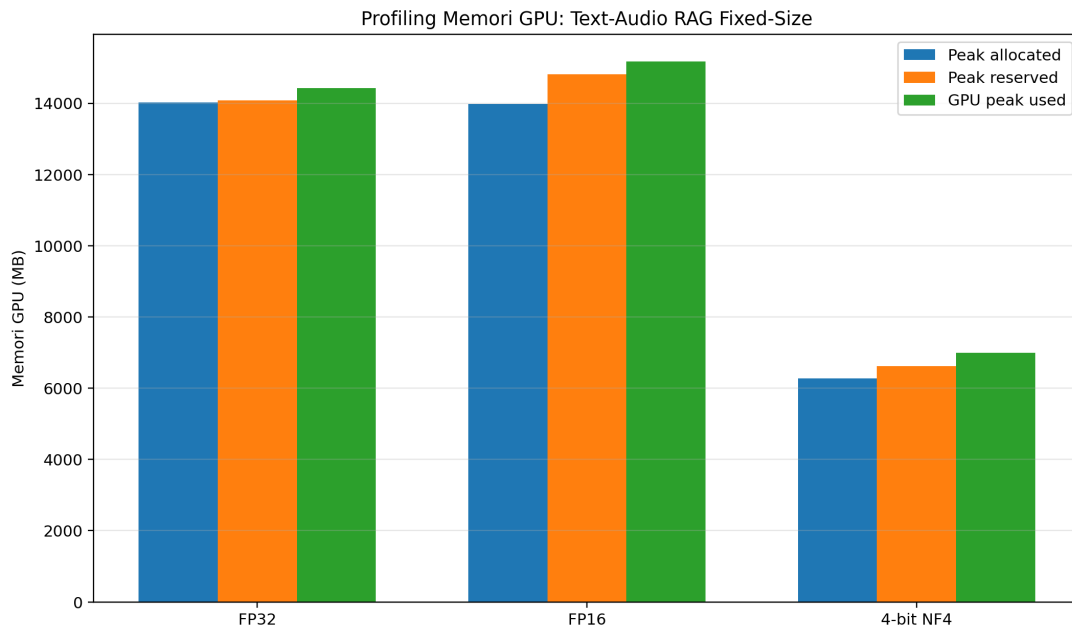
Gambar 4.78: Perbandingan *latency profiling* pada skenario *text-image RAG fixed-size*.

Gambar 4.78 menunjukkan bahwa FP16 memperoleh *latency profiling* terendah, yaitu 0.9293 detik. FP32 memperoleh 1.5616 detik, sedangkan 4-bit NF4 memperoleh 1.4433 detik. Hasil ini menunjukkan bahwa pada skenario *text-image RAG fixed-size*, 4-bit NF4 lebih hemat memori dibandingkan FP16, tetapi FP16 tetap lebih cepat dari sisi *latency*. Dengan demikian,

konfigurasi 4-bit NF4 lebih unggul dari sisi memori, sedangkan FP16 lebih unggul dari sisi waktu respons.

4.9.5 Profiling Text-Audio RAG Fixed-Size

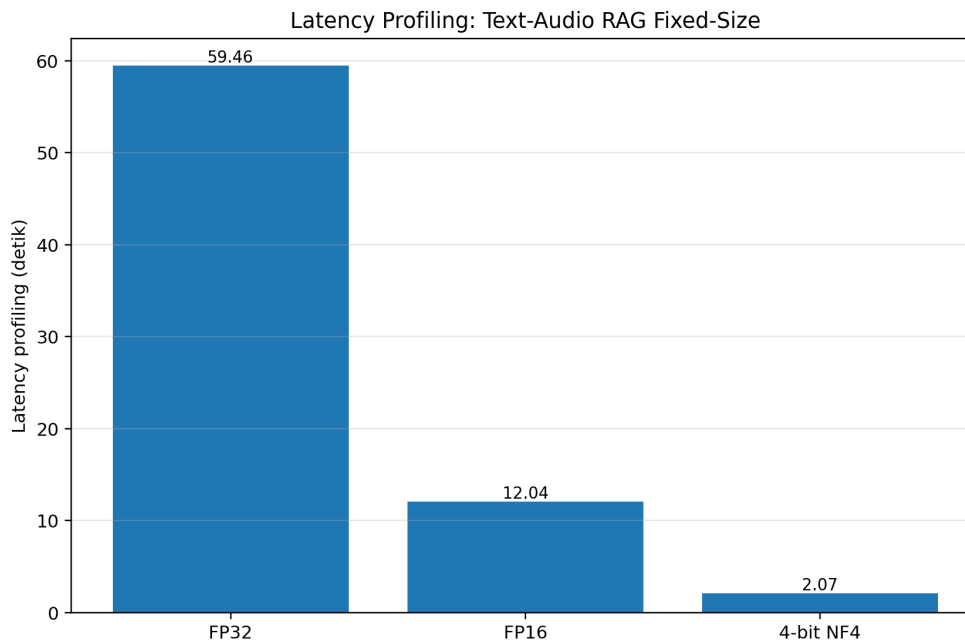
Profiling pada skenario *text-audio RAG fixed-size* dilakukan untuk melihat kebutuhan memori GPU ketika sistem memproses audio, pertanyaan teks, dan konteks dokumen hasil *retrieval*. Skenario ini merupakan salah satu skenario paling berat karena melibatkan model audio dan model teks dalam satu alur pemrosesan.



Gambar 4.79: Profiling kebutuhan memori GPU pada skenario *text-audio RAG fixed-size*.

Gambar 4.79 menunjukkan bahwa skenario *text-audio RAG fixed-size* membutuhkan memori GPU yang tinggi pada FP32 dan FP16. FP32 hanya berhasil memproses 4 dari 10 data karena sebagian data mengalami *OutOfMemoryError*. Pada data yang berhasil, FP32 memiliki rata-rata *GPU peak used* sebesar 14424.5 MB. FP16 berhasil memproses seluruh data dengan rata-rata *GPU peak used* sebesar 15175.4 MB, sedangkan 4-bit NF4 berhasil memproses seluruh data dengan rata-rata *GPU peak used* sebesar 6993.6 MB.

Seperti pada skenario *audio-only*, nilai FP32 pada skenario ini tidak dapat dibandingkan secara penuh dengan FP16 dan 4-bit NF4 karena hanya dihitung dari data yang berhasil. Namun, kegagalan FP32 pada sebagian data menunjukkan bahwa konfigurasi tersebut kurang stabil untuk skenario berbasis audio pada perangkat pengujian. Sebaliknya, 4-bit NF4 mampu menurunkan kebutuhan memori GPU dan tetap berhasil memproses seluruh data uji.



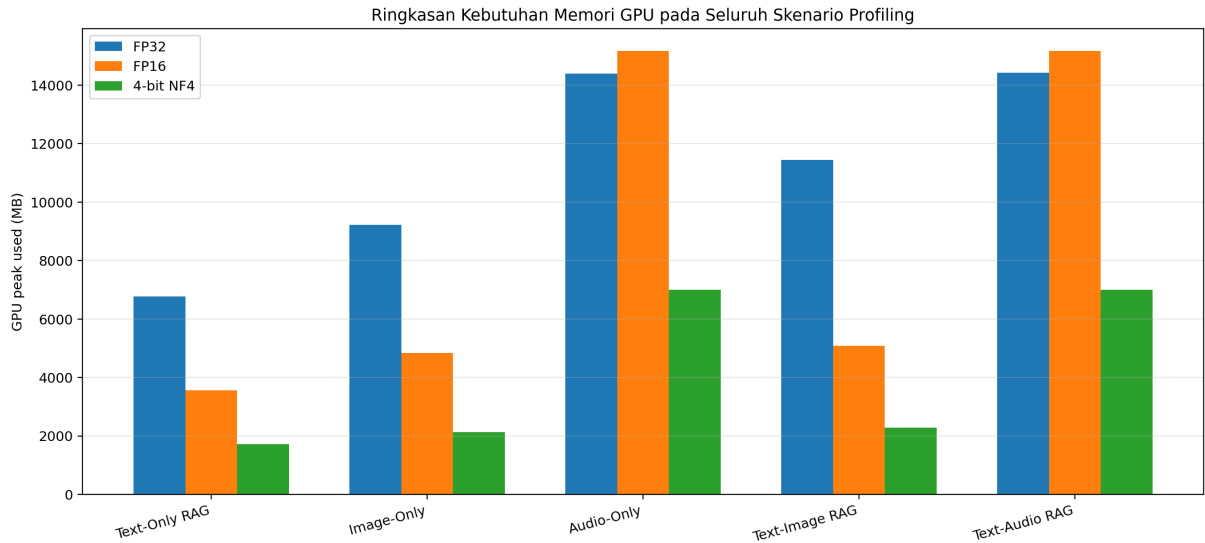
Gambar 4.80: Perbandingan *latency profiling* pada skenario *text-audio RAG fixed-size*.

Gambar 4.80 menunjukkan bahwa 4-bit NF4 memperoleh *latency profiling* paling rendah pada skenario *text-audio RAG fixed-size*, yaitu 2.0724 detik. FP16 memperoleh 12.0362 detik, sedangkan FP32 memperoleh 59.4587 detik pada data yang berhasil. Hasil ini menunjukkan bahwa pada skenario *text-audio RAG fixed-size*, 4-bit NF4 memberikan keuntungan paling jelas karena mampu menurunkan kebutuhan memori GPU sekaligus menurunkan waktu respons.

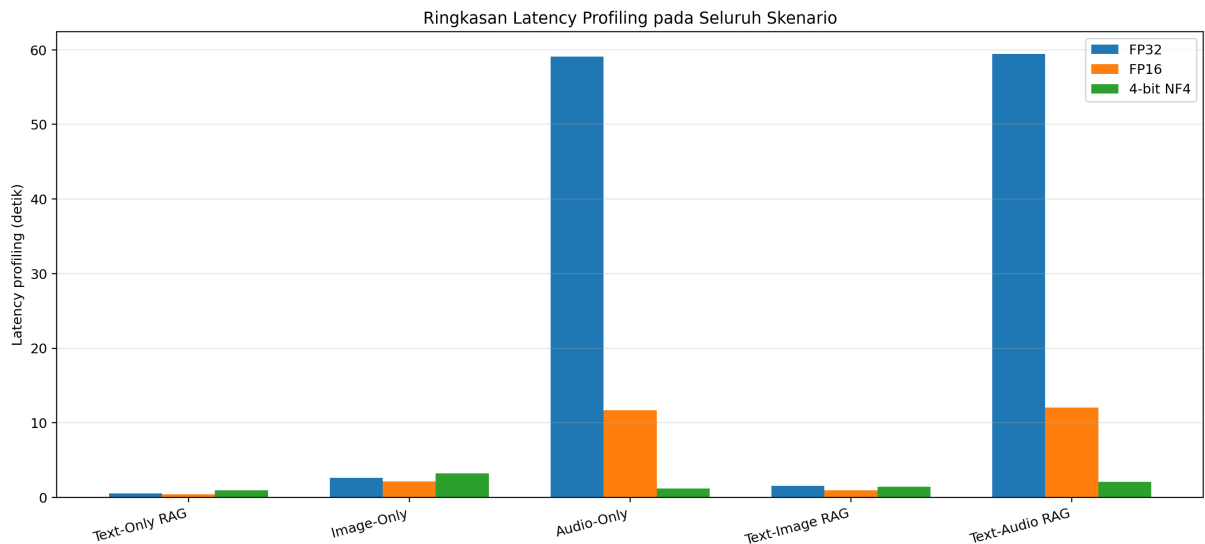
4.9.6 Ringkasan Profiling Memori GPU

Tabel 4.14: Ringkasan hasil profiling kebutuhan memori GPU

Skenario	Konfigurasi Data Valid	Peak Allocated (MB)	Peak Reserved (MB)	GPU Peak Used (MB)	Latency (s)
Text-only RAG fixed-size	FP32 10/10	6181.3364	6401.0000	6771.0000	0.5143
Text-only RAG fixed-size	FP16 10/10	3144.3098	3187.0000	3557.0000	0.3973
Text-only RAG fixed-size	4-bit NF4 10/10	1312.8386	1355.8000	1725.8000	0.9475
Image-only	FP32 10/10	8652.4743	8840.0000	9218.0000	2.5962
Image-only	FP16 10/10	4367.1564	4467.8000	4845.8000	2.1569
Image-only	4-bit NF4 10/10	1609.1466	1760.0000	2138.0000	3.2506
Audio-only	FP32 4/10	14014.9812	14084.0000	14392.5000	59.0846
Audio-only	FP16 10/10	13985.3247	14806.8000	15175.4000	11.6730
Audio-only	4-bit NF4 10/10	6272.7798	6615.6000	6993.6000	1.1678
Text-image RAG fixed-size	FP32 10/10	9554.9619	11067.6000	11445.6000	1.5616
Text-image RAG fixed-size	FP16 10/10	4589.7122	4712.0000	5090.0000	0.9293
Text-image RAG fixed-size	4-bit NF4 10/10	1832.2750	1910.0000	2288.0000	1.4433
Text-audio RAG fixed-size	FP32 4/10	14014.9812	14084.0000	14424.5000	59.4587
Text-audio RAG fixed-size	FP16 10/10	13985.3247	14806.8000	15175.4000	12.0362
Text-audio RAG fixed-size	4-bit NF4 10/10	6272.7798	6615.6000	6993.6000	2.0724



Gambar 4.81: Ringkasan kebutuhan memori GPU pada seluruh skenario profiling.



Gambar 4.82: Ringkasan *latency* pada seluruh skenario profiling.

Gambar 4.81 menunjukkan ringkasan kebutuhan memori GPU pada seluruh skenario profiling. Secara umum, 4-bit NF4 konsisten menghasilkan penggunaan memori GPU paling rendah pada seluruh skenario yang berhasil diproses. Pada skenario *text-only RAG fixed-size*, 4-bit NF4 menurunkan *GPU peak used* dari 6771.0 MB pada FP32 menjadi 1725.8 MB. Pada *image-only*, 4-bit NF4 menurunkan penggunaan memori dari 9218.0 MB menjadi 2138.0 MB. Pada *text-image RAG fixed-size*, 4-bit NF4 menurunkan penggunaan memori dari 11445.6 MB menjadi 2288.0 MB.

Pada skenario berbasis audio, kebutuhan memori jauh lebih tinggi. FP32 tidak berhasil memproses seluruh data pada *audio-only* dan *text-audio RAG fixed-size*, sedangkan FP16 dan 4-bit NF4 berhasil memproses seluruh data. Pada *audio-only*, 4-bit NF4 menggunakan rata-rata *GPU peak used* sebesar 6993.6 MB, jauh lebih rendah dibandingkan FP16 sebesar 15175.4 MB. Pada *text-audio RAG fixed-size*, 4-bit NF4 juga menggunakan 6993.6 MB, sedangkan FP16 menggunakan 15175.4 MB. Hasil ini menunjukkan bahwa 4-bit NF4 memberikan penghematan memori paling jelas pada skenario yang melibatkan model audio.

Gambar 4.82 menunjukkan bahwa penghematan memori tidak selalu diikuti oleh penurunan *latency*. Pada skenario *text-only RAG fixed-size*, 4-bit NF4 memiliki memori paling rendah, tetapi *latency*-nya lebih tinggi dibandingkan FP32 dan FP16. Pada skenario *image-only*, 4-bit NF4 juga paling hemat memori, tetapi *latency*-nya lebih tinggi dibandingkan FP32 dan FP16. Pada skenario *text-image RAG fixed-size*, 4-bit NF4 lebih hemat memori dibandingkan FP16, tetapi FP16 memiliki *latency* lebih rendah.

Sebaliknya, pada skenario berbasis audio, 4-bit NF4 memberikan keuntungan yang lebih kuat. Pada *audio-only*, 4-bit NF4 memperoleh *latency* 1.1678 detik, lebih rendah dibandingkan FP16 sebesar 11.6730 detik dan FP32 sebesar 59.0846 detik pada data yang berhasil. Pada *text-audio RAG fixed-size*, 4-bit NF4 memperoleh *latency* 2.0724 detik, lebih rendah dibandingkan FP16 sebesar 12.0362 detik dan FP32 sebesar 59.4587 detik pada data yang berhasil. Dengan demikian, 4-bit NF4 paling efektif pada skenario audio karena mampu mengurangi kebutuhan memori GPU sekaligus menurunkan waktu respons.

Berdasarkan hasil profiling, konfigurasi 4-bit NF4 terbukti memberikan penghematan memori GPU yang konsisten pada seluruh skenario. Namun, penghematan tersebut tidak selalu menghasilkan *latency* yang lebih rendah. Pada skenario teks dan citra, FP16 sering menjadi konfigurasi dengan *latency* lebih rendah meskipun penggunaan memorinya lebih besar dibandingkan 4-bit NF4. Pada skenario audio, 4-bit NF4 menjadi konfigurasi paling layak karena mampu mengatasi keterbatasan memori, berhasil memproses seluruh data, dan menghasilkan *latency* paling rendah. Oleh karena itu, hasil profiling ini memperkuat bahwa efisiensi memori dan kecepatan respons perlu dianalisis sebagai dua aspek yang berbeda dalam pemilihan konfigurasi representasi numerik model.

4.10 Perbandingan Keseluruhan Konfigurasi Representasi Numerik Model

Bagian ini membandingkan konfigurasi FP32, FP16, dan 4-bit NF4 pada pengujian utama. Perbandingan dilakukan untuk melihat pengaruh konfigurasi representasi numerik model terhadap kualitas keluaran, kualitas *retrieval*, *latency*, dan kebutuhan memori GPU. Perbandingan ini mencakup pengujian non-RAG multimodal, pengujian RAG dengan konfigurasi *fixed-size*, pengujian RAG dengan konfigurasi *recursive*, serta hasil profiling kebutuhan memori GPU. Hasil INT8 dibahas secara terpisah pada bagian pengujian tambahan karena ruang lingkupnya difokuskan pada *latency* konfigurasi *fixed-size* dan tidak menghitung ulang CP maupun CR.

Analisis pada bagian ini tidak diarahkan untuk memilih satu konfigurasi terbaik secara mutlak. Setiap konfigurasi dibaca berdasarkan perannya. FP32 digunakan sebagai konfigurasi representasi numerik penuh, FP16 digunakan sebagai konfigurasi presisi menengah yang umum digunakan untuk inferensi GPU, sedangkan 4-bit NF4 digunakan untuk menurunkan kebutuhan memori model. Oleh karena itu, hasil akhir dianalisis berdasarkan kesesuaian konfigurasi terhadap karakteristik skenario.

Tabel 4.15: Ringkasan hasil pengujian non-RAG multimodal berdasarkan konfigurasi representasi numerik model

Skenario	Konfigurasi	Data Valid	Cosine Similarity	Latency (s)
Image-only	FP32	10/10	0.7175	2.4671
Image-only	FP16	10/10	0.7139	5.8399
Image-only	4-bit NF4	10/10	0.6782	10.3884
Audio-only	FP32	4/10	0.5989	58.1765
Audio-only	FP16	10/10	0.6422	17.2661
Audio-only	4-bit NF4	10/10	0.6441	2.5003

Tabel 4.15 menunjukkan bahwa pengaruh konfigurasi representasi numerik berbeda pada setiap modalitas. Pada skenario *image-only*, FP32 memperoleh *cosine similarity* tertinggi sebesar 0.7175 dan *latency* terendah sebesar 2.4671 detik. FP16 memperoleh kualitas yang sangat dekat, yaitu 0.7139, tetapi *latency*-nya lebih tinggi sebesar 5.8399 detik. Sementara itu, 4-bit NF4 memperoleh *cosine similarity* sebesar 0.6782 dan *latency* sebesar 10.3884 detik. Hasil ini menunjukkan bahwa pada pemahaman citra secara mandiri, konfigurasi dengan presisi lebih rendah tidak selalu memberikan keuntungan waktu respons.

Pada skenario *audio-only*, pola yang muncul berbeda. FP32 hanya berhasil memproses 4 dari 10 data karena sebagian data mengalami *OutOfMemoryError*. FP16 dan 4-bit NF4 berhasil memproses seluruh data. Dari sisi kualitas, 4-bit NF4 memperoleh *cosine similarity* tertinggi sebesar 0.6441, sedikit lebih tinggi dibandingkan FP16 sebesar 0.6422. Dari sisi *latency*, 4-bit NF4 memperoleh waktu paling rendah sebesar 2.5003 detik, sedangkan FP16 membutuhkan 17.2661 detik. Dengan demikian, pada skenario audio, 4-bit NF4 memberikan keuntungan yang lebih jelas dibandingkan pada skenario citra.

Tabel 4.16: Ringkasan hasil pengujian RAG *fixed-size* berdasarkan konfigurasi representasi numerik model

Skenario	Konfigurasi	Data Valid	BLEU-1	ROUGE-L F1	CP	CR	Latency (s)
Text-only RAG fixed-size	FP32	10/10	0.7823	0.8510	0.9560	0.8974	0.6202
Text-only RAG fixed-size	FP16	10/10	0.7823	0.8510	0.9593	0.8974	1.0874
Text-only RAG fixed-size	4-bit NF4	10/10	0.6249	0.7042	0.9454	0.8974	2.9835
Text-image RAG fixed-size	FP32	10/10	0.4951	0.5198	0.5172	0.5000	1.5115
Text-image RAG fixed-size	FP16	10/10	0.7126	0.7594	0.4526	0.7000	2.4276
Text-image RAG fixed-size	4-bit NF4	10/10	0.5024	0.5833	0.4526	0.7000	3.6631
Text-audio RAG fixed-size	FP32	4/10	0.4306	0.4246	0.7667	1.0000	59.4642
Text-audio RAG fixed-size	FP16	10/10	0.3811	0.4170	0.8017	0.9400	19.7073
Text-audio RAG fixed-size	4-bit NF4	10/10	0.4782	0.5182	0.7867	0.8000	5.0453

Tabel 4.16 menunjukkan bahwa pada konfigurasi *fixed-size*, performa terbaik tidak selalu berasal dari konfigurasi yang sama. Pada skenario *text-only RAG fixed-size*, FP32 dan FP16 menghasilkan kualitas jawaban yang sama, yaitu BLEU-1 sebesar 0.7823 dan ROUGE-L F1 sebesar 0.8510. FP16 memiliki *context precision* sedikit lebih tinggi sebesar 0.9593, sedangkan FP32 memiliki *latency* lebih rendah sebesar 0.6202 detik. Pada skenario ini, 4-bit NF4 tetap memiliki kualitas *retrieval* tinggi, tetapi kualitas jawaban dan *latency*-nya lebih rendah dibandingkan FP32 dan FP16.

Pada skenario *text-image RAG fixed-size*, FP16 menghasilkan kualitas jawaban tertinggi dengan BLEU-1 sebesar 0.7126 dan ROUGE-L F1 sebesar 0.7594. FP32 memiliki *latency*

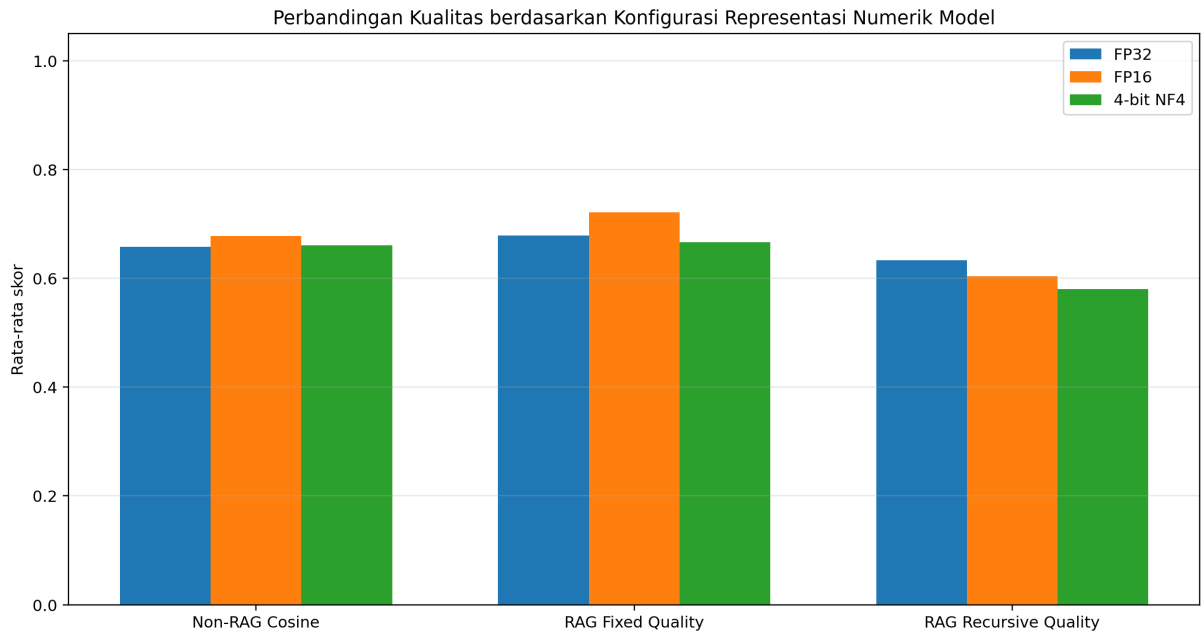
paling rendah sebesar 1.5115 detik, tetapi kualitas jawabannya lebih rendah dibandingkan FP16. Hasil ini menunjukkan adanya perbedaan antara konfigurasi yang paling baik dari sisi kualitas jawaban dan konfigurasi yang paling cepat dari sisi waktu respons. Pada skenario *text-audio RAG fixed-size*, 4-bit NF4 menjadi konfigurasi paling kuat karena berhasil memproses seluruh data, memperoleh BLEU-1 dan ROUGE-L F1 tertinggi, serta memiliki *latency* paling rendah. FP32 hanya valid pada 4 dari 10 data, sehingga interpretasinya perlu dilakukan secara terbatas.

Tabel 4.17: Ringkasan hasil pengujian RAG *recursive* berdasarkan konfigurasi representasi numerik model

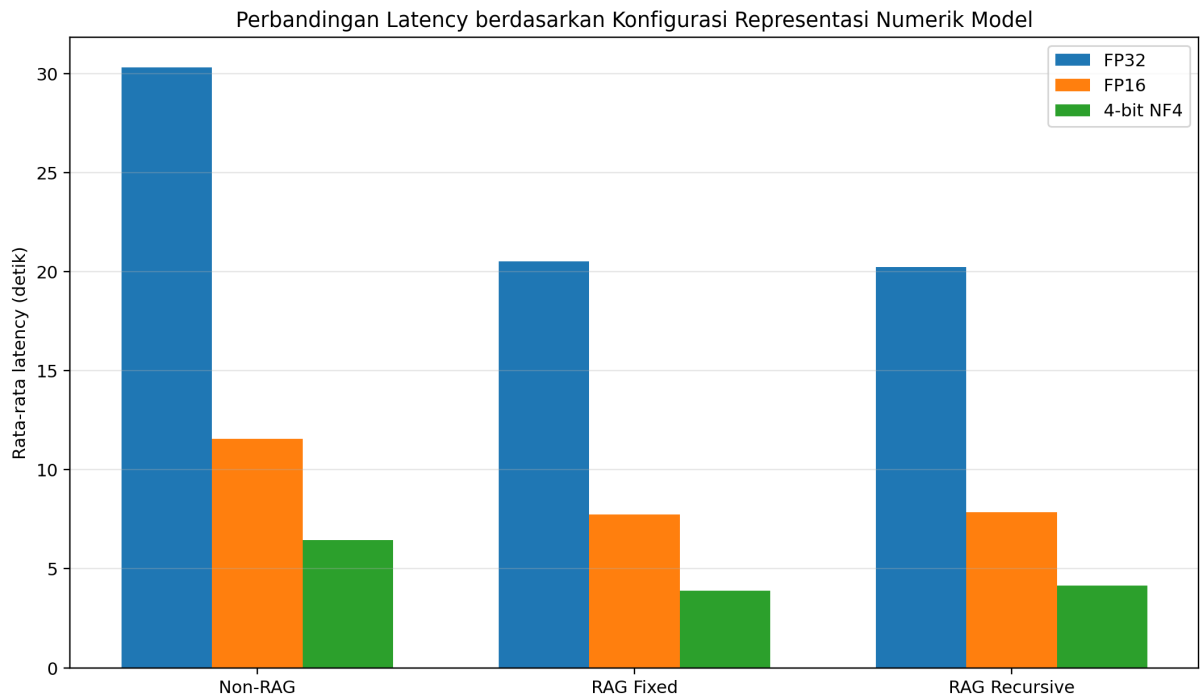
Skenario	Konfigurasi	Data Valid	BLEU-1	ROUGE-L F1	CP	CR	Latency (s)
<i>Text-only RAG recursive</i>	FP32	10/10	0.6697	0.7244	0.8822	0.9000	0.5469
<i>Text-only RAG recursive</i>	FP16	10/10	0.6697	0.7244	0.8739	0.9000	1.0826
<i>Text-only RAG recursive</i>	4-bit NF4	10/10	0.6287	0.6679	0.8739	0.9000	3.1577
<i>Text-image RAG recursive</i>	FP32	10/10	0.5897	0.5638	0.3922	0.4833	1.2388
<i>Text-image RAG recursive</i>	FP16	10/10	0.4594	0.5160	0.3767	0.6000	1.7028
<i>Text-image RAG recursive</i>	4-bit NF4	10/10	0.3280	0.4490	0.3767	0.6000	3.0000
<i>Text-audio RAG recursive</i>	FP32	4/10	0.2863	0.3109	0.7917	1.0000	58.8918
<i>Text-audio RAG recursive</i>	FP16	10/10	0.3322	0.3722	0.7250	0.7000	20.7800
<i>Text-audio RAG recursive</i>	4-bit NF4	10/10	0.3395	0.3732	0.6283	0.8000	6.2795

Tabel 4.17 menunjukkan bahwa pada konfigurasi *recursive*, FP32 cenderung lebih unggul pada skenario teks dan citra, sedangkan 4-bit NF4 lebih layak pada skenario audio. Pada *text-only RAG recursive*, FP32 dan FP16 memiliki kualitas jawaban yang sama, tetapi FP32 memiliki *context precision* dan *latency* yang lebih baik. Pada *text-image RAG recursive*, FP32 menghasilkan BLEU-1 dan ROUGE-L F1 tertinggi, serta *latency* paling rendah. Pada *text-audio RAG recursive*, 4-bit NF4 memiliki BLEU-1 dan ROUGE-L F1 sedikit lebih tinggi dibandingkan FP16, serta *latency* jauh lebih rendah. FP32 kembali hanya valid pada 4 dari 10 data.

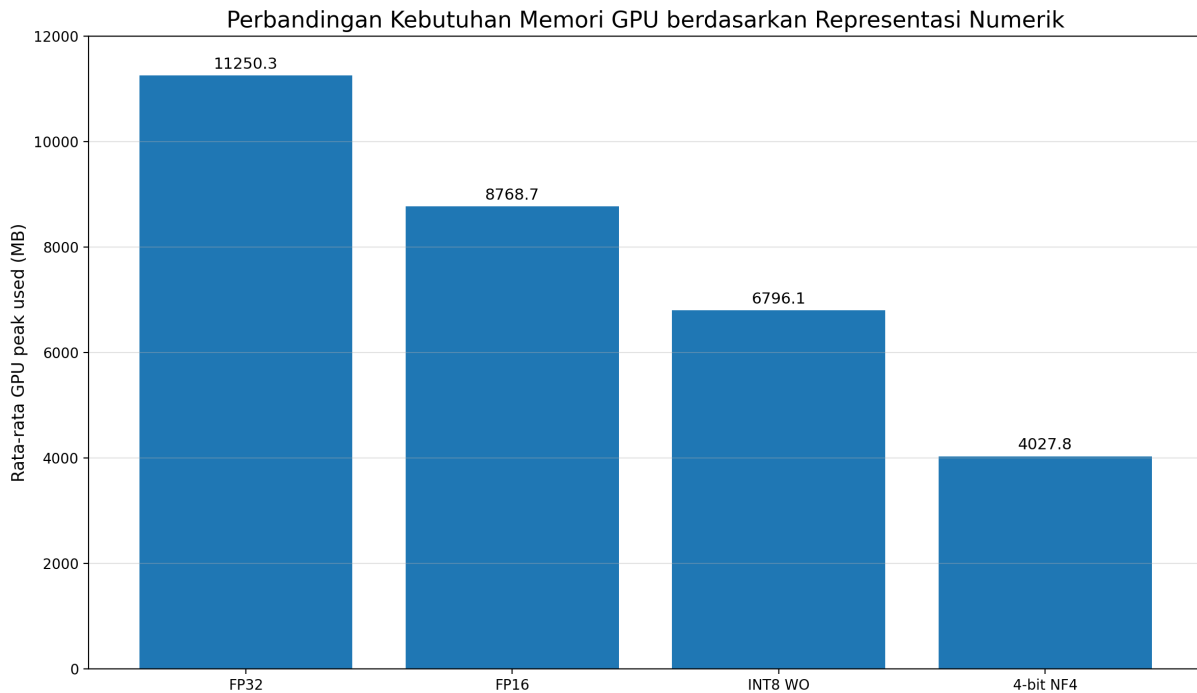
Skor agregat pada Gambar 4.83 digunakan sebagai ringkasan deskriptif untuk memudahkan pembacaan kecenderungan umum. Skor ini tidak diperlakukan sebagai metrik evaluasi tunggal karena kelompok non-RAG menggunakan *cosine similarity*, sedangkan kelompok RAG menggunakan gabungan BLEU-1, ROUGE-L F1, *context precision*, dan *context recall*. Oleh karena itu, interpretasi utama tetap dilakukan berdasarkan hasil per skenario.



Gambar 4.83: Perbandingan kualitas keluaran dan kualitas *retrieval* berdasarkan konfigurasi representasi numerik model.



Gambar 4.84: Perbandingan *latency* berdasarkan konfigurasi representasi numerik model.



Gambar 4.85: Perbandingan kebutuhan memori GPU berdasarkan konfigurasi representasi numerik model, termasuk hasil tambahan INT8 WO A16W8.

Gambar 4.83 merangkum kualitas keluaran dan kualitas *retrieval* berdasarkan konfigurasi representasi numerik model. Pada kelompok non-RAG, FP16 memperoleh rata-rata *cosine similarity* tertinggi sebesar 0.6780, diikuti 4-bit NF4 sebesar 0.6611 dan FP32 sebesar 0.6582. Nilai ini dipengaruhi oleh hasil audio, karena FP32 pada audio hanya valid pada 4 data. Pada kelompok RAG *fixed-size*, FP16 memperoleh rata-rata kualitas tertinggi sebesar 0.7212, diikuti FP32 sebesar 0.6784 dan 4-bit NF4 sebesar 0.6661. Pada kelompok RAG *recursive*, FP32 memperoleh rata-rata kualitas tertinggi sebesar 0.6328, diikuti FP16 sebesar 0.6041 dan 4-bit NF4 sebesar 0.5804.

Gambar 4.84 menunjukkan bahwa 4-bit NF4 secara umum memiliki rata-rata *latency* paling rendah ketika seluruh kelompok pengujian dilihat secara agregat. Pada kelompok non-RAG, 4-bit NF4 memperoleh rata-rata *latency* sebesar 6.4444 detik, lebih rendah dibandingkan FP16 sebesar 11.5530 detik dan FP32 sebesar 30.3218 detik. Pada RAG *fixed-size*, 4-bit NF4 memperoleh rata-rata *latency* sebesar 3.8973 detik, lebih rendah dibandingkan FP16 sebesar 7.7408 detik dan FP32 sebesar 20.5320 detik. Pada RAG *recursive*, 4-bit NF4 memperoleh 4.1457 detik, FP16 memperoleh 7.8551 detik, dan FP32 memperoleh 20.2258 detik. Namun, angka agregat ini perlu dibaca bersama hasil per skenario, karena FP32 pada audio hanya valid pada sebagian data.

Gambar 4.85 menunjukkan perbandingan kebutuhan memori GPU berdasarkan konfigurasi representasi numerik model. Berbeda dengan grafik kualitas dan *latency* agregat yang hanya menggunakan konfigurasi utama FP32, FP16, dan 4-bit NF4, grafik memori GPU juga menampilkan INT8 WO A16W8 sebagai hasil pengujian tambahan pada konfigurasi *fixed-size*. Penambahan INT8 dilakukan untuk melengkapi pembacaan kebutuhan sumber daya pada konfigurasi representasi numerik rendah yang juga digunakan dalam pengujian tambahan *latency*.

Rata-rata *GPU peak used* pada FP32 adalah 11250.3200 MB, FP16 sebesar 8768.7200 MB, INT8 WO A16W8 sebesar 6796.1000 MB, dan 4-bit NF4 sebesar 4027.8000 MB. Hasil ini menunjukkan bahwa INT8 WO A16W8 membutuhkan memori GPU lebih rendah dibandingkan

FP32 dan FP16, tetapi masih lebih tinggi dibandingkan 4-bit NF4. Secara relatif, INT8 WO A16W8 menurunkan kebutuhan memori GPU sekitar 39.6% dibandingkan FP32 dan sekitar 22.5% dibandingkan FP16. Namun, kebutuhan memorinya masih sekitar 68.7% lebih tinggi dibandingkan 4-bit NF4.

Dengan demikian, 4-bit NF4 tetap menjadi konfigurasi paling hemat memori GPU, sedangkan INT8 WO A16W8 menempati posisi menengah antara FP16 dan 4-bit NF4. Temuan ini sejalan dengan karakteristik INT8 WO yang mengurangi presisi bobot model menjadi 8-bit, tetapi tidak menekan kebutuhan memori serendah kuantisasi 4-bit NF4. Oleh karena itu, INT8 lebih tepat dibaca sebagai konfigurasi yang kuat untuk menurunkan *latency* pada pengujian tambahan *fixed-size*, sedangkan 4-bit NF4 tetap menjadi konfigurasi utama untuk prioritas penghematan memori GPU.

Secara keseluruhan, FP32 tidak dapat dianggap selalu paling unggul meskipun menggunakan representasi numerik penuh. FP32 kuat pada skenario teks dan citra tertentu, tetapi kurang stabil pada skenario audio karena keterbatasan memori. FP16 menjadi konfigurasi yang paling seimbang pada banyak skenario RAG, terutama pada konfigurasi *fixed-size*. Sementara itu, 4-bit NF4 tetap menjadi konfigurasi paling penting untuk kebutuhan penghematan memori GPU. Hasil tambahan INT8 WO A16W8 menunjukkan posisi yang berbeda: konfigurasi ini tidak sehemat 4-bit NF4 dari sisi memori, tetapi mampu memberikan *latency* rendah pada pengujian tambahan *fixed-size*. Dengan demikian, pemilihan konfigurasi tidak hanya bergantung pada jumlah bit, tetapi juga pada tujuan optimasi, apakah diarahkan pada kualitas keluaran, waktu respons, atau efisiensi memori GPU.

4.11 Analisis Trade-Off Kualitas, Latency, dan Memori GPU

Bagian ini menganalisis hubungan antara kualitas keluaran, *latency*, dan kebutuhan memori GPU. Analisis dilakukan secara deskriptif-komparatif berdasarkan hasil grafik dan tabel yang telah disajikan pada bagian sebelumnya. Fokus analisis bukan hanya menentukan konfigurasi dengan nilai tertinggi pada satu metrik, melainkan memahami konsekuensi pemilihan konfigurasi terhadap kualitas jawaban, waktu respons, keberhasilan pemrosesan, dan kebutuhan memori GPU.

4.11.1 Trade-Off pada Skenario Teks

Pada skenario teks, konfigurasi FP32 dan FP16 menunjukkan performa kualitas yang paling kuat. Pada *text-only RAG fixed-size*, FP32 dan FP16 memperoleh BLEU-1 sebesar 0.7823 dan ROUGE-L F1 sebesar 0.8510. Nilai ini lebih tinggi dibandingkan 4-bit NF4 yang memperoleh BLEU-1 sebesar 0.6249 dan ROUGE-L F1 sebesar 0.7042. Pada konfigurasi *recursive*, FP32 dan FP16 juga memperoleh nilai yang sama, yaitu BLEU-1 sebesar 0.6697 dan ROUGE-L F1 sebesar 0.7244, sedangkan 4-bit NF4 lebih rendah dengan BLEU-1 sebesar 0.6287 dan ROUGE-L F1 sebesar 0.6679.

Dari sisi *retrieval*, konfigurasi *fixed-size* menghasilkan *context precision* yang lebih tinggi dibandingkan *recursive*. Pada *text-only RAG fixed-size*, nilai *context precision* berada pada rentang 0.9454 sampai 0.9593, sedangkan pada *text-only RAG recursive* berada pada rentang 0.8739 sampai 0.8822. Perbedaan ini menunjukkan bahwa pada pertanyaan teks berbasis dokumen, potongan konteks yang lebih panjang pada konfigurasi *fixed-size* lebih mampu menjaga presisi informasi yang dikembalikan oleh proses *retrieval*.

Dari sisi *latency*, FP32 memiliki waktu respons paling rendah pada hasil evaluasi RAG, sedangkan FP16 memiliki *latency profiling* paling rendah pada profiling *text-only RAG fixed-size*.

Perbedaan ini menunjukkan bahwa hasil *latency* dapat dipengaruhi oleh cara pengukuran, alur evaluasi, dan beban pemrosesan yang terjadi pada saat profiling. Namun, pola umum yang terlihat adalah 4-bit NF4 tidak memberikan keuntungan waktu pada skenario teks. Dengan demikian, untuk skenario teks, FP16 dan FP32 lebih layak dibandingkan 4-bit NF4 apabila prioritas utamanya adalah kualitas jawaban.

4.11.2 Trade-Off pada Skenario Citra

Pada skenario citra, hasil pengujian menunjukkan perbedaan antara kemampuan pemahaman citra secara mandiri dan kualitas jawaban akhir pada *pipeline* RAG multimodal. Pada *image-only*, FP32 memperoleh *cosine similarity* tertinggi sebesar 0.7175 dan *latency* terendah sebesar 2.4671 detik. Namun, pada *text-image RAG fixed-size*, FP16 menghasilkan kualitas jawaban tertinggi dengan BLEU-1 sebesar 0.7126 dan ROUGE-L F1 sebesar 0.7594.

Perbedaan tersebut menunjukkan bahwa kualitas pemahaman citra secara mandiri tidak selalu berbanding lurus dengan kualitas jawaban akhir pada *text-image RAG*. Pada *text-image RAG*, jawaban akhir tidak hanya ditentukan oleh kemampuan model visual, tetapi juga oleh konteks dokumen hasil *retrieval*, penggabungan konteks visual dan tekstual, serta kesesuaian formulasi jawaban terhadap *ground truth*. Oleh karena itu, FP32 yang unggul pada *image-only* tidak otomatis menjadi konfigurasi terbaik pada *text-image RAG fixed-size*.

Dari sisi konfigurasi *chunking*, *fixed-size* lebih unggul dibandingkan *recursive* pada kualitas jawaban dan kualitas *retrieval* secara umum. Pada *text-image RAG fixed-size*, rata-rata BLEU-1 lintas konfigurasi adalah 0.5700 dan ROUGE-L F1 sebesar 0.6209, sedangkan pada *text-image RAG recursive* rata-rata BLEU-1 adalah 0.4590 dan ROUGE-L F1 sebesar 0.5096. Namun, *recursive* memiliki rata-rata *latency* lebih rendah. Dengan demikian, pada skenario citra terdapat trade-off antara kualitas jawaban dan waktu respons, terutama ketika membandingkan konfigurasi *fixed-size* dan *recursive*.

4.11.3 Trade-Off pada Skenario Audio

Pada skenario audio, 4-bit NF4 memberikan keuntungan paling jelas. Pada *audio-only*, 4-bit NF4 berhasil memproses seluruh data, memperoleh *cosine similarity* tertinggi sebesar 0.6441, dan menghasilkan *latency* sebesar 2.5003 detik. FP16 juga berhasil memproses seluruh data, tetapi *latency*-nya lebih tinggi sebesar 17.2661 detik. FP32 hanya berhasil pada 4 dari 10 data dan membutuhkan *latency* sebesar 58.1765 detik pada data yang berhasil.

Pola serupa juga terlihat pada skenario *text-audio RAG*. Pada *text-audio RAG fixed-size*, 4-bit NF4 memperoleh BLEU-1 sebesar 0.4782 dan ROUGE-L F1 sebesar 0.5182, lebih tinggi dibandingkan FP16. Selain itu, 4-bit NF4 memiliki *latency* sebesar 5.0453 detik, jauh lebih rendah dibandingkan FP16 sebesar 19.7073 detik. Pada *text-audio RAG recursive*, 4-bit NF4 juga memiliki *latency* paling rendah sebesar 6.2795 detik dan kualitas jawaban sedikit lebih tinggi dibandingkan FP16.

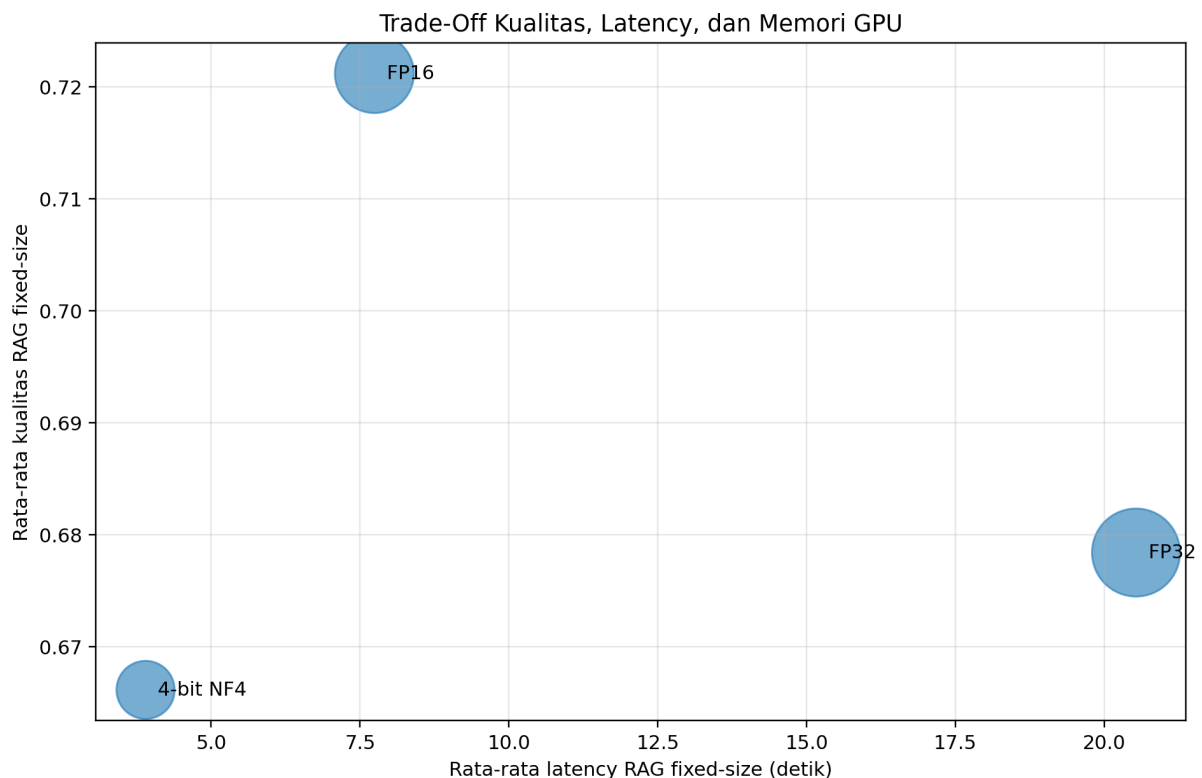
Dari sisi keberhasilan pemrosesan, 4-bit NF4 dan FP16 lebih stabil dibandingkan FP32 pada skenario audio. FP32 gagal pada sebagian data karena *OutOfMemoryError*. Hal ini menunjukkan bahwa skenario audio sangat dipengaruhi oleh kebutuhan memori model. Dengan demikian, pada skenario audio, 4-bit NF4 bukan hanya konfigurasi yang hemat memori, tetapi juga konfigurasi yang lebih layak secara praktis karena mampu memproses seluruh data dengan waktu respons lebih rendah.

4.11.4 Trade-Off Memori GPU dan Latency

Analisis profiling menunjukkan bahwa 4-bit NF4 konsisten menurunkan kebutuhan memori GPU pada seluruh skenario. Rata-rata *GPU peak used* pada 4-bit NF4 adalah 4027.8000 MB, jauh lebih rendah dibandingkan FP16 sebesar 8768.7200 MB dan FP32 sebesar 11250.3200 MB. Hasil ini menegaskan fungsi utama 4-bit NF4 sebagai konfigurasi yang dirancang untuk mengurangi jejak memori GPU.

Namun, penghematan memori tidak selalu menghasilkan *latency* yang lebih rendah. Pada *text-only RAG fixed-size*, 4-bit NF4 memiliki *GPU peak used* paling rendah sebesar 1725.8 MB, tetapi *latency profiling*-nya sebesar 0.9475 detik, lebih tinggi dibandingkan FP32 dan FP16. Pada *image-only*, 4-bit NF4 juga paling hemat memori dengan 2138.0 MB, tetapi *latency profiling*-nya 3.2506 detik, lebih tinggi dibandingkan FP32 dan FP16. Pada *text-image RAG fixed-size*, 4-bit NF4 menggunakan memori paling rendah sebesar 2288.0 MB, tetapi FP16 memiliki *latency profiling* lebih rendah.

Sebaliknya, pada skenario audio, penghematan memori 4-bit NF4 diikuti oleh penurunan *latency*. Pada *audio-only*, 4-bit NF4 menggunakan *GPU peak used* sebesar 6993.6 MB dan memperoleh *latency profiling* sebesar 1.1678 detik. Pada *text-audio RAG fixed-size*, 4-bit NF4 juga menggunakan 6993.6 MB dan memperoleh *latency profiling* sebesar 2.0724 detik. Hasil ini menunjukkan bahwa manfaat 4-bit NF4 paling kuat ketika model yang digunakan memiliki kebutuhan memori besar, seperti model audio.



Gambar 4.86: Analisis *trade-off* antara kualitas keluaran, *latency*, dan kebutuhan memori GPU.

Gambar 4.86 merangkum hubungan antara kualitas, *latency*, dan kebutuhan memori GPU pada konfigurasi *RAG fixed-size*. FP16 memperoleh rata-rata kualitas *RAG fixed-size* tertinggi sebesar 0.7212 dengan rata-rata *latency* sebesar 7.7408 detik dan rata-rata *GPU peak used* sebesar 8768.7200 MB. FP32 memperoleh kualitas sebesar 0.6784 dengan rata-rata *latency*

sebesar 20.5320 detik dan rata-rata *GPU peak used* sebesar 11250.3200 MB. Sementara itu, 4-bit NF4 memperoleh kualitas sebesar 0.6661 dengan rata-rata *latency* paling rendah sebesar 3.8973 detik dan rata-rata *GPU peak used* paling rendah sebesar 4027.8000 MB.

Berdasarkan hubungan tersebut, FP16 dapat dilihat sebagai konfigurasi yang paling seimbang untuk kualitas RAG secara umum, terutama pada skenario teks dan citra. Sementara itu, 4-bit NF4 menjadi konfigurasi yang paling efisien dari sisi memori dan *latency* agregat, tetapi kualitasnya tidak selalu tertinggi pada skenario teks dan citra. FP32 tidak memberikan keuntungan yang konsisten karena kebutuhan memorinya tinggi dan tidak stabil pada skenario audio. Dengan demikian, pemilihan konfigurasi harus disesuaikan dengan prioritas penggunaan dan karakteristik modalitas.

4.12 Keterbatasan Pengujian Tambahan

Pengujian tambahan INT8 pada penelitian ini memiliki ruang lingkup yang terbatas. Pertama, INT8 hanya diuji pada konfigurasi *fixed-size* karena konfigurasi tersebut merupakan konfigurasi utama sistem. Oleh karena itu, hasil INT8 tidak digunakan untuk membandingkan performa pada konfigurasi *recursive*. Kedua, pengujian tambahan INT8 difokuskan pada *latency*, sedangkan CP dan CR tidak dihitung ulang. Hal ini dilakukan karena CP dan CR berkaitan dengan kualitas *retrieval*, sementara pengujian tambahan ini bertujuan membaca pengaruh konfigurasi representasi numerik terhadap waktu respons.

Ketiga, konfigurasi INT8 yang digunakan adalah INT8 *Weight-Only* A16W8 pada pipeline Hugging Face Transformers dan TorchAO. Dengan demikian, hasilnya merepresentasikan performa INT8 pada perangkat lunak dan konfigurasi pipeline penelitian ini, bukan klaim performa optimal seluruh implementasi INT8 pada seluruh mesin inferensi. Keempat, hasil FP32 *strict* digunakan untuk memperjelas pembacaan FP32 penuh tanpa TF32. Hasil ini tidak menghapus hasil pengujian utama, tetapi membantu menjelaskan mengapa FP32 awal dapat terlihat sangat cepat pada beberapa skenario.

Berdasarkan keterbatasan tersebut, hasil pengujian tambahan sebaiknya dibaca sebagai pelengkap analisis *latency*. Hasil utama kualitas jawaban dan kualitas *retrieval* tetap mengacu pada pengujian utama FP32, FP16, dan 4-bit NF4, sedangkan hasil tambahan INT8 digunakan untuk memperkaya pembahasan mengenai efisiensi waktu respons pada konfigurasi utama sistem.

4.13 Rekomendasi Konfigurasi Sistem

Berdasarkan seluruh hasil pengujian, rekomendasi konfigurasi sistem disusun dengan mempertimbangkan prioritas penggunaan. Konfigurasi yang dipilih untuk kualitas jawaban belum tentu sama dengan konfigurasi yang dipilih untuk *latency* rendah atau kebutuhan memori GPU rendah. Oleh karena itu, rekomendasi disajikan berdasarkan kebutuhan penggunaan, bukan berdasarkan satu metrik tunggal.

Tabel 4.18: Rekomendasi konfigurasi sistem berdasarkan prioritas penggunaan

Prioritas Penggunaan	Konfigurasi yang direkomendasikan	Dasar Pertimbangan
Kualitas jawaban pada pertanyaan teks	<i>Text-only RAG fixed-size</i> dengan FP16	FP16 memperoleh kualitas jawaban sangat tinggi pada pengujian utama dengan BLEU-1 sebesar 0.7823 dan ROUGE-L F1 sebesar 0.8510. FP16 juga memiliki CP sedikit lebih tinggi dibanding FP32 pada pengujian utama.
Kualitas jawaban pada masukan citra dalam pipeline RAG	<i>Text-image RAG fixed-size</i> dengan FP16	Pada <i>text-image RAG fixed-size</i> , FP16 memperoleh BLEU-1 tertinggi sebesar 0.7126 dan ROUGE-L F1 tertinggi sebesar 0.7594. Dengan demikian, FP16 paling sesuai ketika prioritasnya adalah kualitas jawaban pada pipeline RAG berbasis citra.
Waktu respons rendah pada konfigurasi utama	INT8 WO A16W8 + TF32 ON pada konfigurasi <i>fixed-size</i>	Pada pengujian tambahan <i>fixed-size</i> , INT8 memperoleh <i>latency</i> terendah pada <i>text-only RAG</i> , <i>image-only</i> , <i>audio-only</i> , <i>text-image RAG</i> , dan <i>text-audio RAG</i> .
Kebutuhan memori GPU rendah	4-bit NF4	4-bit NF4 menghasilkan rata-rata <i>GPU peak used</i> paling rendah pada profiling memori GPU. Konfigurasi ini tetap penting ketika prioritas utama adalah menjalankan model pada GPU dengan kapasitas memori terbatas.
Konfigurasi utama sistem	Konfigurasi <i>fixed-size</i> dengan pemilihan representasi numerik adaptif: FP16 untuk kualitas jawaban, INT8 untuk waktu respons, dan 4-bit NF4 untuk keterbatasan memori	Konfigurasi <i>fixed-size</i> lebih stabil dibandingkan <i>recursive</i> pada sebagian besar metrik kualitas. Pemilihan representasi numerik perlu disesuaikan dengan prioritas penggunaan karena konfigurasi terbaik dari sisi kualitas, <i>latency</i> , dan memori GPU tidak selalu sama.

Tabel 4.18 menunjukkan bahwa konfigurasi sistem yang direkomendasikan tidak bersifat tunggal untuk seluruh skenario. Untuk kualitas jawaban, FP16 tetap menjadi pilihan kuat, terutama pada *text-only RAG* dan *text-image RAG fixed-size*. Untuk waktu respons rendah, hasil pengujian tambahan menunjukkan bahwa INT8 WO A16W8 + TF32 ON menjadi konfigurasi paling efisien pada konfigurasi *fixed-size*. Untuk keterbatasan memori GPU, 4-bit NF4 tetap menjadi pilihan penting karena mampu menurunkan kebutuhan memori model secara konsisten.

Dengan demikian, konfigurasi akhir sistem perlu dipilih secara adaptif berdasarkan prioritas penggunaan.

4.14 Ringkasan Temuan Bab IV

Berdasarkan seluruh pengujian yang telah dilakukan, terdapat beberapa temuan utama yang menjawab fokus penelitian mengenai sistem *question answering* multimodal berbasis Qwen dan RAG pada domain dokumen serta layanan ITS. Temuan pertama adalah bahwa konfigurasi *fixed-size* lebih stabil dibandingkan konfigurasi *recursive* pada sebagian besar skenario RAG. Pada *text-only RAG*, *fixed-size* menghasilkan kualitas jawaban dan *context precision* yang lebih tinggi. Pada *text-image RAG*, *fixed-size* juga menghasilkan kualitas jawaban dan kualitas *retrieval* yang lebih baik. Pada *text-audio RAG*, *fixed-size* unggul dari sisi kualitas jawaban, kualitas *retrieval*, dan *latency* pada konfigurasi FP16 dan 4-bit NF4. Dengan demikian, konfigurasi *fixed-size* layak dipertahankan sebagai konfigurasi *chunking* utama sistem.

Temuan kedua adalah bahwa konfigurasi representasi numerik model memiliki pengaruh yang berbeda pada setiap modalitas. Pada pengujian utama, FP16 menjadi konfigurasi yang kuat untuk kualitas jawaban pada skenario RAG, terutama *text-only RAG fixed-size* dan *text-image RAG fixed-size*. Sementara itu, 4-bit NF4 tetap berperan penting untuk menurunkan kebutuhan memori GPU, khususnya pada skenario yang membutuhkan model dengan ukuran lebih besar seperti audio.

Temuan ketiga adalah bahwa penghematan memori GPU tidak selalu berbanding lurus dengan penurunan *latency*. Hasil profiling menunjukkan bahwa 4-bit NF4 selalu memberikan kebutuhan memori GPU paling rendah. Namun, pada skenario teks dan citra, 4-bit NF4 tidak selalu menjadi konfigurasi tercepat. Dengan demikian, efisiensi memori dan efisiensi waktu perlu dianalisis sebagai dua aspek yang berbeda.

Temuan keempat diperoleh dari pengujian tambahan *latency* pada konfigurasi *fixed-size*. Konfigurasi INT8 WO A16W8 + TF32 ON menghasilkan *latency* terendah pada *text-only RAG fixed-size*, *image-only*, *audio-only*, *text-image RAG fixed-size*, dan *text-audio RAG fixed-size*. Hasil ini menunjukkan bahwa INT8 dapat menjadi pilihan yang efektif ketika prioritas sistem adalah waktu respons rendah. Namun, INT8 tidak selalu menghasilkan kualitas jawaban tertinggi, sehingga pemilihannya perlu disesuaikan dengan prioritas penggunaan.

Temuan kelima adalah bahwa hasil FP32 awal perlu dibedakan dari FP32 *strict*. Pada beberapa skenario, FP32 awal menghasilkan *latency* yang sangat rendah. Setelah pengujian FP32 *strict* dilakukan dengan TF32 dinonaktifkan, *latency* meningkat secara signifikan pada skenario yang diuji ulang. Oleh karena itu, hasil FP32 awal memiliki indikasi kuat dipengaruhi oleh optimasi backend seperti TF32, sedangkan FP32 *strict* digunakan sebagai pembanding FP32 penuh yang lebih hati-hati.

Secara keseluruhan, hasil pengujian menunjukkan bahwa konfigurasi sistem *question answering* multimodal berbasis Qwen dan RAG perlu dipilih berdasarkan *trade-off* antara kualitas keluaran, kualitas *retrieval*, *latency*, dan kebutuhan memori GPU. Konfigurasi *fixed-size* menjadi konfigurasi utama yang paling stabil. FP16 direkomendasikan ketika prioritasnya adalah kualitas jawaban, INT8 WO A16W8 direkomendasikan ketika prioritasnya adalah waktu respons rendah, sedangkan 4-bit NF4 direkomendasikan ketika prioritasnya adalah penghematan memori GPU.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan hasil implementasi, pengujian, dan analisis terhadap sistem *question answering* multimodal berbasis model Qwen dengan metode *Retrieval-Augmented Generation* (RAG), diperoleh beberapa kesimpulan sebagai berikut:

1. Sistem *question answering* multimodal berbasis model Qwen dan RAG berhasil dibangun untuk menangani masukan teks, citra, dan audio. Sistem mampu memproses pertanyaan teks melalui model Qwen2.5, memanfaatkan informasi visual melalui model Qwen2-VL, serta memproses masukan audio menggunakan model Qwen2-Audio. Selain itu, sistem juga dilengkapi fitur manajemen dokumen untuk mendukung proses penambahan, penghapusan, penampilan, dan pembangunan ulang indeks dokumen.
2. Hasil pengujian menunjukkan bahwa performa sistem berbeda pada setiap modalitas. Pada skenario *image-only*, FP32 menghasilkan kualitas semantik dan *latency* terbaik. Pada skenario *audio-only*, 4-bit NF4 menjadi konfigurasi paling layak karena berhasil memproses seluruh data, memperoleh nilai *cosine similarity* tertinggi, dan menghasilkan *latency* paling rendah. Hal ini menunjukkan bahwa pemilihan konfigurasi representasi numerik model perlu mempertimbangkan karakteristik modalitas.
3. Pada skenario berbasis RAG, konfigurasi *fixed-size chunking* lebih stabil dibandingkan konfigurasi *recursive*. Konfigurasi *fixed-size* menghasilkan kualitas jawaban dan kualitas *retrieval* yang lebih baik pada sebagian besar skenario, terutama pada *text-only RAG*, *text-image RAG*, dan *text-audio RAG*. Dengan demikian, konfigurasi *fixed-size* layak dipertahankan sebagai konfigurasi *chunking* utama dalam sistem.
4. Hasil perbandingan FP32, FP16, dan 4-bit NF4 menunjukkan bahwa tidak ada satu konfigurasi yang selalu unggul pada seluruh skenario. FP32 dan FP16 lebih kuat pada skenario teks, FP16 menghasilkan kualitas jawaban terbaik pada *text-image RAG fixed-size*, sedangkan 4-bit NF4 paling efektif pada skenario audio dan kondisi GPU terbatas. Hasil profiling juga menunjukkan bahwa 4-bit NF4 memberikan penghematan memori GPU paling besar, tetapi penghematan memori tidak selalu diikuti oleh penurunan *latency* pada semua skenario.
5. Berdasarkan keseluruhan hasil pengujian, konfigurasi sistem yang paling sesuai adalah penggunaan *fixed-size chunking* dengan pemilihan representasi numerik yang adaptif terhadap modalitas. FP16 direkomendasikan untuk skenario teks dan citra RAG ketika kualitas jawaban menjadi prioritas, sedangkan 4-bit NF4 direkomendasikan untuk skenario audio dan kondisi GPU terbatas. Strategi ini memberikan keseimbangan yang lebih baik antara kualitas keluaran, kualitas *retrieval*, *latency*, dan kebutuhan memori GPU.

5.2 Saran

Berdasarkan hasil penelitian dan keterbatasan yang ditemukan selama proses implementasi serta pengujian, terdapat beberapa saran untuk pengembangan lebih lanjut sebagai berikut:

1. Penelitian selanjutnya dapat memperluas jumlah dan variasi data uji, baik pada pertanyaan

teks, citra, maupun audio. Penambahan data dengan variasi bentuk pertanyaan, kualitas citra, durasi audio, dan kondisi perekaman dapat memberikan gambaran performa sistem yang lebih menyeluruh.

2. Pengembangan berikutnya dapat menguji model Qwen dengan ukuran parameter atau varian yang berbeda. Pengujian terhadap model yang lebih besar, lebih ringan, atau model multimodal lain dapat digunakan untuk melihat pengaruh arsitektur model terhadap kualitas jawaban, *latency*, dan kebutuhan memori GPU.
3. Konfigurasi *chunking* dan *retrieval* dapat dikembangkan lebih lanjut dengan variasi ukuran *chunk*, nilai *overlap*, strategi *reranking*, atau pendekatan *semantic chunking*. Pengembangan ini dapat membantu meningkatkan relevansi konteks yang digunakan dalam proses RAG.
4. Evaluasi sistem dapat dilengkapi dengan penilaian kualitatif terhadap jawaban yang dihasilkan. Selain metrik otomatis seperti BLEU-1, ROUGE-L F1, *context precision*, *context recall*, dan *cosine similarity*, penilaian manusia dapat digunakan untuk mengevaluasi faktualitas, kelengkapan, keterbacaan, dan kesesuaian jawaban terhadap kebutuhan pengguna.
5. Untuk implementasi pada lingkungan nyata, sistem dapat dikembangkan dengan mekanisme pemilihan konfigurasi otomatis berdasarkan modalitas masukan dan ketersediaan sumber daya GPU. Dengan demikian, sistem dapat menggunakan konfigurasi yang lebih sesuai, misalnya FP16 untuk teks dan citra RAG serta 4-bit NF4 untuk audio atau kondisi VRAM terbatas.

DAFTAR PUSTAKA

- [1] K. Peyton, S. Unnikrishnan, and B. Mulligan, “A review of university chatbots for student support: Faqs and beyond,” *Discover Education*, vol. 4, no. 21, 2025. doi: 10.1007/s44217-025-00397-7
- [2] Z. Ji et al., “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.
- [3] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33, Curran Associates, Inc., 2020, pp. 9459–9474. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf
- [4] J. Swacha and M. Gracel, “Retrieval-augmented generation (rag) chatbots for education: A survey of applications,” *Applied Sciences*, vol. 15, no. 8, p. 4234, 2025. doi: 10.3390/app15084234
- [5] S. Yin et al., “A survey on multimodal large language models,” *National Science Review*, vol. 11, no. 12, nwae403, Nov. 2024, issn: 2095-5138. doi: 10.1093/nsr/nwae403 eprint: <https://academic.oup.com/nsr/article-pdf/11/12/nwae403/61201557/nwae403.pdf>. [Online]. Available: <https://doi.org/10.1093/nsr/nwae403>
- [6] A. Yang et al., “Qwen2.5 technical report,” *arXiv preprint arXiv:2412.15115*, 2024.
- [7] P. Wang et al., *Qwen2-vl: Enhancing vision-language model’s perception of the world at any resolution*, 2024. arXiv: 2409.12191 [cs.CV]. [Online]. Available: <https://arxiv.org/abs/2409.12191>
- [8] Y. Chu et al., *Qwen2-audio technical report*, 2024. arXiv: 2407.10759 [eess.AS]. [Online]. Available: <https://arxiv.org/abs/2407.10759>
- [9] Z. Li, Z. Wang, W. Wang, K. Hung, H. Xie, and F. L. Wang, “Retrieval-augmented generation for educational application: A systematic survey,” *Computers and Education: Artificial Intelligence*, vol. 8, p. 100417, 2025, issn: 2666-920X. doi: <https://doi.org/10.1016/j.caeai.2025.100417> [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2666920X25000578>
- [10] J. Gu and D. Qin, “An arxiv paper question-answering system based on qwen and rag,” in *2024 6th International Conference on Frontier Technologies of Information and Computer (ICFTIC)*, 2024, pp. 1354–1361. doi: 10.1109/ICFTIC64248.2024.10913101
- [11] M. Douze et al., “The faiss library,” *arXiv preprint arXiv:2401.08281*, 2024.
- [12] A. Vaswani et al., “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [13] Hugging Face, *Tokenizer*, https://huggingface.co/docs/transformers/main_classes/tokenizer, Diakses 27 Juni 2026, 2026.
- [14] Hugging Face, *Generation*, https://huggingface.co/docs/transformers/main_classes/text_generation, Diakses 27 Juni 2026, 2026.

- [15] Hugging Face, *Cache strategies*, https://huggingface.co/docs/transformers/kv_cache, Diakses 27 Juni 2026, 2026.
- [16] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 2019, pp. 3982–3992.
- [17] Sentence Transformers, *Sentence-transformers/all-minilm-l6-v2*, <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>, Diakses 27 Juni 2026, 2026.
- [18] Facebook Research, *MetricType and distances*, <https://github.com/facebookresearch/faiss/wiki/MetricType-and-distances>, Diakses 27 Juni 2026, 2024.
- [19] S. R. Bhat, M. Rudat, J. Spiekermann, and N. Flores-Herr, *Rethinking chunk size for long-document retrieval: A multi-dataset analysis*, 2025. arXiv: 2505.21700 [cs.IR]. [Online]. Available: <https://arxiv.org/abs/2505.21700>
- [20] S. Taiwo and M. A. Yusoff, *Evaluating chunking strategies for retrieval-augmented generation in oil and gas enterprise documents*, 2026. arXiv: 2603.24556 [cs.IR]. [Online]. Available: <https://arxiv.org/abs/2603.24556>
- [21] PyTorch, *Numerical accuracy*, https://docs.pytorch.org/docs/stable/notes/numerical_accuracy.html, Diakses 27 Juni 2026, 2026.
- [22] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "Qlora: Efficient finetuning of quantized llms," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [23] Hugging Face, *Bitsandbytes*, <https://huggingface.co/docs/transformers/quantization/bitsandbytes>, Diakses 27 Juni 2026, 2026.
- [24] Hugging Face, *Quantization*, https://huggingface.co/docs/transformers/main_classes/quantization, Diakses 27 Juni 2026, 2026.
- [25] PyTorch, *Torch.cuda.memory.max_memory_allocated*, https://docs.pytorch.org/docs/stable/generated/torch.cuda.memory.max_memory_allocated.html, Diakses 27 Juni 2026, 2026.
- [26] PyTorch, *Torch.cuda.memory.max_memory_reserved*, https://docs.pytorch.org/docs/stable/generated/torch.cuda.memory.max_memory_reserved.html, Diakses 27 Juni 2026, 2026.
- [27] PyTorch, *Cuda semantics*, <https://docs.pytorch.org/docs/stable/notes/cuda.html>, Diakses 27 Juni 2026, 2026.
- [28] PyTorch, *Torch.cuda.synchronize*, <https://docs.pytorch.org/docs/stable/generated/torch.cuda.synchronize.html>, Diakses 27 Juni 2026, 2026.
- [29] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "Bleu: A method for automatic evaluation of machine translation," in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, 2002, pp. 311–318. doi: 10.3115/1073083.1073135
- [30] C.-Y. Lin, "Rouge: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*, 2004, pp. 74–81.
- [31] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "Ragas: Automated evaluation of retrieval augmented generation," in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, 2024, pp. 150–158.

- [32] Ragas, *Context precision*, https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/context_precision/, Diakses 27 Juni 2026, 2026.
- [33] Ragas, *Context recall*, https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/context_recall/, Diakses 27 Juni 2026, 2026.
- [34] K. Dong et al., “MC-indexing: Effective long document retrieval via multi-view content-aware indexing,” in *Findings of the Association for Computational Linguistics: EMNLP 2024*, Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 2673–2691. doi: 10.18653/v1/2024.findings-emnlp.150 [Online]. Available: <https://aclanthology.org/2024.findings-emnlp.150/>
- [35] R. Qu, R. Tu, and F. S. Bao, “Is semantic chunking worth the computational cost?” In *Findings of the Association for Computational Linguistics: NAACL 2025*, Albuquerque, New Mexico: Association for Computational Linguistics, Apr. 2025, pp. 2155–2177. doi: 10.18653/v1/2025.findings-naacl.114 [Online]. Available: <https://aclanthology.org/2025.findings-naacl.114/>

[Halaman ini sengaja dikosongkan]

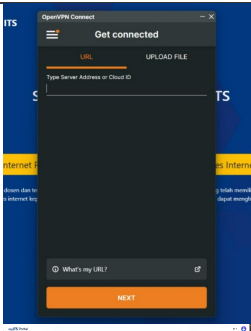
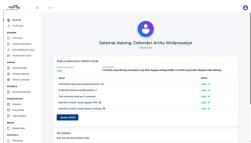
LAMPIRAN

HASIL JAWABAN SISTEM TERPILIH

A.1 Data Media Uji Terpilih

Bagian ini menyajikan data media yang digunakan pada lampiran hasil jawaban sistem. Data citra ditampilkan dalam bentuk pratinjau gambar agar pembaca dapat melihat masukan visual yang diberikan kepada sistem. Data audio dijelaskan melalui isi utama audio karena berkas audio tidak dapat ditampilkan secara visual pada dokumen cetak. Pratinjau citra mengikuti struktur direktori dokumen pada gambar/bab4/, sedangkan data audio dijelaskan berdasarkan isi utamanya tanpa menampilkan jalur berkas karena informasi tersebut tidak diperlukan oleh pembaca dokumen cetak.

Tabel A.1: Data citra terpilih pada lampiran hasil jawaban sistem

ID	Citra	Lokasi Berkas	Deskripsi Isi Citra
IMG03		gambar/bab4/IMG03_vpn	Tampilan aplikasi OpenVPN Connect yang meminta pengguna memasukkan <i>server address</i> atau <i>cloud ID</i> , serta menampilkan tombol <i>Next</i> .
IMG06		gambar/bab4/ IMG06_myits_dashboard	Tampilan dashboard myITS Portal yang memuat beberapa aplikasi layanan ITS, seperti myITS Presensi, myITS Classroom, myITS Student Connect, dan layanan terkait lainnya.
IMG09		gambar/bab4/ IMG09_studentconnect	Tampilan halaman myITS Student Connect yang memuat informasi SKEM, status kelulusan SKEM, keterangan portofolio, dan menu layanan kemahasiswaan.

Tabel A.2: Data audio terpilih pada lampiran hasil jawaban sistem

ID	Isi Utama Audio
AUD03	Audio menjelaskan bahwa VPN ITS digunakan untuk mengakses jaringan kampus dari luar lingkungan ITS.
AUD08	Audio menjelaskan bahwa myITS Student Connect menampilkan informasi SKEM, portofolio, dan layanan kemahasiswaan.
AUD09	Audio menjelaskan bahwa SIAKAD menyediakan informasi akademik seperti biodata mahasiswa, transkrip nilai, dan FRS.

Tabel A.2 – lanjutan

ID	Isi Utama Audio
AUD10	Audio menjelaskan bahwa Office 365 menyediakan akses ke Microsoft Office, OneDrive, dan aplikasi kolaborasi lainnya.

A.2 Hasil Jawaban Skenario Image-Only

Bagian ini hanya menyajikan data uji terpilih yang dinilai lebih representatif dan lebih menantang. Pemilihan dilakukan agar lampiran tidak memuat seluruh Q1–Q10, melainkan hanya contoh yang paling relevan untuk menunjukkan kemampuan sistem pada masing-masing skenario pengujian.

Tiga data uji dipilih karena memuat tampilan aplikasi atau layanan yang membutuhkan pembacaan teks antarmuka dan pemahaman fungsi layanan, yaitu OpenVPN Connect, dashboard myITS Portal, dan myITS Student Connect. Pratinjau citra untuk ketiga data tersebut ditunjukkan pada Tabel A.1.

Tabel A.3: Hasil jawaban terpilih pada skenario image-only

ID	Konfig.	Pertanyaan/Masukan <i>Ground Truth</i>	Jawaban Sistem	Metrik
IMG03	FP32	Deskripsikan gambar Tampilan aplikasi Open-VPN Connect dengan VPN Connect yang sedang berjalan. Di bagian ringkas dalam bahasa latar biru yang meminta atas, ada judul "Get connected" yang me-Indonesia. Fokus pada pengguna memasukkan nunjukkan bahwa aplikasi sedang berusaha antarmuka utama, ob- server address atau cloud menghubungkan ke jaringan. Di bagian ten- jek, teks yang terli- ID, serta tombol Next di hat, dan aktivitas pada bagian bawah. gambar. Jawab dalam satu paragraf tanpa for- mat EVIDENCE, AN- SWER, atau Sumber.	Gambar ini menunjukkan layar aplikasi Open-VPN Connect yang sedang berjalan. Di bagian atas, ada judul "Get connected" yang me-Indonesia. Fokus pada pengguna memasukkan nunjukkan bahwa aplikasi sedang berusaha antarmuka utama, ob- server address atau cloud menghubungkan ke jaringan. Di bagian ten- jek, teks yang terli- ID, serta tombol Next di hat, ada field untuk "URL" yang mungkin digu- nakan untuk menghubungkan ke server, dan di bagian bawah, ada field untuk "UPLOAD FILE" yang mungkin digunakan untuk mengupload file. Di bagian kanan, ada tombol "NEXT" yang mungkin digunakan untuk melanjutkan proses penghubungan. Di samping layar, ada beber- apa teks yang mungkin berisi informasi tentang jaringan atau aplikasi lainnya.	Cosine = 0.6592; La- tency = 2.8648 s
IMG03	FP16	Deskripsikan gambar Tampilan aplikasi Open-VPN Connect dengan VPN Connect yang sedang berjalan. Di bagian ringkas dalam bahasa latar biru yang meminta atas, ada judul "Get connected" yang me-Indonesia. Fokus pada pengguna memasukkan nunjukkan bahwa aplikasi sedang berusaha antarmuka utama, ob- server address atau cloud menghubungkan ke jaringan. Di bagian ten- jek, teks yang terli- ID, serta tombol Next di hat, dan aktivitas pada bagian bawah. gambar. Jawab dalam satu paragraf tanpa for- mat EVIDENCE, AN- SWER, atau Sumber.	Gambar ini menunjukkan layar aplikasi Open-VPN Connect yang sedang berjalan. Di bagian atas, ada judul "Get connected" yang me-Indonesia. Fokus pada pengguna memasukkan nunjukkan bahwa aplikasi sedang berusaha antarmuka utama, ob- server address atau cloud menghubungkan ke jaringan. Di bagian ten- jek, teks yang terli- ID, serta tombol Next di hat, ada field untuk "URL" yang mungkin digu- nakan untuk menghubungkan ke server, dan di bagian bawah, ada field untuk "UPLOAD FILE" yang mungkin digunakan untuk mengupload file. Di bagian kanan, ada tombol "NEXT" yang mungkin digunakan untuk melanjutkan proses penghubungan. Di samping layar, ada beber- apa teks yang mungkin berisi informasi tentang jaringan atau aplikasi lainnya.	Cosine = 0.6591; La- tency = 6.6781 s
IMG03	4-bit NF4	Deskripsikan gambar Tampilan aplikasi Open-VPN Connect dengan Connect yang sedang berjalan. Di bagian atas, ringkas dalam bahasa latar biru yang meminta ada tanda pengaturan dengan dua opsi, "Get Indonesia. Fokus pada pengguna memasukkan connected" dan "Upload file". Di bagian tengah, antarmuka utama, ob- server address atau cloud ada kolom URL dan kolom Upload File. Di jek, teks yang terli- ID, serta tombol Next di hat, dan aktivitas pada bagian bawah. gambar. Jawab dalam satu paragraf tanpa for- mat EVIDENCE, AN- SWER, atau Sumber.	Gambar ini menunjukkan layanan OpenVPN Connect yang sedang berjalan. Di bagian atas, ada tanda pengaturan dengan dua opsi, "Get Indonesia. Fokus pada pengguna memasukkan connected" dan "Upload file". Di bagian tengah, antarmuka utama, ob- server address atau cloud ada kolom URL dan kolom Upload File. Di bagian bawah, ada kolom What's my URL? dan tombol Next.	Cosine = 0.7450; La- tency = 9.0963 s

Tabel A.3 – lanjutan

ID	Konfig.	Pertanyaan/Masukan <i>Ground Truth</i>	Jawaban Sistem	Metrik
IMG06	FP32	Deskripsikan gambar Tampilan dashboard my-ITS Portal dengan menu portal myITS. Di sini, pengguna dapat mengakses berbagai fitur seperti Cek PDDIKTI, myITS Indonesia. Fokus pada berbagai aplikasi seperti Classroom, myITS Thesis, myITS Wali, myITS antarmuka utama, ob-myITS Academics, my-Presensi, myITS Ormawa, myITS StudentConjek, teks yang terli-ITS Classroom, myITS nect, myITS Vote, myITS Research, dan myITS hat, dan aktivitas pada Thesis, myITS Wali, my-Services. Ada juga opsi untuk melihat semua gambar. Jawab dalam ITS Presensi, myITS Ca-pengumuman.	Gambar ini menunjukkan antarmuka utama dari ITS Portal dengan menu portal myITS. Di sini, pengguna dapat mengakses berbagai fitur seperti Cek PDDIKTI, myITS Thesis, myITS Wali, myITS Research, dan myITS Services. Ada juga opsi untuk melihat semua gambar.	Cosine = 0.7700; Latency = 1.8756 s
IMG06	FP16	Deskripsikan gambar Tampilan dashboard my-ITS Portal dengan menu portal myITS. Di sini, pengguna dapat mengakses berbagai fitur seperti Cek PDDIKTI, myITS Indonesia. Fokus pada berbagai aplikasi seperti Classroom, myITS Thesis, myITS Wali, myITS antarmuka utama, ob-myITS Academics, my-Presensi, myITS Ormawa, myITS StudentConjek, teks yang terli-ITS Classroom, myITS nect, myITS Vote, myITS Research, dan myITS hat, dan aktivitas pada Thesis, myITS Wali, my-Services. Ada juga opsi untuk melihat semua gambar. Jawab dalam ITS Presensi, myITS Ca-pengumuman.	Gambar ini menunjukkan antarmuka utama dari ITS Portal dengan menu portal myITS. Di sini, pengguna dapat mengakses berbagai fitur seperti Cek PDDIKTI, myITS Thesis, myITS Wali, myITS Research, dan myITS Services. Ada juga opsi untuk melihat semua gambar.	Cosine = 0.7700; Latency = 4.2696 s
IMG06	4-bit NF4	Deskripsikan gambar Tampilan dashboard my-ITS Portal dengan menu portal myITS. Di sini, pengguna dapat mengakses berbagai fitur seperti akun, pengumuman, antarmuka utama, ob-myITS Academics, my-jek, teks yang terli-ITS Classroom, myITS hat, dan aktivitas pada Thesis, myITS Wali, my-gambar. Jawab dalam ITS Presensi, myITS Ca-satu paragraf tanpa for-reer Portal, myITS Vote, mat EVIDENCE, AN-riset, alumni, dan layanan. SWER, atau Sumber.	Gambar ini menunjukkan antarmuka utama dari ITS Portal dengan menu portal myITS. Di sini, pengguna dapat mengakses berbagai fitur seperti akun, pengumuman, antarmuka utama, ob-myITS Academics, my-jek, teks yang terli-ITS Classroom, myITS hat, dan aktivitas pada Thesis, myITS Wali, my-gambar. Jawab dalam ITS Presensi, myITS Ca-satu paragraf tanpa for-reer Portal, myITS Vote, mat EVIDENCE, AN-riset, alumni, dan layanan. SWER, atau Sumber.	Cosine = 0.6677; Latency = 4.2761 s
IMG09	FP32	Deskripsikan gambar Tampilan halaman myITS Gambar ini menunjukkan halaman utama dari ITS dengan aplikasi "myITS" yang digunakan oleh mahasiswa, seperti beranda, profil saya, beasiswa, magang, antarmuka utama, ob-informasi status SKEM, kedaireka, kegiatan, dan portofolio. Di bagian jek, teks yang terli-total kredit, IP minimum atas, ada tanda selamat datang dan nama penghat, dan aktivitas pada kelulusan SKEM, serta guna. Di bagian bawah, terdapat deskripsi tengambar. Jawab dalam tombol Ajukan SKEM tang status kelulusan SKEM Anda, termasuk satu paragraf tanpa for-dan Portofolio. syarat dan keterangan.	Gambar ini menunjukkan halaman utama dari ITS dengan aplikasi "myITS" yang digunakan oleh mahasiswa, seperti beranda, profil saya, beasiswa, magang, antarmuka utama, ob-informasi status SKEM, kedaireka, kegiatan, dan portofolio. Di bagian jek, teks yang terli-total kredit, IP minimum atas, ada tanda selamat datang dan nama penghat, dan aktivitas pada kelulusan SKEM, serta guna. Di bagian bawah, terdapat deskripsi tengambar. Jawab dalam tombol Ajukan SKEM tang status kelulusan SKEM Anda, termasuk satu paragraf tanpa for-dan Portofolio. syarat dan keterangan.	Cosine = 0.6944; Latency = 2.0956 s
IMG09	FP16	Deskripsikan gambar Tampilan halaman myITS Gambar ini menunjukkan halaman utama dari ITS dengan aplikasi "myITS" yang digunakan oleh mahasiswa, seperti beranda, profil saya, beasiswa, magang, antarmuka utama, ob-informasi status SKEM, kedaireka, kegiatan, dan portofolio. Di bagian jek, teks yang terli-total kredit, IP minimum atas, ada tanda selamat datang dan nama penghat, dan aktivitas pada kelulusan SKEM, serta guna. Di bagian bawah, terdapat deskripsi tengambar. Jawab dalam tombol Ajukan SKEM tang status kelulusan SKEM Anda, termasuk satu paragraf tanpa for-dan Portofolio. syarat dan keterangan.	Gambar ini menunjukkan halaman utama dari ITS dengan aplikasi "myITS" yang digunakan oleh mahasiswa, seperti beranda, profil saya, beasiswa, magang, antarmuka utama, ob-informasi status SKEM, kedaireka, kegiatan, dan portofolio. Di bagian jek, teks yang terli-total kredit, IP minimum atas, ada tanda selamat datang dan nama penghat, dan aktivitas pada kelulusan SKEM, serta guna. Di bagian bawah, terdapat deskripsi tengambar. Jawab dalam tombol Ajukan SKEM tang status kelulusan SKEM Anda, termasuk satu paragraf tanpa for-dan Portofolio. syarat dan keterangan.	Cosine = 0.6944; Latency = 4.6978 s

IMG09 4-bit NF4	Deskripsikan gambar Tampilan halaman myITS Gambar ini menunjukkan antarmuka utama dari Cosine = 0.5013; La- ini secara akurat dan StudentConnect dengan sebuah sistem atau aplikasi, mungkin berhubun- tency = 16.1933 s ringkas dalam bahasa menu navigasi di sisi kiri, gan dengan pendidikan atau bisnis. Berikut Indonesia. Fokus pada nama profil mahasiswa, adalah deskripsi gambar: 1. Di bagian atas, ada antarmuka utama, ob- informasi status SKEM, logo yang mungkin menunjukkan nama atau jek, teks yang terli- total kredit, IP minimum logo dari sistem tersebut. 2. Di bagian tengah, hat, dan aktivitas pada kelulusan SKEM, serta ada judul yang mungkin menunjukkan infor- gambar. Jawab dalam tombol Ajukan SKEM masi tentang pengguna atau status. 3. Di bagian satu paragraf tanpa for- dan Portofolio. bawah, ada tabel yang mungkin menunjukkan data atau informasi tentang pengguna atau porto- folio. Jawaban: Gambar ini menunjukkan an- tarmuka utama dari sebuah sistem atau aplikasi, mungkin berhubungan dengan pendidikan atau bisnis.
-----------------	--

A.3 Hasil Jawaban Skenario Audio-Only

Bagian ini hanya menyajikan data uji terpilih yang dinilai lebih representatif dan lebih menantang. Pemilihan dilakukan agar lampiran tidak memuat seluruh Q1–Q10, melainkan hanya contoh yang paling relevan untuk menunjukkan kemampuan sistem pada masing-masing skenario pengujian.

Tiga data uji dipilih karena memuat informasi layanan yang tidak hanya berupa nama layanan, tetapi juga fungsi layanan kemahasiswaan, layanan akademik, dan fasilitas kolaborasi Office 365.

Tabel A.4: Hasil jawaban terpilih pada skenario audio-only

ID	Konfig.	Pertanyaan/Masukan	Ground Truth	Jawaban Sistem	Metrik
AUD08 FP32		Dengarkan audio ini myITS Student Connect	Tidak menghasilkan jawaban valid karena Cosine = 0.0816; La- dengan teliti. Jelaskan menampilkan informasi proses mengalami OutOfMemoryError: CUDA tency = – s isi utama audio se- SKEM, portofolio, dan out of memory. Tried to allocate 2.38 GiB. GPU cara akurat dalam ba- layanan kemahasiswaan. 0 has a total capacity of 15.46 GiB of which hasa Indonesia. Jawab ringkas dalam satu kali- 1.54 GiB is free. Including non-PyTorch mem- mat atau satu para- ory, this process has 13.88 GiB memory in use. graf pendek. Jan- Of the allocate gan menambahkan for- mat EVIDENCE, AN- SWER, atau Sumber.		
AUD08 FP16		Dengarkan audio ini myITS Student Connect	Isi utama audio adalah tentang akses email test Cosine = 0.5209; La- dengan teliti. Jelaskan menampilkan informasi melalui Microsoft Outlook untuk mengirim dan tency = 15.6830 s isi utama audio se- SKEM, portofolio, dan menerima pesan. cara akurat dalam ba- layanan kemahasiswaan. hasa Indonesia. Jawab ringkas dalam satu kali- mat atau satu para- graf pendek. Jan- gan menambahkan for- mat EVIDENCE, AN- SWER, atau Sumber.		
AUD08 4-bit NF4		Dengarkan audio ini myITS Student Connect	Dalam email test tersebut, Anda dapat men- Cosine = 0.4697; La- dengan teliti. Jelaskan menampilkan informasi gakses Microsoft Outlook untuk mengirim dan tency = 2.1080 s isi utama audio se- SKEM, portofolio, dan menerima pesan. cara akurat dalam ba- layanan kemahasiswaan. hasa Indonesia. Jawab ringkas dalam satu kali- mat atau satu para- graf pendek. Jan- gan menambahkan for- mat EVIDENCE, AN- SWER, atau Sumber.		

Tabel A.4 – lanjutan

ID	Konfig.	Pertanyaan/Masukan <i>Ground Truth</i>	Jawaban Sistem	Metrik
AUD09 FP32		Dengarkan audio ini. SIADAD menyediakan informasi akademik seperti proses mengalami OutOfMemoryError: CUDA tency = - s isi utama audio se-biodata mahasiswa, tran- out of memory. Tried to allocate 2.38 GiB. GPU cara akurat dalam ba- skrip nilai, dan FRS. 0 has a total capacity of 15.46 GiB of which hasa Indonesia. Jawab 1.49 GiB is free. Including non-PyTorch mem- ringkas dalam satu kali- ory, this process has 13.93 GiB memory in use. mat atau satu para- Of the allocate graf pendek. Jan- gan menambahkan for- mat EVIDENCE, AN- SWER, atau Sumber.		
AUD09 FP16		Dengarkan audio ini. SIADAD menyediakan informasi akademik seperti biodata mahasiswa, tran- Cosine = 0.7220; La- dengan teliti. Jelaskan formasi akademik seperti formasi akademik seperti biodata mahasiswa, tency = 17.7050 s isi utama audio se-biodata mahasiswa, tran- transcript, danFRS. cara akurat dalam ba- skrip nilai, dan FRS. hasa Indonesia. Jawab ringkas dalam satu kali- mat atau satu para- graf pendek. Jan- gan menambahkan for- mat EVIDENCE, AN- SWER, atau Sumber.		
AUD09 4-bit NF4		Dengarkan audio ini. SIADAD menyediakan informasi akademik seperti biodata mahasiswa, tran- Cosine = 0.4591; La- dengan teliti. Jelaskan formasi akademik seperti biodata mahasiswa, tran- dio. tency = 2.2630 s isi utama audio se-biodata mahasiswa, tran- cara akurat dalam ba- skrip nilai, dan FRS. hasa Indonesia. Jawab ringkas dalam satu kali- mat atau satu para- graf pendek. Jan- gan menambahkan for- mat EVIDENCE, AN- SWER, atau Sumber.		
AUD10 FP32		Dengarkan audio ini. Office 365 menyediakan informasi akademik seperti biodata mahasiswa, tran- Cosine = 0.1156; La- dengan teliti. Jelaskan akses ke Microsoft Of- proses mengalami OutOfMemoryError: CUDA tency = - s isi utama audio se- fice, OneDrive, dan ap- out of memory. Tried to allocate 2.38 GiB. GPU cara akurat dalam ba- likasi kolaborasi lainnya. 0 has a total capacity of 15.46 GiB of which hasa Indonesia. Jawab 1.49 GiB is free. Including non-PyTorch mem- ringkas dalam satu kali- ory, this process has 13.93 GiB memory in use. mat atau satu para- Of the allocate graf pendek. Jan- gan menambahkan for- mat EVIDENCE, AN- SWER, atau Sumber.		
AUD10 FP16		Dengarkan audio ini. Office 365 menyediakan informasi akademik seperti biodata mahasiswa, tran- Cosine = 0.9522; La- dengan teliti. Jelaskan akses ke Microsoft Of- fice, OneDrive, dan aplikasi kolaborasi lainnya. tency = 16.3640 s isi utama audio se- fice, OneDrive, dan ap- cara akurat dalam ba- likasi kolaborasi lainnya. hasa Indonesia. Jawab ringkas dalam satu kali- mat atau satu para- graf pendek. Jan- gan menambahkan for- mat EVIDENCE, AN- SWER, atau Sumber.		
AUD10 4-bit NF4		Dengarkan audio ini. Office 365 menyediakan informasi akademik seperti biodata mahasiswa, tran- Cosine = 0.8575; La- dengan teliti. Jelaskan akses ke Microsoft Of- berikan akses ke Microsoft Office, OneDrive, tency = 3.0600 s isi utama audio se- fice, OneDrive, dan ap- dan aplikasi kolaborasi lainnya. cara akurat dalam ba- likasi kolaborasi lainnya. hasa Indonesia. Jawab ringkas dalam satu kali- mat atau satu para- graf pendek. Jan- gan menambahkan for- mat EVIDENCE, AN- SWER, atau Sumber.		

A.4 Hasil Jawaban Skenario Text-Only RAG Fixed-Size

Bagian ini hanya menyajikan data uji terpilih yang dinilai lebih representatif dan lebih menantang. Pemilihan dilakukan agar lampiran tidak memuat seluruh Q1–Q10, melainkan hanya contoh yang paling relevan untuk menunjukkan kemampuan sistem pada masing-masing skenario pengujian.

Tiga pertanyaan dipilih karena mewakili kebutuhan pencarian informasi multi-entitas, definisi layanan akademik, dan alasan penggunaan Microsoft Authenticator.

Tabel A.5: Hasil jawaban terpilih pada skenario text-only RAG fixed-size

ID	Konfig.	Pertanyaan/Masukan	Ground Truth	Jawaban Sistem	Metrik
Q2	FP32	Aplikasi penting apa saja yang dapat diakses melalui my ITS Portal?	Melalui my ITS Portal, SI Akademik (SIAKAD), my ITS Classroom, my ITS Presensi, dan my ITS Student Connect.	SI Akademik (SIAKAD), my ITS Classroom, my ITS Presensi, dan my ITS Student Connect.	BLEU-1 = 0.7059; ROUGE-L F1 = 0.7586; CP = 0.8875; CR = 1.0000; Latency = 0.7390 s
Q2	FP16	Aplikasi penting apa saja yang dapat diakses melalui my ITS Portal?	Melalui my ITS Portal, SI Akademik (SIAKAD), my ITS Classroom, my ITS Presensi, dan my ITS Student Connect.	SI Akademik (SIAKAD), my ITS Classroom, my ITS Presensi, dan my ITS Student Connect.	BLEU-1 = 0.7059; ROUGE-L F1 = 0.7586; CP = 0.8875; CR = 1.0000; Latency = 1.3410 s
Q2	4-bit NF4	Aplikasi penting apa saja yang dapat diakses melalui my ITS Portal?	Melalui my ITS Portal, SI Akademik (SIAKAD), my ITS Classroom, my ITS Presensi, dan my ITS Student Connect.	Aplikasi penting yang dapat diakses melalui my ITS Portal termasuk SI Akademik (SIAKAD), my ITS Classroom, my ITS Presensi, dan my ITS Student Connect.	BLEU-1 = 0.5714; ROUGE-L F1 = 0.6957; CP = 0.8875; CR = 1.0000; Latency = 5.0310 s
Q7	FP32	Apa itu SIAKAD ITS?	SIAKAD ITS adalah aplikasi di my ITS Portal yang menyediakan layanan akademik, mulai dari biodata mahasiswa, transaksi nilai, pengisian FRS dan perwalian, hingga pengajuan berbagai surat akademik.	SIAKAD ITS adalah aplikasi di my ITS Portal yang menyediakan layanan akademik, mulai dari biodata mahasiswa, transaksi nilai, pengisian FRS dan perwalian, hingga pengajuan berbagai surat akademik.	BLEU-1 = 0.4828; ROUGE-L F1 = 0.6512; CP = 1.0000; CR = 1.0000; Latency = 1.4420 s
Q7	FP16	Apa itu SIAKAD ITS?	SIAKAD ITS adalah aplikasi di my ITS Portal yang menyediakan layanan akademik, mulai dari biodata mahasiswa, transaksi nilai, pengisian FRS dan perwalian, hingga pengajuan berbagai surat akademik.	SIAKAD ITS adalah aplikasi di my ITS Portal yang menyediakan layanan akademik, mulai dari biodata mahasiswa, transaksi nilai, pengisian FRS dan perwalian, hingga pengajuan berbagai surat akademik.	BLEU-1 = 0.4828; ROUGE-L F1 = 0.6512; CP = 1.0000; CR = 1.0000; Latency = 2.5460 s
Q7	4-bit NF4	Apa itu SIAKAD ITS?	SIAKAD ITS adalah aplikasi di my ITS Portal yang menyediakan layanan akademik, mulai dari biodata mahasiswa, transaksi nilai, pengisian FRS dan perwalian, hingga pengajuan berbagai surat akademik.	SIAKAD ITS adalah aplikasi di my ITS Portal yang menyediakan layanan akademik, mulai dari biodata mahasiswa, transaksi nilai, pengisian FRS dan perwalian, hingga pengajuan berbagai surat akademik.	BLEU-1 = 0.4828; ROUGE-L F1 = 0.6512; CP = 1.0000; CR = 1.0000; Latency = 5.7640 s
Q10	FP32	Mengapa Microsoft Authenticator tidak boleh diinstall?	Microsoft Authenticator tidak boleh diinstall karena ia digunakan selama mengakses portal Office 365.	Microsoft Authenticator tidak boleh diinstall karena ia digunakan selama mengakses portal Office 365.	BLEU-1 = 0.5789; ROUGE-L F1 = 0.6875; CP = 1.0000; CR = 1.0000; Latency = 0.8930 s
Q10	FP16	Mengapa Microsoft Authenticator tidak boleh diinstall?	Microsoft Authenticator tidak boleh diinstall karena ia digunakan selama mengakses portal Office 365.	Microsoft Authenticator tidak boleh diinstall karena ia digunakan selama mengakses portal Office 365.	BLEU-1 = 0.5789; ROUGE-L F1 = 0.6875; CP = 1.0000; CR = 1.0000; Latency = 1.4870 s
Q10	4-bit NF4	Mengapa Microsoft Authenticator tidak boleh diinstall?	Microsoft Authenticator tidak boleh diinstall karena ia digunakan secara aktif selama akses ke portal Office 365.	Microsoft Authenticator tidak boleh diinstall karena ia digunakan secara aktif selama akses ke portal Office 365.	BLEU-1 = 0.6667; ROUGE-L F1 = 0.7143; CP = 1.0000; CR = 1.0000; Latency = 2.9780 s

A.5 Hasil Jawaban Skenario Text-Image RAG Fixed-Size

Bagian ini hanya menyajikan data uji terpilih yang dinilai lebih representatif dan lebih menantang. Pemilihan dilakukan agar lampiran tidak memuat seluruh Q1–Q10, melainkan hanya contoh yang paling relevan untuk menunjukkan kemampuan sistem pada masing-masing

skenario pengujian.

Tiga pertanyaan dipilih karena mengharuskan sistem menggabungkan bukti visual dengan konteks dokumen, yaitu identifikasi VPN, daftar aplikasi pada dashboard portal, serta informasi SKEM dan portofolio. Citra masukan yang digunakan pada skenario ini sama dengan data IMG03, IMG06, dan IMG09 pada Tabel A.1.

Tabel A.6: Hasil jawaban terpilih pada skenario text-image RAG fixed-size

ID	Konfig.	Pertanyaan/Masukan	Ground Truth	Jawaban Sistem	Metrik
Q3	FP32	Berdasarkan gambar dan panduan VPN ITS, aplikasi apa yang terlihat dan apa fungsinya?	Aplikasi yang terlihat adalah OpenVPN Connect Client. Fungsinya adalah untuk mengakses jaringan kampus dan sumber daya dari luar lingkungan kampus melalui VPN.	Aplikasi OpenVPN Connect terlihat pada gambar. Fungsinya adalah untuk menghubungkan komputer Anda ke jaringan internet melalui jaringan private virtual private network (VPN).	BLEU-1 = 0.4545; ROUGE-L F1 = 0.4091; CP = 1.0000; CR = 0.0000; Latency = 1.4660 s
Q3	FP16	Berdasarkan gambar dan panduan VPN ITS, aplikasi apa yang terlihat dan apa fungsinya?	Aplikasi yang terlihat adalah OpenVPN Connect Client. Fungsinya adalah untuk mengakses jaringan kampus dan sumber daya dari luar lingkungan kampus melalui VPN.	Aplikasi OpenVPN Connect Client yang terlihat pada gambar. Fungsinya adalah untuk mengakses jaringan kampus dan sumberdaya yang ada di luar lingkungan kampus.	BLEU-1 = 0.7273; ROUGE-L F1 = 0.6364; CP = -; CR = -; Latency = 2.3930 s
Q3	4-bit NF4	Berdasarkan gambar dan panduan VPN ITS, aplikasi apa yang terlihat dan apa fungsinya?	Aplikasi yang terlihat adalah OpenVPN Connect Client. Fungsinya adalah untuk mengakses jaringan kampus dan sumber daya dari luar lingkungan kampus melalui VPN.	Aplikasi OpenVPN Connect, yang digunakan untuk mengakses jaringan kampus dan sumberdaya yang ada di luar lingkungan kampus.	BLEU-1 = 0.5260; ROUGE-L F1 = 0.5641; CP = -; CR = -; Latency = 3.7950 s
Q6	FP32	Berdasarkan gambar dan panduan melalui myITS Portal dan panduan melalui myITS Portal Akademik, aplikasi utama apa saja yang dapat diakses mahasiswa baru ITS?	Aplikasi utama yang dapat diakses mahasiswa melalui portal myITS Portal Akademik, my ITS Classroom, my ITS Presensi, my ITS Wali, dan my ITS Research.	Aplikasi utama yang dapat diakses mahasiswa melalui portal myITS Portal Akademik, my ITS Classroom, my ITS Presensi, my ITS Wali, dan my ITS Research.	BLEU-1 = 0.6154; ROUGE-L F1 = 0.6250; CP = 0.9167; CR = 1.0000; Latency = 1.5180 s
Q6	FP16	Berdasarkan gambar dan panduan melalui myITS Portal dan panduan melalui myITS Portal Akademik, aplikasi utama apa saja yang dapat diakses mahasiswa baru ITS?	Aplikasi utama yang dapat diakses mahasiswa melalui portal myITS Portal Akademik, my ITS Classroom, my ITS Presensi, my ITS Wali, dan my ITS Research.	Aplikasi utama yang dapat diakses mahasiswa melalui portal myITS Portal Akademik, my ITS Classroom, my ITS Presensi, my ITS Wali, dan my ITS Research.	BLEU-1 = 0.6667; ROUGE-L F1 = 0.7347; CP = -; CR = -; Latency = 2.3960 s
Q6	4-bit NF4	Berdasarkan gambar dan panduan melalui myITS Portal dan panduan melalui myITS Portal Akademik, aplikasi utama apa saja yang dapat diakses mahasiswa baru ITS?	Aplikasi utama yang dapat diakses mahasiswa melalui portal myITS Portal Akademik, my ITS Classroom, my ITS Presensi, my ITS Wali, dan my ITS Research.	Akses ke my ITS Portal meliputi beberapa aplikasi penting seperti SI Akademik, my ITS Classroom, my ITS Presensi, my ITS Student Connect, dan masih banyak lagi.	BLEU-1 = 0.3462; ROUGE-L F1 = 0.3333; CP = -; CR = -; Latency = 3.8230 s
Q9	FP32	Berdasarkan gambar dan konteks kelulusan SKEM, layanan kemaha-siswaan ITS, informasi yang telah SKEM dan portofolio sebagai SKEM, serta apa yang ditampilkan pada halaman tersebut?	Halaman tersebut menampilkan status tang status SKEM Anda, yaitu "Lulus". Untuk Connect dan konteks kelulusan SKEM, syarat portofolio, Anda bisa membuat atau mengelola portofolio Anda.	Halaman tersebut menampilkan informasi ten-tang status SKEM Anda, yaitu "Lulus". Untuk Connect dan konteks kelulusan SKEM, syarat portofolio, Anda bisa membuat atau mengelola portofolio Anda.	BLEU-1 = 0.4944; ROUGE-L F1 = 0.4878; CP = 0.0000; CR = 0.0000; Latency = 1.4160 s

ID	Konfig.	Pertanyaan/Masukan	Ground Truth	Jawaban Sistem	Metrik
Q9	FP16	Berdasarkan gam- Halaman tersebut bar myITS Student menampilkan status Connect dan konteks kelulusan SKEM, syarat layanan kemaha- dan keterangan portofolio siswaan ITS, informasi yang telah SKEM dan portofolio sebagai SKEM, serta apa yang ditampilkan opsi untuk membuat atau pada halaman tersebut? mengelola portofolio.	Halaman tersebut menampilkan status SKEM Anda, syarat dan keterangan tentang portofolio yang telah dihitung sebagai SKEM, CP = -; CR = -; Latency = 2.2540 s	BLEU-1 = 0.9167; ROUGE-L F1 = 0.9565;	
Q9	4-bit NF4	Berdasarkan gam- Halaman tersebut bar myITS Student menampilkan status Connect dan konteks kelulusan SKEM, syarat layanan kemaha- dan keterangan portofolio siswaan ITS, informasi yang telah SKEM dan portofolio sebagai SKEM, serta apa yang ditampilkan opsi untuk membuat atau pada halaman tersebut? mengelola portofolio.	Halaman tersebut menampilkan status SKEM Anda, syarat dan kriteria pengambilan portofolio, dan informasi tentang pen- CP = -; CR = -; Latency = 4.0190 s	BLEU-1 = 0.4494; ROUGE-L F1 = 0.4878;	

Bagian ini hanya menyajikan data uji terpilih yang dinilai lebih representatif dan lebih menantang. Pemilihan dilakukan agar lampiran tidak memuat seluruh Q1–Q10, melainkan hanya contoh yang paling relevan untuk menunjukkan kemampuan sistem pada masing-masing skenario pengujian.

Tabel A.7: Hasil jawaban terpilih pada skenario text-audio RAG fixed-size

136

ID	Konfig.	Pertanyaan/Masukan	Ground Truth	Jawaban Sistem	Metrik
TAUD10FP32		Berdasarkan fasilitas apa yang disediakan Office 365?	audio, Office 365 memberikan akses ke Microsoft Office, OneDrive, dan aplikasi kolaborasi lainnya.	Tidak menghasilkan jawaban valid karena proses mengalami RuntimeError: Audio phase failed: OutOfMemoryError: CUDA out of memory. Tried to allocate 2.38 GiB. GPU 0 has a total capacity of 15.46 GiB of which 1.49 GiB is free. Including non-PyTorch memory, this process has 13.93	BLEU-1 = -; ROUGE-L F1 = -; CP = -; CR = -; Latency = - s
TAUD10FP16		Berdasarkan fasilitas apa yang disediakan Office 365?	audio, Office 365 memberikan akses ke Microsoft Office, OneDrive, dan aplikasi kolaborasi lainnya.	Office 365 menyediakan akses ke Microsoft Office, OneDrive, dan aplikasi kolaborasi lainnya.	BLEU-1 = 0.9167; ROUGE-L F1 = 0.9167; CP = 0.3333; CR = 1.0000; Latency = 18.3250 s
TAUD104-bit NF4		Berdasarkan fasilitas apa yang disediakan Office 365?	audio, Office 365 memberikan akses ke Microsoft Office, OneDrive, dan aplikasi kolaborasi lainnya.	Office 365 memberikan akses ke Microsoft Office, OneDrive, dan aplikasi kolaborasi lainnya.	BLEU-1 = 1.0000; ROUGE-L F1 = 1.0000; CP = 0.5000; CR = 1.0000; Latency = 5.4900 s

Bagian ini hanya menyajikan data uji terpilih yang dinilai lebih representatif dan lebih menantang. Pemilihan dilakukan agar lampiran tidak memuat seluruh Q1–Q10, melainkan hanya contoh yang paling relevan untuk menunjukkan kemampuan sistem pada masing-masing skenario pengujian.

Tabel A.8: Hasil jawaban terpilih pada skenario text-only RAG recursive

137

Tabel A.8 – lanjutan

ID	Konfig.	Pertanyaan/Masukan	Ground Truth	Jawaban Sistem	Metrik
Q10	FP32	Mengapa Microsoft Authenticator tidak boleh diinstall?	Microsoft Authenticator tidak boleh diinstall karena akan selalu digunakan untuk mengakses portal Office 365.	Microsoft Authenticator tidak boleh diinstall karena selalu digunakan selama mengakses portal Office 365 untuk akses internet multi-factor authentication (MFA).	BLEU-1 = 0.6316; ROUGE-L F1 = 0.7500; CP = 0.5000; CR = 1.0000; Latency = 0.6770 s
Q10	FP16	Mengapa Microsoft Authenticator tidak boleh diinstall?	Microsoft Authenticator tidak boleh diinstall karena akan selalu digunakan untuk mengakses portal Office 365.	Microsoft Authenticator tidak boleh diinstall karena selalu digunakan selama mengakses portal Office 365 untuk akses internet multi-factor authentication (MFA).	BLEU-1 = 0.6316; ROUGE-L F1 = 0.7500; CP = 0.5000; CR = 1.0000; Latency = 1.4230 s
Q10	4-bit NF4	Mengapa Microsoft Authenticator tidak boleh diinstall?	Microsoft Authenticator tidak boleh diinstall karena akan selalu digunakan untuk mengakses portal Office 365.	Microsoft Authenticator tidak boleh diinstall karena selalu digunakan selama mengakses portal Office 365. AKSES INTERNET MULTI-FACTOR AUTHENTICATION (MFA) 4 Pan-duan lebih lanjut Akses Internet ITS, silahkan kunjungi: https:// .	BLEU-1 = 0.4444; ROUGE-L F1 = 0.6000; CP = 0.5000; CR = 1.0000; Latency = 6.1960 s

A.8 Hasil Jawaban Skenario Text-Image RAG Recursive

Bagian ini hanya menyajikan data uji terpilih yang dinilai lebih representatif dan lebih menantang. Pemilihan dilakukan agar lampiran tidak memuat seluruh Q1–Q10, melainkan hanya contoh yang paling relevan untuk menunjukkan kemampuan sistem pada masing-masing skenario pengujian.

Tiga pertanyaan dipilih agar hasil recursive chunking dapat dibandingkan pada pertanyaan text-image yang sama dengan skenario fixed-size. Citra masukan yang digunakan tetap mengacu pada data IMG03, IMG06, dan IMG09 pada Tabel A.1.

Tabel A.9: Hasil jawaban terpilih pada skenario text-image RAG recursive

ID	Konfig.	Pertanyaan/Masukan	Ground Truth	Jawaban Sistem	Metrik
Q3	FP32	Berdasarkan gambar dan panduan VPN, apa fungsi aplikasi yang terlihat?	Aplikasi yang terlihat dalam gambar adalah OpenVPN Connect. Fungsinya adalah untuk menghubungkan komputer Anda ke jaringan kampus dan sumber daya dari luar lingkungan kampus melalui VPN.	Aplikasi yang terlihat dalam gambar adalah OpenVPN Connect. Fungsinya adalah untuk menghubungkan komputer Anda ke jaringan kampus menggunakan tunneling SSL/TLS.	BLEU-1 = 0.4995; ROUGE-L F1 = 0.5116; CP = 0.4167; CR = 1.0000; Latency = 1.2330 s
Q3	FP16	Berdasarkan gambar dan panduan VPN, apa fungsi aplikasi yang terlihat?	Aplikasi yang terlihat dalam gambar adalah OpenVPN Connect Client yang digunakan untuk mengakses jaringan kampus dan sumber daya dari luar lingkungan kampus melalui VPN.	Aplikasi OpenVPN Connect Client yang terlihat dalam gambar adalah aplikasi yang digunakan untuk mengakses jaringan kampus dan sumber daya yang ada di luar lingkungan kampus.	BLEU-1 = 0.4250; ROUGE-L F1 = 0.4516; CP = -; CR = -; Latency = 3.2050 s
Q3	4-bit NF4	Berdasarkan gambar dan panduan VPN, apa fungsi aplikasi yang terlihat?	Aplikasi yang terlihat dalam gambar adalah OpenVPN Connect, digunakan untuk menghubungkan ke jaringan kampus.	Aplikasi OpenVPN Connect, digunakan untuk menghubungkan ke jaringan kampus.	BLEU-1 = 0.1573; ROUGE-L F1 = 0.3871; CP = -; CR = -; Latency = 2.0570 s
Q6	FP32	Berdasarkan gambar dashboard myITS, Portal dan panduan mahasiswa baru ITS, aplikasi utama apa saja yang dapat diakses mahasiswa melalui portal tersebut?	Aplikasi utama yang dapat diakses mahasiswa melalui portal myITS adalah myITS Portal Classroom, myITS Presensi, myITS Wali, dan myITS Thesis.	Aplikasi utama yang dapat diakses mahasiswa melalui portal myITS adalah myITS Portal Classroom, myITS Presensi, myITS Wali, dan myITS Thesis.	BLEU-1 = 0.8144; ROUGE-L F1 = 0.8095; CP = 1.0000; CR = 0.8333; Latency = 1.2450 s

Tabel A.9 – lanjutan

ID	Konfig.	Pertanyaan/Masukan	Ground Truth	Jawaban Sistem	Metrik
Q6	FP16	Berdasarkan gambar dashboard myITS dapat diakses melalui Portal dan panduan mahasiswa baru ITS, adalah SI Akademik, myITS Classroom, myITS aplikasi utama apa saja yang dapat diakses mahasiswa melalui Connect, dan myITS portal tersebut?	Aplikasi utama yang dapat diakses mahasiswa melalui Portal Akademik (SIKAD), myITS Classroom, myITS Student Connect, dan myITS ITS Research.	Aplikasi utama yang dapat diakses mahasiswa melalui Portal adalah SI ROUGE-L F1 = 0.6800; CP = -; CR = -; Latency = 2.1090 s	BLEU-1 = 0.6071; ROUGE-L F1 = 0.6800; CP = -; CR = -; Latency = 2.1090 s
Q6	4-bit NF4	Berdasarkan gambar myITS dapat diakses melalui Portal dan panduan mahasiswa baru ITS, adalah SI Akademik, my-aplikasi utama apa saja yang dapat diakses mahasiswa melalui Connect, dan myITS portal tersebut?	Aplikasi utama yang dapat diakses mahasiswa melalui Portal Akademik (SIKAD), myITS Classroom, myITS Student Connect, dan myITS ITS Research.	Akses ke myITS Classroom, myITS Presensi, myITS Student Connect, dan myITS Services.	BLEU-1 = 0.2148; ROUGE-L F1 = 0.2632; CP = -; CR = -; Latency = 2.1760 s
Q9	FP32	Berdasarkan gambar myITS Student Connect dan konteks kelulusan SKEM, syarat man juga menampilkan portofolio yang telah dihitung SKEM dan portofolio sebagai SKEM, serta apa yang ditampilkan opsi untuk membuat atau pada halaman tersebut? mengelola portofolio.	Halaman tersebut menampilkan status kelulusan SKEM Anda, syarat dan kriteria portofolio yang telah diatur sebagai SKEM, serta portofolio yang belum diatur.	Halaman tersebut menampilkan status kelulusan SKEM Anda, syarat dan kriteria portofolio yang telah diatur sebagai SKEM, serta portofolio yang belum diatur.	BLEU-1 = 0.6811; ROUGE-L F1 = 0.6977; CP = 0.0000; CR = 0.0000; Latency = 1.2270 s
Q9	FP16	Berdasarkan gambar myITS Student Connect dan konteks kelulusan SKEM, syarat man juga menampilkan portofolio yang telah dihitung SKEM dan portofolio sebagai SKEM, serta apa yang ditampilkan opsi untuk membuat atau pada halaman tersebut? mengelola portofolio.	Halaman tersebut menampilkan status kelulusan SKEM Anda, yaitu "Lulus". Selain itu, halaman tersebut menampilkan portofolio yang telah dihitung sebagai SKEM.	Halaman tersebut menampilkan status kelulusan SKEM Anda, yaitu "Lulus". Selain itu, halaman tersebut menampilkan portofolio yang telah dihitung sebagai SKEM.	BLEU-1 = 0.5429; ROUGE-L F1 = 0.5714; CP = -; CR = -; Latency = 1.7720 s
Q9	4-bit NF4	Berdasarkan gambar myITS Student Connect dan konteks kelulusan SKEM, syarat man juga menampilkan portofolio yang telah dihitung SKEM dan portofolio sebagai SKEM, serta apa yang ditampilkan opsi untuk membuat atau pada halaman tersebut? mengelola portofolio.	Halaman tersebut menampilkan status kelulusan SKEM, syarat dan kriteria portofolio yang telah diatur sebagai SKEM, serta portofolio yang belum diatur.	Halaman tersebut menampilkan status kelulusan SKEM, syarat dan kriteria portofolio yang telah diatur sebagai SKEM, serta portofolio yang belum diatur.	BLEU-1 = 0.6786; ROUGE-L F1 = 0.7143; CP = -; CR = -; Latency = 3.3010 s

A.9 Hasil Jawaban Skenario Text-Audio RAG Recursive

Bagian ini hanya menyajikan data uji terpilih yang dinilai lebih representatif dan lebih menantang. Pemilihan dilakukan agar lampiran tidak memuat seluruh Q1–Q10, melainkan hanya contoh yang paling relevan untuk menunjukkan kemampuan sistem pada masing-masing skenario pengujian.

Tiga pertanyaan dipilih agar hasil recursive chunking dapat dibandingkan pada pertanyaan text-audio yang sama dengan skenario fixed-size. Audio masukan yang digunakan tetap mengacu pada data AUD03, AUD08, dan AUD10 pada Tabel A.2.

Tabel A.10: Hasil jawaban terpilih pada skenario text-audio RAG recursive

ID	Konfig.	Pertanyaan/Masukan	Ground Truth	Jawaban Sistem	Metrik
TAUD03FP32		Berdasarkan audio, apa fungsi VPN ITS?	VPN ITS digunakan untuk mengakses jaringan kampus dari luar lingkungan ITS.	VPN ITS memberikan akses ke jaringan kampus, memudahkan civitas ITS untuk mengakses informasi dan fasilitas yang mereka butuhkan.	BLEU-1 = 0.3077; ROUGE-L F1 = 0.3243; CP = 1.0000; CR = 1.0000; Latency = 66.8870 s
TAUD03FP16		Berdasarkan audio, apa fungsi VPN ITS?	VPN ITS digunakan untuk mengakses jaringan kampus dari luar lingkungan ITS.	VPN ITS memberikan akses ke jaringan kampus, memudahkan civitas ITS untuk mengakses informasi dan fasilitas yang mereka butuhkan.	BLEU-1 = 0.3077; ROUGE-L F1 = 0.3243; CP = 1.0000; CR = 1.0000; Latency = 22.3210 s

Tabel A.10 – lanjutan

ID	Konfig.	Pertanyaan/Masukan	Ground Truth	Jawaban Sistem	Metrik
TAUD034-bit NF4		Berdasarkan audio, apa fungsi VPN ITS?	VPN ITS digunakan untuk mengakses jaringan kampus dari luar lingkungan ITS.	un-VPN ITS memberikan akses ke jaringan pus dan sumber daya luar lingkungan kampus melalui fitur Search di Outlook.	BLEU-1 = 0.3333; ROUGE-L F1 = 0.4138; CP = 1.0000; CR = 1.0000; Latency = 6.8800 s
TAUD08FP32		Berdasarkan audio, informasi apa yang terse-dia pada myITS Student Connect?	in-myITS Student Connect menampilkan informasi SKEM, portofolio, dan layanan kemahasiswaan.	Tidak menghasilkan jawaban valid karena proses mengalami RuntimeError: Audio phase failed: OutOfMemoryError: CUDA out of memory. Tried to allocate 2.38 GiB. GPU 0 has a total capacity of 15.46 GiB of which 1.54 GiB is free. Including non-PyTorch memory, this process has 13.88	BLEU-1 = -; ROUGE-L F1 = -; CP = -; CR = -; Latency = - s
TAUD08FP16		Berdasarkan audio, informasi apa yang terse-dia pada myITS Student Connect?	in-myITS Student Connect menampilkan informasi SKEM, portofolio, dan layanan kemahasiswaan.	MyITS Student Connect memberikan akses email menggunakan Microsoft Outlook untuk mengirim dan menerima pesan.	BLEU-1 = 0.2857; ROUGE-L F1 = 0.3333; CP = 0.0000; CR = 0.0000; Latency = 19.2050 s
TAUD084-bit NF4		Berdasarkan audio, informasi apa yang terse-dia pada myITS Student Connect?	in-myITS Student Connect menampilkan informasi SKEM, portofolio, dan layanan kemahasiswaan.	Anda dapat mengakses Microsoft Outlook untuk mengirim dan menerima pesan.	BLEU-1 = 0.1000; ROUGE-L F1 = 0.1000; CP = 0.0000; CR = 0.0000; Latency = 4.0300 s
TAUD10FP32		Berdasarkan audio, fasilitas apa yang disediakan Office 365?	Office 365 memberikan akses ke Microsoft fice, OneDrive, dan aplikasi kolaborasi lainnya.	Tidak menghasilkan jawaban valid karena proses mengalami RuntimeError: Audio phase failed: OutOfMemoryError: CUDA out of memory. Tried to allocate 2.38 GiB. GPU 0 has a total capacity of 15.46 GiB of which 1.49 GiB is free. Including non-PyTorch memory, this process has 13.93	BLEU-1 = -; ROUGE-L F1 = -; CP = -; CR = -; Latency = - s
TAUD10FP16		Berdasarkan audio, fasilitas apa yang disediakan Office 365?	Office 365 memberikan akses ke Microsoft fice, OneDrive, dan aplikasi kolaborasi lainnya.	Fasilitas yang disediakan Office 365 mencakup Microsoft Office, OneDrive, dan aplikasi kolaborasi lainnya.	BLEU-1 = 0.6923; ROUGE-L F1 = 0.7200; CP = 1.0000; CR = 1.0000; Latency = 19.6850 s
TAUD104-bit NF4		Berdasarkan audio, fasilitas apa yang disediakan Office 365?	Office 365 memberikan akses ke Microsoft fice, OneDrive, dan aplikasi kolaborasi lainnya.	Office 365 memberikan akses ke Microsoft Of-fice, OneDrive, dan aplikasi kolaborasi lainnya.	BLEU-1 = 1.0000; ROUGE-L F1 = 1.0000; CP = 1.0000; CR = 1.0000; Latency = 5.6630 s

A.10 Ringkasan Pengujian Tambahan Fixed-Size INT8 dan FP32 Strict

Bagian ini menyajikan ringkasan pengujian tambahan pada konfigurasi fixed-size. Pengujian tambahan digunakan untuk memperjelas pengaruh konfigurasi representasi numerik model terhadap waktu respons sistem. Konfigurasi INT8 yang digunakan adalah INT8 Weight-Only A16W8 dengan TF32 aktif pada jalur komputasi yang relevan, sedangkan FP32 strict digunakan dengan TF32 nonaktif sebagai kontrol untuk membaca perilaku FP32 penuh. Pengujian tambahan ini difokuskan pada latency, sehingga context precision dan context recall tidak dihitung ulang pada bagian ini.

Tabel A.11: Ringkasan pengujian tambahan fixed-size INT8 dan FP32 strict

Skenario	Konfigurasi	Fokus Evaluasi	Metrik Kualitas	Latency	Catatan
Text-Only RAG Fixed-Size	INT8 WO TF32 ON A16W8	+ Latency dan kualitas jawaban manual	BLEU-1 = 0.5002; ROUGE-L F1 = 0.6049	0.9000 s	Latency paling rendah pada skenario text-only fixed-size.
Text-Only RAG Fixed-Size	FP32 strict / TF32 OFF	Kontrol FP32 penuh	BLEU-1 = 0.4494; ROUGE-L F1 = 0.5418	2.1554 s	Digunakan untuk membedakan FP32 penuh dari FP32 yang terbantu optimasi backend.
Image-Only	INT8 WO TF32 ON A16W8	+ Latency dan kesesuaian semantik	Cosine = 0.7444	3.8000 s	Menghasilkan cosine similarity tertinggi dan latency lebih rendah daripada FP16, FP32 strict, dan 4-bit NF4.

Tabel A.11 – lanjutan

Skenario	Konfigurasi	Fokus Evaluasi	Metrik Kualitas	Latency	Catatan
Image-Only	FP32 strict / TF32 OFF	Kontrol FP32 penuh	Cosine = 0.7097	7.0000 s	Latency meningkat setelah TF32 dinonaktifkan.
Audio-Only	INT8 WO A16W8 TF32 ON	+ Latency dan kesesuaian semantik	Cosine = 0.6502	1.4719 s	Menjadi konfigurasi paling efisien pada skenario audio-only.
Text-Image Fixed-Size	RAG INT8 WO A16W8 TF32 ON	+ Latency dan kualitas jawaban manual	BLEU-1 = 0.3693; 2.0850 s ROUGE-L F1 = 0.3768		Lebih cepat daripada FP16 dan 4-bit NF4 pada skenario text-image fixed-size.
Text-Image Fixed-Size	RAG FP32 strict / TF32 OFF	Kontrol FP32 penuh	BLEU-1 = 0.3371; 17.4636 s ROUGE-L F1 = 0.3531		Menunjukkan peningkatan latency besar ketika TF32 dinonaktifkan.
Text-Audio Fixed-Size	RAG INT8 WO A16W8 TF32 ON	+ Latency dan kualitas jawaban manual	BLEU-1 = 0.4758; 3.4230 s ROUGE-L F1 = 0.5406		Latency lebih rendah daripada 4-bit NF4, FP16, dan FP32 pada pembandingan fixed-size.

Tabel A.12: Urutan latency hasil pengujian tambahan fixed-size

Skenario	Urutan Latency
Text-Only RAG Fixed-Size	INT8 WO < FP16 < FP32 strict < 4-bit NF4.
Image-Only	INT8 WO < FP16 < FP32 strict < 4-bit NF4.
Audio-Only	INT8 WO < 4-bit NF4 < FP16 < FP32.
Text-Image RAG Fixed-Size	INT8 WO < FP16 < 4-bit NF4 < FP32 strict.
Text-Audio RAG Fixed-Size	INT8 WO < 4-bit NF4 < FP16 < FP32.

[Halaman ini sengaja dikosongkan]

BIOGRAFI PENULIS



Defender Artha Widiprasetyo, lahir di Boyolali pada Mei 2004. Penulis merupakan mahasiswa Program Studi Teknik Komputer, Institut Teknologi Sepuluh Nopember. Selama menempuh pendidikan, penulis memiliki minat pada bidang pengembangan perangkat lunak, pengembangan web, Internet of Things (IoT), kecerdasan buatan, serta penerapan teknologi dalam penyelesaian permasalahan nyata.

Selama masa perkuliahan, penulis memperoleh pengalaman melalui berbagai kegiatan akademik, organisasi, magang, dan proyek profesional. Penulis pernah terlibat dalam pengembangan aplikasi web, sistem monitoring berbasis IoT, dashboard digital, serta pengembangan antarmuka pengguna. Penulis juga aktif dalam kegiatan organisasi kemahasiswaan, khususnya pada bidang kesejahteraan mahasiswa di lingkungan Himpunan Mahasiswa Teknik Komputer ITS.

Tugas akhir ini disusun sebagai salah satu syarat untuk menyelesaikan studi sarjana. Penelitian yang dilakukan berfokus pada pengembangan sistem *question answering* multimodal berbasis model Qwen dengan metode *Retrieval-Augmented Generation* (RAG). Melalui penelitian ini, penulis berharap hasil yang diperoleh dapat memberikan kontribusi dalam pengembangan sistem pencarian dan penjawaban informasi berbasis dokumen, citra, dan audio, khususnya pada lingkungan institusi pendidikan.

[Halaman ini sengaja dikosongkan]